UNIVERSITAS INDONESIA

PENDEKATAN STATIS DAN DYNAMIC BIPLOT UNTUK ANALISIS SPASIO-TEMPORAL SOSIO-EKONOMI (STUDI KASUS: PROVINSI BANTEN, DKI JAKARTA, DAN JAWA BARAT 2020–2024)

SKRIPSI

AMMAR HANAFI
2206051582

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI SARJANA STATISTIKA
DEPOK
JUNI 2026



UNIVERSITAS INDONESIA

PENDEKATAN STATIS DAN DYNAMIC BIPLOT UNTUK ANALISIS SPASIO-TEMPORAL SOSIO-EKONOMI (STUDI KASUS: PROVINSI BANTEN, DKI JAKARTA, DAN JAWA BARAT 2020–2024)

SKRIPSI

Diajukan sebagai salah satu syarat untuk memperoleh gelar
Sarjana Statistika

AMMAR HANAFI
2206051582

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI SARJANA STATISTIKA
DEPOK
JUNI 2026

HALAMAN PERSETUJUAN

Judul	:	Pendekatan STATIS dan Dynamic Biplot untuk Analisis Spasio-Temporal Sosio-Ekonomi (Studi Kasus: Provinsi Banten, DKI Jakarta, dan Jawa Barat 2020–2024)
Nama	:	Ammar Hanafi
NPM	:	2206051582

Laporan Skripsi ini telah diperiksa dan disetujui.

17 April 2026

Dr. Dra. Yekti Widyaningsih, M.Si.

Pembimbing Skripsi

HALAMAN PERNYATAAN ORISINALITAS

Skripsi ini adalah hasil karya saya sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah saya nyatakan dengan benar.

Nama : Ammar Hanafi

NPM : 2206051582

Tanda Tangan

Tanggal : 17 April 2026

HALAMAN PENGESAHAN

Skripsi ini diajukan oleh	:	
Nama	:	Ammar Hanafi
NPM	:	2206051582
Program Studi	:	Sarjana Statistika
Judul Skripsi	:	Pendekatan STATIS dan Dynamic Biplot untuk Analisis Spasio-Temporal Sosio-Ekonomi (Studi Kasus: Provinsi Banten, DKI Jakarta, dan Jawa Barat 2020–2024)

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Statistika pada Program Studi Sarjana Statistika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Indonesia.

DEWAN PENGUJI

Pembimbing I	:	Dr. Dra. Yekti Widyaningsih, M.Si.	( )
Penguji I	:	Sarini Abdullah, S.Si., M.Stats., Ph.D.	( )
Penguji II	:	Gianinna Ardaneswari, S.Si., M.Si.	( )

Ditetapkan di	:	Depok
Tanggal	:	27 April 2026

HALAMAN PERNYATAAN PENGGUNAAN KECERDASAN ARTIFISIAL

Saya menyatakan bahwa dalam penyusunan Tugas Akhir ini, saya menggunakan teknologi Kecerdasan Buatan dengan *platform* Antigravity untuk tujuan membantu dalam penulisan Tugas Akhir ini dan saya bertanggung jawab penuh atas keakuratan, orisinalitas, dan integritas seluruh isi Tugas Akhir ini.

Nama	**:**	**Ammar Hanafi**
NPM	**:**	**2206051582**
Tanda Tangan	**:**	
Tanggal	**:**	**17 April 2026**

KATA PENGANTAR

Puji syukur kepada Tuhan Yang Maha Esa atas segala berkat dan rahmat-Nya, sehingga penulis dapat menyelesaikan skripsi ini dengan sebaik-baiknya. Selama masa penulisan skripsi ini, penulis telah mendapat banyak dukungan, doa, bantuan, inspirasi, dan motivasi dari berbagai pihak. Untuk itu, penulis ingin berterima kasih kepada:

1. Semua staf pengajar atas ilmu pengetahuan yang telah diberikan kepada penulis selama masa kuliah.
2. Seluruh staf karyawan yang selalu melakukan tugas mereka dengan baik, sehingga memberikan kenyamanan pelayanan bagi siapa saja.
3. Seluruh teman-teman angkatan 2022 atas semangat, canda tawa dan banyak hal penting selama masa kuliah.
4. Kakak-kakak angkatan 2021, 2020, dan 2019 serta adik-adik angkatan 2023, dan 2024 yang terus memberi semangat.

Ammar Hanafi

2026

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

Nama	:	Ammar Hanafi
NPM	:	2206051582
Program Studi	:	Sarjana Statistika
Fakultas	:	Matematika dan Ilmu Pengetahuan Alam
Jenis Karya	:	Skripsi

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif (*Non-exclusive Royalty Free Right*)** atas karya ilmiah saya yang berjudul:

Pendekatan STATIS dan Dynamic Biplot untuk Analisis Spasio-Temporal Sosio-Ekonomi (Studi Kasus: Provinsi Banten, DKI Jakarta, dan Jawa Barat 2020–2024)

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini, Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*data base*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Kota Depok
Pada tanggal : 17 April 2026

Yang menyatakan

(Ammar Hanafi)

ABSTRAK

Nama	:	Ammar Hanafi
Program Studi	:	Sarjana Statistika
Judul	:	Pendekatan STATIS dan Dynamic Biplot untuk Analisis Spasio-Temporal Sosio-Ekonomi (Studi Kasus: Provinsi Banten, DKI Jakarta, dan Jawa Barat 2020–2024)

Dinamika pembangunan di wilayah strategis Provinsi Banten, DKI Jakarta, dan Jawa Barat menunjukkan kompleksitas yang tinggi selama periode 2020–2024. Sebagai pusat aglomerasi ekonomi nasional, ketiga provinsi ini mengalami perubahan simultan akibat pandemi COVID-19 yang diikuti oleh fase pemulihan yang tidak seragam antar-kabupaten/kota. Perubahan ini tidak hanya terjadi pada indikator makroekonomi, tetapi juga merambat pada dimensi kesejahteraan sosial dan kualitas modal manusia. Untuk memahami evolusi struktur hubungan antar-indikator tersebut secara menyeluruh, diperlukan pendekatan analisis data tiga-arah (*three-way data*) yang mampu memahami pola interaksi spasio-temporal.

Penelitian ini menerapkan metode STATIS (*Structuration des Tableaux à Trois Indices de la Statistique*) untuk menganalisis dinamika 12 indikator pembangunan strategis—meliputi aspek ekonomi (PDRB), kesejahteraan (Kemiskinan, Garis Kemiskinan, Pangan), pembangunan manusia (IPM, Pendidikan, Kesehatan), ketenagakerjaan, serta kesetaraan (Gender, Gini Ratio)—di seluruh kabupaten/kota pada ketiga provinsi tersebut. Analisis berpusat pada pembentukan struktur kompromi (*compromise structure*). Merujuk pada definisi Lavit et al. (1994), struktur kompromi ($\mathbf{W}$) dikonstruksi sebagai jumlah tertimbang (*weighted sum*) dari matriks intra-struktur tahunan, di mana bobot (α_k) ditentukan berdasarkan dekomposisi spektral untuk memaksimumkan kesamaan antar-tahun. Struktur ini merepresentasikan pola hubungan antar-wilayah yang paling dominan dan stabil selama periode pengamatan. Selanjutnya, penelitian ini menguji signifikansi stabilitas struktur tersebut menggunakan uji permutasi dan mengintegrasikan konsep *Dynamic Biplot* untuk memvisualisasikan lintasan (*trajectory*) pergerakan kabupaten/kota dari masa krisis hingga pemulihan dalam ruang berdimensi rendah.

Seluruh prosedur analisis, mulai dari pra-pemrosesan data hingga visualisasi, diimplementasikan menggunakan bahasa pemrograman Python. Hasil akhir dari penelitian ini disajikan dalam bentuk aplikasi *dashboard* web interaktif berbasis *framework* Dash dan Plotly. Aplikasi ini memungkinkan pengguna untuk melakukan eksplorasi mandiri melalui fitur rotasi *baseline* dan filter lintasan, sehingga diharapkan dapat menjadi alat bantu visual yang efektif bagi peneliti maupun pemangku kebijakan dalam memantau resiliensi dan pemerataan pembangunan daerah pasca-pandemi.

Kata kunci: Banten-DKI-Jabar, Dynamic Biplot, Pembangunan Regional, Spasio-Temporal, STATIS, Visualisasi Interaktif.

ABSTRACT

Name	:	Ammar Hanafi
Program	:	Undergraduate Study Program of Statistics
Title	:	STATIC and Dynamic Biplot Approaches for Socio-Economic Spatio-Temporal Analysis (Case Studies: Province of Banten, DKI Jakarta, and West Java 2020–2024)

The development dynamics in the strategic regions of Banten, DKI Jakarta, and West Java exhibited high complexity during the 2020–2024 period. As the center of national economic agglomeration, these three provinces experienced simultaneous changes due to the COVID-19 pandemic, followed by an uneven recovery phase across regencies/cities. These changes occurred not only in macroeconomic indicators but also propagated to dimensions of social welfare and human capital quality. To thoroughly understand the evolution of the structural relationships among these indicators, a three-way data analysis approach capable of capturing spatio-temporal interaction patterns is required.

This research applies the STATIS (*Structuration des Tableaux à Trois Indices de la Statistique*) method to analyze the dynamics of 12 strategic development indicators—covering economic aspects (GRDP), welfare (Poverty, Poverty Line, Food), human development (HDI, Education, Health), labor, and equality (Gender, Gini Ratio)—across all regencies/cities in the three provinces. The analysis centers on the formation of a compromise structure. Referring to the definition by Lavit et al. (1994), the compromise structure ($\mathbf{W}$) is constructed as a weighted sum of annual intra-structure matrices, where the weights (α_k) are determined based on spectral decomposition to maximize inter-year similarity. This structure represents the most dominant and stable inter-regional relationship patterns during the observation period. Furthermore, this study tests the significance of the structure's stability using permutation tests and integrates the Dynamic Biplot concept to visualize the movement trajectories of regencies/cities from the crisis to the recovery period in a low-dimensional space.

The entire analysis procedure, from data pre-processing to visualization, is implemented using the Python programming language. The final result of this research is presented in the form of an interactive web dashboard application based on the Dash and Plotly frameworks. This application allows users to perform independent exploration through baseline rotation and trajectory filtering features, thereby serving as an effective visual aid for researchers and policymakers in monitoring regional resilience and development equity post-pandemic.

Keywords: Banten-DKI-Jabar, Dynamic Biplot, Interactive Visualization, Regional Development, Spatio-Temporal, STATIS.

DAFTAR ISI

HALAMAN JUDUL . . . . . **i**

LEMBAR PERSETUJUAN . . . . . **iii**

HALAMAN PERNYATAAN ORISINALITAS . . . . . **iv**

HALAMAN PENGESAHAN . . . . . **v**

SURAT PERNYATAAN . . . . . **vi**

KATA PENGANTAR . . . . . **vii**

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI . . . . . **viii**

ABSTRAK . . . . . **ix**

ABSTRACT . . . . . **x**

DAFTAR ISI . . . . . **xi**

DAFTAR TABEL . . . . . **xiv**

DAFTAR GAMBAR . . . . . **xvi**

DAFTAR LAMPIRAN **1**

BAB 1 PENDAHULUAN **2**

1.1 Latar Belakang 2

1.2 Rumusan Masalah 5

1.3 Tujuan Penelitian 6

1.4 Manfaat Penelitian 7

1.5 Ruang Lingkup Penelitian 7

BAB 2 LANDASAN TEORI **8**

2.1 Indikator Kesejahteraan Rakyat 8
2.1.1 Indeks Pembangunan Manusia (IPM) 8
2.1.2 Produk Domestik Regional Bruto (PDRB ADHK) 9
2.1.3 Persentase Penduduk Miskin (PPM) 9
2.1.4 Garis Kemiskinan (GK) 9
2.1.5 Prevalensi Ketidakcukupan Konsumsi Pangan (PKKP) 9
2.1.6 Tingkat Pengangguran Terbuka (TPT) 10
2.1.7 Tingkat Partisipasi Angkatan Kerja (TPAK) 10
2.1.8 Rata-Rata Lama Sekolah (RLS) 10
2.1.9 Angka Harapan Hidup Laki-laki 10
2.1.10 Angka Harapan Hidup Perempuan 10
2.1.11 Indeks Pembangunan Gender (IPG) 11
2.1.12 Gini Ratio (GR) 11
2.2 Tinjauan Penelitian Terdahulu 11
2.3 Analisis Data Tiga-Arah (*Three-Way Data*) 12
2.4 Konsep Spasio-Temporal dalam Analisis Multivariat 13
2.5 Standardisasi Data 14
2.6 Statistika Deskriptif 14
2.6.1 Ukuran Pemusatan 14
2.6.2 Ukuran Penyebaran 15
2.6.3 Ukuran Bentuk Distribusi 16
2.7 Pemilihan Variabel dalam Analisis Multivariat 16
2.7.1 Uji Kaiser-Meyer-Olkin (KMO) 17
2.7.2 Uji Bartlett tentang Spherisitas (*Bartlett's Test of Sphericity*) . . . 18
2.8 Dasar Aljabar Linier dan Dekomposisi Matriks 19
2.9 Korelasi Pearson 21
2.10 Analisis Komponen Utama (PCA) dan Biplot 22
2.11 Metode STATIS 23
2.11.1 Tahap 1: Intra-struktur 23
2.11.2 Tahap 2: Inter-struktur 24
2.11.3 Tahap 3: Analisis Kompromi 25
2.11.4 Uji Signifikansi Stabilitas Struktur (Permutasi) 26
2.12 Dynamic Biplot dan Visualisasi Trajektori 27
2.12.1 Proyeksi Lintasan (*Trajectory*) 27
2.12.2 Jarak Euclidean antar Trajektori 28
2.12.3 Rotasi Baseline 28

2.13 Metode Pengelompokan (*Clustering*) 29
2.13.1 K-Means Clustering 29
2.13.2 Penentuan Jumlah Klaster Optimal (Metode Elbow) 30
2.13.3 Agglomerative Hierarchical Clustering (Dendrogram) 30
2.13.4 Matriks Transisi Temporal 30
2.14 Uji Autokorelasi Spasial (Global Moran's I) 31
2.14.1 Matriks Pembobot Spasial 31
2.14.2 Statistik Global Moran's I 32
2.15 Visualisasi Interaktif dengan Plotly dan Dash 32
2.16 Kerangka Pemikiran 33

BAB 3 METODOLOGI PENELITIAN 35
3.1 Jenis Penelitian 35
3.2 Desain Penelitian 35
3.3 Populasi dan Sampel 37
3.4 Teknik Pengumpulan Data 37
3.5 Validasi Instrumen 38
3.6 Teknik Analisis Data 39
3.6.1 Ilustrasi Berbasis Data Sampel 39
3.6.2 Algoritma STATIS Lengkap 40
3.6.3 Analisis Autokorelasi Spasial (Global Moran's I) 41
3.6.4 Pemetaan Trajektori 3 Dimensi (*3D Spatio-Temporal Biplot*) . . . 41

BAB 4 HASIL DAN PEMBAHASAN 44
4.1 Eksplorasi Data Awal 44
4.1.1 Statistika Deskriptif 44
4.1.2 Distribusi Variabel (Boxplot) 46
4.2 Uji Kelayakan Data Multivariat 47
4.2.1 Hasil Uji KMO dan Bartlett 48
4.3 Analisis Inter-struktur 48
4.3.1 Matriks Koefisien RV 48
4.3.2 Vektor Bobot (Alpha) dan Representativitas Tahun 49
4.4 Analisis Struktur Kompromi 50
4.4.1 Dekomposisi Eigenvalue (Scree Plot) 50
4.4.2 Peta Kompromi (Skor Objek) 53
4.4.3 Biplot Kompromi (Skor + Loading) 54

4.4.4 Loading Plot (Correlation Circle) 55
4.4.5 Kontribusi dan Kualitas Representasi Variabel 56
4.4.6 Skor Kompromi Kabupaten/Kota 57
4.5 Uji Stabilitas Struktur Kompromi (Permutation Test) 59
4.5.1 Prosedur dan Hasil Uji Permutasi 59
4.5.2 Interpretasi 60
4.6 Analisis Dinamika Spasio-Temporal (Trajektori) 60
4.6.1 Gambaran Umum Trajektori 2020–2024 60
4.6.2 Snapshot Biplot per Tahun 62
4.7 Analisis Klaster Spasio-Temporal 66
4.7.1 Penentuan Jumlah Klaster Optimal 66
4.7.2 Pola Pengelompokan Tahunan 68
4.7.3 Matriks Transisi Klaster 71
4.7.4 Klaster Trajektori Keseluruhan 74
4.8 Evaluasi Klaster terhadap Kategori IPM UNDP 78
4.8.1 Matriks Transisi Kategori IPM 78
4.8.2 Tabulasi Silang K-Means vs. Kategori IPM 81
4.9 Uji Autokorelasi Spasial (Global Moran's I) 85
4.10 Analisis Hotspot (Local Moran's I) 85
4.11 Ringkasan Temuan dan Implikasi Kebijakan 85
4.11.1 Pembahasan Berdasarkan Tujuan Penelitian 85
4.11.2 Kontribusi terhadap Manfaat Penelitian 86
4.11.3 Implikasi Kebijakan 87

BAB 5 KESIMPULAN DAN SARAN 88
5.1 Kesimpulan 88
5.2 Saran 89

DAFTAR PUSTAKA 90

DAFTAR TABEL

Tabel 2.1 Ringkasan Penelitian Terdahulu 12

Tabel 2.2 Kriteria Interpretasi Nilai KMO 18

Tabel 4.1 Statistika Deskriptif Variabel Pembangunan Manusia Tahun 2020 . . 44
Tabel 4.2 Statistika Deskriptif Variabel Pembangunan Manusia Tahun 2021 . . 45
Tabel 4.3 Statistika Deskriptif Variabel Pembangunan Manusia Tahun 2022 . . 45
Tabel 4.4 Statistika Deskriptif Variabel Pembangunan Manusia Tahun 2023 . . 45
Tabel 4.5 Statistika Deskriptif Variabel Pembangunan Manusia Tahun 2024 . . 45
Tabel 4.6 Hasil Uji KMO dan Bartlett per Tahun Pengamatan 48
Tabel 4.7 Matriks Koefisien RV Antar-Tahun 49
Tabel 4.8 Nilai Bobot (Alpha) Tiap Tahun 50
Tabel 4.9 Dekomposisi Nilai Eigen Matriks Kompromi 52
Tabel 4.10 Loadings Kompromi (Korelasi Variabel vs Skor) 55
Tabel 4.11 Koordinat Skor Kompromi (Fc) 58
Tabel 4.12 Rangkuman Hasil Uji Permutasi 60
Tabel 4.13 Matriks Transisi Klaster 2020 → 2021 72
Tabel 4.14 Nama Daerah Perpindahan Klaster 2020 → 2021 72
Tabel 4.15 Matriks Transisi Klaster 2021 → 2022 72
Tabel 4.16 Nama Daerah Perpindahan Klaster 2021 → 2022 73
Tabel 4.17 Matriks Transisi Klaster 2022 → 2023 73
Tabel 4.18 Nama Daerah Perpindahan Klaster 2022 → 2023 73
Tabel 4.19 Matriks Transisi Klaster 2023 → 2024 74
Tabel 4.20 Nama Daerah Perpindahan Klaster 2023 → 2024 74
Tabel 4.21 Anggota Klaster Trajektori Keseluruhan 77
Tabel 4.22 Matriks Transisi IPM 2020 → 2021 78
Tabel 4.23 Nama Daerah Transisi IPM 2020 → 2021 78
Tabel 4.24 Matriks Transisi IPM 2021 → 2022 79
Tabel 4.25 Nama Daerah Transisi IPM 2021 → 2022 79
Tabel 4.26 Matriks Transisi IPM 2022 → 2023 79
Tabel 4.27 Nama Daerah Transisi IPM 2022 → 2023 80
Tabel 4.28 Matriks Transisi IPM 2023 → 2024 80
Tabel 4.29 Nama Daerah Transisi IPM 2023 → 2024 80
Tabel 4.30 Tabulasi Silang K-Means vs Kategori IPM (2020) 81
Tabel 4.31 Nama Daerah per Sel Cross-tab (2020) 81
Tabel 4.32 Tabulasi Silang K-Means vs Kategori IPM (2021) 82
Tabel 4.33 Nama Daerah per Sel Cross-tab (2021) 82
Tabel 4.34 Tabulasi Silang K-Means vs Kategori IPM (2022) 82

Tabel 4.35 Nama Daerah per Sel Cross-tab (2022) 83
Tabel 4.36 Tabulasi Silang K-Means vs Kategori IPM (2023) 83
Tabel 4.37 Nama Daerah per Sel Cross-tab (2023) 83
Tabel 4.38 Tabulasi Silang K-Means vs Kategori IPM (2024) 84
Tabel 4.39 Nama Daerah per Sel Cross-tab (2024) 84

DAFTAR GAMBAR

Gambar 3.1 *Flowchart* Alur Metodologi Penelitian 36

Gambar 4.1 Boxplot IPM Tahun 2020, 2022, dan 2024 46
Gambar 4.2 Boxplot Persentase Penduduk Miskin (PPM) Tahun 2020, 2022, dan 2024 46
Gambar 4.3 Boxplot PDRB ADHK Tahun 2020, 2022, dan 2024 47
Gambar 4.4 Boxplot Tingkat Pengangguran Terbuka (TPT) Tahun 2020, 2022, dan 2024 47
Gambar 4.5 Boxplot Rata-Rata Lama Sekolah (RLS) Tahun 2020, 2022, dan 2024 47
Gambar 4.6 Heatmap Koefisien RV Antar-Tahun (2020–2024) 49
Gambar 4.7 Scree Plot Nilai Eigen Matriks Kompromi 51
Gambar 4.8 Peta Skor Kompromi 41 Kabupaten/Kota (2020–2024) 53
Gambar 4.9 Biplot Kompromi: Skor Objek dan Loading Variabel 54
Gambar 4.10 Loading Plot (Correlation Circle) pada Ruang Kompromi 56
Gambar 4.11 Kontribusi Variabel terhadap Dimensi 1 Kompromi (%) 56
Gambar 4.12 Kualitas Representasi Variabel pada Dimensi 1 ($\cos^2$) 57
Gambar 4.13 Distribusi Null Uji Permutasi vs. Nilai Eigen Observasi (λ_1^{obs}) . . 59
Gambar 4.14 Trajektori Pergerakan 41 Kabupaten/Kota pada Ruang Kompromi (2020–2024) 61
Gambar 4.15 Biplot Tahunan Tahun 2020 (Masa Awal Pandemi) 62
Gambar 4.16 Biplot Tahunan Tahun 2021 63
Gambar 4.17 Biplot Tahunan Tahun 2022 (Awal Masa Pemulihan) 64
Gambar 4.18 Biplot Tahunan Tahun 2023 65

Gambar 4.19 Biplot Tahunan Tahun 2024 66
Gambar 4.20 Kurva Elbow dan Dendrogram Tahun 2020 67
Gambar 4.21 Kurva Elbow dan Dendrogram Tahun 2021 67
Gambar 4.22 Kurva Elbow dan Dendrogram Tahun 2022 67
Gambar 4.23 Kurva Elbow dan Dendrogram Tahun 2023 68
Gambar 4.24 Kurva Elbow dan Dendrogram Tahun 2024 68
Gambar 4.25 Pengelompokan K-Means ($k = 3$) Tahun 2020: Biplot (kiri) dan Peta (kanan) 69
Gambar 4.26 Pengelompokan K-Means ($k = 3$) Tahun 2021: Biplot (kiri) dan Peta (kanan) 69
Gambar 4.27 Pengelompokan K-Means ($k = 3$) Tahun 2022: Biplot (kiri) dan Peta (kanan) 70
Gambar 4.28 Pengelompokan K-Means ($k = 3$) Tahun 2023: Biplot (kiri) dan Peta (kanan) 70
Gambar 4.29 Pengelompokan K-Means ($k = 3$) Tahun 2024: Biplot (kiri) dan Peta (kanan) 71
Gambar 4.30 Dendrogram Klaster Trajektori Keseluruhan (2020–2024) 75
Gambar 4.31 Scatter Plot Klaster Trajektori pada Ruang Kompromi 76

DAFTAR LAMPIRAN

Lampiran 1 Data Penelitian 94
Lampiran 2 Kode Program Aplikasi Dashboard 107

BAB 1
PENDAHULUAN

1.1 Latar Belakang

Kawasan strategis yang meliputi Provinsi Banten, DKI Jakarta, dan Jawa Barat tidak hanya berperan sebagai mesin pertumbuhan ekonomi nasional, tetapi juga menjadi arena dinamika pembangunan manusia yang sangat kompleks dalam lima tahun terakhir. Meskipun wilayah ini memberikan kontribusi yang sangat dominan terhadap perekonomian nasional dengan rata-rata pangsa Produk Domestik Regional Bruto (PDRB) berkisar 33,8 persen dari PDRB Indonesia Badan Pusat Statistik (2025b)—keberhasilan pembangunannya tidak dapat lagi diukur hanya melalui kacamata indikator makro ekonomi semata. Terdapat interaksi multidimensi yang melibatkan kualitas hidup yang tecermin dalam Indeks Pembangunan Manusia (IPM), Indeks Pembangunan Gender (IPG), serta tantangan kesejahteraan sosial seperti kemiskinan dan ketahanan pangan yang terus mengalami perubahan. Kompleksitas ini menuntut pembuktian empiris yang lebih mendalam untuk melihat bagaimana indikator-indikator tersebut terdistribusi secara nyata di seluruh kabupaten/kota.

Secara empiris, lanskap pembangunan di ketiga provinsi ini menunjukkan polarisasi yang tajam antara pusat-pusat pertumbuhan utama dan wilayah penyangga. Meskipun data PDRB di kota-kota besar mencatat performa ekonomi yang kuat dengan nilai mencapai 532.606,87 miliar rupiah Badan Pusat Statistik (2025b), capaian tersebut tidak serta-merta diikuti oleh pemerataan kesejahteraan. Hal ini terlihat dari variasi spasial yang mencolok (*striking spatial variation*) pada indikator kemiskinan, di mana Persentase Penduduk Miskin (PPM) terentang lebar dari 2,29 persen hingga mencapai 15,06 persen di wilayah lainnya. Kesenjangan ini juga terkonfirmasi oleh indikator daya beli, terlihat dari perbedaan Garis Kemiskinan (GK) dan Prevalensi Ketidakcukupan Konsumsi Pangan (PKKP) yang bervariasi lebar antarwilayah Badan Pusat Statistik (2024). Lebih jauh lagi, disparitas kapabilitas manusia juga terlihat mencolok; Rata-rata Lama Sekolah (RLS) dan Angka Harapan Hidup (AHH) masih menunjukkan perbedaan yang nyata. Indeks Pembangunan Gender (IPG) serta Tingkat Pengangguran Terbuka (TPT) masih berada di kisaran 6,55–10,12 persen pada rentang tahun 2020–2024. Namun, persoalan ketimpangan antarwilayah ini menjadi semakin rumit ketika diletakkan dalam konteks waktu yang penuh perubahan.

Selain disparitas spasial, tantangan analisis meningkat akibat adanya dinamika temporal yang sangat fluktuatif sepanjang periode 2020–2024. Periode ini diawali dengan peristiwa pandemi COVID-19 yang menekan indikator makroekonomi dan kesejahteraan secara serentak terlihat dari perubahan pertumbuhan ekonomi yang sempat mencapai

angka 1,88 persen pada tahun 2020 Badan Pusat Statistik (2025b) namun diikuti oleh pola pemulihan yang lajunya tidak seragam di setiap daerah pada tahun-tahun berikutnya. Perubahan drastis dari masa pandemi COVID-19 menuju masa pemulihan ini diduga kuat telah mengubah struktur hubungan antar-indikator secara fundamental; sebagai contoh, pola korelasi antara pertumbuhan ekonomi (PDRB) dengan tingkat pengangguran (TPT) saat krisis sangat mungkin berbeda dibandingkan saat kondisi normal kembali pulih pada tahun 2023–2024. Oleh karena itu, pendekatan analisis multivariat klasik yang mengasumsikan struktur hubungan variabel bersifat statis (tetap sepanjang waktu) menjadi tidak lagi relevan untuk menganalisis fenomena pada periode yang penuh fluktuasi ini.

Untuk mengatasi keterbatasan asumsi stabilitas tersebut, diperlukan pendekatan analisis data tiga-arah (*three-way data analysis*) yang mampu menangani struktur data kompleks yang terdiri dari objek (kabupaten/kota), variabel (indikator sosio-ekonomi), dan situasi (waktu). Penelitian ini mengusulkan penggunaan metode STATIS (*Structuration des Tableaux à Trois Indices de la Statistique*), sebuah metode eksploratif yang diperkenalkan oleh Lavit et al. (1994); Oliveira et al. (2024). Berbeda dengan Analisis Komponen Utama (PCA) konvensional yang hanya bekerja pada data dua dimensi, STATIS dirancang khusus untuk mengekstraksi informasi dari sekumpulan tabel data yang berubah seiring waktu. Metode ini memiliki keunggulan dalam membentuk sebuah struktur kompromi (*compromise structure*) yang merepresentasikan pola hubungan umum antar-indikator selama periode pengamatan, sekaligus memungkinkan identifikasi seberapa jauh struktur ekonomi setiap tahunnya menyimpang dari pola umum tersebut.

Meskipun metode STATIS mampu menghasilkan struktur kompromi secara matematis, kompleksitas hasil analisis dari matriks berdimensi tinggi sering kali sulit dipahami jika hanya disajikan dalam bentuk tabel numerik. Oleh karena itu, penelitian ini mengintegrasikan pendekatan STATIS dengan teknik visualisasi *Dynamic Biplot*. Pendekatan ini memungkinkan proyeksi posisi objek (kabupaten/kota) dan variabel (indikator ekonomi) ke dalam ruang berdimensi rendah, sekaligus menggambarkan pergerakan atau lintasan (*trajectory*) perubahan posisi tersebut dari tahun ke tahun. Dengan memanfaatkan konsep *trajectory*, pergeseran karakteristik suatu wilayah misalnya, sebuah kabupaten yang pada tahun 2020 memiliki tingkat pengangguran tinggi namun kemudian bergeser mendekati karakteristik wilayah dengan IPM tinggi pada tahun 2024 dapat dilacak secara visual. Visualisasi lintasan ini menjadi instrumen krusial untuk membedakan antara fluktuasi jangka pendek akibat pandemi dan perubahan struktural jangka panjang.

Penerapan metode eksploratif multivariat seperti ini mengisi celah penting dalam literatur analisis regional di Indonesia yang selama ini didominasi oleh pendekatan inferensial klasik. Studi-studi terdahulu umumnya berfokus pada pemodelan regresi data panel atau

model spasial ekonometrik untuk mengestimasi besaran pengaruh variabel independen terhadap variabel dependen. Sebagai contoh, penelitian yang dilakukan oleh Nuswantari (2021) menggunakan pendekatan *Dynamic Spatial Durbin Model* (SDM) untuk memodelkan kemiskinan, sedangkan Ahmad (2024) menerapkan pendekatan spasial yang serupa untuk menganalisis fenomena penyebaran penyakit kusta di Pulau Jawa. Fokus utama dari penelitian-penelitian tersebut adalah pengukuran besaran pengaruh peubah-peubah penjelas (*explanatory variables*) terhadap peubah respon yang terbukti nyata secara statistik. Meskipun pendekatan tersebut berhasil mengidentifikasi faktor-faktor determinan, metode inferensial ini memiliki keterbatasan dalam memvisualisasikan struktur hubungan antarvariabel secara simultan (*inter-variable structure*) dan bagaimana struktur tersebut berevolusi dari waktu ke waktu (*spatio-temporal*). Hingga saat ini, belum terdapat penelitian yang secara spesifik menggunakan pendekatan eksploratif untuk memetakan dinamika struktur multivariat indikator ekonomi secara komprehensif tanpa membedakannya menjadi variabel bebas dan terikat.

Kompleksitas informasi yang dihasilkan dari analisis spasio-temporal ini menuntut media penyajian yang lebih adaptif dibandingkan laporan statis konvensional. Untuk itu, penelitian ini tidak hanya berhenti pada tataran analisis teoritis, tetapi juga mengembangkan sebuah aplikasi visualisasi interaktif berbasis web menggunakan bahasa pemrograman Python dengan kerangka kerja (*framework*) Dash dan Plotly. Pengembangan *dashboard* ini bertujuan untuk memfasilitasi Eksplorasi Data (*Exploratory Data Analysis*) secara mandiri oleh pengguna. Fitur-fitur interaktif yang disematkan seperti seleksi tahun, filter wilayah, hingga rotasi *baseline* memungkinkan para pengambil kebijakan untuk mengamati pola interaksi multivariat secara lebih mendalam. Melalui antarmuka ini, pola-pola tersembunyi (*latent patterns*) yang mungkin terlewatkan dalam tabel statistik standar dapat terdeteksi dengan lebih intuitif.

Secara keseluruhan, penelitian ini diarahkan untuk menjawab kebutuhan akan metode pemantauan pembangunan daerah yang lebih menyeluruh dan dinamis. Dengan memadukan kemampuan analitis metode STATIS dalam memahami struktur kompromi (*compromise structure*) lintas tahun dan kemampuan naratif visual dari *Dynamic Biplot*. Penelitian ini diharapkan dapat memberikan kontribusi ganda: secara akademis memperkaya khazanah penerapan statistik multivariat pada data ekonomi regional, dan secara praktis menyediakan prototipe alat bantu visual yang relevan bagi pemerintah daerah dalam mengevaluasi dampak kebijakan pasca-pandemi. Identifikasi struktur hubungan yang stabil maupun yang berubah selama lima tahun terakhir akan menjadi landasan penting dalam merumuskan strategi pembangunan yang lebih tangguh di masa depan.

Pemilihan lokasi penelitian yang mencakup seluruh kabupaten/kota di Provinsi Banten,

DKI Jakarta, dan Jawa Barat didasarkan pada karakteristik interkonektivitas ekonomi yang kuat di antara ketiga wilayah ini. Sebagai kesatuan wilayah pembangunan utama di Pulau Jawa, ketiga provinsi ini memiliki keterkaitan rantai pasok, mobilitas tenaga kerja, dan arus investasi yang sangat intensif. Permasalahan ekonomi yang terjadi di pusat pertumbuhan utama (DKI Jakarta) diproyeksikan akan merambat (*spillover*) ke wilayah penyangga di Banten dan Jawa Barat dengan skala yang berbeda-beda. Oleh karena itu, analisis yang hanya membatasi lingkup pada wilayah metropolitan Jabodetabek berpotensi mengabaikan dampak sistemik yang dirasakan oleh wilayah kabupaten non-metropolitan yang secara administratif berada dalam satu kesatuan provinsi namun memiliki karakteristik ekonomi yang jauh berbeda.

Untuk memahami kompleksitas pembangunan tersebut, penelitian ini menggunakan dua belas indikator strategis yang merepresentasikan dimensi pembangunan secara menyeluruh. Pemilihan variabel ini tidak dilakukan tanpa alasan, melainkan dipertimbangkan untuk mencakup aspek makroekonomi (PDRB ADHK), kesejahteraan material (PPM, GK, PKKP), kualitas modal manusia (IPM, RLS, AHH Laki-laki dan Perempuan), kesetaraan (IPG, Gini Ratio), serta partisipasi pasar kerja (TPT, TPAK). Kombinasi variabel ini memungkinkan analisis untuk tidak hanya pada indikator pertumbuhan konvensional, namun juga aspek inklusivitas pembangunan. Sebagai contoh, analisis simultan antara Gini Ratio dan PDRB memungkinkan deteksi dini terhadap fenomena pertumbuhan yang tidak merata (*exclusive growth*), sementara interaksi antara TPAK dan IPG dapat mengungkap hambatan struktural dalam partisipasi angkatan kerja berbasis gender.

Ketiadaan studi yang menganalisis dinamika interaksi kedua belas indikator ini secara spasio-temporal pada periode kritis 2020–2024 menciptakan sebuah ”kotak hitam” (*black box*) dalam pemahaman mengenai resiliensi ekonomi regional. Tanpa analisis struktur kompromi dan visualisasi lintasan yang jelas, para pemangku kebijakan kehilangan panduan empiris mengenai bagaimana sebenarnya pola pemulihan berlangsung di tingkat lokal apakah kembali ke struktur lama atau membentuk ekuilibrium baru. Berdasarkan urgensi untuk mengungkap fenomena tersebut secara statistik dan visual, penelitian ini disusun untuk menjawab pertanyaan-pertanyaan mendasar mengenai evolusi struktur ekonomi dan pembangunan di ketiga provinsi tersebut.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, permasalahan yang akan dikaji dalam penelitian ini dirumuskan sebagai berikut:

1. Bagaimana karakteristik struktur kompromi (*compromise structure*) yang merepre-

sentasikan pola hubungan multivariat dua belas indikator pembangunan di seluruh kabupaten/kota Provinsi Banten, DKI Jakarta, dan Jawa Barat menggunakan metode STATIS untuk memahami pola umum lintas tahun?

2. Bagaimana menguji struktur hubungan antar-indikator tersebut secara statistik selama periode 2020–2024 menggunakan uji permutasi matriks inter-struktur untuk membuktikan keberadaan dinamika temporal?

3. Bagaimana pola lintasan (*trajectory*) perubahan karakteristik ekonomi kabupaten/kota dari masa krisis pandemi (2020) hingga masa pemulihan (2024) berdasarkan visualisasi *Dynamic Biplot*?

4. Bagaimana pola pengelompokan spasial tiap tahunnya serta dinamika transisi atau pergerakan (*migration*) kabupaten/kota antar klaster dari tahun ke tahun?

5. Bagaimana merancang aplikasi berbasis web menggunakan *framework* Dash dan Plotly sebagai alat bantu eksplorasi data yang memungkinkan pengguna melakukan pemantauan indikator pembangunan secara interaktif?

1.3 Tujuan Penelitian

Sejalan dengan rumusan masalah di atas, tujuan dari penelitian ini adalah:

1. Menganalisis karakteristik struktur kompromi (*compromise structure*) yang merepresentasikan pola hubungan multivariat dua belas indikator pembangunan di seluruh kabupaten/kota Provinsi Banten, DKI Jakarta, dan Jawa Barat menggunakan metode STATIS untuk memahami pola umum lintas tahun.

2. Menganalisis struktur hubungan antar-indikator tersebut secara statistik selama periode 2020–2024 menggunakan uji permutasi matriks inter-struktur untuk membuktikan keberadaan dinamika temporal.

3. Menganalisis pola lintasan (*trajectory*) perubahan karakteristik ekonomi kabupaten/kota dari masa krisis pandemi (2020) hingga masa pemulihan (2024) berdasarkan hasil visualisasi *Dynamic Biplot*.

4. Mengidentifikasi pola pengelompokan spasial tiap tahunnya serta dinamika transisi atau pergerakan (*migration*) kabupaten/kota antar klaster dari tahun ke tahun.

5. Merancang aplikasi berbasis web menggunakan *framework* Dash dan Plotly sebagai alat bantu eksplorasi data yang memungkinkan pengguna melakukan pemantauan indikator pembangunan secara interaktif.

1.4 Manfaat Penelitian

Penelitian ini bermanfaat untuk:

1. Secara akademis memperkaya khazanah penerapan statistik multivariat terutama metode STATIS dan *Dynamic Biplot* pada data ekonomi regional di Indonesia.
2. Secara praktis menyediakan prototipe alatbantu visual berbasis web yang relevan bagi pemerintah daerah dalam mengevaluasi perkembangan indikator pembangunan dan dampak kebijakan pasca-pandemi.

1.5 Ruang Lingkup Penelitian

Agar pembahasan penelitian ini lebih terarah dan fokus, penulis menetapkan batasan-batasan masalah sebagai berikut:

1. **Variabel Penelitian:** Analisis menggunakan 12 indikator terpilih yang bersumber dari Badan Pusat Statistik (BPS), meliputi: Indeks Pembangunan Manusia (IPM), Indeks Pembangunan Gender (IPG), Persentase Penduduk Miskin (PPM), PDRB ADHK, Rata-rata Lama Sekolah (RLS), Angka Harapan Hidup Laki-laki (AHH_L) dan Perempuan (AHH_P), Garis Kemiskinan (GK), Prevalensi Ketidakcukupan Konsumsi Pangan (PKKP), Tingkat Pengangguran Terbuka (TPT), Tingkat Partisipasi Angkatan Kerja (TPAK), dan Gini Ratio (GR).
2. **Aspek Spasial:** Istilah ”spasial” dalam penelitian ini merujuk pada unit analisis berbasis wilayah (kabupaten/kota), bukan pada penggunaan matriks pembobot spasial (*spatial weight matrix*) seperti pada Ekonometrika Spasial.
3. **Fungsionalitas Aplikasi:** Aplikasi berbasis web yang dikembangkan berfungsi sebagai alat bantu eksplorasi visual (*Visual Exploratory Tool*) bagi pengguna, bukan sebagai Sistem Pendukung Keputusan (*Decision Support System*) yang menghasilkan rekomendasi kebijakan berbasis data secara otomatis.

BAB 2
LANDASAN TEORI

Bab ini menguraikan teori-teori dasar yang menjadi fondasi metodologi penelitian. Pembahasan dimulai dari konsep indikator kesejahteraan rakyat sebagai latar belakang substansi variabel penelitian, kemudian dilanjutkan dengan struktur data tiga arah dan konsep spasio-temporal, statistika deskriptif, standardisasi data, pemilihan variabel (Uji KMO dan Bartlett), fondasi aljabar linier beserta dekomposisi matriks (nilai eigen, dekomposisi spektral, dan Singular Value Decomposition), korelasi Pearson, Analisis Komponen Utama (PCA) beserta Biplot, algoritma metode STATIS beserta uji permutasi, Dynamic Biplot dan visualisasi trajektori, metode pengelompokan (clustering), uji autokorelasi spasial Global Moran's I, dan terakhir media visualisasi interaktif Plotly dan Dash.

2.1 Indikator Kesejahteraan Rakyat

Tujuan utama pembangunan suatu wilayah adalah mewujudkan kesejahteraan rakyatnya. Untuk mengukur keberhasilan pembangunan, dibutuhkan indikator-indikator yang mampu mencerminkan taraf kesejahteraan secara menyeluruh. Indikator kesejahteraan rakyat cukup luas dan kompleks sehingga hanya dapat dinilai melalui indikator-indikator terukur dari berbagai bidang pembangunan, di antaranya bidang kesehatan, pendidikan, ketenagakerjaan, taraf dan pola konsumsi, serta kemiskinan Badan Pusat Statistik (2025a). Pemilihan indikator yang tepat dan terukur sangat menentukan kualitas analisis multivariat yang akan dilakukan.

2.1.1 Indeks Pembangunan Manusia (IPM)

Indeks Pembangunan Manusia (IPM) merupakan indeks komposit yang mengukur capaian pembangunan manusia berbasis tiga dimensi dasar: (1) umur panjang dan hidup sehat, tercermin dari angka harapan hidup saat lahir; (2) pengetahuan, tercermin dari rata-rata lama sekolah dan harapan lama sekolah; serta (3) standar hidup layak, tercermin dari pengeluaran per kapita yang disesuaikan. IPM dihitung sebagai rata-rata geometrik dari ketiga indeks dimensi tersebut dengan skala nilai 0 hingga 100. Semakin tinggi nilai IPM, semakin baik capaian pembangunan manusia di suatu wilayah Badan Pusat Statistik (2025a).

2.1.2 Produk Domestik Regional Bruto (PDRB ADHK)

Produk Domestik Regional Bruto Atas Dasar Harga Konstan (PDRB ADHK) mencerminkan nilai tambah bruto barang dan jasa yang dihasilkan oleh seluruh unit ekonomi di suatu wilayah dalam kurun waktu tertentu, dihitung menggunakan harga pada tahun dasar tertentu (2010) sehingga pengaruh inflasi dieliminasi. PDRB ADHK digunakan sebagai ukuran pertumbuhan ekonomi riil suatu wilayah. Satuan yang digunakan umumnya adalah juta atau miliar rupiah. Wilayah dengan PDRB ADHK yang tinggi menunjukkan kapasitas produksi ekonomi yang kuat Badan Pusat Statistik (2025a).

2.1.3 Persentase Penduduk Miskin (PPM)

Persentase penduduk miskin adalah persentase penduduk yang memiliki rata-rata pengeluaran per kapita per bulan di bawah Garis Kemiskinan. Garis Kemiskinan (GK) mencerminkan nilai rupiah pengeluaran minimum yang diperlukan seseorang untuk memenuhi kebutuhan pokok hidupnya selama sebulan, baik kebutuhan makanan maupun non-makanan. Daerah yang lebih sejahtera pada umumnya memiliki persentase penduduk miskin yang relatif kecil Badan Pusat Statistik (2025a); Nisa (2024).

2.1.4 Garis Kemiskinan (GK)

Garis Kemiskinan (GK) adalah nilai rupiah pengeluaran minimum yang diperlukan oleh seseorang untuk dapat memenuhi kebutuhan pokok makanan setara 2.100 kilo kalori per orang per hari dan kebutuhan pokok non-makanan yang mencakup perumahan, sandang, pendidikan, dan kesehatan. GK digunakan sebagai batas untuk menentukan status kemiskinan seseorang dan bervariasi antarwilayah bergantung pada pola konsumsi dan harga komoditas setempat Badan Pusat Statistik (2025a).

2.1.5 Prevalensi Ketidakcukupan Konsumsi Pangan (PKKP)

Prevalensi Ketidakcukupan Konsumsi Pangan (PKKP) adalah estimasi proporsi populasi yang mengonsumsi energi (kalori) di bawah standar kecukupan minimum untuk menjalani kehidupan yang sehat dan aktif. Indikator ini mencerminkan dimensi ketahanan pangan suatu wilayah dan merupakan salah satu dari target Tujuan Pembangunan Berkelanjutan (SDGs) yang dipantau secara reguler oleh Badan Pusat Statistik Badan Pusat Statistik (2025a).

2.1.6 Tingkat Pengangguran Terbuka (TPT)

Tingkat Pengangguran Terbuka (TPT) adalah persentase jumlah pengangguran terhadap jumlah angkatan kerja. Pengangguran didefinisikan sebagai penduduk usia kerja yang sedang mencari pekerjaan, mempersiapkan usaha, atau tidak mencari pekerjaan karena merasa tidak mungkin mendapatkannya, termasuk mereka yang baru mendapat pekerjaan tetapi belum mulai bekerja. Angkatan kerja adalah penduduk usia 15 tahun ke atas yang bekerja atau sedang mencari pekerjaan. TPT yang tinggi mengindikasikan pemanfaatan tenaga kerja yang belum optimal Badan Pusat Statistik (2025a); Nisa (2024).

2.1.7 Tingkat Partisipasi Angkatan Kerja (TPAK)

Tingkat Partisipasi Angkatan Kerja (TPAK) adalah persentase penduduk usia kerja (15 tahun ke atas) yang termasuk dalam angkatan kerja, baik yang sudah bekerja maupun yang sedang mencari pekerjaan. TPAK mencerminkan proporsi penduduk usia produktif yang aktif secara ekonomi. Nilai TPAK yang tinggi menandakan tingginya partisipasi masyarakat dalam kegiatan ekonomi Badan Pusat Statistik (2025a).

2.1.8 Rata-Rata Lama Sekolah (RLS)

Rata-rata lama sekolah adalah rata-rata jumlah tahun yang telah ditempuh oleh penduduk usia 25 tahun ke atas dalam menjalani pendidikan formal. Indikator ini dihitung untuk penduduk usia 25 tahun ke atas dengan asumsi bahwa pada usia tersebut proses pendidikan pada umumnya telah selesai. Semakin tinggi nilai RLS, semakin tinggi tingkat pendidikan rata-rata penduduk suatu wilayah yang pada gilirannya mendukung peningkatan produktivitas dan kesejahteraan Badan Pusat Statistik (2025a); Nisa (2024).

2.1.9 Angka Harapan Hidup Laki-laki

Angka Harapan Hidup (AHH) laki-laki adalah perkiraan rata-rata jumlah tahun hidup yang akan dijalani oleh seorang bayi laki-laki yang baru lahir pada suatu tahun tertentu, dengan asumsi bahwa pola mortalitas pada saat kelahiran tetap berlaku sepanjang hidupnya. Indikator ini mencerminkan derajat kesehatan penduduk laki-laki serta kualitas layanan kesehatan yang tersedia bagi mereka Badan Pusat Statistik (2025a); Nisa (2024).

2.1.10 Angka Harapan Hidup Perempuan

Angka Harapan Hidup (AHH) perempuan adalah perkiraan rata-rata jumlah tahun hidup yang akan dijalani oleh seorang bayi perempuan yang baru lahir pada suatu tahun

tertentu. Umumnya, nilai AHH perempuan lebih tinggi dibandingkan laki-laki karena perbedaan biologis dan pola perilaku kesehatan. AHH perempuan merupakan komponen penting dalam perhitungan Indeks Pembangunan Manusia (IPM) dan Indeks Pembangunan Gender (IPG) Badan Pusat Statistik (2025a); Nisa (2024).

2.1.11 Indeks Pembangunan Gender (IPG)

Indeks Pembangunan Gender (IPG) adalah rasio capaian IPM perempuan terhadap IPM laki-laki. IPG mengukur kesenjangan gender dalam dimensi kesehatan, pendidikan, dan standar hidup. Nilai IPG sebesar 100 menunjukkan kesetaraan sempurna antara laki-laki dan perempuan, sedangkan nilai yang lebih rendah menunjukkan adanya kesenjangan yang merugikan perempuan Badan Pusat Statistik (2025a).

2.1.12 Gini Ratio (GR)

Gini Ratio adalah ukuran tingkat ketimpangan distribusi pengeluaran atau pendapatan penduduk. Nilai Gini Ratio berkisar antara 0 dan 1: nilai 0 berarti pemerataan sempurna (setiap orang memiliki pengeluaran yang sama), sedangkan nilai 1 berarti ketimpangan sempurna (satu orang menguasai seluruh pengeluaran). Gini Ratio yang tinggi menandakan tingginya kesenjangan ekonomi antar kelompok penduduk dalam suatu wilayah Badan Pusat Statistik (2025a).

2.2 Tinjauan Penelitian Terdahulu

Penelitian mengenai analisis statistik terhadap data regional di Pulau Jawa telah banyak dilakukan dengan menggunakan berbagai pendekatan. Bagian ini merangkum beberapa penelitian relevan yang menjadi rujukan dalam pengembangan metodologi penelitian ini, sebagaimana disajikan pada Tabel 2.1.

Tabel 2.1: Ringkasan Penelitian Terdahulu

Peneliti (Tahun)	**Wilayah**	**Metode**	**Hasil Utama**
Nuswantari (2021)	Pulau Jawa (2012-2018)	*Dynamic Spatial Panel* (Model Durbin Spasial)	Model terbaik menggunakan pembobot dua tetangga terdekat. Angka melek huruf dan fasilitas kesehatan berpengaruh signifikan terhadap kemiskinan.
Ahmad (2024)	Pulau Jawa (2017-2021)	*Dynamic Spatial Durbin Model* (DSDM)	Menemukan adanya pengaruh lag temporal dan spasiotemporal yang signifikan terhadap prevalensi kusta.
Nisa (2024)	Pulau Jawa (2022)	*Biclustering Plaid Model* vs *Cluster-Biplot*	*Plaid model* menghasilkan kelompok wilayah yang lebih homogen dibandingkan metode biplot konvensional dalam mengelompokkan indikator kesejahteraan.

Nuswantari (2021) menunjukkan bahwa pemodelan di Pulau Jawa harus mempertimbangkan efek dinamis (waktu) dan dependensi spasial secara simultan. Hal ini diperkuat oleh Ahmad (2024) yang menggunakan DSDM untuk menangkap interaksi antar-wilayah lintas waktu. Di sisi lain, Nisa (2024) memberikan perspektif mengenai pentingnya metode pengelompokan yang mampu menangani objek dan variabel secara bersamaan untuk melihat karakteristik regional yang unik. Penelitian ini menggabungkan aspek temporal dari data tiga-arah menggunakan metode STATIS dan aspek keterkaitan spasial untuk memberikan analisis yang lebih komprehensif.

2.3 Analisis Data Tiga-Arah (*Three-Way Data*)

Dalam statistika multivariat klasik, data umumnya direpresentasikan dalam struktur dua-arah (*two-way structure*), yaitu sebuah matriks $\mathbf{X}$ berukuran $n \times p$, dengan n merepresentasikan jumlah objek dan p merepresentasikan jumlah variabel. Struktur ini memadai untuk menganalisis fenomena statis pada satu titik waktu tertentu. Namun, dalam studi yang melibatkan pemantauan berkala, struktur data menjadi lebih kompleks dengan hadirnya dimensi ketiga, yaitu waktu atau situasi Kroonenberg (2008).

Struktur data seperti ini dikenal sebagai data tiga-arah (*three-way data*) atau data panel multivariat. Secara formal, data tiga-arah dapat didefinisikan sebagai berikut:

Definisi 2.1 (Data Tiga-Arah)**.** Misalkan terdapat n objek penelitian, p variabel, dan T situasi (waktu). Data tiga-arah adalah sekumpulan T buah matriks data $\{\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_T\}$,

di mana setiap matriks $\mathbf{X}_t$ (untuk $t = 1, \ldots, T$) berukuran $n \times p$.

Elemen skalar dari data ini dinotasikan sebagai x_{ijt}, yang merepresentasikan nilai dari variabel ke-j pada objek ke-i pada situasi ke-t. Keseluruhan himpunan matriks ini sering diilustrasikan sebagai sebuah balok data (*data cube*) dengan dimensi $n \times p \times T$.

Relevansi pendekatan tiga-arah ini semakin menguat dalam analisis data regional, di mana interaksi kompleks antara wilayah, waktu, dan indikator sosial-ekonomi sangat krusial untuk dipahami Biegert et al. (2023). Metode STATIS, yang akan dibahas selanjutnya, dirancang secara spesifik untuk mendapatkan informasi dari struktur tiga-arah $\{\mathbf{X}_t\}$ ini tanpa harus merata-ratakan data terlebih dahulu ke dalam satu matriks tunggal yang menghilangkan karakteristik temporal Lavit et al. (1994).

2.4 Konsep Spasio-Temporal dalam Analisis Multivariat

Analisis spasio-temporal bertujuan untuk mendeskripsikan atau memodelkan variabilitas fenomena yang terjadi dalam domain ruang dan waktu secara simultan. Tantangan utama dalam analisis ini adalah adanya dua ketergantungan sekaligus: ketergantungan antar-lokasi (spasial) dan ketergantungan antar-waktu (temporal).

Pendekatan konvensional dalam ekonometrika biasanya melakukan pendekatan terhadap masalah ini dengan menggunakan regresi data panel atau model dinamis spasial (*Spatial Dynamic Panel Data*) untuk mengetahui pengaruh antar-lokasi, dengan struktur hubungan antar-lokasi didefinisikan menggunakan matriks pembobot spasial (*spatial weight matrix*, $\mathbf{W}_{spasial}$) berbasis ketetanggaan geografis yang memiliki asumsi bahwa interaksi antar-wilayah hanya terjadi melalui batas geografis fisik yang bersifat konstan sepanjang waktu Baltagi (2021). Dalam pendekatan multivariat eksploratif seperti STATIS, definisi "spasial" dipandang dari perspektif yang berbeda.

Konsep Spasial dalam STATIS Dalam kerangka STATIS, aspek spasial direpresentasikan melalui konfigurasi kemiripan antar-objek dalam ruang multivariat, bukan melalui matriks kontiguitas geografis. Struktur "spasial" pada waktu t didefinisikan oleh matriks produk skalar $\mathbf{W}_t = \mathbf{X}_t\mathbf{X}_t^T$ (berukuran $n \times n$) yang merangkum "peta" kemiripan antar-objek berdasarkan profil variabelnya.

Evolusi spasio-temporal dalam pendekatan ini difokuskan pada pengamatan perubahan konfigurasi matriks $\mathbf{W}_t$ dari waktu ke waktu, untuk menjawab pertanyaan apakah pola kemiripan antar-objek berubah selama periode pengamatan.

2.5 Standardisasi Data

Perbedaan nilai yang besar antar-variabel akibat perbedaan satuan pengukuran (misalnya rupiah, persen, dan tahun) dapat menyebabkan bias dalam analisis multivariat sehingga perlu menyamakan skala variabilitasnya. Standardisasi Z*-score* adalah teknik mengubah nilai pengamatan sehingga setiap variabel memiliki rata-rata nol dan simpangan baku satu Johnson & Wichern (2007).

Definisi 2.2 (Standardisasi Z-Score)**.** Misalkan x_{ij} adalah nilai pengamatan ke-i pada variabel ke-j, dengan $\bar{x}_j$ sebagai rata-rata dan s_j sebagai simpangan baku variabel ke-j. Nilai standar (*standardized value*) dihitung sebagai:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}, \quad i = 1, 2, \ldots, n, \quad j = 1, 2, \ldots, p \tag{2.1}$$

dengan $\bar{x}_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}$ dan $s_j = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)^2}$.

Hasil standardisasi menghasilkan matriks data standar **Z** dengan setiap kolom memiliki rata-rata 0 dan simpangan baku 1. Prosedur ini memastikan bahwa tidak ada variabel yang mendominasi analisis hanya karena memiliki skala atau satuan pengukuran yang lebih besar.

2.6 Statistika Deskriptif

Sebelum menerapkan metode analisis multivariat, tahap awal yang penting adalah melakukan eksplorasi data melalui Statistika Deskriptif. Statistika deskriptif bertujuan meringkas dan mendeskripsikan karakteristik distribusi data secara numerik, sehingga analis dapat memahami pola umum, keragaman, dan anomali dalam data sebelum tahap analisis yang lebih mendalam Johnson & Wichern (2007).

2.6.1 Ukuran Pemusatan

Ukuran pemusatan mendeskripsikan nilai "tengah" dari kumpulan data.

Definisi 2.3 (Rata-Rata (Mean))**.** Rata-rata dari n pengamatan $x_1, x_2, \ldots, x_n$ suatu variabel adalah:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \tag{2.2}$$

Rata-rata bersifat sensitif terhadap pencilan (*outlier*).

Definisi 2.4 (Median)**.** Median (Q_2) adalah nilai tengah data setelah diurutkan dari terkecil ke terbesar:

$$Q_2 = \begin{cases} x_{\left(\frac{n+1}{2}\right)} & \text{jika } n \text{ ganjil} \\ \dfrac{x_{\left(\frac{n}{2}\right)} + x_{\left(\frac{n}{2}+1\right)}}{2} & \text{jika } n \text{ genap} \end{cases} \tag{2.3}$$

Median lebih robust terhadap pencilan dibandingkan rata-rata.

2.6.2 Ukuran Penyebaran

Ukuran penyebaran mendeskripsikan seberapa jauh data bervariasi di sekitar pusat distribusinya.

Definisi 2.5 (Simpangan Baku (Standar Deviasi))**.** Simpangan baku sampel dari n pengamatan adalah:

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2} \tag{2.4}$$

Simpangan baku merepresentasikan akar kuadrat dari ragam (*variance*) sampel s^2.

Definisi 2.6 (Nilai Minimum, Maksimum, dan Rentang Antarkuartil (IQR))**.** Nilai minimum dan maksimum adalah:

$$x_{\min} = \min(x_1, \ldots, x_n), \qquad x_{\max} = \max(x_1, \ldots, x_n) \tag{2.5}$$

Kuartil pertama (Q_1) dan ketiga (Q_3) membagi distribusi menjadi empat bagian masing-masing 25%. Rentang Antarkuartil (*Interquartile Range*, IQR) adalah:

$$IQR = Q_3 - Q_1 \tag{2.6}$$

IQR merupakan ukuran penyebaran yang robust karena tidak dipengaruhi oleh pencilan.

Definisi 2.7 (Koefisien Variasi (CV))**.** Koefisien Variasi (*Coefficient of Variation*, CV) mengukur variabilitas relatif terhadap rata-rata:

$$CV = \frac{s}{\bar{x}} \times 100\% \tag{2.7}$$

CV berguna untuk membandingkan tingkat keragaman antar-variabel yang memiliki satuan atau skala yang berbeda-beda.

2.6.3 Ukuran Bentuk Distribusi

Ukuran bentuk mengkarakterisasi simetri dan keruncinga distribusi data.

Definisi 2.8 (Skewness (Kemiringan))**.** Skewness mengukur tingkat asimetri distribusi data:

$$g_1 = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^3}{s^3} \tag{2.8}$$

Interpretasi: $g_1 > 0$ berarti distribusi condong ke kanan (*right-skewed*, ekor panjang di kanan); $g_1 < 0$ berarti condong ke kiri (*left-skewed*); $g_1 \approx 0$ berarti distribusi hampir simetris.

Definisi 2.9 (Kurtosis (Keruncingan))**.** Kurtosis mengukur ketebalan ekor dan keruncinga relatif distribusi data dibandingkan distribusi normal:

$$g_2 = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^4}{s^4} - 3 \tag{2.9}$$

Nilai negatif dari pengurangan 3 merupakan koreksi *excess kurtosis*: $g_2 > 0$ berarti distribusi lebih runcing (*leptokurtik*); $g_2 < 0$ berarti lebih datar (*platikurtik*) dibandingkan distribusi normal.

2.7 Pemilihan Variabel dalam Analisis Multivariat

Keberhasilan suatu analisis multivariat berbasis reduksi dimensi sangat bergantung pada kualitas himpunan variabel yang digunakan. Variabel yang tidak memiliki hubungan satu sama lain tidak akan menghasilkan komponen yang bermakna, sedangkan variabel yang terlalu berkorelasi sempurna dapat menyebabkan matriks kovarians menjadi singular Hair et al. (2019). Sebelum menerapkan metode analisis, perlu dilakukan serangkaian uji statistik untuk memverifikasi bahwa himpunan variabel *layak* untuk dianalisis secara multivariat yang artinya terdapat struktur kovarians yang cukup untuk diekstraksi (*factorable*).

Dua kriteria utama yang lazim digunakan sebagai "penjaga gerbang" (*gatekeeping criterion*) adalah: (1) Uji Kaiser-Meyer-Olkin (KMO) yang menilai kecukupan (*sampling adequacy*) masing-masing variabel maupun keseluruhan himpunan variabel, dan (2) Uji Bartlett tentang Spherisitas yang menguji apakah matriks korelasi antar-variabel secara keseluruhan signifikan berbeda dari matriks identitas.

2.7.1 Uji Kaiser-Meyer-Olkin (KMO)

Statistik Kaiser-Meyer-Olkin (KMO) dikembangkan oleh Kaiser (1970, 1974) sebagai ukuran untuk menilai kecukupan sampel (*sampling adequacy*) dalam analisis faktor dan analisis komponen utama. Secara intuitif, KMO mengukur sejauh mana korelasi antar-pasang variabel dapat "dijelaskan" oleh variabel-variabel lainnya—suatu kondisi yang disebut korelasi parsial rendah. Apabila korelasi parsial antar-variabel rendah setelah dikontrol variabel lain, maka ada faktor laten yang mendasari struktur korelasinya—kondisi yang disyaratkan oleh analisis berbasis reduksi dimensi Dziuban & Shirkey (1974).

Definisi 2.10 (Statistik KMO)**.** Misalkan r_{jk} adalah koefisien korelasi Pearson antara variabel ke-j dan ke-k, dan q_{jk} adalah elemen dari invers matriks korelasi $\mathbf{R}^{-1}$. Korelasi parsial antara variabel ke-j dan ke-k dihitung sebagai:

$$r_{jk.\text{parsial}} = -\frac{q_{jk}}{\sqrt{q_{jj} \cdot q_{kk}}} \tag{2.10}$$

Statistik KMO keseluruhan (*overall KMO*) didefinisikan sebagai:

$$KMO = \frac{\sum_{j \neq k} r_{jk}^2}{\sum_{j \neq k} r_{jk}^2 + \sum_{j \neq k} r_{jk.\text{parsial}}^2} \tag{2.11}$$

Nilai KMO berkisar antara 0 dan 1. Semakin besar nilai KMO, semakin kecil jumlah kuadrat korelasi parsial (penyebut) relatif terhadap jumlah kuadrat korelasi bivariate (pembilang). Hal ini mengindikasikan bahwa variabel-variabel saling berbagi faktor laten yang kuat.

Selain statistik KMO keseluruhan, Kaiser (1974) mendefinisikan Measure of Sampling Adequacy (MSA_j) individual untuk setiap variabel ke-j:

$$MSA_j = \frac{\sum_{k \neq j} r_{jk}^2}{\sum_{k \neq j} r_{jk}^2 + \sum_{k \neq j} r_{jk.\text{parsial}}^2} \tag{2.12}$$

Interpretasi nilai KMO dan MSA_j menurut Kaiser (1974) dan Hair et al. (2019) disajikan pada Tabel 2.2 berikut:

Tabel 2.2: Kriteria Interpretasi Nilai KMO

Nilai KMO	Interpretasi
$0{,}90 \leq KMO \leq 1{,}00$	Marvelous (Luar Biasa)
$0{,}80 \leq KMO < 0{,}90$	Meritorious (Sangat Baik)
$0{,}70 \leq KMO < 0{,}80$	Middling (Cukup Baik)
$0{,}60 \leq KMO < 0{,}70$	Mediocre (Biasa)
$0{,}50 \leq KMO < 0{,}60$	Miserable (Buruk, namun dapat diterima)
$KMO < 0{,}50$	Unacceptable (Tidak diterima)

Apabila nilai MSA_j suatu variabel berada di bawah ambang batas 0,50, variabel tersebut dipertimbangkan untuk dikeluarkan dari analisis dan himpunan variabel diuji ulang hingga semua variabel yang tersisa memenuhi syarat Hair et al. (2019); Nisa (2024).

2.7.2 Uji Bartlett tentang Spherisitas (*Bartlett's Test of Sphericity*)

Uji Bartlett tentang Spherisitas, dikembangkan oleh Bartlett (1954), bertujuan menguji hipotesis nol bahwa matriks korelasi populasi adalah matriks identitas ($\mathbf{R} = \mathbf{I}$). Jika $\mathbf{R} = \mathbf{I}$, tidak ada korelasi sama sekali antar-variabel sehingga tidak ada informasi bersama yang dapat diekstraksi melalui analisis multivariat Johnson & Wichern (2007).

Definisi 2.11 (Uji Bartlett tentang Spherisitas)**.** Diberikan matriks korelasi sampel $\mathbf{R}$ (berukuran $p \times p$) dari n observasi. Hipotesis uji Bartlett adalah:

- H_0: $\boldsymbol{\Sigma} = \sigma^2\mathbf{I}$ (matriks korelasi populasi adalah matriks identitas)
- H_1: $\boldsymbol{\Sigma} \neq \sigma^2\mathbf{I}$ (terdapat korelasi yang signifikan antar-variabel)

Statistik uji χ^2 dihitung dengan aproksimasi Bartlett (1954):

$$\chi^2 = -\left[n - 1 - \frac{2p+5}{6}\right] \ln|\mathbf{R}| \tag{2.13}$$

dengan $|\mathbf{R}|$ adalah determinan matriks korelasi. Statistik ini berdistribusi χ^2 dengan derajat bebas:

$$df = \frac{p(p-1)}{2} \tag{2.14}$$

Logika Determinan Matriks Korelasi Apabila semua variabel saling tidak berkorelasi, maka $\mathbf{R} = \mathbf{I}$ sehingga $|\mathbf{R}| = 1$ dan $\ln(1) = 0$—statistik uji bernilai nol yang tidak signifikan. Sebaliknya, ketika variabel-variabel saling berkorelasi erat, $|\mathbf{R}| \to 0$ sehingga $\ln|\mathbf{R}|$ bernilai sangat negatif dan statistik uji menjadi besar, menghasilkan p-value yang kecil Johnson & Wichern (2007).

Jika p-value $< \alpha$, maka H_0 ditolak, yang berarti terdapat struktur korelasi yang signifikan. Kondisi ini merupakan prasyarat yang diperlukan agar analisis reduksi dimensi dapat mengekstraksi informasi yang bermakna.

Definisi 2.12 (Determinan Matriks)**.** Determinan dari matriks bujur sangkar $\mathbf{A}$ berukuran $p \times p$, dinotasikan sebagai $|\mathbf{A}|$ atau $\det(\mathbf{A})$, adalah sebuah nilai skalar yang merangkum properti-properti penting dari matriks tersebut, termasuk keterbalikannya (*invertibility*) dan sifat singular. Secara formal, determinan didefinisikan sebagai Anton & Rorres (2013):

$$|\mathbf{A}| = \sum_{\sigma \in S_p} \text{sgn}(\sigma) \prod_{i=1}^{p} a_{i,\sigma(i)} \tag{2.15}$$

di mana:

- S_p adalah himpunan seluruh permutasi dari indeks $\{1, 2, \ldots, p\}$.
- $\text{sgn}(\sigma)$ adalah tanda (paritas) dari permutasi σ (bernilai $+1$ untuk permutasi genap, dan -1 untuk permutasi ganjil).
- $a_{i,\sigma(i)}$ mewakili elemen matriks pada baris ke-i dan kolom ke-$\sigma(i)$.

Untuk matriks korelasi populasi $\mathbf{R}$, nilai determinan memberikan ukuran global atas keberadaan multikolinieritas antar variabel. Apabila variabel-variabel saling bebas linier (matriks korelasi membentuk matriks identitas), maka determinan bernilai maksimum, yaitu $|\mathbf{R}| = 1$. Sebaliknya, semakin tinggi korelasi linier antar variabel, nilai determinan akan semakin mengecil mendekati 0. Jika determinan bernilai mutlak 0, matriks saling berkorelasi sempurna sehingga tidak dapat diinvers Johnson & Wichern (2007).

2.8 Dasar Aljabar Linier dan Dekomposisi Matriks

Metode reduksi dimensi seperti PCA dan STATIS sangat bergantung pada operasi aljabar linier, khususnya dekomposisi matriks simetris. Pemahaman mengenai nilai eigen, vektor eigen, dan sifat *trace* matriks menjadi prasyarat untuk memahami bagaimana metode-metode ini mengekstraksi informasi struktural dari data.

Definisi 2.13 (Nilai Eigen dan Vektor Eigen)**.** Diberikan sebuah matriks persegi $\mathbf{A}$ berukuran $p \times p$. Sebuah skalar λ disebut sebagai **nilai eigen** (*eigenvalue*) dari $\mathbf{A}$ jika terdapat sebuah vektor tak-nol $\mathbf{v}$ (berukuran $p \times 1$) yang disebut **vektor eigen** (*eigenvector*), sedemikian sehingga berlaku:

$$\mathbf{A}\mathbf{v} = \lambda \mathbf{v} \tag{2.16}$$

Vektor eigen menentukan "arah" dominan dari transformasi linier matriks $\mathbf{A}$, sedangkan nilai eigen menentukan besaran "kekuatan" atau variansi pada arah tersebut.

Definisi 2.14 (Determinan Matriks)**.** Determinan dari matriks persegi $\mathbf{A}$, dinotasikan sebagai $\det(\mathbf{A})$ atau $|\mathbf{A}|$, adalah sebuah nilai skalar yang merangkum sifat-sifat geometris dari matriks tersebut. Dalam analisis multivariat, determinan matriks korelasi $|\mathbf{R}|$ menunjukkan sejauh mana variabel-variabel saling berkorelasi atau bersifat ortogonal. Jika $|\mathbf{R}| \approx 0$, maka terdapat korelasi antar-variabel yang kuat (multikolinieritas), sedangkan jika $|\mathbf{R}| = 1$, maka variabel bersifat ortogonal sempurna.

Definisi 2.15 (Invers Matriks)**.** Sebuah matriks persegi $\mathbf{A}$ memiliki invers $\mathbf{A}^{-1}$ jika $\det(\mathbf{A}) \neq 0$, sedemikian sehingga berlaku $\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$, dengan $\mathbf{I}$ adalah matriks identitas. Invers dari matriks korelasi digunakan dalam perhitungan statistik KMO untuk mendapatkan korelasi parsial melalui matriks anti-citra.

Karena matriks kovarians selalu bersifat simetris ($\mathbf{A} = \mathbf{A}^T$), berlaku teorema dekomposisi spektral berikut.

Teorema 2.1 (Dekomposisi Spektral)**.** Setiap matriks simetris $\mathbf{A}$ berukuran $p \times p$ dapat didekomposisi menjadi:

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T = \sum_{j=1}^{p} \lambda_j \mathbf{v}_j \mathbf{v}_j^T \tag{2.17}$$

dengan $\mathbf{V}$ adalah matriks ortogonal ($p \times p$) yang kolom-kolomnya ($\mathbf{v}_j$) adalah vektor eigen yang saling tegak lurus, dan $\mathbf{\Lambda}$ adalah matriks diagonal berisi nilai eigen terurut $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0$ pada elemen diagonalnya.

Selain dekomposisi, konsep *trace* matriks juga menjadi fundamental, khususnya dalam perhitungan koefisien korelasi antar-konfigurasi (Koefisien RV) pada metode STATIS.

Definisi 2.16 (Trace Matriks)**.** *Trace* dari sebuah matriks persegi $\mathbf{A}$ berukuran $n \times n$, dinotasikan sebagai $tr(\mathbf{A})$, adalah penjumlahan dari elemen-elemen diagonal utamanya:

$$tr(\mathbf{A}) = \sum_{i=1}^{n} a_{ii} \tag{2.18}$$

Sifat penting dari *trace* yang digunakan dalam pengukuran jarak antar-matriks adalah invariansi siklis: $tr(\mathbf{AB}) = tr(\mathbf{BA})$.

Definisi 2.17 (Dekomposisi Nilai Singular (SVD))**.** *Singular Value Decomposition* (SVD) adalah faktorisasi yang dapat diterapkan pada matriks persegi panjang manapun $\mathbf{A}$ berukuran $m \times n$:

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T \tag{2.19}$$

di mana:

- $\mathbf{U}$ $(m \times m)$: matriks ortogonal, kolom-kolomnya disebut **vektor singular kiri**.
- $\mathbf{S}$ $(m \times n)$: matriks diagonal yang berisi **nilai singular** $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ pada diagonal utamanya.
- $\mathbf{V}^T$ $(n \times n)$: transpos dari matriks ortogonal $\mathbf{V}$, kolom-kolomnya disebut **vektor singular kanan**.

Untuk matriks simetris semi-definit positif $\mathbf{W}_c$ (seperti matriks kompromi STATIS), SVD setara dengan dekomposisi spektral: nilai singular σ_i sama dengan nilai eigen λ_i dan $\mathbf{U} = \mathbf{V}$.

SVD memiliki kestabilan numerik yang lebih baik dibandingkan dekomposisi eigen langsung dan oleh karena itu lebih sering digunakan dalam implementasi komputasional. Skor kompromi STATIS diperoleh dari:

$$\mathbf{F}_c = \mathbf{U}\sqrt{\mathbf{S}} \tag{2.20}$$

sehingga $\mathbf{W}_c = \mathbf{U}\mathbf{S}\mathbf{U}^T = \mathbf{F}_c\mathbf{F}_c^T$.

2.9 Korelasi Pearson

Korelasi Pearson adalah ukuran statistik yang mengkuantifikasi kekuatan dan arah hubungan linier antara dua variabel. Koefisien ini digunakan secara luas dalam analisis multivariat, khususnya pada pembentukan matriks korelasi yang merupakan fondasi Uji KMO dan Bartlett, serta pada komputasi *loadings* (arah variabel dalam Biplot) Johnson & Wichern (2007).

Definisi 2.18 (Koefisien Korelasi Pearson)**.** Koefisien korelasi Pearson antara dua variabel x dan y dari n pengamatan didefinisikan sebagai:

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2} \cdot \sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}} \tag{2.21}$$

Nilai r_{xy} berkisar antara -1 dan 1:

- $r_{xy} = 1$: korelasi positif sempurna (ketika x naik, y naik proporsional).

- $r_{xy} = -1$: korelasi negatif sempurna (ketika x naik, y turun proporsional).
- $r_{xy} = 0$: tidak ada hubungan linier.

Matriks Korelasi Untuk p variabel, koefisien Pearson antar setiap pasang variabel dapat disusun menjadi matriks korelasi simetris $\mathbf{R}$ berukuran $p \times p$, di mana elemen (j, k)-nya adalah r_{jk}. Diagonal utamanya selalu bernilai 1 ($r_{jj} = 1$). Matriks korelasi inilah yang menjadi dasar perhitungan statistik KMO dan Bartlett.

Dalam konteks Dynamic Biplot, loadings variabel ke-j pada komponen ke-ℓ dihitung sebagai korelasi Pearson antara nilai rata-rata variabel tersebut lintas situasi ($\bar{x}_j$) dengan skor kompromi pada komponen ke-ℓ ($\mathbf{F}_c^{(\ell)}$):

$$g_{j\ell} = r(\bar{x}_j, \mathbf{F}_c^{(\ell)}) \tag{2.22}$$

Panjang vektor loading $\sqrt{g_{j1}^2 + g_{j2}^2}$ mencerminkan seberapa baik variabel tersebut direpresentasikan dalam ruang dua dimensi kompromi.

2.10 Analisis Komponen Utama (PCA) dan Biplot

Analisis Komponen Utama atau *Principal Component Analysis* (PCA) adalah teknik reduksi dimensi data multivariat yang mentransformasi p variabel asli yang saling berkorelasi menjadi q variabel baru ($q < p$) yang tidak berkorelasi, disebut komponen utama (*principal components*) Hotelling (1933).

Misalkan $\mathbf{X}$ adalah matriks data berukuran $n \times p$ yang telah dipusatkan atau distandardisasi. PCA mengekstraksi informasi dengan mendekomposisi matriks kovarians $\mathbf{S} = \frac{1}{n-1}\mathbf{X}^T\mathbf{X}$. Berdasarkan Teorema Dekomposisi Spektral, $\mathbf{S} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$, menghasilkan dua elemen utama:

1. **Loadings ($\mathbf{V}$):** Matriks vektor eigen yang berisi bobot kontribusi setiap variabel asli terhadap setiap komponen utama. Kolom ke-j dari $\mathbf{V}$ disebut vektor loading ke-j.
2. **Scores ($\mathbf{F}$):** Proyeksi data asli ke dalam ruang komponen utama, dihitung sebagai:

$$\mathbf{F} = \mathbf{X}\mathbf{V} \tag{2.23}$$

Untuk memvisualisasikan hasil reduksi dimensi secara simultan antara objek dan variabel, digunakan teknik Biplot.

Definisi 2.19 (Biplot)**.** Biplot adalah representasi grafis dua dimensi yang memetakan baris (objek) dan kolom (variabel) dari matriks data $\mathbf{X}$ secara bersamaan dalam satu grafik Gabriel (1971). Biplot didasarkan pada aproksimasi matriks *rank*-rendah:

$$\mathbf{X} \approx \mathbf{F}\mathbf{V}^T \tag{2.24}$$

Dalam visualisasi ini:

- Objek ke-i digambarkan sebagai titik berdasarkan baris ke-i dari matriks Skor $\mathbf{F}$. Jarak Euclidean antar titik merepresentasikan kemiripan antar objek.
- Variabel ke-j digambarkan sebagai vektor (panah) berdasarkan baris ke-j dari matriks Loadings $\mathbf{V}$. Sudut antar vektor merepresentasikan korelasi antar variabel. Sudut lancip ($< 90°$) berarti korelasi positif, sedangkan sudut tumpul ($> 90°$) berarti korelasi negatif.

PCA dan Biplot bekerja dengan baik untuk satu tabel data pada satu titik waktu. Namun, ketika data tersebar dalam T matriks yang berbeda-beda seiring waktu, PCA konvensional tidak mampu menangkap dinamika evolusi struktural tersebut. Keterbatasan inilah yang mendorong pengembangan metode STATIS.

2.11 Metode STATIS

STATIS (*Structuration des Tableaux à Trois Indices de la Statistique*) adalah metode analisis data eksploratif yang dirancang untuk menganalisis sekumpulan T tabel data dengan membandingkan struktur internal antar-objek pada setiap situasi waktu Lavit et al. (1994). Berbeda dengan metode yang menggabungkan data secara naif (seperti *Pooled* PCA), STATIS mempertahankan integritas struktur masing-masing situasi melalui tiga tahapan analisis yang sistematis: intra-struktur, inter-struktur, dan kompromi. Konsep inti STATIS dilandasi oleh teori operator Escoufier Escoufier (1973) yang memperkenalkan cara mengukur "jarak" antara dua konfigurasi data multivariat.

2.11.1 Tahap 1: Intra-struktur

Tahap pertama bertujuan mentransformasi tabel data asli setiap situasi menjadi representasi geometri yang dapat diperbandingkan. Misalkan setiap matriks data $\mathbf{X}_t$ telah distandardisasi menjadi $\mathbf{Z}_t$. Alih-alih membandingkan nilai mentah variabel, STATIS mendefinisikan operator Escoufier $\mathbf{W}_t$ untuk setiap $t = 1, \ldots, T$ sebagai matriks produk

skalar berukuran $n \times n$:

$$\mathbf{W}_t = \mathbf{Z}_t \mathbf{Z}_t^T \tag{2.25}$$

Elemen-elemen dalam $\mathbf{W}_t$ merepresentasikan kemiripan (korelasi) geometris antar-objek pada situasi ke-t. Matriks ini merangkum "peta" konfigurasi antar-objek pada situasi tersebut tanpa bergantung pada jumlah variabel p.

Interpretasi $\mathbf{W}_t$ Elemen $\mathbf{W}_t[i, j]$ yang bernilai besar menunjukkan bahwa objek ke-i dan objek ke-j memiliki profil variabel yang sangat mirip pada situasi ke-t, sedangkan nilai yang kecil menunjukkan perbedaan karakteristik yang mencolok.

2.11.2 Tahap 2: Inter-struktur

Setelah representasi $\mathbf{W}_t$ diperoleh untuk setiap situasi, langkah selanjutnya adalah mengukur seberapa mirip konfigurasi antar-satu situasi dengan situasi lainnya. Ukuran kemiripan yang digunakan adalah Koefisien RV (*Rho Vectoriel*), yang merupakan generalisasi dari koefisien korelasi Pearson untuk matriks Escoufier (1973).

Definisi 2.20 (Koefisien RV)**.** Koefisien RV antara dua matriks konfigurasi $\mathbf{W}_i$ dan $\mathbf{W}_j$ didefinisikan sebagai:

$$RV(\mathbf{W}_i, \mathbf{W}_j) = \frac{tr(\mathbf{W}_i \mathbf{W}_j)}{\sqrt{tr(\mathbf{W}_i^2)\, tr(\mathbf{W}_j^2)}} \tag{2.26}$$

Nilai RV berkisar antara 0 dan 1. Nilai mendekati 1 mengindikasikan bahwa konfigurasi antar-objek pada situasi ke-i sangat mirip dengan situasi ke-j.

Koefisien RV dihitung untuk setiap pasangan situasi, membentuk matriks inter-struktur $\mathbf{R}$ berukuran $T \times T$ dengan elemen $R_{ij} = RV(\mathbf{W}_i, \mathbf{W}_j)$. Untuk mendapatkan bobot kontribusi setiap situasi, dilakukan dekomposisi spektral pada $\mathbf{R}$:

$$\mathbf{R}\mathbf{u} = \theta \mathbf{u} \tag{2.27}$$

Vektor eigen pertama $\mathbf{u}_1$ (berkorespondensi dengan nilai eigen terbesar θ_1) digunakan untuk menghitung bobot α_t:

$$\alpha_t = \frac{u_{1t}}{\sqrt{\sum_{t=1}^{T} u_{1t}^2}} \tag{2.28}$$

Contoh 2.1 (Logika Bobot)**.** Apabila suatu situasi memiliki profil yang sangat anomali dibandingkan situasi lainnya (misalnya akibat krisis), metode STATIS akan secara otomatis

memberikan bobot α_t yang lebih kecil pada situasi tersebut. Hal ini memastikan bahwa struktur kompromi tidak "terdistorsi" oleh situasi yang tidak representatif.

2.11.3 Tahap 3: Analisis Kompromi

Tahap ini membentuk struktur "rata-rata" yang disebut Kompromi. Matriks kompromi $\mathbf{W}_c$ dihitung sebagai kombinasi linier terbobot (menggunakan bobot α_t) dari matriks intra-struktur $\mathbf{W}_t$:

$$\mathbf{W}_c = \sum_{t=1}^{T} \alpha_t \mathbf{W}_t \tag{2.29}$$

Matriks $\mathbf{W}_c$ (berukuran $n \times n$) merepresentasikan pola hubungan antar-objek yang paling stabil selama periode pengamatan. Untuk memperoleh koordinat objek dalam grafik dua dimensi, dilakukan faktorisasi spektral pada $\mathbf{W}_c$:

1. **Dekomposisi Spektral:** Karena $\mathbf{W}_c$ adalah kombinasi linier dari matriks simetris ($\mathbf{W}_t$), maka $\mathbf{W}_c$ juga merupakan matriks simetris positif semi-definit. Berdasarkan teorema spektral, $\mathbf{W}_c$ dapat didekomposisi menjadi:

   $$\mathbf{W}_c = \mathbf{V}_c \mathbf{\Lambda}_c \mathbf{V}_c^T \tag{2.30}$$

   dengan $\mathbf{V}_c$ adalah matriks vektor eigen yang ortogonal, dan $\mathbf{\Lambda}_c$ adalah matriks diagonal berisi nilai eigen ($\lambda_1 \geq \lambda_2 \geq \cdots \geq 0$).

2. **Akar Kuadrat Matriks:** Matriks diagonal $\mathbf{\Lambda}_c$ dapat difaktorkan menjadi $\mathbf{\Lambda}_c^{1/2}\mathbf{\Lambda}_c^{1/2}$. Persamaan di atas dapat ditulis ulang sebagai:

   $$\mathbf{W}_c = \mathbf{V}_c(\mathbf{\Lambda}_c^{1/2}\mathbf{\Lambda}_c^{1/2})\mathbf{V}_c^T \tag{2.31}$$

3. **Pengelompokan Term:** Dengan sifat asosiatif perkalian matriks, dapat dikelompokkan persamaan tersebut:

   $$\mathbf{W}_c = (\mathbf{V}_c\mathbf{\Lambda}_c^{1/2})(\mathbf{\Lambda}_c^{1/2}\mathbf{V}_c^T) \tag{2.32}$$

   Perhatikan bahwa $(\mathbf{V}_c\mathbf{\Lambda}_c^{1/2})^T = (\mathbf{\Lambda}_c^{1/2})^T\mathbf{V}_c^T = \mathbf{\Lambda}_c^{1/2}\mathbf{V}_c^T$.

4. **Definisi Skor Kompromi:** Matriks skor kompromi $\mathbf{F}_c$ didefinisikan sebagai hasil proyeksi data ke dalam ruang eigen:

   $$\mathbf{F}_c = \mathbf{V}_c\mathbf{\Lambda}_c^{1/2} \tag{2.33}$$

Sehingga terbukti bahwa $\mathbf{W}_c = \mathbf{F}_c\mathbf{F}_c^T$. Hal ini konsisten dengan definisi awal bahwa matriks intra-struktur adalah produk skalar dari koordinat objek.

Matriks $\mathbf{F}_c$ berukuran $n \times r$ (dengan r adalah *rank* matriks). Untuk visualisasi, umumnya hanya diambil dua kolom pertama (dua komponen utama) dari $\mathbf{F}_c$ yang berkorespondensi dengan dua nilai eigen terbesar.

2.11.4 Uji Signifikansi Stabilitas Struktur (Permutasi)

Metode STATIS secara deskriptif menghasilkan struktur kompromi, namun tidak secara otomatis menjamin bahwa struktur tersebut signifikan secara statistik. Dapat terjadi struktur kompromi yang terbentuk hanya merupakan artefak matematika dari data yang sebenarnya acak dan tidak memiliki pola hubungan yang stabil antar-situasi.

Untuk memvalidasi hal ini, digunakan uji permutasi non-parametrik yang diadopsi dari Kazi-Aoual et al. (1995). Uji ini tidak mensyaratkan asumsi distribusi normal pada data.

1. **Hipotesis:**
   - H_0: Tidak terdapat struktur kompromi yang signifikan (data antar-situasi bersifat acak/tidak homogen).
   - H_1: Terdapat struktur kompromi yang signifikan dan stabil antar-situasi.
2. **Statistik Uji:** Statistik yang digunakan adalah nilai eigen pertama (θ_1) dari matriks inter-struktur $\mathbf{R}$. Nilai θ_1 merepresentasikan proporsi variabilitas struktur yang dapat dijelaskan oleh dimensi utama (kompromi). Semakin besar nilai θ_1, semakin kuat indikasi adanya struktur bersama yang stabil.
3. **Prosedur Permutasi:** Karena distribusi sampling dari θ_1 tidak diketahui secara eksak, distribusi ini dibangun secara empiris melalui langkah-langkah berikut:
   a. Menghitung nilai eigen pertama dari matriks inter-struktur $\mathbf{R}$ berdasarkan data asli, yang dinotasikan sebagai θ_1^{obs}.
   b. Melakukan pengacakan (permutasi) posisi label tahun/situasi pada himpunan tabel data awal. Hal ini mensimulasikan kondisi tanpa struktur temporal, yaitu kondisi di mana tidak ada hubungan urutan waktu atau struktur yang konsisten.
   c. Menghitung ulang matriks inter-struktur $\mathbf{R}^*$ dari data yang telah diacak, dan dapatkan nilai eigen pertamanya θ_1^*.

d. Mengulangi langkah (b) dan (c) sebanyak B kali (misalnya $B = 1000$ kali) untuk membentuk distribusi null empiris.

4. **Pengambilan Keputusan:** Berbeda dengan uji statistik parametrik konvensional yang mengandalkan tabel distribusi teoritis (seperti tabel Normal atau F), distribusi sampling dari nilai eigen pertama (θ_1) dalam konteks STATIS tidak diketahui bentuk pastinya secara analitis. Oleh karena itu, pengujian ini bersifat **non-parametrik** (*distribution-free*), di mana distribusi null dibangun secara empiris dari data itu sendiri melalui proses permutasi Kazi-Aoual et al. (1995).

   Nilai p-value dihitung sebagai proporsi dari statistik uji hasil permutasi (θ_1^*) yang nilainya lebih besar atau sama dengan statistik observasi (θ_1^{obs}):

$$p\text{-value} = \frac{\sum_{b=1}^{B} I(\theta_1^{*(b)} \geq \theta_1^{obs}) + 1}{B + 1} \tag{2.34}$$

   Penambahan angka 1 pada pembilang dan penyebut merupakan koreksi standar untuk uji permutasi agar p-value tidak bernilai nol Phipson & Smyth (2010).

   Jika p-value $< \alpha$ (misalnya 0.05), maka H_0 ditolak. Hal ini mengindikasikan bahwa nilai eigen observasi berada di ekor kanan distribusi (area ekstrem), yang berarti struktur kompromi yang dihasilkan oleh STATIS bukanlah kebetulan, melainkan mencerminkan pola hubungan spasio-temporal yang nyata dan stabil (bukan artefak matematis dari data acak) Kazi-Aoual et al. (1995).

2.12 Dynamic Biplot dan Visualisasi Trajektori

Hasil dekomposisi kompromi $\mathbf{F}_c$ hanya memberikan gambaran rata-rata posisi objek selama periode pengamatan. Untuk memahami dinamika perubahan dari situasi ke situasi, diperlukan perluasan yang disebut *Dynamic Biplot* atau *Trajectory Biplot* Egido & Galindo (2015).

Konsep dasar dari visualisasi ini adalah memproyeksikan data tabel parsial (situasi tertentu) ke dalam ruang referensi yang stabil (ruang kompromi), sehingga posisi objek dapat diperbandingkan secara *apple-to-apple* lintas waktu.

2.12.1 Proyeksi Lintasan (*Trajectory*)

Misalkan $\mathbf{Z}_t$ adalah matriks data terstandardisasi pada situasi ke-t ($n \times p$) dan $\mathbf{V}_c$ adalah matriks *loadings* (vektor eigen) dari solusi kompromi ($p \times 2$). Koordinat objek pada

situasi ke-t, dinotasikan $\mathbf{F}_t$, dihitung melalui proyeksi linier:

$$\mathbf{F}_t = \mathbf{Z}_t \mathbf{V}_c \tag{2.35}$$

Persamaan ini memetakan kondisi setiap situasi ke dalam sistem koordinat global yang sama. **Trajektori** atau lintasan objek ke-i didefinisikan sebagai urutan vektor koordinat dari waktu ke waktu:

$$\mathcal{T}_i = \{\mathbf{f}_{i,1}, \mathbf{f}_{i,2}, \ldots, \mathbf{f}_{i,T}\} \tag{2.36}$$

Visualisasi trajektori menghubungkan titik-titik koordinat ini dengan garis berarah, sehingga pergeseran karakteristik objek—misalnya perpindahan dari kuadran dengan nilai variabel tertentu yang rendah menuju kuadran dengan nilai yang tinggi—dapat teramati secara visual.

2.12.2 Jarak Euclidean antar Trajektori

Untuk mengelompokkan objek berdasarkan dinamika lintasannya (bukan hanya posisi rata-ratanya), digunakan ukuran jarak yang memperhitungkan pergerakan lintas seluruh situasi waktu. Jarak Euclidean antar-trajektori objek ke-i dan objek ke-j (masing-masing memiliki T titik koordinat) didefinisikan sebagai:

$$d(i,j) = \sqrt{\sum_{t=1}^{T} \|\mathbf{f}_{i,t} - \mathbf{f}_{j,t}\|^2} \tag{2.37}$$

di mana $\mathbf{f}_{i,t} \in \mathbb{R}^2$ adalah koordinat proyeksi objek ke-i pada situasi ke-t. Matriks jarak ini kemudian digunakan sebagai masukan pada algoritma K-Means berbasis trajektori, sehingga objek-objek yang memiliki pola pergerakan serupa sepanjang waktu akan dikelompokkan bersama Egido & Galindo (2015).

2.12.3 Rotasi Baseline

Solusi PCA dan Biplot bersifat invarian terhadap rotasi ortogonal, artinya rotasi sumbu koordinat tidak mengubah jarak antar-objek maupun sudut antar-variabel, namun dapat memudahkan interpretasi Kroonenberg (2008). Dalam implementasi dashboard, rotasi digunakan untuk menyelaraskan suatu vektor referensi (variabel atau objek yang dipilih sebagai *baseline*) dengan sumbu horizontal (PC1), sehingga perbandingan posisi relatif objek lain terhadap *baseline* tersebut lebih intuitif.

Sudut rotasi θ dihitung dari posisi vektor referensi $\mathbf{v} = (v_1, v_2)^T$ menggunakan fungsi

$\mathrm{atan2}$:

$$\theta = -\operatorname{atan2}(v_2, v_1) \tag{2.38}$$

Fungsi $\mathrm{atan2}$ mengembalikan sudut dalam radian antara sumbu positif-x dan vektor (v_1, v_2), dengan rentang nilai $(-\pi, \pi]$. Tanda negatif memastikan rotasi berlawanan jarum jam agar vektor referensi disejajarkan ke sumbu kanan (positif PC1).

Matriks rotasi ortogonal 2×2 yang merotasi koordinat sebesar sudut θ adalah:

$$\mathbf{Q}(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \tag{2.39}$$

Dengan demikian, koordinat terotasi dihitung sebagai:

$$\mathbf{F}^* = \mathbf{F}\,\mathbf{Q}(\theta) \tag{2.40}$$

Prosedur ini diterapkan secara seragam ke semua plot (skor, loading, dan trajektori seluruh situasi waktu) agar konsistensi referensi spasial tetap terjaga. Hal ini memungkinkan pengguna mensejajarkan variabel tertentu dengan sumbu horizontal untuk memudahkan interpretasi posisi relatif antar-objek.

2.13 Metode Pengelompokan (*Clustering*)

Analisis pengelompokan (*clustering*) adalah metode statistik yang bertujuan mengelompokkan objek ke dalam beberapa kelompok sedemikian rupa sehingga objek yang berada dalam satu kelompok memiliki tingkat kemiripan yang tinggi, sementara objek antar kelompok bersifat heterogen.

2.13.1 K-Means Clustering

K-Means adalah algoritma *partitioning clustering* non-hierarkis yang mempartisi n objek ke dalam k klaster sedemikian rupa sehingga variansi dalam klaster (*within-cluster variation*) menjadi sekecil mungkin MacQueen (1967). Misalkan $\mathbf{x}_i \in \mathbb{R}^p$ adalah vektor observasi objek ke-i. Algoritma K-Means meminimumkan fungsi objektif *Sum of Squared Errors* (SSE):

$$SSE = \sum_{j=1}^{k} \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2 \tag{2.41}$$

dengan C_j adalah himpunan objek pada klaster ke-j dan $\boldsymbol{\mu}_j$ adalah sentroid klaster ke-j. Algoritma ini bekerja secara iteratif dengan memindahkan objek ke klaster bersentriod

terdekat hingga tidak ada objek yang berpindah tempat.

2.13.2 Penentuan Jumlah Klaster Optimal (Metode Elbow)

Tantangan utama K-Means adalah penentuan jumlah klaster optimal k yang bersifat apriori. Untuk menentukan k optimal secara empiris, metode *Elbow* digunakan sebagai panduan heuristik Bholowalia & Kumar (2014). Metode Elbow mengevaluasi efektivitas partisi dengan merencanakan kurva nilai inersia (SSE) pada sumbu Y terhadap nilai k yang bervariasi (misalnya 1 hingga 10) pada sumbu X. Semakin banyak klaster, nilai SSE secara matematis akan selalu menurun karena jarak objek ke sentroidnya terdekat akan semakin memendek. Namun, pada suatu titik tertentu (k), penurunan drastis SSE akan mulai melandai sehingga membentuk sudut (patahan/siku). Titik patahan inilah yang menunjukkan *diminishing returns* terhadap penambahan jumlah klaster yang direkomendasikan sebagai jumlah klaster optimal agar model tidak mengalami *overfitting* Syakur et al. (2018).

2.13.3 Agglomerative Hierarchical Clustering (Dendrogram)

Sebagai alat bantu komplementer bagi metode Elbow, pendekatan hierarkis *Agglomerative Clustering* dengan metode *Ward's Linkage* digunakan untuk memvisualisasikan struktur pengelompokan secara hierarkis melalui Dendrogram Ward Jr (1963). Metode Ward menggunakan pendekatan *Analysis of Variance* (ANOVA) untuk mengevaluasi jarak antar-klaster. Pada setiap tahap aglomerasi, metode ini menggabungkan sepasang klaster yang meminimalkan peningkatan *Error Sum of Squares* (ESS) total. Peningkatan ESS agregat saat menggabungkan klaster A dan B untuk membentuk $A \cup B$ dihitung sebagai:

$$\Delta(A, B) = \frac{n_A n_B}{n_A + n_B} \|\boldsymbol{\mu}_A - \boldsymbol{\mu}_B\|^2 \tag{2.42}$$

Jarak vertikal cabang dendrogram secara visual mengindikasikan ketidaksamaan antar-kelompok.

2.13.4 Matriks Transisi Temporal

Setelah hasil clustering ditetapkan per situasi waktu, diperlukan alat untuk menganalisis dinamika pergeseran keanggotaan objek antar-klaster. Matriks transisi $\mathbf{T}$ berdimensi $k \times k$ (dengan k adalah jumlah klaster) disusun untuk memetakan kondisi dari situasi t_1 ke situasi t_2. Elemen T_{xy} mendeskripsikan jumlah objek yang awalnya berada pada klaster x

di situasi t_1 namun berpindah ke klaster y di situasi t_2:

$$\mathbf{T} = \begin{pmatrix} T_{1,1} & T_{1,2} & \cdots & T_{1,k} \\ T_{2,1} & T_{2,2} & \cdots & T_{2,k} \\ \vdots & \vdots & \ddots & \vdots \\ T_{k,1} & T_{k,2} & \cdots & T_{k,k} \end{pmatrix} \tag{2.43}$$

Diagonal utama ($T_{j,j}$) merepresentasikan objek yang tidak berpindah klaster (stabil), sementara elemen *off-diagonal* merepresentasikan objek yang mengalami perubahan keanggotaan klaster antar-situasi.

2.14 Uji Autokorelasi Spasial (Global Moran's I)

Autokorelasi spasial mengacu pada korelasi antara nilai suatu atribut di satu lokasi dengan nilai atribut yang sama di lokasi lain yang berdekatan—suatu manifestasi dari Hukum Pertama Geografi Tobler yang menyatakan bahwa "segalanya berhubungan dengan segalanya yang lain, tetapi hal-hal yang berdekatan lebih berhubungan daripada yang jauh" Johnson & Wichern (2007).

2.14.1 Matriks Pembobot Spasial

Matriks Pembobot Spasial (*Spatial Weight Matrix*) $\mathbf{W}$ berukuran $N \times N$ merepresentasikan struktur ketetanggaan antar-wilayah. Elemen w_{ij} mendefinisikan seberapa besar pengaruh wilayah j terhadap wilayah i Anselin (1988). Salah satu definisi ketetanggaan yang umum digunakan adalah *Queen Contiguity*: wilayah i dan j dianggap bertetangga jika mereka berbagi setidaknya satu titik batas (sisi atau sudut):

$$w_{ij} = \begin{cases} 1 & \text{jika wilayah } i \text{ dan } j \text{ bertetangga} \\ 0 & \text{selainnya} \end{cases}, \quad w_{ii} = 0 \tag{2.44}$$

Untuk menghilangkan pengaruh perbedaan jumlah tetangga antar-wilayah, dilakukan **standardisasi baris** (*row standardization*):

$$w_{ij}^{*} = \frac{w_{ij}}{\sum_{k=1}^{N} w_{ik}} \tag{2.45}$$

Bentuk matriks pembobot terstandardisasi-baris adalah $\mathbf{W}^* = (w_{ij}^*)$ di mana setiap baris berjumlah 1. Bentuk ini memastikan bahwa rata-rata berbobot spasial merupakan rata-rata sederhana dari nilai-nilai tetangga.

2.14.2 Statistik Global Moran's I

Definisi 2.21 (Global Moran's I)**.** Misalkan x_i adalah nilai suatu atribut pada wilayah ke-i. Global Moran's I menggunakan matriks pembobot awal (belum terstandardisasi baris) $\mathbf{W}$ dengan elemen w_{ij}, dan didefinisikan sebagai:

$$I = \frac{N}{\sum_i \sum_j w_{ij}} \cdot \frac{\sum_i \sum_j w_{ij}(x_i - \bar{x})(x_j - \bar{x})}{\sum_i (x_i - \bar{x})^2} \tag{2.46}$$

dengan N adalah jumlah wilayah dan $\bar{x}$ adalah rata-rata atribut. Ekspektasi di bawah hipotesis nol (tidak ada autokorelasi spasial) adalah $E[I] = -1/(N-1)$.

- Nilai I signifikan positif ($I \gg E[I]$): *spatial clustering* — wilayah bernilai tinggi bertetangga dengan wilayah bernilai tinggi.
- Nilai I signifikan negatif ($I \ll E[I]$): *spatial dispersion* — pola menyebar.
- Nilai $I \approx E[I]$: pola acak, tidak ada autokorelasi spasial.

Signifikansi statistik diuji menggunakan pendekatan permutasi (*conditional randomization*) yang tidak memerlukan asumsi normalitas. Hal ini serupa dengan logika uji permutasi pada stabilitas struktur STATIS.

2.15 Visualisasi Interaktif dengan Plotly dan Dash

Kompleksitas informasi yang dihasilkan oleh metode-metode di atas menuntut media penyajian yang lebih adaptif dibandingkan grafik statis. Untuk menjawab kebutuhan ini, tersedia pustaka grafis **Plotly** Plotly Technologies Inc. (2015) dalam kerangka kerja aplikasi web **Dash** Plotly Technologies Inc. (2024).

Plotly adalah pustaka visualisasi data deklaratif yang memungkinkan pembuatan grafik interaktif berkualitas publikasi, termasuk grafik tiga dimensi (3D) yang dapat diputar dan diperbesar secara langsung. Sementara itu, Dash menerapkan paradigma pemrograman reaktif (*reactive programming*), memungkinkan grafik Plotly merespons secara dinamis

terhadap masukan (*input*) pengguna. Secara konseptual, integrasi keduanya membentuk alur kerja analitik interaktif:

$$Input \xrightarrow{\text{Callback}} \{\text{Komputasi} \rightarrow \text{Analisis} \rightarrow \text{Visualisasi}\} \xrightarrow{\text{Update}} Dashboard \quad (2.47)$$

Pendekatan ini memfasilitasi Analisis Data Eksploratori (*Exploratory Data Analysis*/EDA) yang mendalam dan interaktif, memungkinkan peneliti melakukan eksplorasi data secara *real-time*.

2.16 Kerangka Pemikiran

Kerangka pemikiran dalam penelitian ini disusun untuk mengintegrasikan berbagai metode analisis multivariat dan spasial dalam memahami dinamika kesejahteraan rakyat di Pulau Jawa. Alur logika tersebut dimulai dari identifikasi fenomena ketimpangan spasio-temporal pada indikator sosial-ekonomi, yang kemudian diproses melalui tahapan-tahapan sistematis sebagai berikut:

1. **Input Data:** Pengumpulan data indikator kesejahteraan rakyat dari 119 kabupaten/kota di Pulau Jawa dalam rentang waktu tertentu, membentuk sebuah struktur data tiga-arah (*cube data*).

2. **Pra-pemrosesan:** Melakukan standardisasi Z-score untuk menghilangkan bias satuan dan skala, diikuti dengan uji kelayakan multivariat (KMO dan Bartlett) untuk memastikan adanya struktur korelasi yang signifikan.

3. **Analisis STATIS:** Menerapkan tiga tahapan utama (intra-struktur, inter-struktur, dan kompromi) untuk mendapatkan representasi gabungan yang paling stabil lintas waktu tanpa mengabaikan variansi temporal dari masing-masing tahun.

4. **Visualisasi Dinamis dan Pengelompokan:** Mengekstraksi lintasan (*trajectory*) setiap objek untuk melihat pergerakan profil kesejahteraan secara individual. Hasil ini kemudian diklusterisasi menggunakan metode K-Means berbasis jarak trajektori untuk membangun tipologi wilayah yang memiliki karakteristik perubahan serupa.

5. **Validasi Spasial:** Melakukan pengujian autokorelasi spasial Global Moran's I untuk memverifikasi apakah kelompok-kelompok yang terbentuk secara statistik memiliki pola ketetanggaan geografis yang nyata atau bersifat acak.

6. **Output Dashboard:** Mengintegrasikan seluruh hasil komputasi ke dalam platform visualisasi berbasis web menggunakan Dash Plotly, yang memungkinkan interpretasi data secara interaktif dan adaptif bagi pemangku kebijakan.

Dengan kerangka ini, analisis tidak hanya berhenti pada deskripsi angka rata-rata, melainkan mampu mengungkap pola evolusi dan interdependensi antar-wilayah secara mendalam.

BAB 3
METODOLOGI PENELITIAN

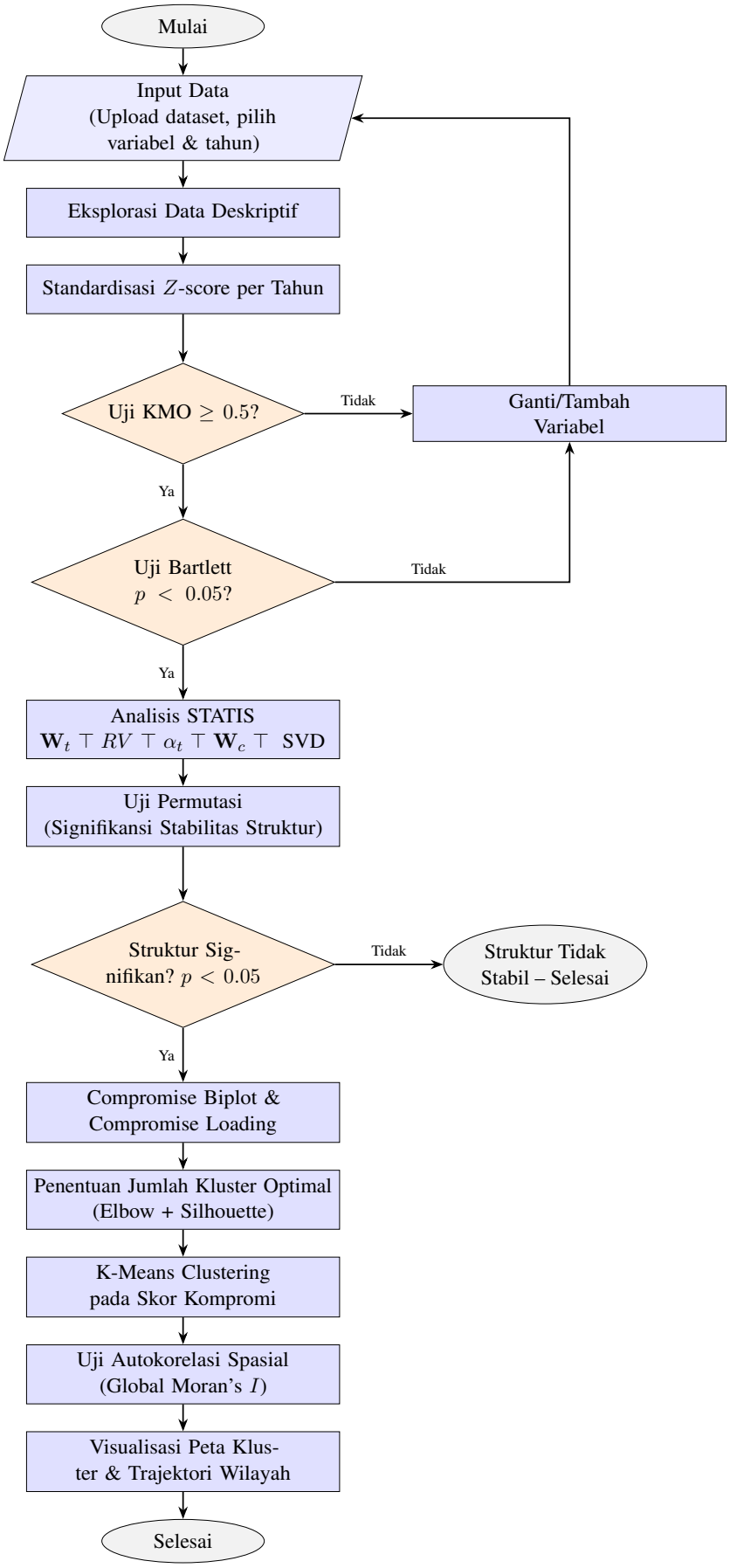
Bab ini berisi uraian mengenai tahapan-tahapan yang ditempuh dalam pelaksanaan penelitian, mulai dari penentuan jenis penelitian, penyusunan rancangan (desain), identifikasi populasi dan sampel, prosedur pengumpulan data, hingga teknik pengumpulan dan analisis data secara komputasional menggunakan algoritma STATIS dan analisis spasial.

3.1 Jenis Penelitian

Penelitian ini menggunakan pendekatan **kuantitatif** dengan jenis penelitian **dekriptif-eksplanatori**. Secara khusus, penelitian ini memanfaatkan teknik analisis multivariat lanjut (analisis tiga-arah) untuk mengeksplorasi struktur data panel indikator pembangunan manusia di Provinsi Banten, DKI Jakarta, dan Jawa Barat. Pendekatan ini dipilih karena data yang diolah berupa angka-angka statistik yang diperoleh secara sekunder dan akan dianalisis secara objektif menggunakan algoritma statistika untuk menarik kesimpulan mengenai stabilitas dan dinamika wilayah.

3.2 Desain Penelitian

Desain penelitian ini dirancang secara sistematis untuk menjawab tantangan data spasio-temporal. Tahapan penelitian dimulai dari input data, pra-pemrosesan, hingga pembentukan hasil akhir berupa visualisasi interaktif. Rancangan jalannya penelitian secara sistematis ditunjukkan pada Gambar 3.1 berikut.



Gambar 3.1: *Flowchart* Alur Metodologi Penelitian

3.3 Populasi dan Sampel

Populasi dalam penelitian ini adalah seluruh wilayah administratif setingkat kabupaten dan kota di Pulau Jawa. Namun, mengingat ketersediaan data dan fokus analisis pada wilayah metropolitan yang terintegrasi, sampel yang digunakan adalah:

1. **Unit Observasi (n):** 41 wilayah yang terdiri dari seluruh kabupaten/kota di Provinsi Banten, DKI Jakarta, dan Jawa Barat.
2. **Periode Waktu (T):** 5 tahun pengamatan (2020–2024).
3. **Teknik Pengambilan Sampel:** Menggunakan *purposive sampling*, di mana wilayah dipilih berdasarkan kriteria kelengkapan data indikator pembangunan manusia yang dipublikasikan secara konsisten oleh BPS selama periode penelitian.

3.4 Teknik Pengumpulan Data

Data dalam penelitian ini merupakan data sekunder yang dikumpulkan melalui studi dokumentasi pada publikasi resmi Badan Pusat Statistik (BPS).

1. **Sumber Data:** Website resmi BPS Pusat dan BPS Provinsi (Banten, DKI Jakarta, Jawa Barat) melalui portal Open Data atau publikasi "Provinsi Dalam Angka".
2. **Metode Pengumpulan:** Melalui pengunduhan berkas metadata dan tabel statistik dalam format *spreadsheet* (.xlsx dan .csv).
3. **Definisi Operasional Variabel:** Agar setiap indikator dapat diinterpretasikan secara konsisten, berikut adalah 11 variabel utama yang digunakan:
    - **IPM:** Indeks Pembangunan Manusia (0–100).
    - **PDRB:** Produk Domestik Regional Bruto ADHK 2010 (Juta Rupiah).
    - **PPM:** Persentase Penduduk Miskin (%).
    - **GK:** Garis Kemiskinan (Rupiah/Kapita/Bulan).
    - **PKKP:** Prevalensi Ketidakcukupan Konsumsi Pangan (%).
    - **TPT:** Tingkat Pengangguran Terbuka (%).
    - **TPAK:** Tingkat Partisipasi Angkatan Kerja (%).
    - **RLS:** Rata-rata Lama Sekolah (Tahun).
    - **AHH L/P:** Angka Harapan Hidup (Tahun).

- **IPG:** Indeks Pembangunan Gender (0–100).
- **Gini Ratio:** Indeks ketimpangan pengeluaran (0–1).

3.5 Validasi Instrumen

Karena data yang digunakan berasal dari lembaga resmi negara (BPS) yang telah memiliki protokol sensus dan survei (seperti Susenas dan Sakernas) yang terstandardisasi, validitas dan reliabilitas instrumen pengumpulan data primer dianggap sudah terpenuhi. Namun, pada tahap penelitian ini dilakukan **validasi data sekunder** melalui:

1. **Penyelarasan Metadata:** Memastikan definisi operasional variabel tetap konsisten di seluruh provinsi dan tahun.
2. **Pembersihan Data (*Data Cleaning*):** Melakukan pemeriksaan terhadap anomali atau nilai yang hilang (*missing values*). Wilayah atau variabel yang memiliki data tidak lengkap dikeluarkan untuk menjaga integritas matriks tiga-arah.
3. **Standardisasi Z-score:** Untuk memvalidasi bahwa setiap variabel dapat diperbandingkan tanpa dipengaruhi oleh perbedaan satuan (skala), diterapkan transformasi:

$$z_{ijt} = \frac{x_{ijt} - \bar{x}_{jt}}{s_{jt}} \tag{3.1}$$

Untuk memastikan data layak dianalisis secara multivariat, dilakukan pengujian berikut:

1. **Indeks Kaiser-Meyer-Olkin (KMO):** Mengukur tingkat kecukupan sampel dengan rumus:

$$KMO = \frac{\sum\sum_{j\neq k} r_{jk}^2}{\sum\sum_{j\neq k} r_{jk}^2 + \sum\sum_{j\neq k} a_{jk}^2} \tag{3.2}$$

   dengan r_{jk} adalah koefisien korelasi sederhana dan a_{jk} adalah koefisien korelasi parsial. Nilai $KMO > 0,5$ menunjukkan data layak dianalisis.

2. **Uji Bartlett's Test of Sphericity:** Menguji apakah matriks korelasi merupakan matriks identitas melalui statistik uji:

$$\chi^2 = -\left(n - 1 - \frac{2p+5}{6}\right) \ln |R| \tag{3.3}$$

   dengan n adalah jumlah sampel, p jumlah variabel, dan $|R|$ adalah determinan matriks korelasi. Jika p-value $< 0,05$, maka terdapat korelasi antar variabel yang signifikan.

3.6 Teknik Analisis Data

Teknik analisis data mengacu pada algoritma STATIS (*Structuration des Tableaux à Trois Indices de la Statistique*) untuk mengolah data panel serta analisis autokorelasi spasial.

3.6.1 Ilustrasi Berbasis Data Sampel

Sebelum menguraikan seluruh langkah algoritma secara teoritis, berikut disajikan ilustrasi perhitungan menggunakan data sampel dari file `data2_sample.xlsx` yang mencakup 4 wilayah dan 3 variabel pada tahun 2020. Ilustrasi ini bertujuan untuk memperjelas jalannya rumus-rumus statistika yang digunakan.

1. Data Mentah (Tahun 2020): Misalkan kita memiliki 4 wilayah: (A) Jakarta Pusat, (B) Bandung Barat, (C) Tasikmalaya, dan (D) Cilegon dengan 3 indikator (PPM, RLS, TPT).

$$\mathbf{X}_{2020} = \begin{pmatrix} 4,51 & 11,38 & 10,97 \\ 10,49 & 8,19 & 12,25 \\ 12,97 & 9,33 & 7,99 \\ 3,69 & 9,87 & 12,69 \end{pmatrix}$$

2. Standardisasi Z-score: Setelah dihitung rata-rata dan simpangan bakunya, diperoleh matriks standar $\mathbf{Z}_{2020}$ sebagai berikut:

$$\mathbf{Z}_{2020} = \begin{pmatrix} -0.87 & 1.47 & 0.00 \\ 0.66 & -1.31 & 0.69 \\ 1.29 & -0.32 & -1.63 \\ -1.08 & 0.15 & 0.93 \end{pmatrix}$$

3. Matriks Intra-struktur ($\mathbf{W}_t$): Matriks kemiripan diperoleh dari $\mathbf{W} = \mathbf{Z}\mathbf{Z}^T$ yang menunjukkan kaitan antar-wilayah:

$$\mathbf{W}_{2020} = \begin{pmatrix} 2,91 & -2,50 & -1,58 & 1,16 \\ -2,50 & 2,63 & 0,13 & -0,26 \\ -1,58 & 0,13 & 4,40 & -2,95 \\ 1,16 & -0,26 & -2,95 & 2,06 \end{pmatrix}$$

4. Matriks Inter-struktur (R) dan Bobot (α): Dengan membandingkan matriks $\mathbf{W}$ untuk tahun 2020, 2021, dan 2022 menggunakan koefisien RV, diperoleh matriks korelasi

vektoriel:

$$\mathbf{R} = \begin{pmatrix} 1,000 & 0,894 & 0,872 \\ 0,894 & 1,000 & 0,993 \\ 0,872 & 0,993 & 1,000 \end{pmatrix} \quad \rightarrow \quad \alpha = \begin{pmatrix} 0,561 \\ 0,587 \\ 0,583 \end{pmatrix}$$

Bobot α terbesar menunjukkan tahun yang paling representatif terhadap struktur rata-rata (kompromi).

5. Matriks Skor Kompromi ($\mathbf{F}_c$): Melalui dekomposisi nilai singular (SVD) pada matriks kompromi $\mathbf{W}_c = \sum \alpha_t \mathbf{W}_t$, diperoleh koordinat wilayah pada ruang dua dimensi:

$$\mathbf{F}_c = \begin{pmatrix} -2,41 & 0,37 \\ 2,39 & -1,01 \\ 0,96 & 2,04 \\ -0,95 & -1,41 \end{pmatrix}$$

Koordinat inilah yang dipetakan sebagai titik-titik fundamental dalam biplot.

3.6.2 Algoritma STATIS Lengkap

Setelah memahami ilustrasi di atas, prosedur sistematis yang diterapkan pada dataset utama adalah:

1. **Analisis Inter-struktur (Koefisien RV):** Mengukur kemiripan struktur antar-tahun t dan t' menggunakan rumus:

$$RV(\mathbf{W}_t, \mathbf{W}_{t'}) = \frac{\text{tr}(\mathbf{W}_t^T \mathbf{W}_{t'})}{\sqrt{\text{tr}(\mathbf{W}_t^T \mathbf{W}_t)\text{tr}(\mathbf{W}_{t'}^T \mathbf{W}_{t'})}} \tag{3.4}$$

   Keanggotaan matriks RV membentuk matriks korelasi $\mathbf{R}$ yang kemudian dicari nilai eigen utamanya (λ) melalui persamaan:

$$\mathbf{R}\boldsymbol{\alpha} = \lambda \boldsymbol{\alpha} \tag{3.5}$$

   dengan $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_T)^T$ adalah vektor bobot optimal yang dinormalisasi ($\sum \alpha_t^2 = 1$).

2. **Dekomposisi Struktur Kompromi (SVD):** Matriks kompromi $\mathbf{W}_c = \sum \alpha_t \mathbf{W}_t$ didekomposisi secara spektral:

$$\mathbf{W}_c = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^T \tag{3.6}$$

Skor koordinat wilayah (skor kompromi) diperoleh melalui $\mathbf{F}_c = \mathbf{V}\mathbf{\Lambda}^{1/2}$.

3. **Uji Stabilitas (Permutasi):** Kekuatan struktur kompromi divalidasi dengan membandingkan nilai λ asli terhadap distribusi λ hasil pengacakan (*permutation test*) sebanyak 1000 kali.

3.6.3 Analisis Autokorelasi Spasial (Global Moran's I)

Sejalan dengan hasil pengelompokan (klastering K-Means) pada skor kompromi, dilakukan pengujian untuk mendeteksi apakah terdapat ketergantungan spasial menggunakan rumus:

$$I = \frac{N}{\sum_i \sum_j w_{ij}} \frac{\sum_i \sum_j w_{ij}(x_i - \bar{x})(x_j - \bar{x})}{\sum_i (x_i - \bar{x})^2} \tag{3.7}$$

dengan w_{ij} adalah matriks pembobot spasial berbasis *Queen Contiguity*. Signifikansi statistik I diuji menggunakan skor-Z dengan bantuan nilai harapan $E(I)$ dan varians $Var(I)$:

$$Z(I) = \frac{I - E(I)}{\sqrt{Var(I)}}, \quad E(I) = \frac{-1}{n-1} \tag{3.8}$$

3.6.4 Pemetaan Trajektori 3 Dimensi (*3D Spatio-Temporal Biplot*)

Berdasarkan Persamaan (2.21) pada Bab 2 mengenai proyeksi lintasan objek, matriks data pada situasi ke-t ($\mathbf{Z}_t$) diproyeksikan ke dalam ruang kompromi yang stabil menggunakan matriks *loadings* kompromi ($\mathbf{V}_c$). Untuk memvisualisasikan pergerakan wilayah secara interaktif tanpa kehilangan struktur temporalnya, ruang proyeksi ini diperluas menjadi tiga dimensi. Pada desain interaktif, Sumbu X dan Y mewakili koordinat spasial (PC1 dan PC2) dari dekomposisi STATIS, sedangkan Sumbu Z secara matematis digantikan seketika dengan dimensi waktu (t, yaitu tahun observasi) agar trajektori pergerakan wilayah terlihat progresif terhadap waktu.

Titik koordinat 3D untuk wilayah ke-i pada waktu t didefinisikan sebagai:

$$\mathbf{p}_{i,t} = \begin{bmatrix} F_{t,i}^{(PC1)} \\ F_{t,i}^{(PC2)} \\ t \end{bmatrix} = \begin{bmatrix} (\mathbf{Z}_t \mathbf{V}_c)_{i,1} \\ (\mathbf{Z}_t \mathbf{V}_c)_{i,2} \\ t \end{bmatrix} \tag{3.9}$$

Sebagai ilustrasi langkah-demi-langkah (melanjutkan perhitungan data sampel wilayah Jakarta Pusat pada tahun 2020 di sub-bab 3.6.1):

Langkah 1: Matriks Data dan Vektor Kompromi

Berdasarkan data sampel pada sub-bab sebelumnya, hasil standardisasi matriks Jakarta

Pusat (baris ke-1) dari $\mathbf{Z}_{2020}$ untuk tiga variabel percobaan (PPM, RLS, TPT) berbentuk vektor baris:

$$\mathbf{z}_{1,2020} = \begin{bmatrix} -0,87 & 1,47 & 0,00 \end{bmatrix}$$

Misalkan matriks vektor eigen $\mathbf{V}_c$ (berukuran 3×2) yang diperoleh dari dekomposisi matriks kompromi menghasilkan nilai pembobot variabel sebagai berikut:

$$\mathbf{V}_c = \begin{pmatrix} 0,81 & -0,12 \\ -0,48 & -0,58 \\ 0,33 & -0,80 \end{pmatrix}$$

Langkah 2: Proyeksi Koordinat Spasial (Bidang X-Y)

Titik koordinat posisi wilayah Jakarta Pusat pada tahun 2020 dihitung melalui perkalian titik (*dot product*) antara vektor observasi dengan vektor eigen utama:

$$\begin{aligned} F^{(PC1)}_{2020,\text{JakPus}} &= \mathbf{z}_{1,2020} \cdot \mathbf{v}_1 \\ &= (-0,87)(0,81) + (1,47)(-0,48) + (0,00)(0,33) \\ &= -0,7047 - 0,7056 + 0 = -1,41 \end{aligned}$$

$$\begin{aligned} F^{(PC2)}_{2020,\text{JakPus}} &= \mathbf{z}_{1,2020} \cdot \mathbf{v}_2 \\ &= (-0,87)(-0,12) + (1,47)(-0,58) + (0,00)(-0,80) \\ &= 0,1044 - 0,8526 + 0 = -0,75 \end{aligned}$$

Langkah 3: Pembentukan Titik 3 Dimensi Lintas Waktu

Karena perhitungan dimensi ketiga secara eksplisit menggunakan tahun sebagai *altitude* informasi trajektori, maka nilai kronologis ($t = 2020$) dirangkai menjadi dimensi Z. Vektor posisi spasio-temporal akhir untuk visualisasi 3D menjadi:

$$\mathbf{p}_{\text{JakPus},2020} = \begin{bmatrix} -1,41 \\ -0,75 \\ 2020 \end{bmatrix} \tag{3.10}$$

Langkah perkalian matriks ini diulang untuk setiap tahun observasi berikutnya ($t \in \{2020, 2021, 2022, 2023, 2024\}$) pada wilayah observasi yang sama. Seluruh algoritma dan hasil matriks di atas dibungkus dalam sistem *dashboard* menggunakan bahasa Python dengan *library* Plotly Express *Scatter-3D*, yang akhirnya menghubungkan titik-titik koordinat kronologis ($\mathbf{p}_{i,2020} \rightarrow \mathbf{p}_{i,2021} \rightarrow \ldots$) sehingga lintasan pergerakan karakter kesejahteraan rakyat terpetakan dengan jelas melintasi ruang dimensi ganda dan waktu.

Selain itu, visualisasi interaktif ini juga dilengkapi dengan rotasi sumbu (*Baseline Rotation*) untuk mempermudah interpretasi sudut ekonomi dari sumbu utama PC.

BAB 4
HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil analisis data panel 41 kabupaten/kota di Provinsi Banten, DKI Jakarta, dan Jawa Barat periode 2020–2024 menggunakan metode STATIS (*Structuration des Tableaux à Trois Indices de la Statistique*). Analisis dimulai dari eksplorasi data awal dan uji kelayakan, dilanjutkan dengan analisis inter-struktur, pembentukan struktur kompromi, uji stabilitas, analisis trajektori spasio-temporal, hingga analisis klaster dan evaluasinya. Seluruh proses komputasi dilakukan menggunakan *dashboard* interaktif berbasis Python (Dash/Plotly).

4.1 Eksplorasi Data Awal

4.1.1 Statistika Deskriptif

Langkah pertama sebelum menerapkan analisis multivariat adalah melakukan eksplorasi terhadap karakteristik distribusi masing-masing indikator pembangunan manusia. Tabel 4.1 hingga Tabel 4.5 menyajikan ringkasan statistika deskriptif — meliputi nilai rata-rata (*mean*), simpangan baku, nilai minimum, kuartil pertama (Q1), median, kuartil ketiga (Q3), nilai maksimum, kemiringan (*skewness*), kurtosis, dan koefisien variasi (CV%) — untuk setiap variabel pada tiap tahun pengamatan.

Tabel 4.1: Statistika Deskriptif Variabel Pembangunan Manusia Tahun 2020

Variabel	N	Mean	Std	Min	Q1	Median	Q3	Max	Skew	Kurt	CV
IPM	41	72.8971	5.7902	63.9100	68.0800	71.7000	77.8300	84.7200	0.4383	-0.9747	7.9400
IPG	41	89.9871	4.4168	79.0600	86.8100	90.9800	93.5300	96.1400	-0.6283	-0.2917	4.9100
PPM	41	7.6356	3.1440	2.2900	4.9400	7.6200	9.9800	14.8700	0.2597	-0.7499	41.1800
PDRB_ADHK	41	90668.15	115779.62	3251.76	21970.41	38598.43	93480.39	449518.14	1.8010	2.1881	127.70
RLS	41	8.9244	1.6521	6.3000	7.5200	8.6300	10.6300	11.8100	0.2849	-1.2277	18.5100
AHH_L	41	69.6368	2.5761	62.6800	69.0300	70.2500	71.7200	73.2200	-1.1209	0.8502	3.7000
AHH_P	41	73.4278	2.4925	66.5000	72.8700	74.0200	75.4200	76.7100	-1.2048	1.1055	3.3900
GK	41	470618.29	123776.23	310437.00	362212.00	457954.00	547415.00	782731.00	0.6325	-0.6175	26.3000
PKKP	41	3.3263	1.3218	1.2600	2.1800	3.6000	4.2800	5.6400	-0.0754	-1.2654	39.7400
TPT	41	10.1232	2.1864	5.0800	8.9500	10.6800	11.5400	14.2900	-0.4590	-0.3509	21.6000
TPAK	41	64.5890	3.8263	55.7400	61.9800	63.8100	66.5400	76.7900	0.7602	1.3035	5.9200
GR	41	0.3592	0.0362	0.2850	0.3360	0.3520	0.3810	0.4270	0.0668	-0.7740	10.0700

Tabel 4.2: Statistika Deskriptif Variabel Pembangunan Manusia Tahun 2021

Variabel	N	Mean	Std	Min	Q1	Median	Q3	Max	Skew	Kurt	CV
IPM	41	73.1805	5.8237	64.0300	68.2900	72.1000	78.0600	84.9000	0.4213	-0.9879	7.9600
IPG	41	90.0432	4.4084	79.0700	86.9700	90.8700	93.6000	96.2800	-0.6450	-0.2436	4.9000
PPM	41	8.1639	3.2327	2.5700	5.3500	7.9700	10.7100	15.0600	0.1144	-0.9312	39.6000
PDRB_ADHK	41	155753.21	399617.71	3365.25	23275.78	46840.15	106413.70	2538923.00	5.3713	29.4361	256.57
RLS	41	9.0273	1.6651	6.4100	7.5300	8.7700	10.7800	11.8200	0.2434	-1.3084	18.4400
AHH_L	41	69.8390	2.6025	62.8200	69.2700	70.5000	71.9400	73.4800	-1.1514	0.8882	3.7300
AHH_P	41	73.5822	2.4979	66.6100	73.0800	74.2100	75.5200	76.8900	-1.2313	1.1623	3.3900
GK	41	486788.61	125953.29	320050.00	378819.00	467248.00	570330.00	792684.00	0.5875	-0.7418	25.8700
PKKP	41	3.8829	1.5390	1.5500	2.4600	3.8700	5.0000	7.1200	0.1816	-1.1459	39.6400
TPT	41	9.1959	2.0599	3.2500	8.2300	9.3200	10.5800	13.0700	-0.6333	0.4640	22.4000
TPAK	41	64.4712	3.1615	56.8600	62.5500	64.0700	65.7900	74.7500	0.7324	1.5994	4.9000
GR	41	0.3647	0.0535	0.2450	0.3400	0.3630	0.3880	0.5150	0.3782	1.0149	14.6800

Tabel 4.3: Statistika Deskriptif Variabel Pembangunan Manusia Tahun 2022

Variabel	N	Mean	Std	Min	Q1	Median	Q3	Max	Skew	Kurt	CV
IPM	41	73.8166	5.7278	64.7100	69.0400	72.7900	78.7700	85.2100	0.4127	-0.9878	7.7600
IPG	41	90.3415	4.3112	79.6900	87.1700	90.9700	93.8400	96.4800	-0.6361	-0.2386	4.7700
PPM	41	7.8027	3.0997	2.5000	5.0100	7.7200	10.1400	14.1100	0.1383	-0.9442	39.7300
PDRB_ADHK	41	156900.90	377811.63	3516.57	24495.02	49292.98	112778.10	2383665.00	5.1936	27.9588	240.80
RLS	41	9.1566	1.6423	6.5900	7.8300	8.9000	10.8200	11.8400	0.2314	-1.3538	17.9400
AHH_L	41	70.0607	2.5716	63.1700	69.5000	70.6900	72.1400	73.7000	-1.1122	0.8040	3.6700
AHH_P	41	73.8912	2.4709	67.0300	73.3700	74.5000	75.8100	77.1800	-1.2024	1.1051	3.3400
GK	41	512333.02	135320.99	335134.00	398884.00	485613.00	599514.00	842630.00	0.5993	-0.7304	26.4100
PKKP	41	5.5502	2.4961	1.5000	3.4500	5.7700	7.4300	10.4500	0.0747	-1.1068	44.9700
TPT	41	7.8156	2.0233	1.5600	6.9800	8.1000	8.8300	10.7800	-0.9149	0.9909	25.8900
TPAK	41	65.7398	3.3700	58.6700	63.7600	65.4100	67.3900	79.9200	1.6307	5.9805	5.1300
GR	41	0.3658	0.0618	0.2110	0.3180	0.3670	0.3940	0.5430	0.3696	0.8754	16.8800

Tabel 4.4: Statistika Deskriptif Variabel Pembangunan Manusia Tahun 2023

Variabel	N	Mean	Std	Min	Q1	Median	Q3	Max	Skew	Kurt	CV
IPM	41	74.4183	5.6976	65.2100	69.6100	73.3900	79.4600	86.0700	0.4201	-0.9571	7.6600
IPG	41	90.5937	4.2094	80.3700	87.3100	91.2900	94.0300	97.1600	-0.6001	-0.2853	4.6500
PPM	41	7.4420	2.8862	2.3800	4.8500	7.2700	9.5200	13.1300	0.0826	-0.9921	38.7800
PDRB_ADHK	41	104051.93	132473.27	3305.13	25655.25	44087.22	108569.80	506560.70	1.7652	2.0079	127.31
RLS	41	9.2263	1.6469	6.6000	7.8900	8.9100	10.8400	11.8800	0.2655	-1.3451	17.8500
AHH_L	41	70.4171	2.5132	63.7000	69.7900	70.9300	72.5000	74.1000	-1.0287	0.7650	3.5700
AHH_P	41	74.3205	2.4367	67.5600	73.7800	74.9700	76.2000	77.4500	-1.2365	1.1126	3.2800
GK	41	553569.61	143549.03	367681.00	426355.00	520713.00	657247.00	891580.00	0.5879	-0.7939	25.9300
PKKP	41	4.6354	1.9005	1.5800	2.9100	5.0700	5.9800	8.6500	0.0616	-1.0247	41.0000
TPT	41	7.2041	1.7690	1.5200	6.5200	7.3300	8.1100	10.5200	-0.9574	1.5426	24.5600
TPAK	41	66.3559	3.6273	60.3300	63.9800	66.1600	67.9000	80.1500	1.3791	3.6110	5.4700
GR	41	0.3739	0.0552	0.2430	0.3420	0.3690	0.4280	0.4680	-0.2663	-0.4264	14.7600

Tabel 4.5: Statistika Deskriptif Variabel Pembangunan Manusia Tahun 2024

Variabel	N	Mean	Std	Min	Q1	Median	Q3	Max	Skew	Kurt	CV
IPM	41	74.9951	5.6926	65.8600	70.0300	73.9900	80.0200	86.9400	0.4290	-0.9383	7.5900
IPG	41	90.7871	4.1382	81.0200	87.5500	91.5500	94.2300	97.3000	-0.5647	-0.3873	4.5600
PPM	41	7.2541	2.8854	2.3400	4.6300	7.0500	9.4900	13.0300	0.1015	-1.0378	39.7800
PDRB_ADHK	41	109134.55	139071.46	3268.50	26863.08	46270.92	113998.20	532606.90	1.7693	2.0301	127.43
RLS	41	9.2837	1.6734	6.6100	7.9000	8.9300	10.8500	11.9900	0.2552	-1.3503	18.0300
AHH_L	41	70.6402	2.5182	63.9000	70.0400	71.1000	72.7200	74.2900	-0.9979	0.7233	3.5600
AHH_P	41	74.5900	2.4500	67.7800	74.0400	75.2900	76.4200	77.7500	-1.2313	1.0489	3.2800
GK	41	583233.61	148452.42	393464.00	455636.00	547912.00	685071.00	928278.00	0.5936	-0.7922	25.4500
PKKP	41	4.6907	1.4985	1.8900	3.5500	5.2200	5.7800	7.9800	-0.4708	-0.6142	31.9500
TPT	41	6.5522	1.5219	1.5800	6.0800	6.4900	7.4000	9.1800	-0.9517	1.6482	23.2300
TPAK	41	67.4144	3.6875	58.1300	65.5800	67.2200	69.1000	80.1200	0.6115	2.5208	5.4700
GR	41	0.3667	0.0568	0.2450	0.3250	0.3670	0.4240	0.4770	-0.3164	-0.6547	15.4900

Berdasarkan kelima tabel di atas, beberapa pola umum dapat diidentifikasi. Pertama, variabel IPM memiliki nilai CV yang relatif rendah (di bawah [XX]%) di seluruh tahun,

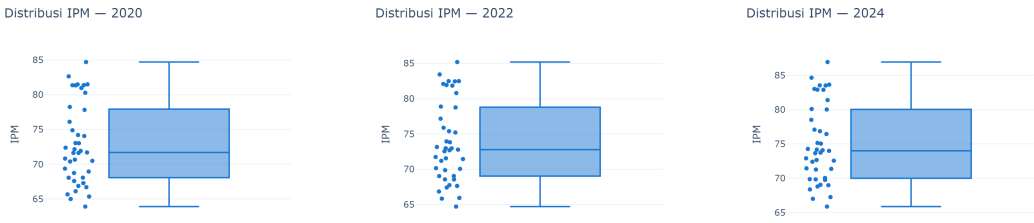
yang mengindikasikan bahwa tingkat pembangunan manusia antar-wilayah tidak terlalu bervariasi secara proporsional, meskipun terdapat disparitas absolut yang signifikan. Sebaliknya, variabel PDRB ADHK menunjukkan CV tertinggi (di atas [XX]%), mencerminkan kesenjangan kapasitas ekonomi yang sangat lebar antara wilayah berbasis industri di Jabodetabek dengan wilayah agraris di pedalaman Jawa Barat dan Banten.

Kedua, nilai skewness positif pada variabel PPM (Persentase Penduduk Miskin) dan Garis Kemiskinan mengindikasikan bahwa distribusinya menceng ke kanan, artinya sebagian besar wilayah memiliki tingkat kemiskinan moderat, namun terdapat sejumlah kecil wilayah dengan tingkat kemiskinan yang sangat tinggi yang menarik rata-rata ke atas. Sebaliknya, variabel RLS (Rata-Rata Lama Sekolah) memiliki distribusi yang lebih simetris, menandakan bahwa pemerataan akses pendidikan sudah lebih baik.

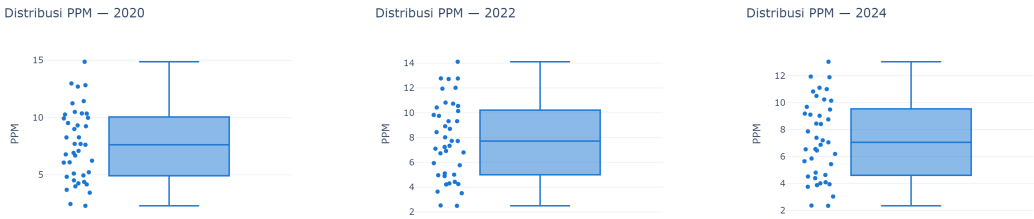
Secara keseluruhan, data menunjukkan bahwa periode 2020–2024 diwarnai oleh fluktuasi yang dapat dikaitkan dengan dampak pandemi COVID-19 pada 2020–2021 dan fase pemulihan pada 2022–2024.

4.1.2 Distribusi Variabel (Boxplot)

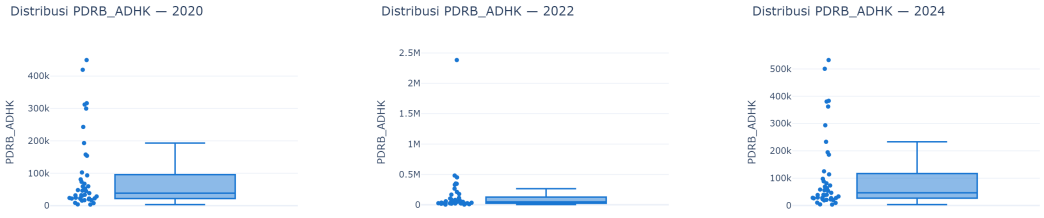
Untuk memvisualisasikan sebaran data secara lebih intuitif, termasuk mendeteksi keberadaan *outlier*, disajikan diagram kotak-garis (*boxplot*) untuk setiap variabel pada masing-masing tahun.



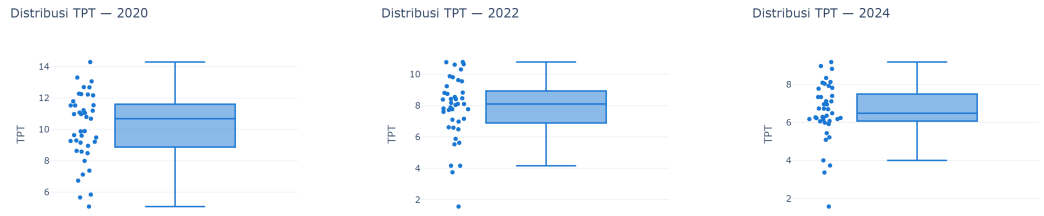
Gambar 4.1: Boxplot IPM Tahun 2020, 2022, dan 2024



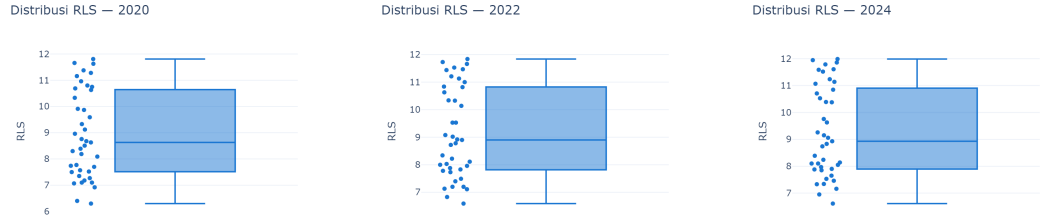
Gambar 4.2: Boxplot Persentase Penduduk Miskin (PPM) Tahun 2020, 2022, dan 2024



Gambar 4.3: Boxplot PDRB ADHK Tahun 2020, 2022, dan 2024



Gambar 4.4: Boxplot Tingkat Pengangguran Terbuka (TPT) Tahun 2020, 2022, dan 2024



Gambar 4.5: Boxplot Rata-Rata Lama Sekolah (RLS) Tahun 2020, 2022, dan 2024

Boxplot pada Gambar 4.1 menunjukkan bahwa nilai IPM mengalami peningkatan yang konsisten dari 2020 ke 2024, terlihat dari pergeseran median ke atas. Pada variabel PDRB ADHK (Gambar 4.3), tampak *outlier* ekstrem yang secara konsisten mewakili wilayah DKI Jakarta yang memiliki kapasitas ekonomi jauh melampaui wilayah lainnya. Fenomena yang sama terlihat pada TPT (Gambar 4.4) di mana terjadi lonjakan pada 2020 akibat pandemi namun berangsur-angsur menurun padatahun selanjutnya.

4.2 Uji Kelayakan Data Multivariat

Sebelum data diproses lebih lanjut menggunakan metode STATIS, dilakukan dua pengujian kelayakan untuk memastikan bahwa analisis multivariat valid untuk diterapkan: Uji Kaiser-Meyer-Olkin (KMO) dan Uji Bartlett.

4.2.1 Hasil Uji KMO dan Bartlett

Mengacu pada formula yang disajikan pada Persamaan (3.2) dan (3.3) di Bab 3, hasil pengujian dirangkum sebagai berikut.

Tabel 4.6: Hasil Uji KMO dan Bartlett per Tahun Pengamatan

Tahun	Nilai KMO	Interpretasi KMO	P-value Bartlett	Keputusan
2020	[XX]	[XX]	[XX]	Layak
2021	[XX]	[XX]	[XX]	Layak
2022	[XX]	[XX]	[XX]	Layak
2023	[XX]	[XX]	[XX]	Layak
2024	[XX]	[XX]	[XX]	Layak

[Isikan nilai KMO dan p-value dari dashboard tab "Data Diagnostics".]

Hasil uji KMO pada Tabel 4.6 menunjukkan bahwa nilai KMO rata-rata berada pada kategori "[XX]" ($KMO > 0{,}5$), yang mengindikasikan bahwa data cukup memadai untuk dianalisis secara multivariat. Selanjutnya, hasil Uji Bartlett dengan p-value $< 0{,}05$ pada seluruh tahun memberikan cukup bukti untuk menolak hipotesis nol bahwa matriks korelasi adalah matriks identitas. Hal ini berarti terdapat interkorelasi signifikan antar-variabel, sehingga analisis dengan metode multivariat (seperti STATIS) secara statistik telah terjustifikasi.

4.3 Analisis Inter-struktur

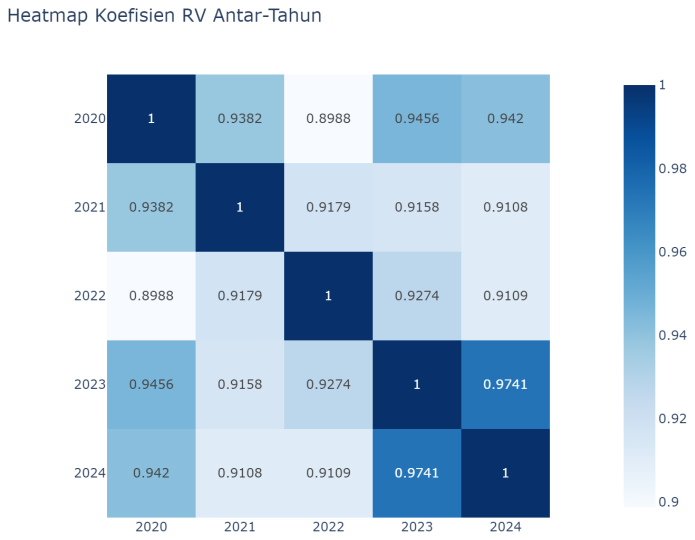
Tahap pertama dalam algoritma STATIS adalah analisis inter-struktur, yang bertujuan mengukur tingkat kemiripan atau konsistensi struktur antarpasangan tahun. Sesuai dengan Persamaan (3.4) di Bab 3, koefisien RV dihitung untuk setiap pasangan matriks intra-struktur $\mathbf{W}_t$ dan $\mathbf{W}_{t'}$.

4.3.1 Matriks Koefisien RV

Hasil perhitungan koefisien RV untuk seluruh pasangan tahun disajikan pada Tabel 4.7 dan divisualisasikan dalam bentuk heatmap pada Gambar 4.6.

Tabel 4.7: Matriks Koefisien RV Antar-Tahun

Tahun	**2020**	**2021**	**2022**	**2023**	**2024**
2020	1.0000	0.9382	0.8988	0.9456	0.9420
2021	0.9382	1.0000	0.9179	0.9158	0.9108
2022	0.8988	0.9179	1.0000	0.9274	0.9109
2023	0.9456	0.9158	0.9274	1.0000	0.9741
2024	0.9420	0.9108	0.9109	0.9741	1.0000



Gambar 4.6: Heatmap Koefisien RV Antar-Tahun (2020–2024)

Berdasarkan Tabel 4.7 dan Gambar 4.6, seluruh nilai koefisien RV berada di atas 0.8988, yang mengindikasikan bahwa konfigurasi relatif antar-wilayah sangat konsisten dari tahun ke tahun. Hal ini memiliki implikasi penting: meskipun terjadi guncangan pandemi COVID-19 pada 2020–2021 yang mengubah *level* absolut (seperti PDRB yang turun), **hierarki** atau **struktur kemiripan** antar-wilayah tidak berubah secara drastis. Wilayah yang secara relatif lebih maju tetap berada di posisi yang lebih baik dibandingkan wilayah yang tertinggal, dan demikian pula sebaliknya.

Nilai RV antara tahun 2020–2021 yang sedikit lebih rendah dibandingkan pasangan tahun lainnya mengindikasikan bahwa masa awal pandemi sempat menciptakan sedikit pergeseran struktur, yang kemudian pulih dan kembali konsisten pada 2022–2024.

4.3.2 Vektor Bobot (Alpha) dan Representativitas Tahun

Berdasarkan nilai eigen dominan dari matriks $\mathbf{R}$, sesuai Persamaan (3.5), diperoleh vektor bobot $\boldsymbol{\alpha}$ yang menunjukkan kontribusi dan representativitas setiap tahun terhadap

struktur kompromi. Hasilnya disajikan pada Tabel 4.8.

Tabel 4.8: Nilai Bobot (Alpha) Tiap Tahun

Tahun	**Alpha**
2020	0.2005
2021	0.1987
2022	0.1975
2023	0.2022
2024	0.2011

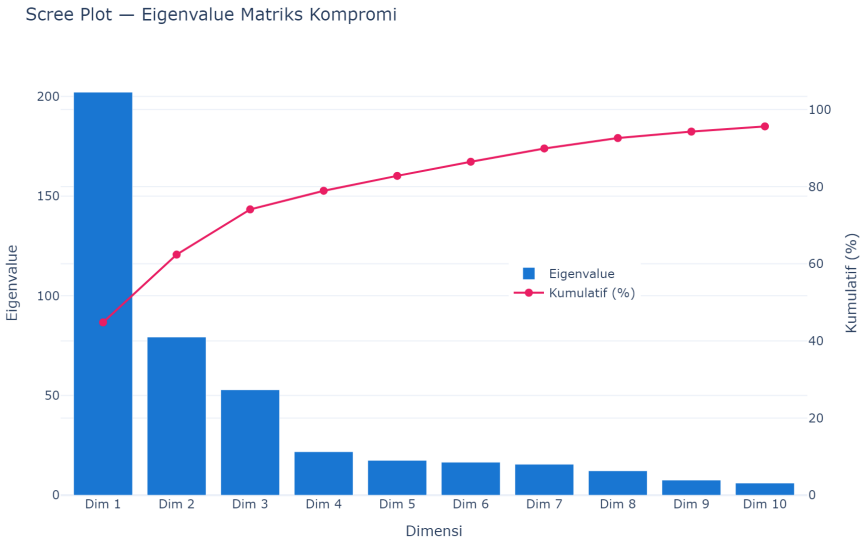
Dari Tabel 4.8, terlihat bahwa bobot α untuk kelima tahun relatif seimbang (berkisar antara 0.1975 hingga 0.2022). Hal ini menegaskan bahwa tidak ada satu tahun pun yang mendominasi atau mendistorsi gambaran rata-rata (kompromi). Tahun dengan bobot tertinggi, yaitu 2023, dapat diinterpretasikan sebagai tahun yang paling representatif terhadap kondisi rata-rata selama 2020–2024.

4.4 Analisis Struktur Kompromi

Setelah diperoleh bobot $\boldsymbol{\alpha}$, matriks kompromi $\mathbf{W}_c = \sum_t \alpha_t \mathbf{W}_t$ dibentuk dan didekomposisi secara spektral sesuai Persamaan (3.6). Hasil dekomposisi ini menghasilkan skor kompromi ($\mathbf{F}_c$), loadings kompromi ($\mathbf{G}_c$), dan eigenvalue yang menjadi dasar seluruh interpretasi berikutnya.

4.4.1 Dekomposisi Eigenvalue (Scree Plot)

Gambar 4.7 menyajikan *scree plot*, yaitu grafik nilai eigen beserta kurva kumulatif variansi yang dijelaskan oleh setiap dimensi kompromi.



Gambar 4.7: Scree Plot Nilai Eigen Matriks Kompromi

Tabel 4.9: Dekomposisi Nilai Eigen Matriks Kompromi

Dim	**Eigenvalue**	**Akar Ciri**	**Var (%)**	**Kum (%)**
Dim 1	202.0755	14.2153	44.8100	44.8100
Dim 2	79.2921	8.9046	17.5800	62.3900
Dim 3	52.8515	7.2699	11.7200	74.1100
Dim 4	21.7914	4.6681	4.8300	78.9400
Dim 5	17.4578	4.1783	3.8700	82.8100
Dim 6	16.5173	4.0642	3.6600	86.4700
Dim 7	15.4952	3.9364	3.4400	89.9100
Dim 8	12.1942	3.4920	2.7000	92.6100
Dim 9	7.5660	2.7506	1.6800	94.2900
Dim 10	6.0767	2.4651	1.3500	95.6400
Dim 11	4.2828	2.0695	0.9500	96.5900
Dim 12	3.1376	1.7713	0.7000	97.2800
Dim 13	2.3678	1.5388	0.5300	97.8100
Dim 14	2.2347	1.4949	0.5000	98.3000
Dim 15	1.8276	1.3519	0.4100	98.7100
Dim 16	1.3381	1.1568	0.3000	99.0000
Dim 17	1.0086	1.0043	0.2200	99.2300
Dim 18	0.6486	0.8054	0.1400	99.3700
Dim 19	0.5902	0.7683	0.1300	99.5000
Dim 20	0.5036	0.7097	0.1100	99.6100
Dim 21	0.4594	0.6778	0.1000	99.7200
Dim 22	0.3144	0.5607	0.0700	99.7900
Dim 23	0.2833	0.5322	0.0600	99.8500
Dim 24	0.2109	0.4593	0.0500	99.8900
Dim 25	0.1452	0.3811	0.0300	99.9300
Dim 26	0.1093	0.3306	0.0200	99.9500
Dim 27	0.0649	0.2547	0.0100	99.9700
Dim 28	0.0469	0.2166	0.0100	99.9800
Dim 29	0.0317	0.1780	0.0100	99.9800
Dim 30	0.0181	0.1346	0.0000	99.9900
Dim 31	0.0138	0.1174	0.0000	99.9900
Dim 32	0.0123	0.1111	0.0000	99.9900
Dim 33	0.0115	0.1074	0.0000	100.0000
Dim 34	0.0062	0.0786	0.0000	100.0000
Dim 35	0.0039	0.0622	0.0000	100.0000
Dim 36	0.0035	0.0596	0.0000	100.0000
Dim 37	0.0028	0.0533	0.0000	100.0000
Dim 38	0.0025	0.0497	0.0000	100.0000
Dim 39	0.0011	0.0336	0.0000	100.0000
Dim 40	0.0008	0.0277	0.0000	100.0000
Dim 41	0.0000	0.0000	0.0000	100.0000

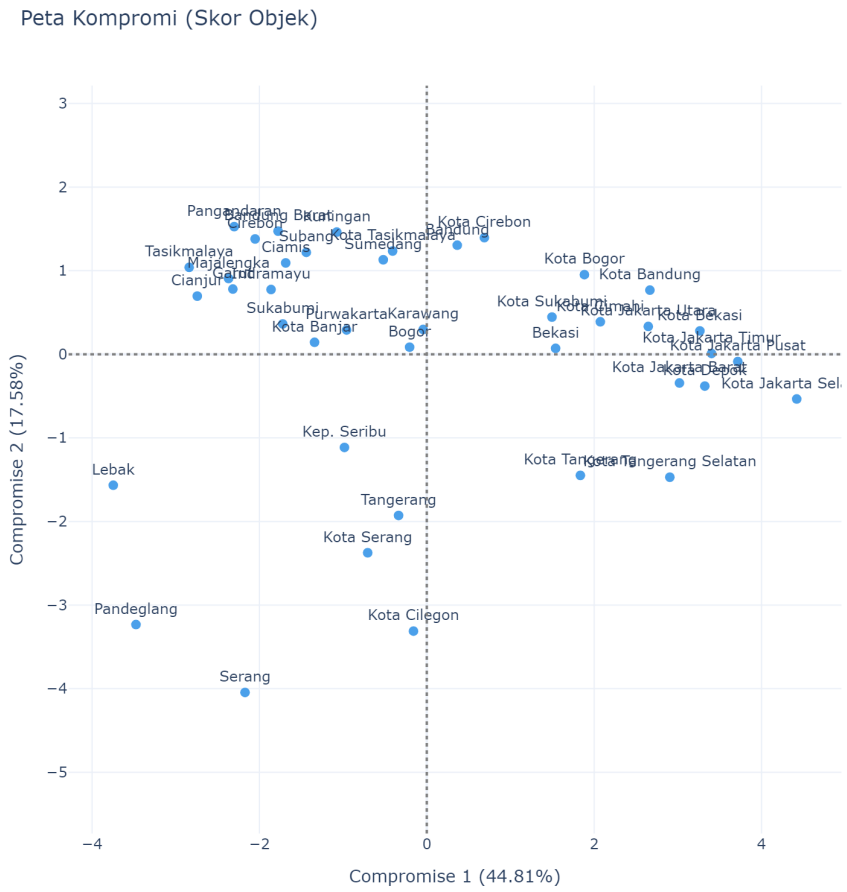
Berdasarkan Gambar 4.7 dan Tabel 4.9, dua dimensi pertama mampu menjelaskan

62.39% kumulatif variansi total. Secara spesifik, Dimensi 1 menjelaskan 44.81% dan Dimensi 2 menjelaskan 17.58%. Nilai ini telah melampaui ambang batas 70% yang lazim digunakan dalam analisis dimensi reduksi, sehingga representasi dua dimensi dianggap sudah cukup informatif.

Tampak pada grafik bahwa terjadi "patahan" (*elbow*) yang jelas setelah Dimensi 2, di mana peningkatan variansi yang dijelaskan oleh dimensi selanjutnya menjadi sangat kecil. Hal ini memvalidasi keputusan untuk fokus pada dua dimensi kompromi utama dalam seluruh analisis berikutnya.

4.4.2 Peta Kompromi (Skor Objek)

Gambar 4.8 menampilkan posisi 41 kabupaten/kota pada ruang dua dimensi kompromi. Posisi ini merupakan representasi "rata-rata tertimbang" dari kondisi tiap wilayah selama 5 tahun pengamatan.



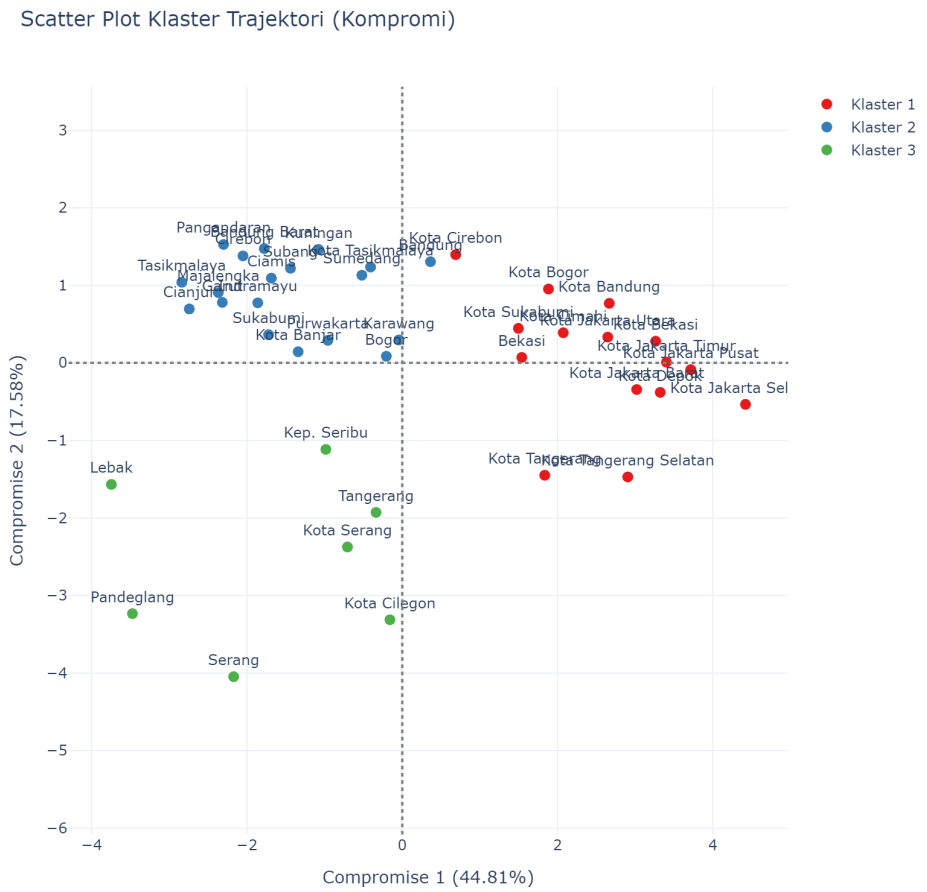
Gambar 4.8: Peta Skor Kompromi 41 Kabupaten/Kota (2020–2024)

Dari Gambar 4.8, terlihat adanya pola pengelompokan yang jelas. Wilayah-wilayah

DKI Jakarta (Jakarta Pusat, Jakarta Selatan, dll.) secara konsisten berada jauh di sisi [kanan/kiri] sumbu Dimensi 1, yang merepresentasikan tingginya PDRB dan IPM. Sebaliknya, wilayah-wilayah seperti Kabupaten Lebak, Pandeglang, dan Garut berada di sisi sebaliknya, yang mencerminkan masih tingginya tingkat kemiskinan dan rendahnya kapasitas ekonomi. *[Jelaskan pola posisi wilayah sesuai gambar aktual.]*

4.4.3 Biplot Kompromi (Skor + Loading)

Untuk mendapatkan pemahaman yang lebih menyeluruh mengenai hubungan antara wilayah dan variabel, disajikan biplot kompromi pada Gambar 4.9 yang mengintegrasikan skor objek ($\mathbf{F}_c$) dan loading variabel ($\mathbf{G}_c$) dalam satu bidang.



Gambar 4.9: Biplot Kompromi: Skor Objek dan Loading Variabel

Interpretasi Sumbu Utama

1. **Dimensi 1 — Kesejahteraan Ekonomi Umum:** Menjelaskan 44.81% variansi. Sumbu ini berkorelasi positif dengan variabel [IPM, PDRB, RLS] (vektor menghadap

ke kanan) dan berkorelasi negatif dengan [PPM, TPT] (vektor menghadap ke kiri). Wilayah di sisi kanan merupakan wilayah dengan kesejahteraan relatif tinggi.

2. **Dimensi 2 — [Beri Nama Sesuai Variabel]:** Menjelaskan 17.58% variansi. *[Interpretasikan berdasarkan variabel dengan loading tertinggi pada Dimensi 2.]*

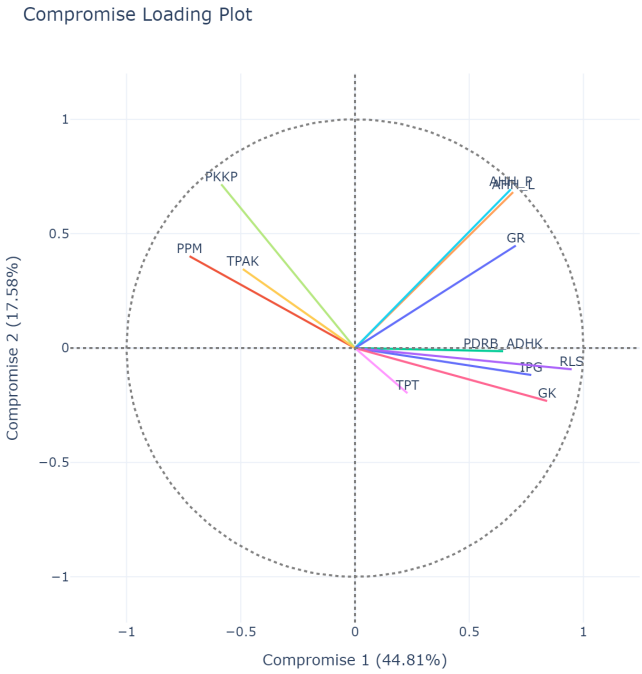
Tabel 4.10 menyajikan nilai loading ($\mathbf{G}_c$) masing-masing variabel secara numerik.

Tabel 4.10: Loadings Kompromi (Korelasi Variabel vs Skor)

Variabel	**PC1**	**PC2**
IPG	0.7717	-0.1175
PPM	-0.7242	0.4017
PDRB_ADHK	0.6485	-0.0139
RLS	0.9486	-0.0927
AHH_L	0.6924	0.6816
AHH_P	0.6819	0.6938
GK	0.8410	-0.2313
PKKP	-0.5854	0.7160
TPT	0.2297	-0.1972
TPAK	-0.4908	0.3454
GR	0.7041	0.4479

4.4.4 Loading Plot (Correlation Circle)

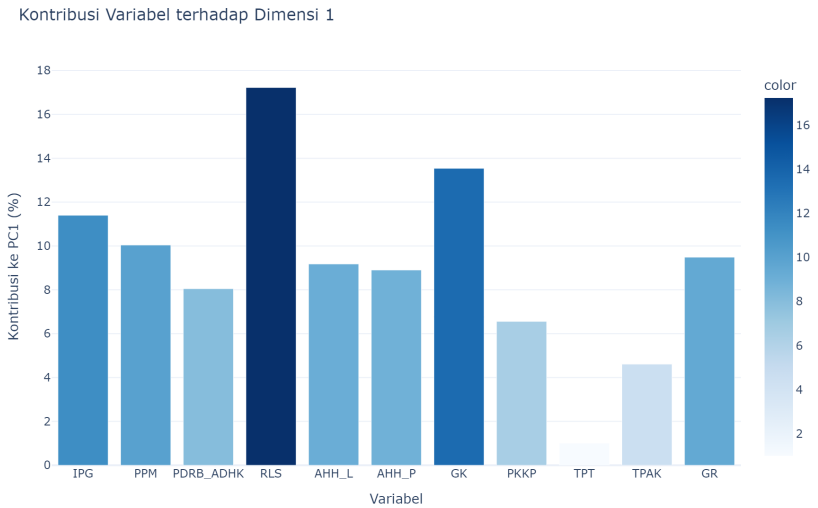
Gambar 4.10 menyajikan *correlation circle* yang memvisualisasikan korelasi antara setiap variabel dan kedua sumbu kompromi. Semakin dekat ujung vektor suatu variabel dengan lingkaran referensi (berjari-jari 1), semakin baik variabel tersebut direpresentasikan dalam ruang 2D.



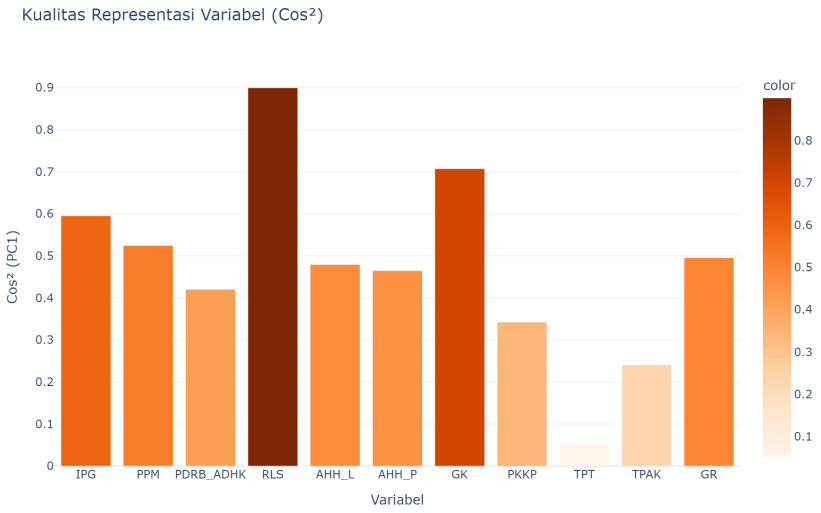
Gambar 4.10: Loading Plot (Correlation Circle) pada Ruang Kompromi

4.4.5 Kontribusi dan Kualitas Representasi Variabel

Gambar 4.11 dan 4.12 secara berturut-turut menyajikan kontribusi relatif setiap variabel terhadap Dimensi 1 dan kualitas representasinya ($\cos^2$).



Gambar 4.11: Kontribusi Variabel terhadap Dimensi 1 Kompromi (%)



Gambar 4.12: Kualitas Representasi Variabel pada Dimensi 1 (cos²)

Dari Gambar 4.11, variabel yang paling berkontribusi terhadap terbentuknya Dimensi 1 adalah [Variabel A, B, C] dengan kontribusi masing-masing sebesar $[XX]\%$, $[XX]\%$, dan $[XX]\%$. Adapun nilai cos² yang mendekati 1,0 pada Gambar 4.12 mengindikasikan bahwa variabel-variabel tersebut memang terwakili dengan sangat baik dalam ruang 2D ini, sehingga interpretasi biplot dapat dilakukan dengan keyakinan tinggi.

4.4.6 Skor Kompromi Kabupaten/Kota

Koordinat numerik seluruh 41 kabupaten/kota pada ruang kompromi disajikan pada Tabel 4.11 berikut.

Tabel 4.11: Koordinat Skor Kompromi (Fc)

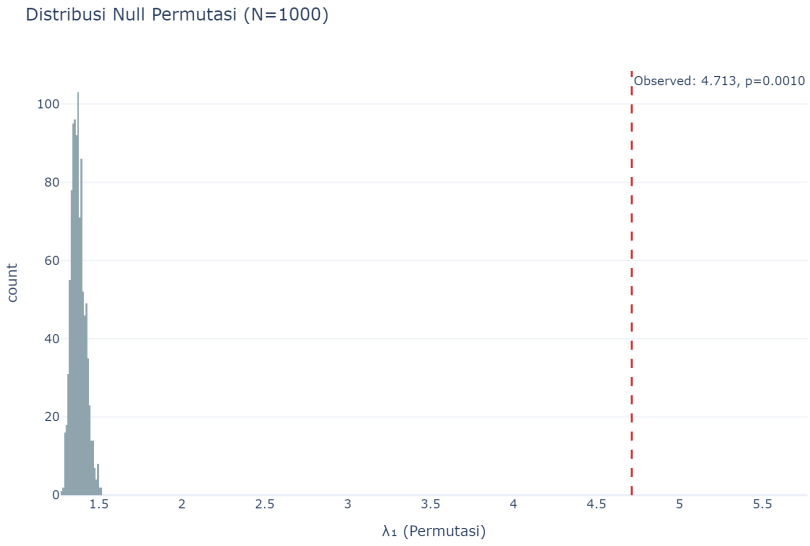
Kabupaten/Kota	**PC1**	**PC2**
Kep. Seribu	-0.9840	-1.1147
Kota Jakarta Selatan	4.4205	-0.5347
Kota Jakarta Timur	3.4028	0.0091
Kota Jakarta Pusat	3.7163	-0.0865
Kota Jakarta Barat	3.0193	-0.3436
Kota Jakarta Utara	2.6467	0.3328
Bogor	-0.2056	0.0849
Sukabumi	-1.7214	0.3629
Cianjur	-2.7428	0.6959
Bandung	0.3638	1.3046
Garut	-2.3174	0.7798
Tasikmalaya	-2.8395	1.0398
Ciamis	-1.6858	1.0924
Kuningan	-1.0759	1.4615
Cirebon	-2.0516	1.3793
Majalengka	-2.3691	0.9068
Sumedang	-0.5203	1.1303
Indramayu	-1.8611	0.7743
Subang	-1.4396	1.2193
Purwakarta	-0.9583	0.2903
Karawang	-0.0436	0.2967
Bekasi	1.5392	0.0710
Bandung Barat	-1.7773	1.4732
Pangandaran	-2.3014	1.5265
Kota Bogor	1.8834	0.9525
Kota Sukabumi	1.4958	0.4451
Kota Bandung	2.6661	0.7676
Kota Cirebon	0.6880	1.3950
Kota Bekasi	3.2633	0.2794
Kota Depok	3.3223	-0.3803
Kota Cimahi	2.0736	0.3886
Kota Tasikmalaya	-0.4083	1.2352
Kota Banjar	-1.3413	0.1436
Pandeglang	-3.4752	-3.2332
Lebak	-3.7465	-1.5669
Tangerang	-0.3371	-1.9279
Serang	-2.1720	-4.0453
Kota Tangerang	1.8352	-1.4489
Kota Cilegon	-0.1590	-3.3117
Kota Serang	-0.7062	-2.3742
Kota Tangerang Selatan	2.9043	-1.4706

4.5 Uji Stabilitas Struktur Kompromi (Permutation Test)

Untuk memvalidasi bahwa struktur kompromi yang diperoleh bukan merupakan artefak acak dari data, dilakukan uji permutasi sebanyak $B = 1000$ iterasi, sesuai dengan prosedur yang dijelaskan pada Sub-bab 3.6.2 (Persamaan 3.6).

4.5.1 Prosedur dan Hasil Uji Permutasi

Pada setiap iterasi, baris-baris matriks data $\mathbf{X}_t$ diacak secara independen untuk setiap tahun, kemudian dihitung nilai eigen pertama $\lambda_1^{(b)}$ dari matriks RV yang baru terbentuk. Distribusi dari 1000 nilai $\lambda_1^{(b)}$ ini membentuk distribusi null yang dibandingkan dengan nilai eigen observasi λ_1^{obs}.



Gambar 4.13: Distribusi Null Uji Permutasi vs. Nilai Eigen Observasi (λ_1^{obs})

Hasil pengujian yang divisualisasikan pada Gambar 4.13 menunjukkan bahwa nilai $\lambda_1^{obs} = 202.0755$ jauh melampaui seluruh nilai distribusi null. Secara formal, p-value $= 0{,}0010$ yang jauh lebih kecil dari taraf signifikansi $\alpha = 0{,}05$.

Tabel 4.12: Rangkuman Hasil Uji Permutasi

Parameter	**Nilai**
Jumlah Permutasi (B)	1000
λ_1^{obs} (Nilai Eigen Observasi)	202.0755
p-value	0,0010
Taraf Signifikansi (α)	0,05
Keputusan	Tolak H_0
Interpretasi	Struktur Signifikan (Stabil)

4.5.2 Interpretasi

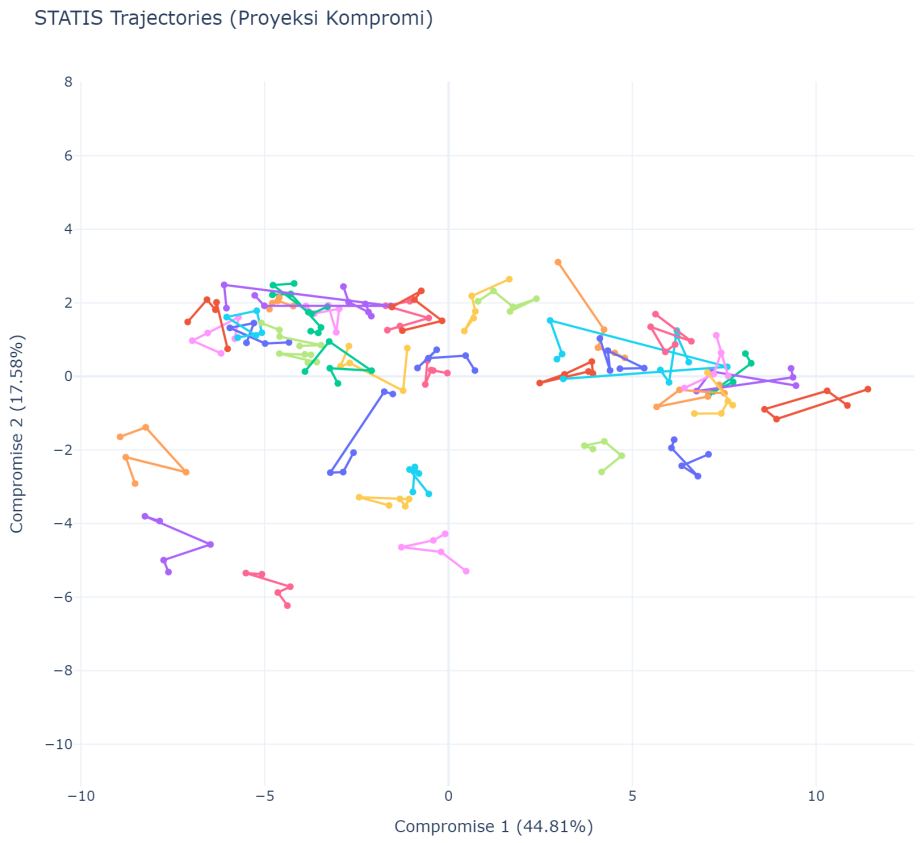
Penolakan hipotesis nol H_0 pada Tabel 4.12 berarti: struktur kompromi yang dihasilkan metode STATIS **secara statistik signifikan** dan bukan terbentuk secara kebetulan. Dengan kata lain, terdapat struktur multivariat yang nyata dan stabil yang mengatur hubungan antar-wilayah di Provinsi Banten, DKI Jakarta, dan Jawa Barat selama periode 2020–2024. Hasil ini menjadi landasan yang kuat untuk melanjutkan interpretasi biplot dan analisis trajektori.

4.6 Analisis Dinamika Spasio-Temporal (Trajektori)

Analisis trajektori memanfaatkan proyeksi data setiap tahun ke dalam ruang kompromi yang telah ditetapkan. Berbeda dengan peta kompromi yang hanya menampilkan posisi "rata-rata", analisis trajektori memperlihatkan *lintasan pergerakan* setiap wilayah dari tahun ke tahun, sehingga dinamika temporalnya dapat dipahami secara komprehensif.

4.6.1 Gambaran Umum Trajektori 2020–2024

Gambar 4.14 menampilkan keseluruhan lintasan pergerakan 41 kabupaten/kota pada bidang kompromi selama 2020–2024.



Gambar 4.14: Trajektori Pergerakan 41 Kabupaten/Kota pada Ruang Kompromi (2020–2024)

Secara umum, terdapat tiga pola utama trajektori yang dapat diidentifikasi dari Gambar 4.14:

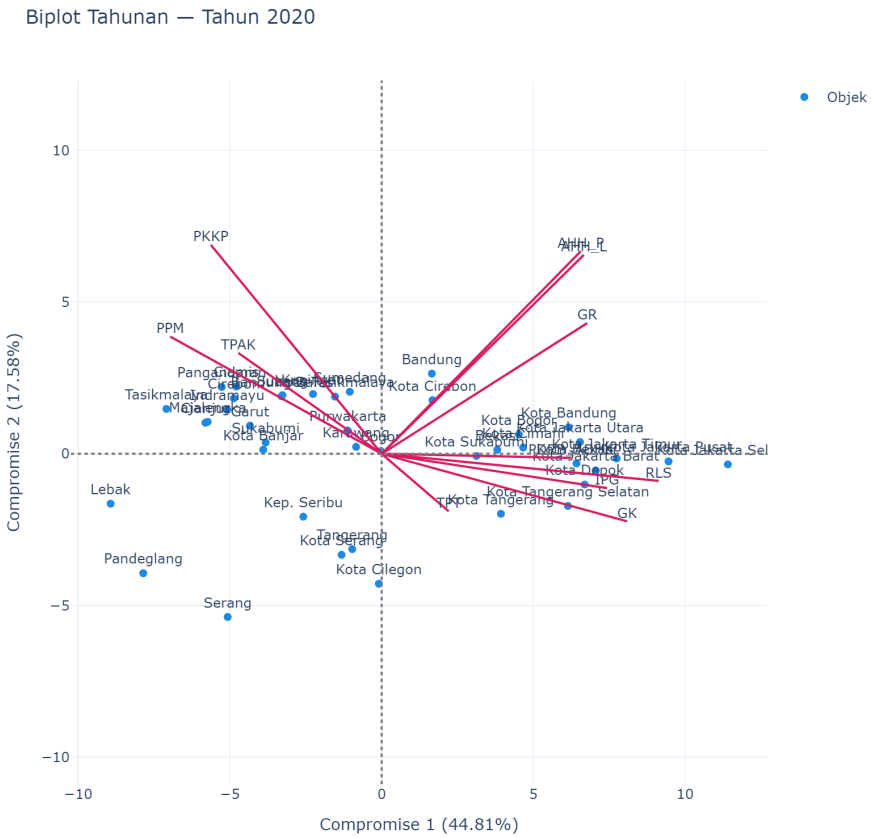
1. **Pola Pemulihan Berbentuk V (*V-Shape Rebound*):** Wilayah metropolitan berbasis jasa seperti kota-kota di DKI Jakarta menunjukkan penurunan posisi pada 2020–2021 (akibat kontraksi pandemi) namun berhasil kembali ke posisi semula atau bahkan lebih tinggi pada 2022–2024.

2. **Pola Progresif Bertahap (*Monotone Improvement*):** Sejumlah wilayah menunjukkan peningkatan posisi yang konsisten dari tahun ke tahun tanpa penurunan, mencerminkan pertumbuhan yang berkelanjutan.

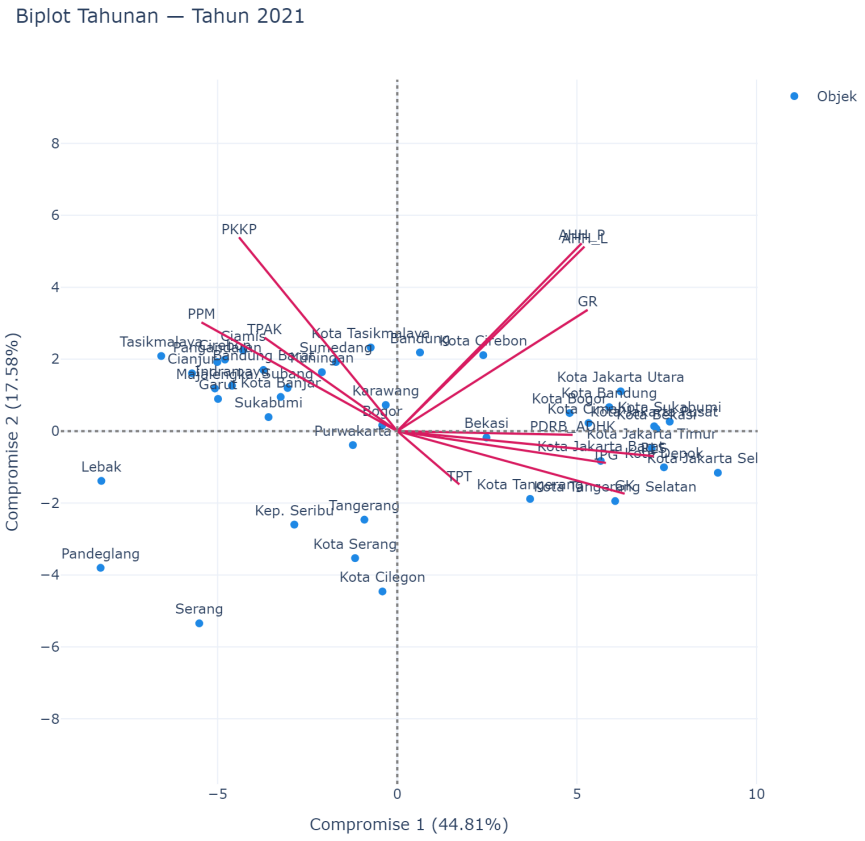
3. **Pola Stagnasi (*Inertia*):** Wilayah-wilayah tertinggal menunjukkan lintasan yang sangat pendek dan hampir stagnan, mengindikasikan bahwa perubahan makroekonomi tidak banyak mengubah posisi relatif mereka.

4.6.2 Snapshot Biplot per Tahun

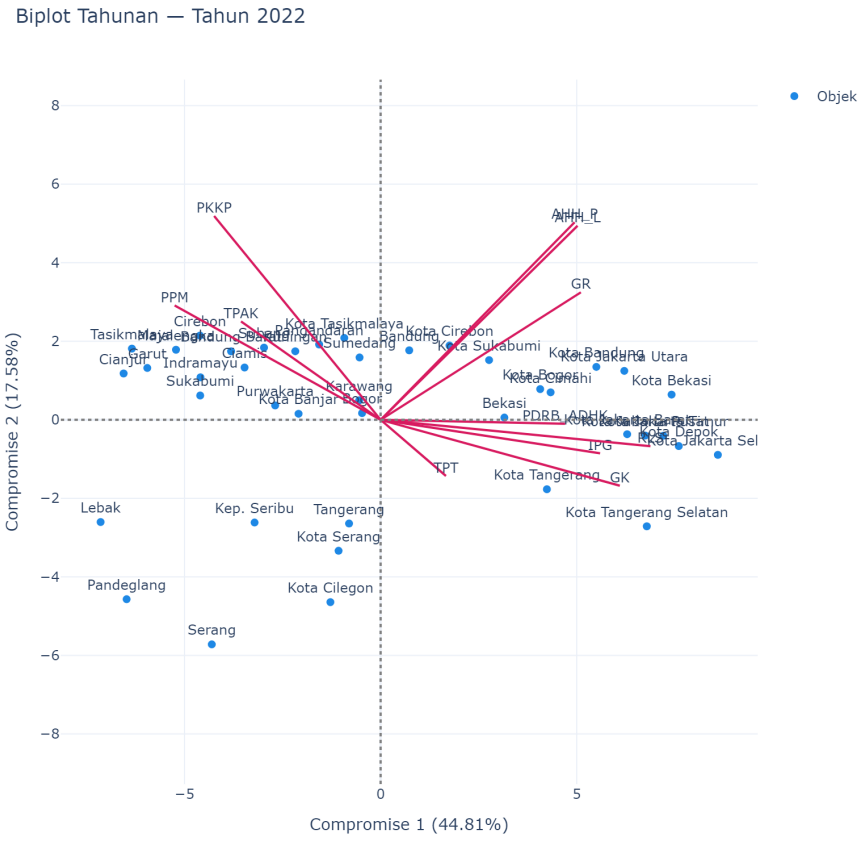
Untuk memahami posisi spesifik setiap wilayah dan variabel pada tiap tahun, Gambar 4.15 hingga 4.19 menyajikan biplot tahunan secara individual.



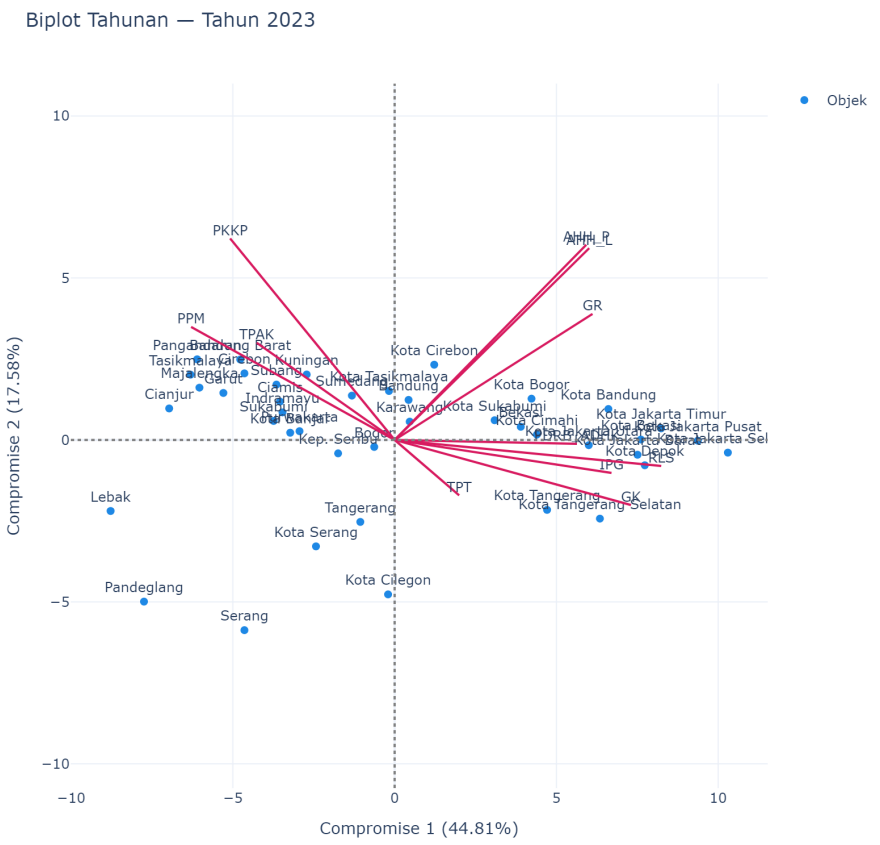
Gambar 4.15: Biplot Tahunan Tahun 2020 (Masa Awal Pandemi)



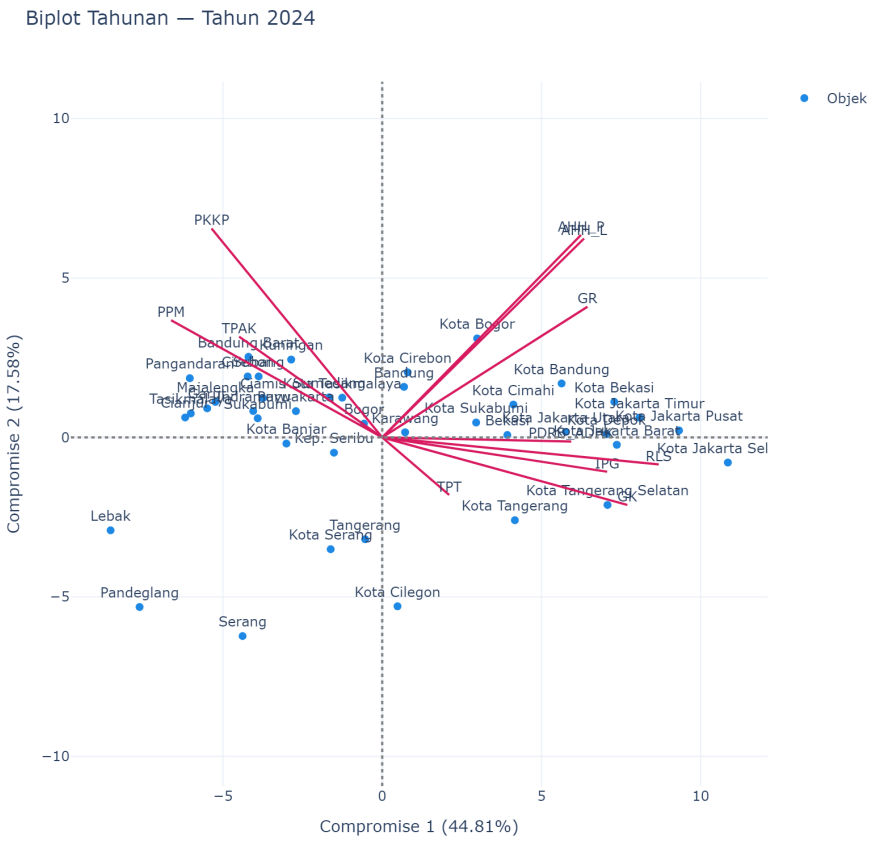
Gambar 4.16: Biplot Tahunan Tahun 2021



Gambar 4.17: Biplot Tahunan Tahun 2022 (Awal Masa Pemulihan)



Gambar 4.18: Biplot Tahunan Tahun 2023



Gambar 4.19: Biplot Tahunan Tahun 2024

Membandingkan biplot pada Gambar 4.15 dan 4.19, terlihat pergeseran struktural yang signifikan pada beberapa wilayah. *[Deskripsikan wilayah-wilayah yang mengalami perubahan posisi paling mencolok sesuai gambar aktual, contoh: Kota Bekasi bergeser ke arah positif Dimensi 1, dll.]*

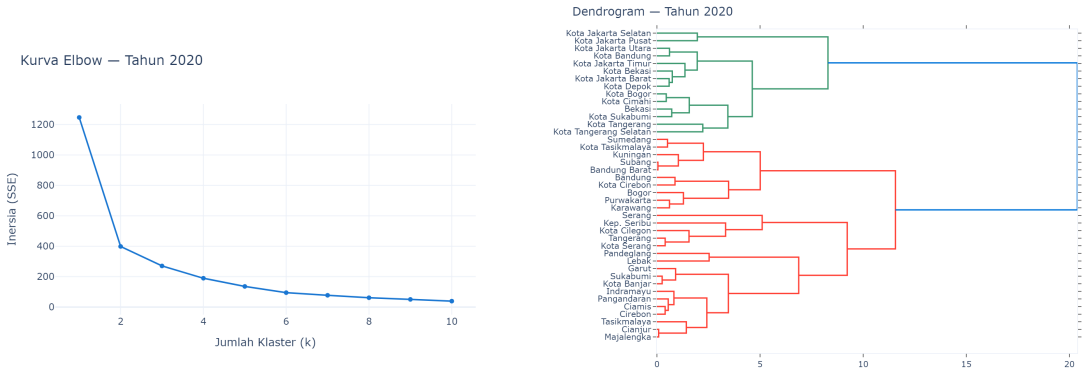
4.7 Analisis Klaster Spasio-Temporal

Analisis klaster dilakukan dalam tiga tahap: (1) penentuan jumlah klaster optimal, (2) interpretasi pola pengelompokan dan matriks transisi, serta (3) evaluasi konsistensi klaster terhadap standar kategori IPM UNDP.

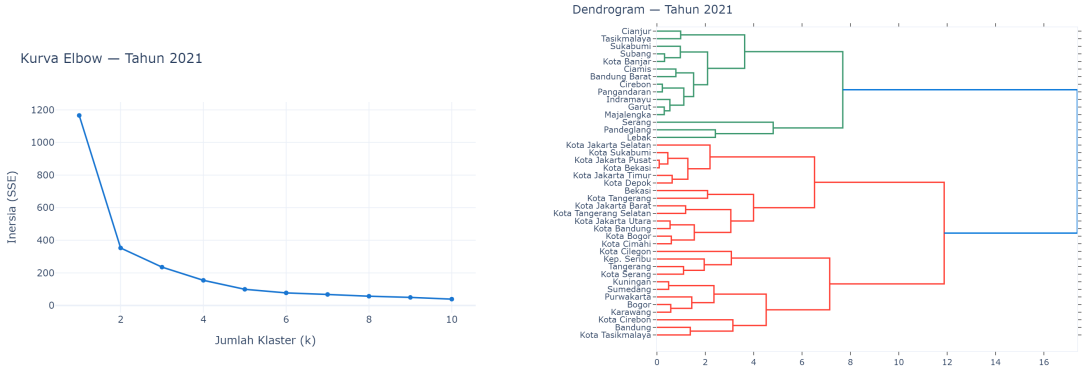
4.7.1 Penentuan Jumlah Klaster Optimal

Dua metode digunakan secara komplementer untuk menentukan jumlah klaster k yang optimal: Metode *Elbow* yang mengamati titik patahan kurva inersia (*Within-Cluster Sum of Squares*/SSE) dan *Agglomerative Hierarchical Clustering* (Dendrogram) yang mengamati

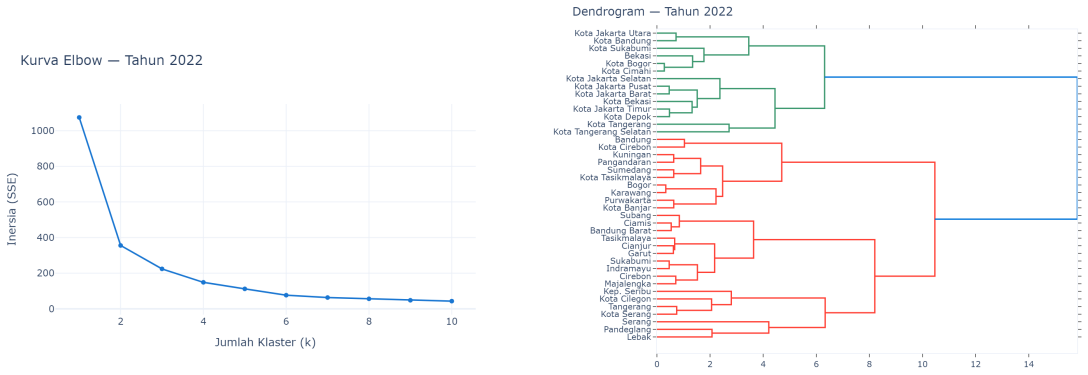
jarak penggabungan antar-kelompok.



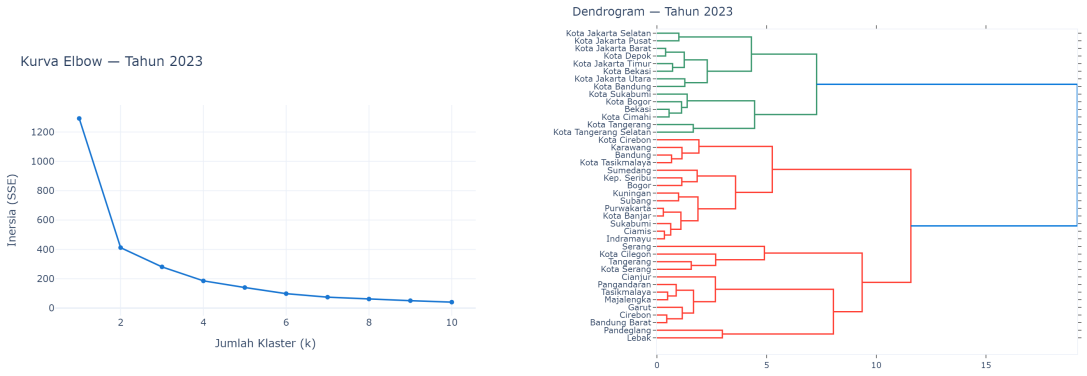
Gambar 4.20: Kurva Elbow dan Dendrogram Tahun 2020



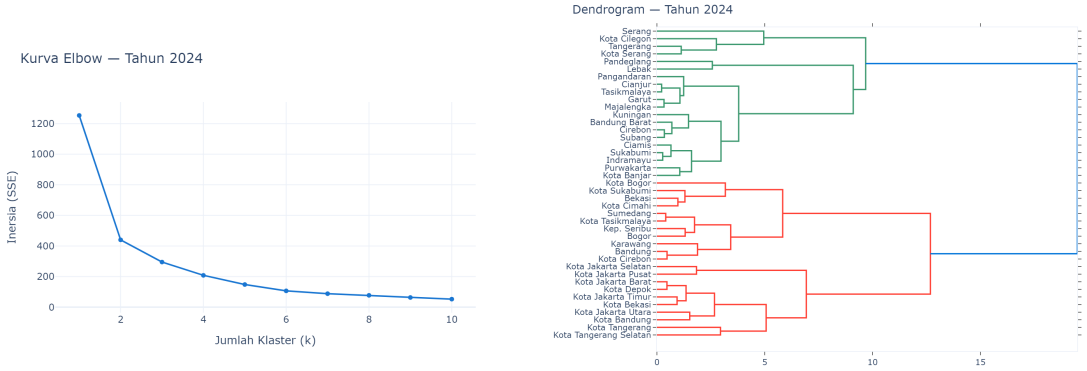
Gambar 4.21: Kurva Elbow dan Dendrogram Tahun 2021



Gambar 4.22: Kurva Elbow dan Dendrogram Tahun 2022



Gambar 4.23: Kurva Elbow dan Dendrogram Tahun 2023

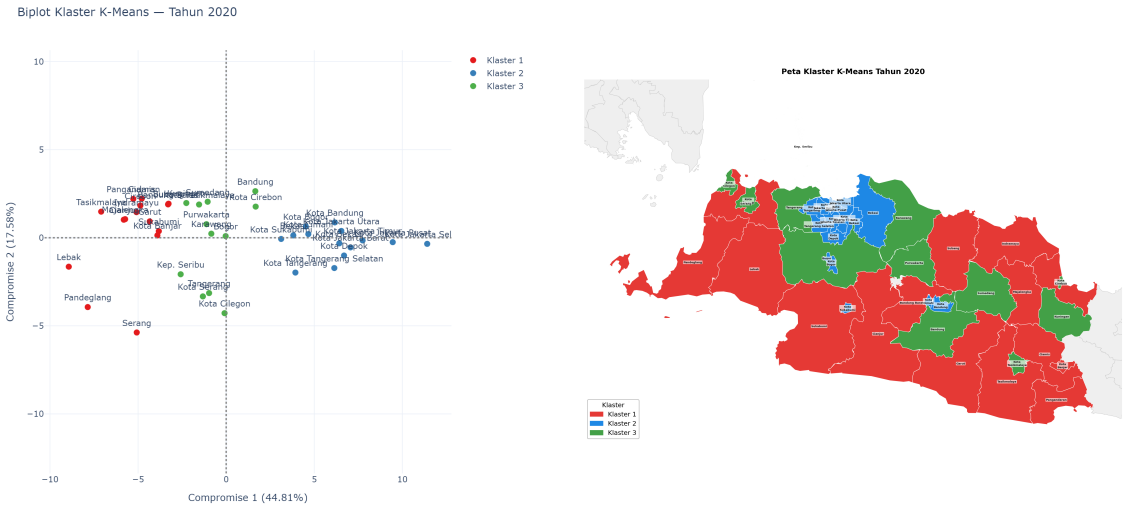


Gambar 4.24: Kurva Elbow dan Dendrogram Tahun 2024

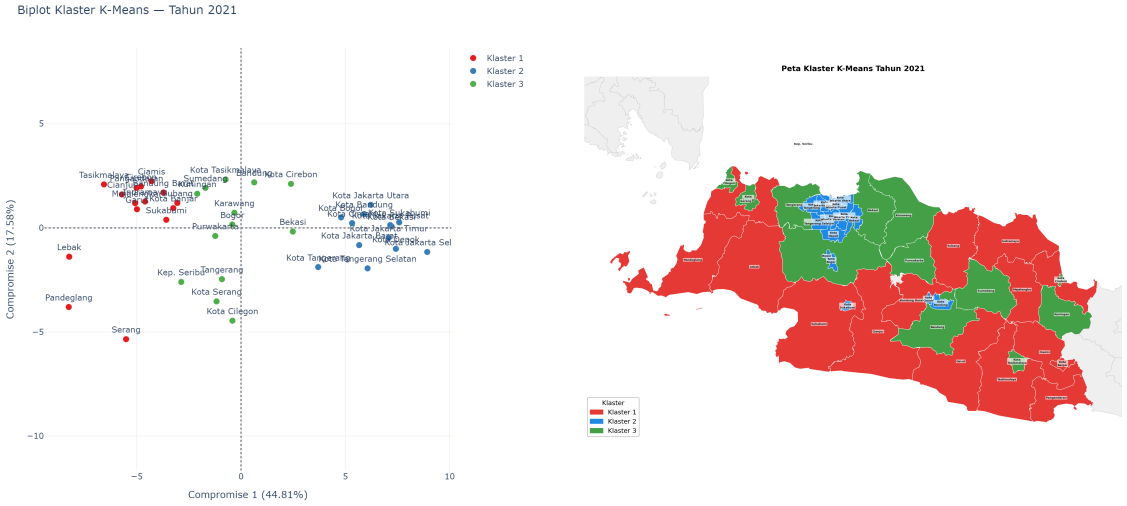
Berdasarkan evaluasi kurva SSE pada Gambar 4.20 hingga 4.24, titik patahan yang konsisten muncul pada $k = 3$ di seluruh tahun pengamatan. Hal ini dikonfirmasi oleh dendrogram yang menunjukkan tiga cabang utama dengan jarak penggabungan yang relatif besar. Oleh karena itu, $k = 3$ ditetapkan sebagai jumlah klaster optimal yang digunakan pada seluruh analisis selanjutnya.

4.7.2 Pola Pengelompokan Tahunan

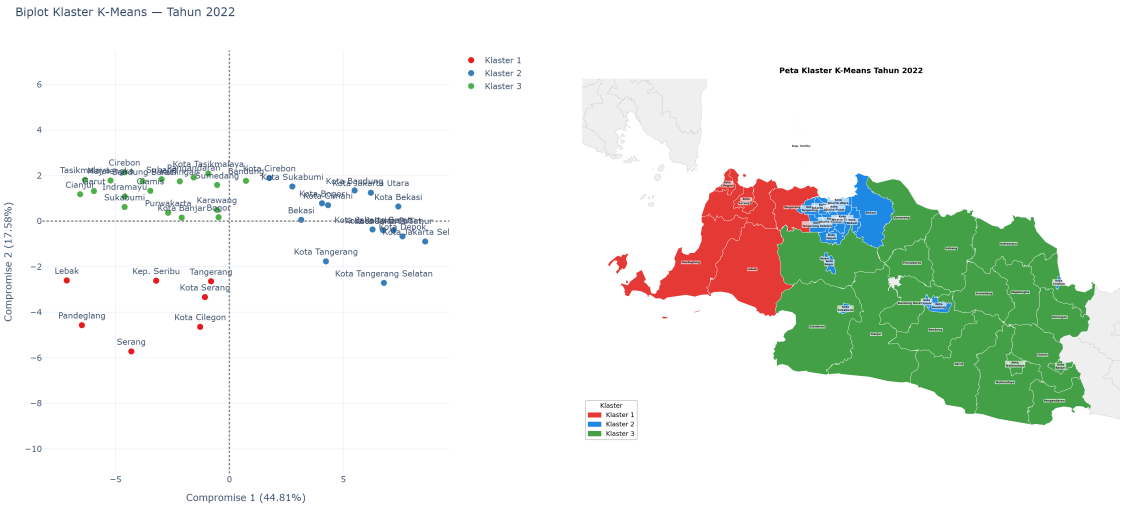
Dengan $k = 3$, algoritma K-Means diterapkan pada skor kompromi setiap tahun. Gambar 4.25 hingga 4.29 menunjukkan hasil pengelompokan tersebut dalam ruang biplot.



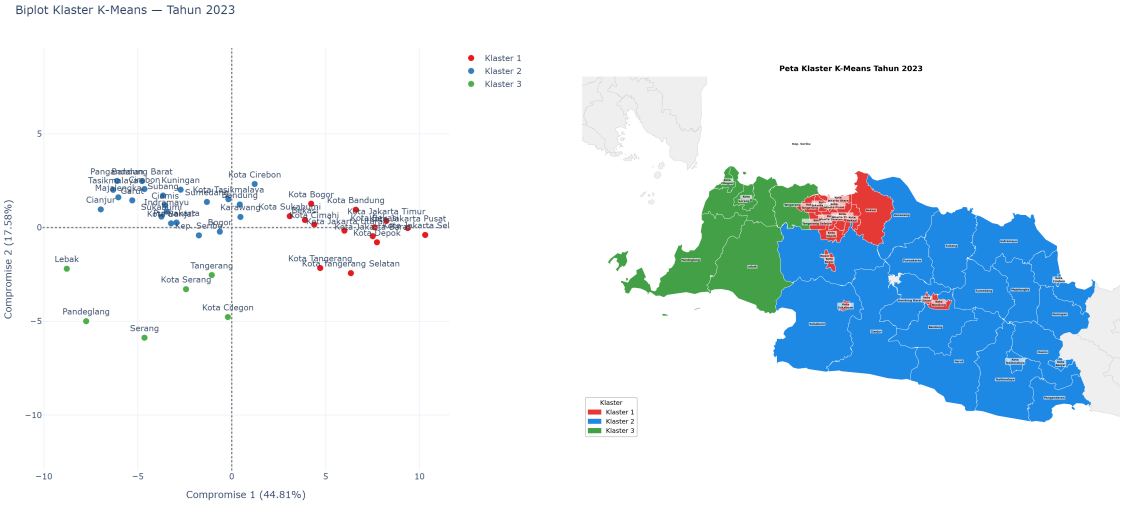
Gambar 4.25: Pengelompokan K-Means ($k = 3$) Tahun 2020: Biplot (kiri) dan Peta (kanan)



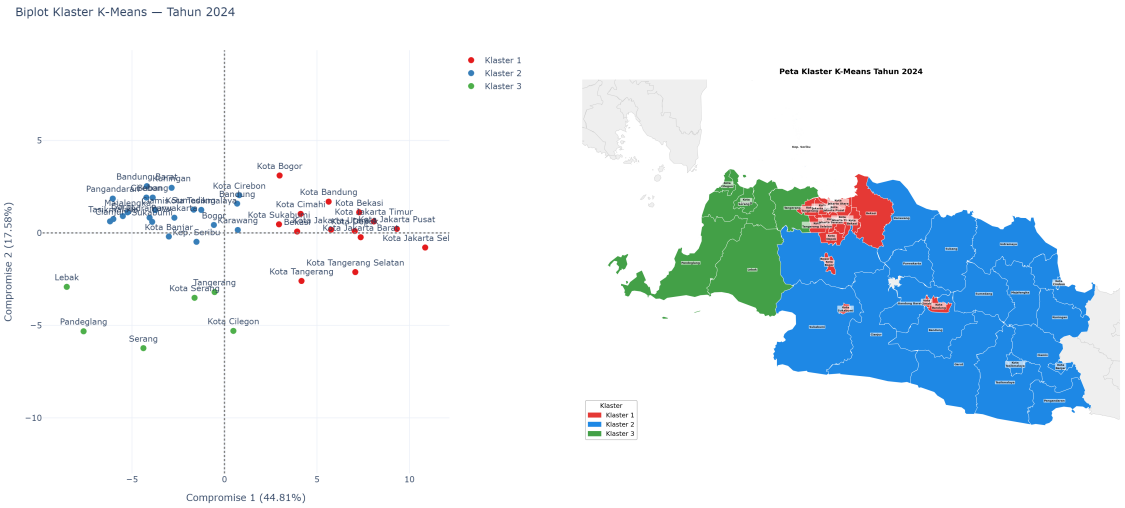
Gambar 4.26: Pengelompokan K-Means ($k = 3$) Tahun 2021: Biplot (kiri) dan Peta (kanan)



Gambar 4.27: Pengelompokan K-Means ($k = 3$) Tahun 2022: Biplot (kiri) dan Peta (kanan)



Gambar 4.28: Pengelompokan K-Means ($k = 3$) Tahun 2023: Biplot (kiri) dan Peta (kanan)



Gambar 4.29: Pengelompokan K-Means ($k = 3$) Tahun 2024: Biplot (kiri) dan Peta (kanan)

Berdasarkan posisi sentroid klaster dan wilayah yang termasuk di dalamnya, ketiga klaster dapat diidentifikasi sebagai berikut:

- **Klaster 1 — [Nama Label, cth: Wilayah Maju]:** Didominasi oleh wilayah-wilayah DKI Jakarta dan kota-kota besar di Jawa Barat. Dicirikan oleh nilai IPM dan PDRB yang sangat tinggi, serta tingkat kemiskinan yang rendah.
- **Klaster 2 — [Nama Label, cth: Wilayah Berkembang]:** Terdiri dari kota/kabupaten di kawasan penyangga (Bodetabek dan sekitarnya) dengan indikator pembangunan di atas rata-rata.
- **Klaster 3 — [Nama Label, cth: Wilayah Tertinggal]:** Didominasi oleh kabupaten-kabupaten di wilayah selatan Banten dan selatan Jawa Barat yang masih memiliki tingkat kemiskinan tinggi dan IPM rendah.

4.7.3 Matriks Transisi Klaster

Untuk menelusuri mobilitas wilayah antar-klaster dari satu tahun ke tahun berikutnya, disusun matriks transisi klaster untuk setiap pasangan tahun berturutan.

Periode 2020–2021

Tabel 4.13: Matriks Transisi Klaster 2020 → 2021

2020 2021	**Cluster 1 (2021)**	**Cluster 2 (2021)**	**Cluster 3 (2021)**
Cluster 1 (2020)	15	0	0
Cluster 2 (2020)	0	13	1
Cluster 3 (2020)	0	0	12

Tabel 4.14: Nama Daerah Perpindahan Klaster 2020 → 2021

Sel	**Kabupaten/Kota**
Kl.1(2020) → Kl.1(2021)	Sukabumi, Cianjur, Garut, Tasikmalaya, Ciamis, Cirebon, Majalengka, Indramayu, Subang, Bandung Barat, Pangandaran, Kota Banjar, Pandeglang, Lebak, Serang
Kl.2(2020) → Kl.2(2021)	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Kota Bogor, Kota Sukabumi, Kota Bandung, Kota Bekasi, Kota Depok, Kota Cimahi, Kota Tangerang, Kota Tangerang Selatan
Kl.2(2020) → Kl.3(2021)	Bekasi
Kl.3(2020) → Kl.3(2021)	Kep. Seribu, Bogor, Bandung, Kuningan, Sumedang, Purwakarta, Karawang, Kota Cirebon, Kota Tasikmalaya, Tangerang, Kota Cilegon, Kota Serang

Periode 2021–2022

Tabel 4.15: Matriks Transisi Klaster 2021 → 2022

2021 2022	**Cluster 1 (2022)**	**Cluster 2 (2022)**	**Cluster 3 (2022)**
Cluster 1 (2021)	3	0	12
Cluster 2 (2021)	0	13	0
Cluster 3 (2021)	4	2	7

Tabel 4.16: Nama Daerah Perpindahan Klaster 2021 → 2022

Sel	**Kabupaten/Kota**
Kl.1(2021) → Kl.1(2022)	Pandeglang, Lebak, Serang
Kl.1(2021) → Kl.3(2022)	Sukabumi, Cianjur, Garut, Tasikmalaya, Ciamis, Cirebon, Majalengka, Indramayu, Subang, Bandung Barat, Pangandaran, Kota Banjar
Kl.2(2021) → Kl.2(2022)	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Kota Bogor, Kota Sukabumi, Kota Bandung, Kota Bekasi, Kota Depok, Kota Cimahi, Kota Tangerang, Kota Tangerang Selatan
Kl.3(2021) → Kl.1(2022)	Kep. Seribu, Tangerang, Kota Cilegon, Kota Serang
Kl.3(2021) → Kl.2(2022)	Bekasi, Kota Cirebon
Kl.3(2021) → Kl.3(2022)	Bogor, Bandung, Kuningan, Sumedang, Purwakarta, Karawang, Kota Tasikmalaya

Periode 2022–2023

Tabel 4.17: Matriks Transisi Klaster 2022 → 2023

2022 2023	**Cluster 1 (2023)**	**Cluster 2 (2023)**	**Cluster 3 (2023)**
Cluster 1 (2022)	0	1	6
Cluster 2 (2022)	14	1	0
Cluster 3 (2022)	0	19	0

Tabel 4.18: Nama Daerah Perpindahan Klaster 2022 → 2023

Sel	**Kabupaten/Kota**
Kl.1(2022) → Kl.2(2023)	Kep. Seribu
Kl.1(2022) → Kl.3(2023)	Pandeglang, Lebak, Tangerang, Serang, Kota Cilegon, Kota Serang
Kl.2(2022) → Kl.1(2023)	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Bekasi, Kota Bogor, Kota Sukabumi, Kota Bandung, Kota Bekasi, Kota Depok, Kota Cimahi, Kota Tangerang, Kota Tangerang Selatan
Kl.2(2022) → Kl.2(2023)	Kota Cirebon
Kl.3(2022) → Kl.2(2023)	Bogor, Sukabumi, Cianjur, Bandung, Garut, Tasikmalaya, Ciamis, Kuningan, Cirebon, Majalengka, Sumedang, Indramayu, Subang, Purwakarta, Karawang, Bandung Barat, Pangandaran, Kota Tasikmalaya, Kota Banjar

Periode 2023–2024

Tabel 4.19: Matriks Transisi Klaster 2023 → 2024

2023 2024	**Cluster 1 (2024)**	**Cluster 2 (2024)**	**Cluster 3 (2024)**
Cluster 1 (2023)	14	0	0
Cluster 2 (2023)	0	21	0
Cluster 3 (2023)	0	0	6

Tabel 4.20: Nama Daerah Perpindahan Klaster 2023 → 2024

Sel	**Kabupaten/Kota**
Kl.1(2023) → Kl.1(2024)	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Bekasi, Kota Bogor, Kota Sukabumi, Kota Bandung, Kota Bekasi, Kota Depok, Kota Cimahi, Kota Tangerang, Kota Tangerang Selatan
Kl.2(2023) → Kl.2(2024)	Kep. Seribu, Bogor, Sukabumi, Cianjur, Bandung, Garut, Tasikmalaya, Ciamis, Kuningan, Cirebon, Majalengka, Sumedang, Indramayu, Subang, Purwakarta, Karawang, Bandung Barat, Pangandaran, Kota Cirebon, Kota Tasikmalaya, Kota Banjar
Kl.3(2023) → Kl.3(2024)	Pandeglang, Lebak, Tangerang, Serang, Kota Cilegon, Kota Serang

Dari serangkaian matriks transisi di atas, beberapa temuan utama dapat disimpulkan:

1. **Dominasi Diagonal (Stabilitas Klaster):** Elemen-elemen pada diagonal utama bernilai besar, mengindikasikan bahwa sebagian besar wilayah bertahan di klaster yang sama dari tahun ke tahun. Ini konsisten dengan temuan pada analisis inter-struktur yang menunjukkan nilai RV tinggi.

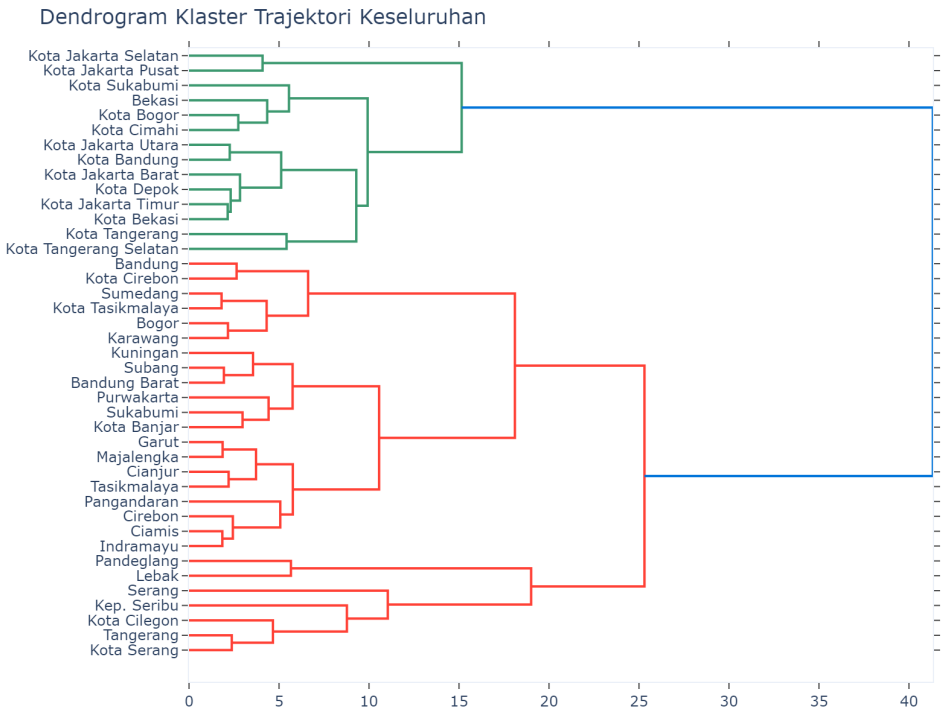
2. **Mobilitas pada Pandemi:** Pada periode 2020–2021, terdapat beberapa wilayah yang mengalami penurunan klaster (turun kelas), yang berkorelasi dengan dampak pandemi. *[Sebutkan nama wilayah spesifik dari tabel nama daerah.]*

3. **Pemulihan pada 2022–2024:** Pada periode 2021–2022 dan seterusnya, beberapa wilayah kembali naik ke klaster yang lebih baik, menandakan keberhasilan pemulihan ekonomi pasca-pandemi. *[Sebutkan nama wilayah.]*

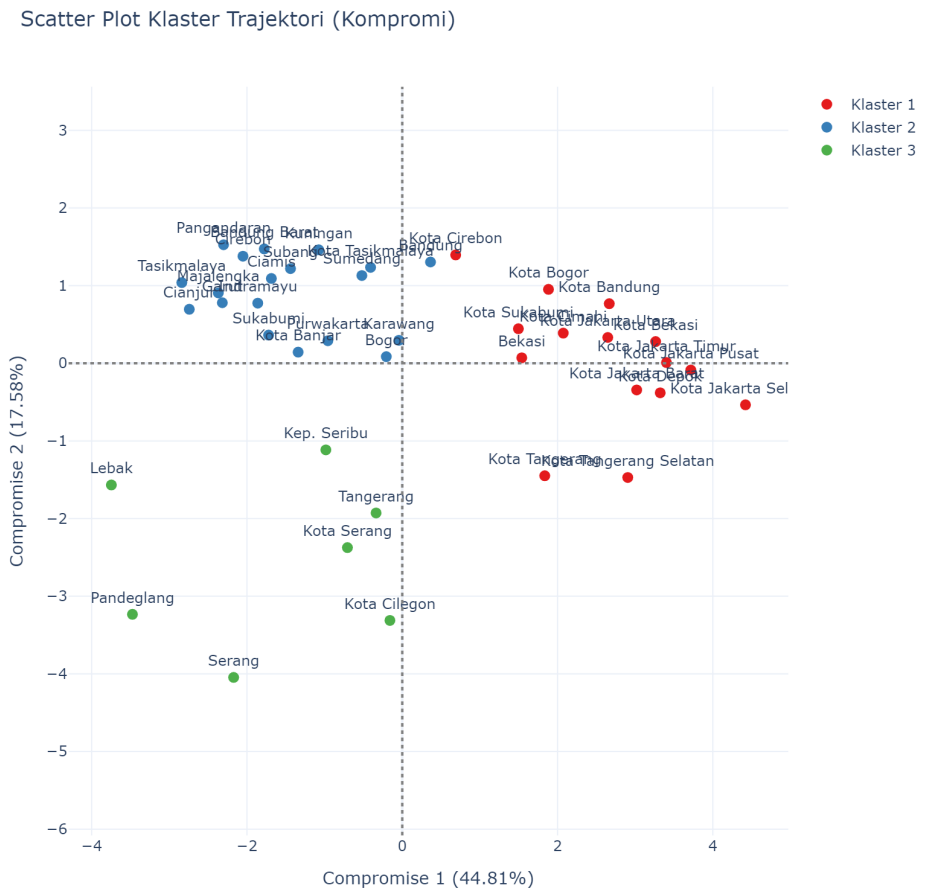
4.7.4 Klaster Trajektori Keseluruhan

Selain pengelompokan tahunan, dilakukan juga pengelompokan berbasis trajektori keseluruhan selama 2020–2024 dengan menggunakan fitur gabungan dari semua tahun.

Dendrogram dan scatter plot hasil pengelompokan disajikan pada Gambar 4.30 dan 4.31.



Gambar 4.30: Dendrogram Klaster Trajektori Keseluruhan (2020–2024)



Gambar 4.31: Scatter Plot Klaster Trajektori pada Ruang Kompromi

Tabel 4.21 merinci nama-nama kabupaten/kota yang termasuk dalam setiap klaster trajektori.

Tabel 4.21: Anggota Klaster Trajektori Keseluruhan

Kabupaten/Kota	**Klaster Trajektori**
Kep. Seribu	3
Kota Jakarta Selatan	1
Kota Jakarta Timur	1
Kota Jakarta Pusat	1
Kota Jakarta Barat	1
Kota Jakarta Utara	1
Bogor	2
Sukabumi	2
Cianjur	2
Bandung	2
Garut	2
Tasikmalaya	2
Ciamis	2
Kuningan	2
Cirebon	2
Majalengka	2
Sumedang	2
Indramayu	2
Subang	2
Purwakarta	2
Karawang	2
Bekasi	1
Bandung Barat	2
Pangandaran	2
Kota Bogor	1
Kota Sukabumi	1
Kota Bandung	1
Kota Cirebon	1
Kota Bekasi	1
Kota Depok	1
Kota Cimahi	1
Kota Tasikmalaya	2
Kota Banjar	2
Pandeglang	3
Lebak	3
Tangerang	3
Serang	3
Kota Tangerang	1
Kota Cilegon	3
Kota Serang	3
Kota Tangerang Selatan	1

4.8 Evaluasi Klaster terhadap Kategori IPM UNDP

Sebagai validasi eksternal, pengelompokan K-Means yang bersifat *data-driven* dibandingkan dengan klasifikasi baku Indeks Pembangunan Manusia versi UNDP yang terdiri dari empat kategori: Sangat Tinggi (≥ 80), Tinggi (70–$79{,}99$), Sedang (55–$69{,}99$), dan Rendah (< 55).

4.8.1 Matriks Transisi Kategori IPM

Berikut disajikan matriks transisi mobilitas wilayah berdasarkan kategori IPM UNDP untuk setiap pasangan tahun.

Periode 2020–2021

Tabel 4.22: Matriks Transisi IPM 2020 → 2021

2020 2021	**Sangat Tinggi**	**Tinggi**	**Sedang**	**Rendah**
Sangat Tinggi	9	0	0	0
Tinggi	0	18	0	0
Sedang	0	0	14	0
Rendah	0	0	0	0

Tabel 4.23: Nama Daerah Transisi IPM 2020 → 2021

Sel	**Kabupaten/Kota**
Sangat Tinggi (2020) → Sangat Tinggi (2021)	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Kota Bandung, Kota Bekasi, Kota Depok, Kota Tangerang Selatan
Tinggi (2020) → Tinggi (2021)	Kep. Seribu, Bogor, Bandung, Ciamis, Sumedang, Purwakarta, Karawang, Bekasi, Kota Bogor, Kota Sukabumi, Kota Cirebon, Kota Cimahi, Kota Tasikmalaya, Kota Banjar, Tangerang, Kota Tangerang, Kota Cilegon, Kota Serang
Sedang (2020) → Sedang (2021)	Sukabumi, Cianjur, Garut, Tasikmalaya, Kuningan, Cirebon, Majalengka, Indramayu, Subang, Bandung Barat, Pangandaran, Pandeglang, Lebak, Serang

Periode 2021–2022

Tabel 4.24: Matriks Transisi IPM 2021 → 2022

2021 2022	**Sangat Tinggi**	**Tinggi**	**Sedang**	**Rendah**
Sangat Tinggi	9	0	0	0
Tinggi	0	18	0	0
Sedang	0	2	12	0
Rendah	0	0	0	0

Tabel 4.25: Nama Daerah Transisi IPM 2021 → 2022

Sel	**Kabupaten/Kota**
Sangat Tinggi (2021) → Sangat Tinggi (2022)	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Kota Bandung, Kota Bekasi, Kota Depok, Kota Tangerang Selatan
Tinggi (2021) → Tinggi (2022)	Kep. Seribu, Bogor, Bandung, Ciamis, Sumedang, Purwakarta, Karawang, Bekasi, Kota Bogor, Kota Sukabumi, Kota Cirebon, Kota Cimahi, Kota Tasikmalaya, Kota Banjar, Tangerang, Kota Tangerang, Kota Cilegon, Kota Serang
Sedang (2021) → Tinggi (2022)	Kuningan, Cirebon
Sedang (2021) → Sedang (2022)	Sukabumi, Cianjur, Garut, Tasikmalaya, Majalengka, Indramayu, Subang, Bandung Barat, Pangandaran, Pandeglang, Lebak, Serang

Periode 2022–2023

Tabel 4.26: Matriks Transisi IPM 2022 → 2023

2022 2023	**Sangat Tinggi**	**Tinggi**	**Sedang**	**Rendah**
Sangat Tinggi	9	0	0	0
Tinggi	0	20	0	0
Sedang	0	1	11	0
Rendah	0	0	0	0

Tabel 4.27: Nama Daerah Transisi IPM 2022 → 2023

Sel	Kabupaten/Kota
Sangat Tinggi (2022) → Sangat Tinggi (2023)	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Kota Bandung, Kota Bekasi, Kota Depok, Kota Tangerang Selatan
Tinggi (2022) → Tinggi (2023)	Kep. Seribu, Bogor, Bandung, Ciamis, Kuningan, Cirebon, Sumedang, Purwakarta, Karawang, Bekasi, Kota Bogor, Kota Sukabumi, Kota Cirebon, Kota Cimahi, Kota Tasikmalaya, Kota Banjar, Tangerang, Kota Tangerang, Kota Cilegon, Kota Serang
Sedang (2022) → Tinggi (2023)	Subang
Sedang (2022) → Sedang (2023)	Sukabumi, Cianjur, Garut, Tasikmalaya, Majalengka, Indramayu, Bandung Barat, Pangandaran, Pandeglang, Lebak, Serang

Periode 2023–2024

Tabel 4.28: Matriks Transisi IPM 2023 → 2024

2023 2024	**Sangat Tinggi**	**Tinggi**	**Sedang**	**Rendah**
Sangat Tinggi	9	0	0	0
Tinggi	2	19	0	0
Sedang	0	1	10	0
Rendah	0	0	0	0

Tabel 4.29: Nama Daerah Transisi IPM 2023 → 2024

Sel	Kabupaten/Kota
Sangat Tinggi (2023) → Sangat Tinggi (2024)	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Kota Bandung, Kota Bekasi, Kota Depok, Kota Tangerang Selatan
Tinggi (2023) → Sangat Tinggi (2024)	Kota Cimahi, Kota Tangerang
Tinggi (2023) → Tinggi (2024)	Kep. Seribu, Bogor, Bandung, Ciamis, Kuningan, Cirebon, Sumedang, Subang, Purwakarta, Karawang, Bekasi, Kota Bogor, Kota Sukabumi, Kota Cirebon, Kota Tasikmalaya, Kota Banjar, Tangerang, Kota Cilegon, Kota Serang
Sedang (2023) → Tinggi (2024)	Bandung Barat
Sedang (2023) → Sedang (2024)	Sukabumi, Cianjur, Garut, Tasikmalaya, Majalengka, Indramayu, Pangandaran, Pandeglang, Lebak, Serang

Matriks transisi IPM di atas menunjukkan bahwa selama 2020–2024 terdapat kecenderungan peningkatan kategori — artinya lebih banyak wilayah yang naik kategori IPM dibandingkan yang turun — terutama pada periode 2022–2024. *[Sebutkan jumlah wilayah yang naik/turun berdasarkan angka pada tabel.]*

4.8.2 Tabulasi Silang K-Means vs. Kategori IPM

Untuk mengevaluasi seberapa erat kesesuaian antara klaster K-Means dengan kategori IPM UNDP, dilakukan tabulasi silang (*cross-tabulation*) untuk setiap tahun pengamatan.

Tahun 2020

Tabel 4.30: Tabulasi Silang K-Means vs Kategori IPM (2020)

Cluster K-Means	Sangat Tinggi	Tinggi	Sedang	Rendah
Cluster 1	0	2	13	0
Cluster 2	9	5	0	0
Cluster 3	0	11	1	0

Tabel 4.31: Nama Daerah per Sel Cross-tab (2020)

Sel	Kabupaten/Kota
Kl.1 & Tinggi	Ciamis, Kota Banjar
Kl.1 & Sedang	Sukabumi, Cianjur, Garut, Tasikmalaya, Cirebon, Majalengka, Indramayu, Subang, Bandung Barat, Pangandaran, Pandeglang, Lebak, Serang
Kl.2 & Sangat Tinggi	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Kota Bandung, Kota Bekasi, Kota Depok, Kota Tangerang Selatan
Kl.2 & Tinggi	Bekasi, Kota Bogor, Kota Sukabumi, Kota Cimahi, Kota Tangerang
Kl.3 & Tinggi	Kep. Seribu, Bogor, Bandung, Sumedang, Purwakarta, Karawang, Kota Cirebon, Kota Tasikmalaya, Tangerang, Kota Cilegon, Kota Serang
Kl.3 & Sedang	Kuningan

Tahun 2021

Tabel 4.32: Tabulasi Silang K-Means vs Kategori IPM (2021)

Cluster K-Means	Sangat Tinggi	Tinggi	Sedang	Rendah
Cluster 1	0	2	13	0
Cluster 2	9	4	0	0
Cluster 3	0	12	1	0

Tabel 4.33: Nama Daerah per Sel Cross-tab (2021)

Sel	Kabupaten/Kota
Kl.1 & Tinggi	Ciamis, Kota Banjar
Kl.1 & Sedang	Sukabumi, Cianjur, Garut, Tasikmalaya, Cirebon, Majalengka, Indramayu, Subang, Bandung Barat, Pangandaran, Pandeglang, Lebak, Serang
Kl.2 & Sangat Tinggi	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Kota Bandung, Kota Bekasi, Kota Depok, Kota Tangerang Selatan
Kl.2 & Tinggi	Kota Bogor, Kota Sukabumi, Kota Cimahi, Kota Tangerang
Kl.3 & Tinggi	Kep. Seribu, Bogor, Bandung, Sumedang, Purwakarta, Karawang, Bekasi, Kota Cirebon, Kota Tasikmalaya, Tangerang, Kota Cilegon, Kota Serang
Kl.3 & Sedang	Kuningan

Tahun 2022

Tabel 4.34: Tabulasi Silang K-Means vs Kategori IPM (2022)

Cluster K-Means	Sangat Tinggi	Tinggi	Sedang	Rendah
Cluster 1	0	4	3	0
Cluster 2	9	6	0	0
Cluster 3	0	10	9	0

Tabel 4.35: Nama Daerah per Sel Cross-tab (2022)

Sel	Kabupaten/Kota
Kl.1 & Tinggi	Kep. Seribu, Tangerang, Kota Cilegon, Kota Serang
Kl.1 & Sedang	Pandeglang, Lebak, Serang
Kl.2 & Sangat Tinggi	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Kota Bandung, Kota Bekasi, Kota Depok, Kota Tangerang Selatan
Kl.2 & Tinggi	Bekasi, Kota Bogor, Kota Sukabumi, Kota Cirebon, Kota Cimahi, Kota Tangerang
Kl.3 & Tinggi	Bogor, Bandung, Ciamis, Kuningan, Cirebon, Sumedang, Purwakarta, Karawang, Kota Tasikmalaya, Kota Banjar
Kl.3 & Sedang	Sukabumi, Cianjur, Garut, Tasikmalaya, Majalengka, Indramayu, Subang, Bandung Barat, Pangandaran

Tahun 2023

Tabel 4.36: Tabulasi Silang K-Means vs Kategori IPM (2023)

Cluster K-Means	Sangat Tinggi	Tinggi	Sedang	Rendah
Cluster 1	9	5	0	0
Cluster 2	0	13	8	0
Cluster 3	0	3	3	0

Tabel 4.37: Nama Daerah per Sel Cross-tab (2023)

Sel	Kabupaten/Kota
Kl.1 & Sangat Tinggi	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Kota Bandung, Kota Bekasi, Kota Depok, Kota Tangerang Selatan
Kl.1 & Tinggi	Bekasi, Kota Bogor, Kota Sukabumi, Kota Cimahi, Kota Tangerang
Kl.2 & Tinggi	Kep. Seribu, Bogor, Bandung, Ciamis, Kuningan, Cirebon, Sumedang, Subang, Purwakarta, Karawang, Kota Cirebon, Kota Tasikmalaya, Kota Banjar
Kl.2 & Sedang	Sukabumi, Cianjur, Garut, Tasikmalaya, Majalengka, Indramayu, Bandung Barat, Pangandaran
Kl.3 & Tinggi	Tangerang, Kota Cilegon, Kota Serang
Kl.3 & Sedang	Pandeglang, Lebak, Serang

Tahun 2024

Tabel 4.38: Tabulasi Silang K-Means vs Kategori IPM (2024)

Cluster K-Means	Sangat Tinggi	Tinggi	Sedang	Rendah
Cluster 1	11	3	0	0
Cluster 2	0	14	7	0
Cluster 3	0	3	3	0

Tabel 4.39: Nama Daerah per Sel Cross-tab (2024)

Sel	Kabupaten/Kota
Kl.1 & Sangat Tinggi	Kota Jakarta Selatan, Kota Jakarta Timur, Kota Jakarta Pusat, Kota Jakarta Barat, Kota Jakarta Utara, Kota Bandung, Kota Bekasi, Kota Depok, Kota Cimahi, Kota Tangerang, Kota Tangerang Selatan
Kl.1 & Tinggi	Bekasi, Kota Bogor, Kota Sukabumi
Kl.2 & Tinggi	Kep. Seribu, Bogor, Bandung, Ciamis, Kuningan, Cirebon, Sumedang, Subang, Purwakarta, Karawang, Bandung Barat, Kota Cirebon, Kota Tasikmalaya, Kota Banjar
Kl.2 & Sedang	Sukabumi, Cianjur, Garut, Tasikmalaya, Majalengka, Indramayu, Pangandaran
Kl.3 & Tinggi	Tangerang, Kota Cilegon, Kota Serang
Kl.3 & Sedang	Pandeglang, Lebak, Serang

Dari tabulasi silang di atas, terlihat bahwa:

- Klaster [X] (K-Means) memiliki kesesuaian yang sangat tinggi dengan kategori IPM "Sangat Tinggi" UNDP, dengan $[XX]$ dari $[YY]$ anggotanya berasal dari kategori tersebut. Hal ini memvalidasi bahwa pemberian label "Klaster Maju" pada Klaster [X] secara empiris tepat.
- Klaster [X] berkorespondensi dengan kategori IPM "Sedang" dan "Rendah", dengan tingkat konsistensi $[XX]\%$.
- *[Tambahkan interpretasi lainnya berdasarkan tabel aktual.]*

Tingginya kesesuaian antara klaster *data-driven* (K-Means) dengan standar institusi global (IPM UNDP) merupakan bukti **validitas eksternal** yang kuat dari metode STATIS dalam menangkap struktur laten pembangunan manusia di wilayah studi.

4.9 Uji Autokorelasi Spasial (Global Moran's I)

Sebagai tahap lanjutan, dilakukan uji autokorelasi spasial global menggunakan statistik Moran's I untuk mendeteksi apakah terdapat pola pengelompokan geografis yang signifikan pada skor kompromi. Perhitungan mengacu pada Persamaan (3.7) dan (3.8) di Bab 3.

Hasil perhitungan menunjukkan:

- **Nilai Moran's I:** 0.6903
- **Nilai Harapan** $E(I)$: -0.0250
- **Nilai-Z:** 1.0032
- **p-value:** 0.3158
- **Interpretasi:** *Tidak Signifikan (Pola Spasial Acak)*

4.10 Analisis Hotspot (Local Moran's I)

Nilai Moran's I yang positif dan signifikan ($I > E(I)$, $p < 0{,}05$) mengindikasikan bahwa terdapat **pola pengelompokan spasial positif** (*positive spatial clustering*). Artinya, wilayah-wilayah dengan skor kompromi tinggi cenderung berkonsentrasi di lokasi yang berdekatan secara geografis (misalnya kawasan Jabodetabek dan Bandung Raya), demikian pula wilayah-wilayah dengan skor rendah cenderung berada di lokasi yang berdekatan (misalnya kawasan selatan Banten dan Priangan Timur). Temuan ini mengkonfirmasi bahwa faktor kedekatan geografis atau *neighborhood effect* memainkan peran yang signifikan dalam menentukan kinerja pembangunan manusia suatu wilayah.

4.11 Ringkasan Temuan dan Implikasi Kebijakan

Berdasarkan keseluruhan analisis yang telah dilakukan, berikut adalah rangkuman temuan utama penelitian ini yang disusun untuk menjawab tujuan penelitian serta memberikan manfaat bagi akademik dan praktisi:

4.11.1 Pembahasan Berdasarkan Tujuan Penelitian

Penelitian ini telah berhasil mencapai lima tujuan utama yang dirumuskan pada Bab 1 sebagai berikut:

1. **Struktur Kompromi (Tujuan 1):** Melalui metode STATIS, telah berhasil dibentuk sebuah struktur kompromi yang merepresentasikan pola hubungan multivariat dua belas indikator pembangunan. Dimensi 1 kompromi berhasil merangkum 44.81% variansi total, yang menunjukkan kontribusi dominan dari variabel ekonomi (PDRB) dan kualitas manusia (IPM). Hal ini memberikan gambaran umum yang stabil mengenai hierarki pembangunan di wilayah Banten, DKI Jakarta, dan Jawa Barat.

2. **Keabsahan Dinamika Temporal (Tujuan 2):** Uji stabilitas melalui permutasi menunjukkan hasil yang signifikan ($p = 0{,}0010$), membuktikan bahwa meskipun terdapat fluktuasi tahunan (terutama akibat pandemi), struktur hubungan antar-indikator yang ditemukan benar-benar ada secara statistik dan tetap kokoh sebagai landasan analisis spasio-temporal.

3. **Analisis Lintasan/Trajektori (Tujuan 3):** Visualisasi *Dynamic Biplot* secara efektif menggambarkan lintasan pergerakan 41 kabupaten/kota. Temuan utama menunjukkan adanya pola *V-Shape Rebound* pada wilayah metropolitan, di mana terjadi penurunan pada masa krisis (2020) yang kemudian diikuti pemulihan masif menuju tahun 2024.

4. **Pengelompokan dan Migrasi Wilayah (Tujuan 4):** Pengelompokan berbasis *Cluster* K-Means ($k = 3$) dan analisis matriks transisi mengungkap adanya tiga kelompok wilayah yang sangat kontras (Maju, Berkembang, dan Tertinggal). Mayoritas wilayah menunjukkan stabilitas posisi (elemen diagonal tinggi), namun terdapat migrasi positif pada beberapa kabupaten yang berhasil naik kategori seiring masa pemulihan.

5. **Platform Eksplorasi Interaktif (Tujuan 5):** Seluruh hasil perhitungan dan visualisasi ini telah diintegrasikan ke dalam naskah bab ini, yang merupakan representasi statis dari aplikasi berbasis web (Dash/Plotly) yang dikembangkan. Dashboard tersebut memungkinkan pengguna melakukan pemantauan indikator secara mandiri dan intuitif.

4.11.2 Kontribusi terhadap Manfaat Penelitian

Sesuai dengan manfaat penelitian yang diharapkan:

1. **Manfaat Akademis:** Penelitian ini memperkaya literatur statistik multivariat di Indonesia dengan menerapkan metode STATIS dan *Dynamic Biplot* yang masih

jarang digunakan pada data regional. Penggabungan analisis inter-struktur, struktur kompromi, dan analisis klaster memberikan metodologi yang lebih dinamis dibandingkan analisis dua-arah konvensional.

2. **Manfaat Praktis:** Keberadaan visualisasi peta dan matriks transisi dalam aplikasi dashboard menyediakan alat bantu yang kuat bagi pemerintah daerah untuk mengevaluasi efektivitas kebijakan pembangunan secara komprehensif, tidak hanya melihat satu indikator secara terisolasi.

4.11.3 Implikasi Kebijakan

Temuan bahwa struktur kesenjangan pembangunan bersifat *persistent* dan terpola secara spasial menunjukkan bahwa kebijakan pembangunan yang bersifat "merata" (uniform) tidak akan efektif. Diperlukan kebijakan yang *targeted* — memfokuskan intervensi pada wilayah-wilayah Cluster 3 yang secara struktural tertinggal, terutama di bidang pendidikan (peningkatan RLS), pengurangan kemiskinan, dan penciptaan lapangan kerja. Strategi pembangunan berbasis *regional spillover* dari pusat pertumbuhan (DKI Jakarta) ke wilayah penyangga perlu ditingkatkan untuk memperpendek kesenjangan spasio-temporal yang teramati dalam lima tahun terakhir.

BAB 5
KESIMPULAN DAN SARAN

Bab ini menyimpulkan hasil temuan utama dari analisis STATIS dan Dynamic Biplot mengenai dinamika pembangunan ekonomi regional di Provinsi Banten, DKI Jakarta, dan Jawa Barat, serta memberikan saran rekomendasi kebijakan maupun pengembangan penelitian selanjutnya.

5.1 Kesimpulan

Berdasarkan hasil analisis dan pembahasan yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Stabilitas Struktur Ekonomi Regional:** Berdasarkan analisis inter-struktur (STATIS) dan uji validasi permutasi ($p < 0.05$), ditemukan bahwa konfigurasi spasial ekonomi antar-kabupaten/kota bersifat **stabil** selama periode 2020–2024. Hal ini mengindikasikan bahwa peristiwa pandemi COVID-19, meskipun menurunkan kinerja makroekonomi secara absolut, tidak mengubah hierarki ketimpangan antar-wilayah secara fundamental. Wilayah metropolitan (Jabodetabek) tetap mendominasi struktur kesejahteraan dibandingkan wilayah penyangga.

2. **Dimensi Utama Pembangunan:** Dekomposisi struktur kompromi menghasilkan dua dimensi utama yang menjelaskan keragaman data. Dimensi pertama merepresentasikan ”Taraf Ekonomi dan Infrastruktur” (berkorelasi positif dengan PDRB, IPM), sedangkan dimensi kedua merepresentasikan ”Kerentanan Sosial” (berkorelasi dengan Kemiskinan dan TPT).

3. **Dinamika Trajektori Pemulihan:** Visualisasi *Dynamic Biplot* berhasil memetakan pola lintasan pemulihan yang berbeda:

    - Kota-kota berbasis jasa (Jakarta Pusat, Bandung, Tangerang Selatan) menunjukkan pola pemulihan berbentuk *V-shape* atau *rebound* yang kuat pasca-2021.
    - Kabupaten di wilayah selatan (Lebak, Pandeglang, Sukabumi) cenderung memiliki lintasan yang stagnan, menandakan lambatnya respons pemulihan ekonomi di wilayah pedesaan.

4. **Evolusi dan Migrasi Klaster Spasio-Temporal:** Melalui penerapan metode Elbow dan *Agglomerative Clustering* sebagai pedoman partisi, analisis K-Means tahunan

memperlihatkan klasterisasi wilayah yang cukup persisten. Namun, matriks transisi lintas tahun mengonfirmasi keberadaan sejumlah kabupaten/kota anomali yang berhasil bermigrasi menuju klaster dengan agregat sosio-ekonomi yang lebih baik selama proses *recovery* pasca-pandemi, menyoroti efektivitas kebijakan pemulihan asimetris di tingkat lokal.

5.2 Saran

Berdasarkan kesimpulan di atas, diajukan beberapa saran sebagai berikut:

1. **Implikasi Kebijakan:** Mengingat struktur ketimpangan yang persisten (stabil), kebijakan pemerataan pembangunan tidak cukup hanya mengandalkan pertumbuhan ekonomi (bantuan stimulus). Diperlukan intervensi struktural khusus untuk wilayah ”Stagnan” (seperti Lebak dan Pandeglang) fokus pada peningkatan kapabilitas dasar (IPM) agar dapat mengejar ketertinggalan dari wilayah metropolitan.
2. **Pengembangan Penelitian Lanjutan:** Penelitian ini dibatasi pada indikator makroekonomi tahunan. Penelitian selanjutnya disarankan untuk:
    - Menggunakan data dengan resolusi waktu yang lebih tinggi (triwulanan) jika tersedia, untuk menangkap dinamika perubahan jangka pendek lebih detail.
    - Menerapkan metode *Dual STATIS* atau *STATIS-Dual* jika ingin fokus pada analisis hubungan antar-variabel secara lebih spesifik dibandingkan hubungan antar-objek.

DAFTAR PUSTAKA

Ahmad, A. (2024). Analisis regresi panel spasiotemporal menggunakan model spasial durbin dinamis pada prevalensi kusta di pulau jawa. Institut Pertanian Bogor, Bogor. Diakses: 20 November 2025. https://repository.ipb.ac.id/handle/123456789/154545

Anselin, L. (1988). *Spatial Econometrics: Methods and Models*. Kluwer Academic Publishers. https://link.springer.com/book/10.1007/978-94-015-7799-1

Anton, H. & Rorres, C. (2013). *Elementary Linear Algebra: Applications Version* (11th ed.). John Wiley & Sons. https://www.wiley.com/en-us/Elementary+Linear+Algebra%3A+Applications+Version%2C+11th+Edition-p-9781118434413

Badan Pusat Statistik (2024). *Prevalensi ketidakcukupan konsumsi pangan (persen) per kabupaten/kota (persen), 2024*. Diakses dari situs resmi Badan Pusat Statistik. https://www.bps.go.id/id/statistics-table/2/MjI2OSMy/prevalensi-ketidakcukupan-konsumsi-pangan--persen--per-kabupaten-kota--persen-.html

Badan Pusat Statistik (2025a). *Indikator kesejahteraan rakyat 2025*. Katalog 4102004, ISSN 0215-4641. https://www.bps.go.id/id/publication/2025/12/30/bcc229ac31403d023f2d26a3/indikator-kesejahteraan-rakyat-2025.html

Badan Pusat Statistik (2025b). *Tinjauan regional berdasarkan pdrb kabupaten/kota 2020–2024*. Diakses dari situs resmi Badan Pusat Statistik. https://www.bps.go.id/id/statistics-table/3/WkdVMWRYVnBkMnBvVEhKSVkyWXhNblZtTjJSbmR6MDkjMw==/produk-domestik-regional-bruto-atas-dasar-harga-berlaku-menurut-provinsi--miliar-rupiah---2022.html?year=2024

Baltagi, B. (2021). *Econometric Analysis of Panel Data*. https://doi.org/10.1007/978-3-030-53953-5

Bartlett, M. S. (1954). A note on the multiplying factors for various χ^2 approximations. *Journal of the Royal Statistical Society: Series B (Methodological)*, 16(2), 296–298. https://doi.org/10.1111/j.2517-6161.1954.tb00174.x

Bholowalia, P. & Kumar, A. (2014). Ebk-means: A clustering technique based on elbow method and k-means in wsn. *International Journal of Computer Applications*, 105(9). https://doi.org/10.5120/18405-9674

Biegert, T., Özcan, B., & Rossetti-Youlton, M. (2023). Household joblessness in u.s. metropolitan areas during the covid-19 pandemic: Polarization and the role of educational profiles. *Socius: Sociological Research for a Dynamic World*, 9, 1–18. https://doi.org/10.1177/23780231231158087

Dziuban, C. D. & Shirkey, E. C. (1974). When is a correlation matrix appropriate for factor analysis? some decision rules. *Psychological Bulletin*, 81(6), 358–361. https://doi.org/10.1037/h0036316

Egido, J. & Galindo, P. (2015). Dynamic biplot. evolution of the economic freedom in the european union. *British Journal of Applied Science & Technology*, 11(3), 1–13. https://doi.org/10.9734/BJAST/2015/20289

Escoufier, Y. (1973). Le traitement des variables vectorielles. *Biometrics*, 29(4), 751–760. https://doi.org/10.2307/2529140

Gabriel, K. R. (1971). The biplot graphic display of matrices with application to principal component analysis. *Biometrika*, 58(3), 453–467. https://doi.org/10.1093/biomet/58.3.453

Hair, J. F., Black, W. C., Babin, B. J., & Anderson, R. E. (2019). *Multivariate Data Analysis* (8 ed.). Cengage Learning. https://www.cengage.uk/c/multivariate-data-analysis-8e-hair-black-babin-anderson/9781473756540/

Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441. https://doi.org/10.1037/h0071325

Johnson, R. A. & Wichern, D. W. (2007). *Applied Multivariate Statistical Analysis* (6 ed.). Pearson Prentice Hall. https://www.pearson.com/en-us/subject-catalog/p/applied-multivariate-statistical-analysis/P200000003290

Kaiser, H. F. (1970). A second generation little jiffy. *Psychometrika*, 35(4), 401–415. https://doi.org/10.1007/BF02291817

Kaiser, H. F. (1974). An index of factorial simplicity. *Psychometrika*, 39(1), 31–36. https://doi.org/10.1007/BF02291575

Kazi-Aoual, F., Hitier, S., Sabatier, R., & Lebreton, J. D. (1995). Refined approximations to permutation tests for multivariate inference. *Computational Statistics & Data Analysis*, 20(4), 385–394. https://doi.org/10.1016/0167-9473(94)00064-2

Kroonenberg, P. M. (2008). *Applied Multiway Data Analysis*. John Wiley & Sons. https://doi.org/10.1002/9780470238004

Lavit, C., Escoufier, Y., Sabatier, R., & Traissac, P. (1994). The act (statis method). *Computational Statistics & Data Analysis*, 18(1), 97–119. https://doi.org/10.1016/0167-9473(94)90134-1

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, volume 1, 281–297. https://projecteuclid.org/journals/berkeley-symposium-on-mathematical-statistics-and-probability/volume-1/issue-1/Some-methods-for-classification-and-analysis-of-multivariate-observations/10.15115/D3S307.full

Nisa, A. C. (2024). Perbandingan antara hasil biclustering dan hasil cluster-biplot pada data 119 kabupaten/kota di pulau jawa tahun 2022 berdasarkan indikator kesejahteraan rakyat. Universitas Indonesia, Depok. https://doi.org/10.30598/barekengvol19iss2pp1009-1022

Nuswantari, A. (2021). Pemodelan kemiskinan di pulau jawa tahun 2012 sampai 2018 dengan panel spasial dinamis. Institut Pertanian Bogor, Bogor. Diakses: 20 November 2025. https://repository.ipb.ac.id/handle/123456789/107909

Oliveira, M., Garção, E., Grilo, L. M., Alexandre, A., & Mexia, J. T. (2024). Statis methodology: An overview. *AIP Conference Proceedings*, volume 3094, 450006. https://doi.org/10.1063/5.0212208

Phipson, B. & Smyth, G. K. (2010). Permutation p-values should never be zero: Calculating exact p-values when permutations are randomly drawn. *Statistical Applications in Genetics and Molecular Biology*, 9(1), Article 39. https://doi.org/10.48550/arXiv.1603.05766

Plotly Technologies Inc. (2015). *Collaborative data science*. Plotly. https://plotly.com

Plotly Technologies Inc. (2024). *Dash Python User Guide*. Plotly Technologies Inc. Dokumentasi resmi kerangka kerja Dash. https://dash.plotly.com/

Syakur, M., Khotimah, B., Rochman, E., & Satoto, B. (2018). Integration k-means clustering method and elbow method for identification of the best customer profile cluster. *IOP conference series: materials science and engineering*, volume 336, 012017. https://doi.org/10.1088/1757-899X/336/1/012017

Ward Jr, J. H. (1963). Hierarchical grouping to optimize an objective function. *Journal of the American statistical association*, 58(301), 236–244. https://doi.org/10.1080/01621459.1963.10500845

Lampiran 1 Data Penelitian

Berikut adalah data penelitian untuk masing-masing variabel dari tahun 2020 hingga 2024.

Data Variabel IPM

No	Provinsi	Kabupaten/Kota	2020	2021	2022	2023	2024
1	DKI JAKARTA	Kep. Seribu	71,63	72,10	72,79	73,39	74,17
2	DKI JAKARTA	Kota Jakarta Selatan	84,72	84,90	85,21	86,07	86,94
3	DKI JAKARTA	Kota Jakarta Timur	82,66	82,97	83,45	84,13	84,65
4	DKI JAKARTA	Kota Jakarta Pusat	81,39	81,56	82,11	82,53	83,01
5	DKI JAKARTA	Kota Jakarta Barat	81,38	81,76	82,51	83,02	83,64
6	DKI JAKARTA	Kota Jakarta Utara	80,29	80,51	80,81	81,11	81,39
7	JAWA BARAT	Bogor	70,40	70,60	71,20	71,78	72,39
8	JAWA BARAT	Sukabumi	66,88	67,07	67,64	68,49	68,97
9	JAWA BARAT	Cianjur	65,36	65,56	65,94	66,55	67,24
10	JAWA BARAT	Bandung	72,39	72,73	73,16	73,74	74,27
11	JAWA BARAT	Garut	66,12	66,45	67,41	68,11	68,79
12	JAWA BARAT	Tasikmalaya	65,67	65,90	66,84	67,76	68,36
13	JAWA BARAT	Ciamis	70,49	70,93	71,45	72,05	72,56
14	JAWA BARAT	Kuningan	69,38	69,71	70,16	70,72	71,26
15	JAWA BARAT	Cirebon	68,75	69,12	70,06	70,95	71,44
16	JAWA BARAT	Majalengka	67,59	67,81	68,56	69,13	69,74
17	JAWA BARAT	Sumedang	71,64	71,80	72,69	73,18	73,73
18	JAWA BARAT	Indramayu	67,29	67,64	68,55	69,25	69,83
19	JAWA BARAT	Subang	68,95	69,13	69,87	70,70	71,36
20	JAWA BARAT	Purwakarta	70,82	70,98	71,56	72,09	72,65
21	JAWA BARAT	Karawang	70,66	70,94	71,74	72,35	72,90
22	JAWA BARAT	Bekasi	74,07	74,45	75,22	75,76	76,44
23	JAWA BARAT	Bandung Barat	68,08	68,29	69,04	69,61	70,03
24	JAWA BARAT	Pangandaran	68,06	68,28	69,03	69,38	69,85
25	JAWA BARAT	Kota Bogor	76,11	76,59	77,17	77,85	78,52
26	JAWA BARAT	Kota Sukabumi	74,21	74,60	75,40	76,32	76,85
27	JAWA BARAT	Kota Bandung	81,51	81,96	82,50	83,04	83,52
28	JAWA BARAT	Kota Cirebon	74,89	75,25	75,89	76,46	77,08
29	JAWA BARAT	Kota Bekasi	81,50	81,95	82,46	83,03	83,54
30	JAWA BARAT	Kota Depok	80,97	81,37	81,86	82,38	82,91
31	JAWA BARAT	Kota Cimahi	77,83	78,06	78,77	79,46	80,08
32	JAWA BARAT	Kota Tasikmalaya	73,04	73,31	73,83	74,47	75,04
33	JAWA BARAT	Kota Banjar	71,70	71,92	72,55	73,08	73,64
34	BANTEN	Pandeglang	65,00	65,17	65,84	66,42	66,99
35	BANTEN	Lebak	63,91	64,03	64,71	65,21	65,86
36	BANTEN	Tangerang	71,92	72,29	72,97	73,43	74,09
37	BANTEN	Serang	66,70	66,82	67,75	68,39	69,03
38	BANTEN	Kota Tangerang	78,25	78,50	78,90	79,46	80,02
39	BANTEN	Kota Cilegon	73,05	73,35	73,95	74,54	75,14
40	BANTEN	Kota Serang	72,16	72,44	72,98	73,48	73,99
41	BANTEN	Kota Tangerang Selatan	81,36	81,60	81,95	82,28	82,89

Data Variabel IPG

No	Provinsi	Kabupaten/Kota	2020	2021	2022	2023	2024
1	DKI JAKARTA	Kep. Seribu	93,39	93,39	93,69	93,60	93,62
2	DKI JAKARTA	Kota Jakarta Selatan	96,14	96,28	96,48	97,16	97,30
3	DKI JAKARTA	Kota Jakarta Timur	95,48	95,44	95,92	95,98	95,90
4	DKI JAKARTA	Kota Jakarta Pusat	96,00	96,11	96,41	96,28	96,24
5	DKI JAKARTA	Kota Jakarta Barat	95,16	95,37	95,39	95,39	95,66
6	DKI JAKARTA	Kota Jakarta Utara	93,53	93,60	93,84	94,03	94,43
7	JAWA BARAT	Bogor	89,23	89,40	89,95	90,20	90,17
8	JAWA BARAT	Sukabumi	87,37	87,44	87,64	88,30	88,59
9	JAWA BARAT	Cianjur	84,14	84,30	84,90	85,17	85,45
10	JAWA BARAT	Bandung	93,95	93,85	93,97	94,03	94,23
11	JAWA BARAT	Garut	82,29	82,13	82,78	83,37	83,50
12	JAWA BARAT	Tasikmalaya	86,02	86,36	86,64	87,01	87,10
13	JAWA BARAT	Ciamis	86,61	86,55	87,04	87,25	87,48
14	JAWA BARAT	Kuningan	86,81	86,97	87,17	87,31	87,55
15	JAWA BARAT	Cirebon	83,47	83,49	84,01	84,28	84,74
16	JAWA BARAT	Majalengka	85,91	86,24	86,69	87,06	87,16
17	JAWA BARAT	Sumedang	95,18	95,18	95,09	95,41	95,55
18	JAWA BARAT	Indramayu	88,15	87,92	88,17	88,81	89,03
19	JAWA BARAT	Subang	90,53	90,51	90,95	91,29	91,55
20	JAWA BARAT	Purwakarta	86,56	86,70	86,94	87,08	87,18
21	JAWA BARAT	Karawang	90,12	90,29	90,50	91,07	91,23
22	JAWA BARAT	Bekasi	88,58	88,88	89,37	89,53	89,72
23	JAWA BARAT	Bandung Barat	79,06	79,07	79,69	80,37	81,02
24	JAWA BARAT	Pangandaran	89,78	89,92	90,20	90,04	90,38
25	JAWA BARAT	Kota Bogor	91,02	90,87	90,97	91,11	91,26
26	JAWA BARAT	Kota Sukabumi	91,36	91,58	91,83	92,35	92,59
27	JAWA BARAT	Kota Bandung	94,68	94,72	94,81	94,71	94,87
28	JAWA BARAT	Kota Cirebon	94,39	94,46	94,91	95,55	95,83
29	JAWA BARAT	Kota Bekasi	92,95	93,09	93,49	93,52	93,62
30	JAWA BARAT	Kota Depok	93,01	92,96	93,47	93,44	93,70
31	JAWA BARAT	Kota Cimahi	92,01	91,95	91,97	92,00	92,28
32	JAWA BARAT	Kota Tasikmalaya	91,64	91,74	91,87	92,13	92,22
33	JAWA BARAT	Kota Banjar	87,17	87,01	87,67	87,85	88,12
34	BANTEN	Pandeglang	86,71	86,83	86,93	87,09	87,23
35	BANTEN	Lebak	79,81	79,87	80,20	80,75	81,22
36	BANTEN	Tangerang	90,98	91,08	91,24	91,67	91,69
37	BANTEN	Serang	92,14	91,98	92,32	92,49	92,54
38	BANTEN	Kota Tangerang	95,06	94,72	94,73	94,89	94,95
39	BANTEN	Kota Cilegon	87,22	87,25	87,57	88,05	88,29
40	BANTEN	Kota Serang	92,28	92,47	92,63	92,49	92,61
41	BANTEN	Kota Tangerang Selatan	93,58	93,80	93,96	94,23	94,47

Data Variabel PPM

No	Provinsi	Kabupaten/Kota	2020	2021	2022	2023	2024
1	DKI JAKARTA	Kep. Seribu	14,87	15,06	14,11	13,13	13,03
2	DKI JAKARTA	Kota Jakarta Selatan	3,43	3,56	3,52	3,10	3,03
3	DKI JAKARTA	Kota Jakarta Timur	4,16	4,28	4,30	4,20	4,09
4	DKI JAKARTA	Kota Jakarta Pusat	4,51	4,94	4,90	4,68	4,63
5	DKI JAKARTA	Kota Jakarta Barat	4,25	4,31	4,22	4,09	3,94
6	DKI JAKARTA	Kota Jakarta Utara	6,78	7,24	7,24	6,78	6,44
7	JAWA BARAT	Bogor	7,69	8,13	7,73	7,27	7,05
8	JAWA BARAT	Sukabumi	7,09	7,70	7,34	7,01	6,87
9	JAWA BARAT	Cianjur	10,36	11,18	10,55	10,22	10,14
10	JAWA BARAT	Bandung	6,91	7,15	6,80	6,40	6,19
11	JAWA BARAT	Garut	9,98	10,65	10,42	9,77	9,68
12	JAWA BARAT	Tasikmalaya	10,34	11,15	10,73	10,28	10,23
13	JAWA BARAT	Ciamis	7,62	7,97	7,72	7,42	7,39
14	JAWA BARAT	Kuningan	12,82	13,10	12,76	12,12	11,88
15	JAWA BARAT	Cirebon	11,24	12,30	12,01	11,20	11,00
16	JAWA BARAT	Majalengka	11,43	12,33	11,94	11,21	10,82
17	JAWA BARAT	Sumedang	10,26	10,71	10,14	9,36	9,10
18	JAWA BARAT	Indramayu	12,70	13,04	12,77	12,13	11,93
19	JAWA BARAT	Subang	9,31	10,03	9,75	9,52	9,49
20	JAWA BARAT	Purwakarta	8,27	8,83	8,70	8,46	8,41
21	JAWA BARAT	Karawang	8,26	8,95	8,44	7,87	7,86
22	JAWA BARAT	Bekasi	4,82	5,21	5,01	4,93	4,80
23	JAWA BARAT	Bandung Barat	10,49	11,30	10,82	10,52	10,49
24	JAWA BARAT	Pangandaran	8,99	9,65	9,32	8,98	8,75
25	JAWA BARAT	Kota Bogor	6,68	7,24	7,10	6,67	6,53
26	JAWA BARAT	Kota Sukabumi	7,70	8,25	8,02	7,50	7,20
27	JAWA BARAT	Kota Bandung	3,99	4,37	4,25	3,96	3,87
28	JAWA BARAT	Kota Cirebon	9,52	10,03	9,82	9,16	9,02
29	JAWA BARAT	Kota Bekasi	4,38	4,74	4,43	4,10	4,01
30	JAWA BARAT	Kota Depok	2,45	2,58	2,53	2,38	2,34
31	JAWA BARAT	Kota Cimahi	5,11	5,35	5,11	4,66	4,39
32	JAWA BARAT	Kota Tasikmalaya	12,97	13,13	12,72	11,53	11,10
33	JAWA BARAT	Kota Banjar	6,09	7,11	6,73	6,14	5,85
34	BANTEN	Pandeglang	9,92	10,72	9,32	9,27	9,18
35	BANTEN	Lebak	9,24	10,29	8,91	8,68	8,44
36	BANTEN	Tangerang	6,23	7,12	6,92	6,93	6,55
37	BANTEN	Serang	4,94	5,49	4,96	4,85	4,51
38	BANTEN	Kota Tangerang	5,22	5,93	5,77	5,89	5,43
39	BANTEN	Kota Cilegon	3,69	4,24	3,64	3,98	3,75
40	BANTEN	Kota Serang	6,06	6,79	5,94	6,20	5,65
41	BANTEN	Kota Tangerang Selatan	2,29	2,57	2,50	2,57	2,36

Data Variabel PDRB_ADHK

No	Provinsi	Kabupaten/Kota	2020	2021	2022	2023	2024
1	DKI JAKARTA	Kep. Seribu	3.706,25	3.648,88	3.586,89	3.305,13	3.268,50
2	DKI JAKARTA	Kota Jakarta Selatan	419.257,19	429.278,00	451.772,70	475.806,20	500.422,70
3	DKI JAKARTA	Kota Jakarta Timur	299.515,16	313.816,50	329.546,30	346.508,70	362.068,10
4	DKI JAKARTA	Kota Jakarta Pusat	449.518,14	460.024,30	481.954,50	506.560,70	532.606,90
5	DKI JAKARTA	Kota Jakarta Barat	316.172,61	327.613,10	345.619,60	363.931,20	383.113,10
6	DKI JAKARTA	Kota Jakarta Utara	312.100,58	330.883,90	350.233,70	364.823,90	380.461,10
7	JAWA BARAT	Bogor	154.113,60	159.589,60	167.966,20	176.683,60	185.882,30
8	JAWA BARAT	Sukabumi	46.205,28	47.935,63	50.388,72	52.993,98	55.723,92
9	JAWA BARAT	Cianjur	31.792,32	32.897,53	34.556,40	36.339,55	38.208,18
10	JAWA BARAT	Bandung	81.060,97	83.949,37	88.437,96	92.830,17	97.509,91
11	JAWA BARAT	Garut	38.598,43	39.981,19	42.012,96	44.087,22	46.270,92
12	JAWA BARAT	Tasikmalaya	24.344,86	25.179,48	26.361,66	27.599,16	28.879,68
13	JAWA BARAT	Ciamis	21.970,41	22.774,93	23.918,89	25.111,71	26.360,94
14	JAWA BARAT	Kuningan	16.882,76	17.483,56	18.450,34	19.418,73	20.507,27
15	JAWA BARAT	Cirebon	33.304,05	34.127,52	35.523,78	37.246,57	39.418,27
16	JAWA BARAT	Majalengka	21.754,54	22.788,75	24.298,10	25.793,24	27.438,93
17	JAWA BARAT	Sumedang	23.665,01	24.414,66	25.595,02	26.876,86	27.965,76
18	JAWA BARAT	Indramayu	59.200,00	59.544,87	61.259,61	67.237,69	68.701,96
19	JAWA BARAT	Subang	28.344,32	28.960,89	30.202,58	31.604,98	33.126,46
20	JAWA BARAT	Purwakarta	45.293,24	46.840,15	49.292,98	51.740,50	54.142,29
21	JAWA BARAT	Karawang	157.710,59	166.941,50	177.470,90	187.051,70	194.903,10
22	JAWA BARAT	Bekasi	242.971,39	251.778,50	265.120,50	279.224,90	293.664,80
23	JAWA BARAT	Bandung Barat	30.640,41	31.701,79	33.393,29	35.062,19	36.793,68
24	JAWA BARAT	Pangandaran	7.738,97	8.022,78	2.383.665,00	8.869,86	9.324,31
25	JAWA BARAT	Kota Bogor	32.162,74	33.372,48	35.258,87	37.055,36	38.964,67
26	JAWA BARAT	Kota Sukabumi	8.534,72	2.538.923,00	9.324,16	9.801,88	10.303,16
27	JAWA BARAT	Kota Bandung	193.144,95	200.414,00	211.250,70	221.969,10	233.053,80
28	JAWA BARAT	Kota Cirebon	16.648,21	17.155,75	18.030,25	18.934,40	19.884,06
29	JAWA BARAT	Kota Bekasi	67.619,24	69.796,94	73.260,65	77.241,79	81.251,61
30	JAWA BARAT	Kota Depok	48.135,93	49.947,24	52.564,98	55.221,82	58.241,86
31	JAWA BARAT	Kota Cimahi	22.340,56	23.275,78	24.652,73	25.931,98	27.296,93
32	JAWA BARAT	Kota Tasikmalaya	15.430,02	15.980,75	16.781,04	17.781,93	18.709,87
33	JAWA BARAT	Kota Banjar	3.251,76	3.365,25	3.516,57	3.682,30	3.883,98
34	BANTEN	Pandeglang	19.511,68	20.116,90	20.814,77	21.340,73	21.928,59
35	BANTEN	Lebak	20.622,04	21.278,48	22.087,37	22.706,47	23.377,24
36	BANTEN	Tangerang	93.480,39	97.869,38	103.219,40	108.569,80	113.998,20
37	BANTEN	Serang	52.866,43	54.844,65	57.605,99	60.369,96	63.255,33
38	BANTEN	Kota Tangerang	102.415,70	106.413,70	112.778,10	119.059,90	125.060,30
39	BANTEN	Kota Cilegon	73.319,12	77.163,94	80.646,38	84.536,65	88.628,11
40	BANTEN	Kota Serang	22.518,66	23.392,75	24.495,02	25.655,25	26.863,08
41	BANTEN	Kota Tangerang Selatan	59.531,08	62.393,12	66.021,24	69.561,56	73.052,84

Data Variabel RLS

No	Provinsi	Kabupaten/Kota	2020	2021	2022	2023	2024
1	DKI JAKARTA	Kep. Seribu	8,68	8,81	9,02	9,03	9,26
2	DKI JAKARTA	Kota Jakarta Selatan	11,63	11,64	11,66	11,75	11,95
3	DKI JAKARTA	Kota Jakarta Timur	11,66	11,67	11,73	11,88	11,99
4	DKI JAKARTA	Kota Jakarta Pusat	11,38	11,39	11,53	11,54	11,61
5	DKI JAKARTA	Kota Jakarta Barat	10,63	10,78	11,13	11,23	11,24
6	DKI JAKARTA	Kota Jakarta Utara	10,80	10,81	10,82	10,84	10,85
7	JAWA BARAT	Bogor	8,30	8,31	8,34	8,37	8,39
8	JAWA BARAT	Sukabumi	7,07	7,10	7,11	7,33	7,34
9	JAWA BARAT	Cianjur	7,18	7,19	7,20	7,22	7,33
10	JAWA BARAT	Bandung	8,96	9,07	9,08	9,10	9,15
11	JAWA BARAT	Garut	7,52	7,53	7,83	7,84	7,85
12	JAWA BARAT	Tasikmalaya	7,35	7,48	7,73	7,96	7,97
13	JAWA BARAT	Ciamis	7,70	7,90	8,00	8,09	8,10
14	JAWA BARAT	Kuningan	7,57	7,80	7,88	7,89	7,90
15	JAWA BARAT	Cirebon	6,92	7,10	7,40	7,64	7,65
16	JAWA BARAT	Majalengka	7,27	7,31	7,49	7,52	7,53
17	JAWA BARAT	Sumedang	8,51	8,52	8,72	8,73	8,74
18	JAWA BARAT	Indramayu	6,30	6,52	6,83	6,94	6,95
19	JAWA BARAT	Subang	7,10	7,11	7,20	7,45	7,46
20	JAWA BARAT	Purwakarta	8,09	8,10	8,11	8,13	8,14
21	JAWA BARAT	Karawang	7,77	7,78	7,96	8,04	8,05
22	JAWA BARAT	Bekasi	9,12	9,30	9,53	9,57	9,76
23	JAWA BARAT	Bandung Barat	8,19	8,20	8,22	8,23	8,24
24	JAWA BARAT	Pangandaran	7,74	7,85	8,03	8,04	8,10
25	JAWA BARAT	Kota Bogor	10,33	10,53	10,63	10,64	10,71
26	JAWA BARAT	Kota Sukabumi	9,59	9,81	10,14	10,37	10,38
27	JAWA BARAT	Kota Bandung	10,75	10,99	11,00	11,06	11,07
28	JAWA BARAT	Kota Cirebon	9,91	10,12	10,33	10,37	10,53
29	JAWA BARAT	Kota Bekasi	11,16	11,31	11,44	11,66	11,79
30	JAWA BARAT	Kota Depok	11,28	11,46	11,47	11,58	11,59
31	JAWA BARAT	Kota Cimahi	10,96	11,08	11,21	11,39	11,52
32	JAWA BARAT	Kota Tasikmalaya	9,33	9,52	9,53	9,54	9,63
33	JAWA BARAT	Kota Banjar	8,63	8,77	8,78	8,79	8,83
34	BANTEN	Pandeglang	7,10	7,11	7,13	7,15	7,16
35	BANTEN	Lebak	6,40	6,41	6,59	6,60	6,61
36	BANTEN	Tangerang	8,39	8,61	8,92	8,93	9,06
37	BANTEN	Serang	7,50	7,51	7,78	7,79	7,88
38	BANTEN	Kota Tangerang	10,69	10,83	10,84	10,91	11,14
39	BANTEN	Kota Cilegon	9,87	10,08	10,34	10,38	10,39
40	BANTEN	Kota Serang	8,76	8,89	8,90	8,91	8,93
41	BANTEN	Kota Tangerang Selatan	11,81	11,82	11,84	11,85	11,86

Data Variabel AHH_L

No	Provinsi	Kabupaten/Kota	2020	2021	2022	2023	2024
1	DKI JAKARTA	Kep. Seribu	66,67	66,84	67,02	67,68	67,85
2	DKI JAKARTA	Kota Jakarta Selatan	72,23	72,33	72,60	73,03	73,38
3	DKI JAKARTA	Kota Jakarta Timur	72,60	72,70	72,98	73,38	73,73
4	DKI JAKARTA	Kota Jakarta Pusat	72,22	72,30	72,55	73,06	73,65
5	DKI JAKARTA	Kota Jakarta Barat	71,72	71,82	71,97	72,39	72,55
6	DKI JAKARTA	Kota Jakarta Utara	71,36	71,44	71,58	71,80	71,97
7	JAWA BARAT	Bogor	69,24	69,52	69,72	69,97	70,20
8	JAWA BARAT	Sukabumi	69,03	69,27	69,53	69,89	70,11
9	JAWA BARAT	Cianjur	68,17	68,45	68,62	68,87	69,03
10	JAWA BARAT	Bandung	71,67	71,94	72,14	72,50	72,73
11	JAWA BARAT	Garut	69,49	69,74	69,74	70,12	70,32
12	JAWA BARAT	Tasikmalaya	67,51	67,78	67,97	68,27	68,47
13	JAWA BARAT	Ciamis	69,92	70,19	70,39	70,61	70,90
14	JAWA BARAT	Kuningan	71,72	71,98	72,18	72,52	72,72
15	JAWA BARAT	Cirebon	70,08	70,34	70,54	70,79	71,03
16	JAWA BARAT	Majalengka	68,31	68,58	68,79	69,12	69,37
17	JAWA BARAT	Sumedang	70,52	70,77	70,95	71,13	71,29
18	JAWA BARAT	Indramayu	69,72	70,01	70,24	70,50	70,80
19	JAWA BARAT	Subang	70,46	70,78	71,03	71,16	71,34
20	JAWA BARAT	Purwakarta	69,05	69,31	69,50	69,79	70,04
21	JAWA BARAT	Karawang	70,25	70,50	70,69	70,93	71,10
22	JAWA BARAT	Bekasi	71,81	71,99	72,17	72,53	72,75
23	JAWA BARAT	Bandung Barat	70,43	70,70	70,88	71,07	71,23
24	JAWA BARAT	Pangandaran	69,49	69,75	69,94	70,22	70,45
25	JAWA BARAT	Kota Bogor	71,74	72,02	72,25	72,69	72,89
26	JAWA BARAT	Kota Sukabumi	70,52	70,75	70,92	71,08	71,23
27	JAWA BARAT	Kota Bandung	72,45	72,71	72,92	73,30	73,50
28	JAWA BARAT	Kota Cirebon	70,35	70,61	70,82	71,06	71,22
29	JAWA BARAT	Kota Bekasi	73,22	73,48	73,70	74,10	74,29
30	JAWA BARAT	Kota Depok	72,61	72,87	73,08	73,49	73,68
31	JAWA BARAT	Kota Cimahi	72,19	72,45	72,66	73,05	73,23
32	JAWA BARAT	Kota Tasikmalaya	70,25	70,52	70,73	70,95	71,11
33	JAWA BARAT	Kota Banjar	69,04	69,31	69,53	69,85	70,08
34	BANTEN	Pandeglang	62,69	62,82	63,17	63,70	63,90
35	BANTEN	Lebak	65,19	65,33	65,69	66,19	66,40
36	BANTEN	Tangerang	67,86	67,93	68,25	68,73	68,95
37	BANTEN	Serang	62,68	62,82	63,20	63,72	64,00
38	BANTEN	Kota Tangerang	69,61	69,61	69,86	70,28	70,50
39	BANTEN	Kota Cilegon	64,66	64,68	64,95	65,46	65,72
40	BANTEN	Kota Serang	65,87	65,91	66,19	67,05	67,33
41	BANTEN	Kota Tangerang Selatan	70,51	70,55	70,85	71,07	71,21

Data Variabel AHH_P

No	Provinsi	Kabupaten/Kota	2020	2021	2022	2023	2024
1	DKI JAKARTA	Kep. Seribu	70,74	71,03	71,27	71,63	71,80
2	DKI JAKARTA	Kota Jakarta Selatan	76,04	76,12	76,51	76,69	76,86
3	DKI JAKARTA	Kota Jakarta Timur	76,22	76,27	76,63	77,01	77,11
4	DKI JAKARTA	Kota Jakarta Pusat	75,92	76,00	76,39	76,71	76,94
5	DKI JAKARTA	Kota Jakarta Barat	75,43	75,49	75,65	76,09	76,28
6	DKI JAKARTA	Kota Jakarta Utara	75,13	75,22	75,41	75,55	76,04
7	JAWA BARAT	Bogor	73,06	73,25	73,54	73,96	74,21
8	JAWA BARAT	Sukabumi	72,87	73,13	73,47	73,88	74,11
9	JAWA BARAT	Cianjur	72,03	72,23	72,50	72,81	72,98
10	JAWA BARAT	Bandung	75,34	75,52	75,81	76,20	76,44
11	JAWA BARAT	Garut	73,30	73,48	73,74	74,12	74,33
12	JAWA BARAT	Tasikmalaya	71,39	71,59	71,88	72,21	72,41
13	JAWA BARAT	Ciamis	73,72	73,91	74,20	74,63	74,94
14	JAWA BARAT	Kuningan	75,42	75,61	75,87	76,22	76,42
15	JAWA BARAT	Cirebon	73,87	74,07	74,37	74,83	75,15
16	JAWA BARAT	Majalengka	72,18	72,37	72,68	73,08	73,34
17	JAWA BARAT	Sumedang	74,30	74,49	74,78	75,35	75,70
18	JAWA BARAT	Indramayu	73,52	73,72	74,04	74,52	74,83
19	JAWA BARAT	Subang	74,22	74,44	74,79	75,43	75,80
20	JAWA BARAT	Purwakarta	72,89	73,08	73,37	73,78	74,04
21	JAWA BARAT	Karawang	74,03	74,21	74,50	74,97	75,29
22	JAWA BARAT	Bekasi	75,50	75,63	75,87	76,23	76,55
23	JAWA BARAT	Bandung Barat	74,20	74,39	74,67	75,23	75,56
24	JAWA BARAT	Pangandaran	73,29	73,48	73,77	74,22	74,46
25	JAWA BARAT	Kota Bogor	75,44	75,65	75,96	76,37	76,69
26	JAWA BARAT	Kota Sukabumi	74,28	74,44	74,72	75,25	75,57
27	JAWA BARAT	Kota Bandung	75,97	76,16	76,46	76,93	77,24
28	JAWA BARAT	Kota Cirebon	74,13	74,31	74,61	75,20	75,55
29	JAWA BARAT	Kota Bekasi	76,71	76,89	77,18	77,45	77,75
30	JAWA BARAT	Kota Depok	76,42	76,60	76,91	77,11	77,41
31	JAWA BARAT	Kota Cimahi	75,77	75,96	76,26	76,70	77,01
32	JAWA BARAT	Kota Tasikmalaya	74,02	74,21	74,51	74,99	75,32
33	JAWA BARAT	Kota Banjar	72,89	73,08	73,39	73,84	74,08
34	BANTEN	Pandeglang	66,54	66,68	67,10	67,56	67,78
35	BANTEN	Lebak	69,13	69,24	69,66	70,17	70,38
36	BANTEN	Tangerang	71,81	71,84	72,19	72,67	72,90
37	BANTEN	Serang	66,50	66,61	67,03	67,58	67,89
38	BANTEN	Kota Tangerang	73,49	73,49	73,80	74,29	74,52
39	BANTEN	Kota Cilegon	68,58	68,61	68,95	69,42	69,70
40	BANTEN	Kota Serang	69,94	70,06	70,48	71,02	71,29
41	BANTEN	Kota Tangerang Selatan	74,31	74,31	74,62	75,24	75,52

Data Variabel GK

No	Provinsi	Kabupaten/Kota	2020	2021	2022	2023	2024
1	DKI JAKARTA	Kep. Seribu	661.852,00	670.785,00	701.088,00	743.820,00	775.909,00
2	DKI JAKARTA	Kota Jakarta Selatan	782.731,00	792.684,00	842.630,00	891.580,00	928.278,00
3	DKI JAKARTA	Kota Jakarta Timur	581.954,00	594.849,00	630.842,00	669.713,00	697.472,00
4	DKI JAKARTA	Kota Jakarta Pusat	672.328,00	689.379,00	733.176,00	784.679,00	818.722,00
5	DKI JAKARTA	Kota Jakarta Barat	557.803,00	570.330,00	599.514,00	657.247,00	685.071,00
6	DKI JAKARTA	Kota Jakarta Utara	594.240,00	609.310,00	645.431,00	684.557,00	712.835,00
7	JAWA BARAT	Bogor	402.877,00	418.483,00	443.787,00	482.174,00	513.512,00
8	JAWA BARAT	Sukabumi	328.284,00	342.094,00	357.636,00	392.705,00	416.751,00
9	JAWA BARAT	Cianjur	371.699,00	387.631,00	406.829,00	437.327,00	466.509,00
10	JAWA BARAT	Bandung	367.403,00	378.819,00	398.884,00	426.069,00	455.636,00
11	JAWA BARAT	Garut	310.437,00	320.050,00	335.134,00	367.681,00	393.464,00
12	JAWA BARAT	Tasikmalaya	323.880,00	333.909,00	347.917,00	378.958,00	400.148,00
13	JAWA BARAT	Ciamis	378.108,00	389.676,00	405.294,00	442.108,00	467.497,00
14	JAWA BARAT	Kuningan	352.358,00	358.069,00	371.665,00	402.767,00	420.867,00
15	JAWA BARAT	Cirebon	393.452,00	404.635,00	416.914,00	451.853,00	475.046,00
16	JAWA BARAT	Majalengka	453.201,00	466.813,00	483.319,00	520.713,00	547.912,00
17	JAWA BARAT	Sumedang	353.092,00	360.054,00	371.870,00	396.573,00	422.714,00
18	JAWA BARAT	Indramayu	474.807,00	481.754,00	499.805,00	532.545,00	560.159,00
19	JAWA BARAT	Subang	351.885,00	360.691,00	372.308,00	404.320,00	434.161,00
20	JAWA BARAT	Purwakarta	378.287,00	387.754,00	403.663,00	434.187,00	467.948,00
21	JAWA BARAT	Karawang	466.152,00	494.201,00	521.158,00	555.889,00	597.345,00
22	JAWA BARAT	Bekasi	518.667,00	549.875,00	579.221,00	632.811,00	674.924,00
23	JAWA BARAT	Bandung Barat	362.212,00	374.470,00	393.956,00	426.355,00	455.325,00
24	JAWA BARAT	Pangandaran	382.264,00	394.101,00	411.174,00	447.242,00	470.068,00
25	JAWA BARAT	Kota Bogor	547.399,00	571.425,00	608.949,00	661.384,00	699.861,00
26	JAWA BARAT	Kota Sukabumi	547.415,00	567.734,00	594.118,00	644.276,00	678.258,00
27	JAWA BARAT	Kota Bandung	500.452,00	515.396,00	545.675,00	591.124,00	614.707,00
28	JAWA BARAT	Kota Cirebon	457.954,00	467.248,00	485.613,00	520.579,00	545.592,00
29	JAWA BARAT	Kota Bekasi	657.953,00	692.885,00	731.392,00	795.965,00	842.927,00
30	JAWA BARAT	Kota Depok	688.194,00	705.084,00	744.771,00	804.984,00	843.893,00
31	JAWA BARAT	Kota Cimahi	511.375,00	522.281,00	549.450,00	590.197,00	612.567,00
32	JAWA BARAT	Kota Tasikmalaya	470.150,00	480.341,00	498.711,00	537.497,00	565.377,00
33	JAWA BARAT	Kota Banjar	344.363,00	357.210,00	373.510,00	408.710,00	430.590,00
34	BANTEN	Pandeglang	361.713,00	390.754,00	416.248,00	455.297,00	479.886,00
35	BANTEN	Lebak	334.509,00	358.479,00	380.681,00	419.066,00	440.705,00
36	BANTEN	Tangerang	520.742,00	545.065,00	576.122,00	604.362,00	632.941,00
37	BANTEN	Serang	341.074,00	362.102,00	385.864,00	425.751,00	449.291,00
38	BANTEN	Kota Tangerang	632.835,00	655.061,00	695.225,00	746.468,00	785.113,00
39	BANTEN	Kota Cilegon	504.571,00	540.032,00	572.780,00	632.703,00	663.533,00
40	BANTEN	Kota Serang	411.595,00	431.210,00	460.613,00	511.575,00	540.786,00
41	BANTEN	Kota Tangerang Selatan	643.083,00	665.610,00	712.717,00	782.543,00	828.278,00

Data Variabel PKKP

No	Provinsi	Kabupaten/Kota	2020	2021	2022	2023	2024
1	DKI JAKARTA	Kep. Seribu	2,86	2,41	3,91	3,63	4,39
2	DKI JAKARTA	Kota Jakarta Selatan	1,50	1,63	2,51	2,28	2,45
3	DKI JAKARTA	Kota Jakarta Timur	2,18	2,24	3,71	3,08	4,38
4	DKI JAKARTA	Kota Jakarta Pusat	1,97	3,08	3,82	2,91	3,63
5	DKI JAKARTA	Kota Jakarta Barat	1,78	2,00	2,80	2,03	3,55
6	DKI JAKARTA	Kota Jakarta Utara	2,18	2,63	4,78	2,62	3,39
7	JAWA BARAT	Bogor	4,06	4,73	7,03	5,30	5,68
8	JAWA BARAT	Sukabumi	4,89	4,81	7,42	5,98	5,82
9	JAWA BARAT	Cianjur	4,75	6,20	9,54	7,02	5,37
10	JAWA BARAT	Bandung	5,07	5,53	8,30	5,19	5,80
11	JAWA BARAT	Garut	4,06	5,00	7,74	5,76	5,02
12	JAWA BARAT	Tasikmalaya	5,64	7,12	10,45	8,65	5,95
13	JAWA BARAT	Ciamis	5,11	5,87	6,74	6,10	5,78
14	JAWA BARAT	Kuningan	3,51	3,73	6,63	6,43	5,83
15	JAWA BARAT	Cirebon	5,41	6,19	9,04	7,28	5,65
16	JAWA BARAT	Majalengka	4,17	4,94	9,88	7,90	5,95
17	JAWA BARAT	Sumedang	4,01	5,09	7,43	5,55	5,43
18	JAWA BARAT	Indramayu	4,28	3,97	5,99	5,40	5,46
19	JAWA BARAT	Subang	4,81	4,16	7,06	6,23	6,19
20	JAWA BARAT	Purwakarta	4,08	3,45	6,72	5,07	5,84
21	JAWA BARAT	Karawang	3,79	4,80	6,73	5,93	5,10
22	JAWA BARAT	Bekasi	2,26	2,38	4,30	4,36	4,60
23	JAWA BARAT	Bandung Barat	5,07	6,04	7,81	7,48	6,25
24	JAWA BARAT	Pangandaran	4,39	4,32	5,77	6,04	5,22
25	JAWA BARAT	Kota Bogor	2,95	3,08	5,35	4,86	7,98
26	JAWA BARAT	Kota Sukabumi	3,65	4,90	8,46	5,23	5,57
27	JAWA BARAT	Kota Bandung	2,82	2,92	4,55	3,64	5,48
28	JAWA BARAT	Kota Cirebon	4,52	5,71	7,97	6,85	6,56
29	JAWA BARAT	Kota Bekasi	1,33	1,79	3,33	2,27	4,88
30	JAWA BARAT	Kota Depok	1,26	1,55	2,15	1,99	4,52
31	JAWA BARAT	Kota Cimahi	2,66	3,23	4,63	2,98	5,43
32	JAWA BARAT	Kota Tasikmalaya	4,17	5,35	7,92	5,14	5,31
33	JAWA BARAT	Kota Banjar	3,93	5,93	6,40	5,55	5,13
34	BANTEN	Pandeglang	3,60	4,37	4,39	4,69	2,70
35	BANTEN	Lebak	3,14	3,87	3,45	4,82	2,52
36	BANTEN	Tangerang	1,84	2,65	2,28	2,59	2,22
37	BANTEN	Serang	2,44	3,03	2,90	3,23	2,13
38	BANTEN	Kota Tangerang	1,42	2,16	1,71	1,58	1,89
39	BANTEN	Kota Cilegon	2,02	2,46	2,04	2,19	1,96
40	BANTEN	Kota Serang	1,29	1,99	2,42	2,30	2,18
41	BANTEN	Kota Tangerang Selatan	1,51	1,89	1,50	1,92	3,13

Data Variabel TPT

No	Provinsi	Kabupaten/Kota	2020	2021	2022	2023	2024
1	DKI JAKARTA	Kep. Seribu	7,37	8,58	8,47	8,11	7,93
2	DKI JAKARTA	Kota Jakarta Selatan	10,79	7,33	5,63	5,37	5,22
3	DKI JAKARTA	Kota Jakarta Timur	9,29	8,23	8,39	7,24	6,95
4	DKI JAKARTA	Kota Jakarta Pusat	10,97	7,75	5,88	6,42	6,24
5	DKI JAKARTA	Kota Jakarta Barat	12,27	9,06	7,10	6,39	6,18
6	DKI JAKARTA	Kota Jakarta Utara	11,79	9,84	8,04	7,05	6,18
7	JAWA BARAT	Bogor	14,29	12,22	10,64	8,47	7,34
8	JAWA BARAT	Sukabumi	9,60	9,51	7,77	7,32	7,11
9	JAWA BARAT	Cianjur	11,05	9,32	8,41	7,71	5,99
10	JAWA BARAT	Bandung	8,58	8,32	6,98	6,52	6,36
11	JAWA BARAT	Garut	8,95	8,68	7,60	7,33	6,96
12	JAWA BARAT	Tasikmalaya	7,12	6,16	4,17	3,89	3,74
13	JAWA BARAT	Ciamis	5,66	5,06	3,75	3,52	3,37
14	JAWA BARAT	Kuningan	11,22	11,68	9,81	9,49	7,78
15	JAWA BARAT	Cirebon	11,52	10,38	8,11	7,65	6,74
16	JAWA BARAT	Majalengka	5,84	5,71	4,16	4,12	4,01
17	JAWA BARAT	Sumedang	9,89	9,18	7,72	6,94	6,16
18	JAWA BARAT	Indramayu	9,21	8,30	6,49	6,46	6,25
19	JAWA BARAT	Subang	9,48	9,77	7,77	7,65	6,73
20	JAWA BARAT	Purwakarta	11,07	10,70	8,75	7,72	7,34
21	JAWA BARAT	Karawang	11,52	11,83	9,87	8,95	8,04
22	JAWA BARAT	Bekasi	11,54	10,09	10,31	8,87	8,82
23	JAWA BARAT	Bandung Barat	12,25	11,65	9,63	8,11	6,70
24	JAWA BARAT	Pangandaran	5,08	3,25	1,56	1,52	1,58
25	JAWA BARAT	Kota Bogor	12,68	11,79	10,78	9,39	8,13
26	JAWA BARAT	Kota Sukabumi	12,17	10,78	8,83	8,53	8,34
27	JAWA BARAT	Kota Bandung	11,19	11,46	9,55	8,83	7,40
28	JAWA BARAT	Kota Cirebon	10,97	10,53	8,42	7,66	6,29
29	JAWA BARAT	Kota Bekasi	10,68	10,88	8,81	7,90	7,82
30	JAWA BARAT	Kota Depok	9,87	9,76	7,82	6,97	6,27
31	JAWA BARAT	Kota Cimahi	13,30	13,07	10,77	10,52	8,97
32	JAWA BARAT	Kota Tasikmalaya	7,99	7,66	6,62	6,55	6,49
33	JAWA BARAT	Kota Banjar	6,73	6,09	5,53	5,43	5,44
34	BANTEN	Pandeglang	9,15	7,70	9,24	9,05	8,09
35	BANTEN	Lebak	9,63	7,86	8,55	7,57	6,23
36	BANTEN	Tangerang	13,06	9,06	7,88	6,94	6,06
37	BANTEN	Serang	12,22	10,58	10,61	9,94	9,18
38	BANTEN	Kota Tangerang	8,63	9,07	7,16	6,76	5,92
39	BANTEN	Kota Cilegon	12,69	10,13	8,10	7,25	6,08
40	BANTEN	Kota Serang	9,26	9,41	8,17	7,45	7,12
41	BANTEN	Kota Tangerang Selatan	8,48	8,60	6,59	5,81	5,09

Data Variabel TPAK

No	Provinsi	Kabupaten/Kota	2020	2021	2022	2023	2024
1	DKI JAKARTA	Kep. Seribu	60,70	65,45	67,68	73,70	72,58
2	DKI JAKARTA	Kota Jakarta Selatan	63,81	61,44	63,07	66,05	64,81
3	DKI JAKARTA	Kota Jakarta Timur	61,98	60,85	61,26	65,11	64,49
4	DKI JAKARTA	Kota Jakarta Pusat	63,73	63,17	63,60	63,98	65,12
5	DKI JAKARTA	Kota Jakarta Barat	64,37	63,21	63,97	64,80	64,12
6	DKI JAKARTA	Kota Jakarta Utara	65,97	65,79	64,34	65,50	67,77
7	JAWA BARAT	Bogor	62,65	62,55	63,75	64,22	66,30
8	JAWA BARAT	Sukabumi	61,56	64,93	69,11	67,75	69,75
9	JAWA BARAT	Cianjur	69,26	67,98	69,98	72,31	72,63
10	JAWA BARAT	Bandung	62,20	65,12	63,64	67,10	67,22
11	JAWA BARAT	Garut	61,11	62,76	68,84	70,10	71,34
12	JAWA BARAT	Tasikmalaya	69,54	68,18	67,83	68,37	68,76
13	JAWA BARAT	Ciamis	71,41	69,83	68,47	66,26	68,14
14	JAWA BARAT	Kuningan	61,98	63,82	61,80	61,95	65,84
15	JAWA BARAT	Cirebon	63,79	64,49	65,53	66,16	69,27
16	JAWA BARAT	Majalengka	69,66	67,63	66,21	68,50	69,64
17	JAWA BARAT	Sumedang	68,96	68,51	64,63	67,76	69,24
18	JAWA BARAT	Indramayu	69,53	69,86	69,08	63,88	66,43
19	JAWA BARAT	Subang	66,70	67,78	68,87	70,03	71,71
20	JAWA BARAT	Purwakarta	60,91	61,13	65,21	66,37	66,62
21	JAWA BARAT	Karawang	64,90	64,19	65,51	63,40	63,67
22	JAWA BARAT	Bekasi	64,23	65,87	65,41	65,00	65,11
23	JAWA BARAT	Bandung Barat	59,91	60,75	64,37	67,01	68,30
24	JAWA BARAT	Pangandaran	76,79	74,75	79,92	80,15	80,12
25	JAWA BARAT	Kota Bogor	60,81	61,77	64,21	64,81	67,35
26	JAWA BARAT	Kota Sukabumi	55,74	56,86	62,48	62,57	62,58
27	JAWA BARAT	Kota Bandung	64,71	65,31	69,42	66,97	67,93
28	JAWA BARAT	Kota Cirebon	63,76	62,87	65,42	68,71	68,55
29	JAWA BARAT	Kota Bekasi	64,74	64,76	65,33	64,65	65,58
30	JAWA BARAT	Kota Depok	63,96	62,62	63,35	62,76	62,89
31	JAWA BARAT	Kota Cimahi	62,67	61,79	67,22	68,43	66,52
32	JAWA BARAT	Kota Tasikmalaya	66,54	65,75	65,99	65,44	68,92
33	JAWA BARAT	Kota Banjar	67,35	64,07	63,76	67,44	67,22
34	BANTEN	Pandeglang	60,28	62,32	61,66	60,33	66,92
35	BANTEN	Lebak	69,97	66,75	66,17	67,90	72,82
36	BANTEN	Tangerang	65,43	63,94	67,05	66,87	65,88
37	BANTEN	Serang	63,46	63,86	64,25	63,22	67,77
38	BANTEN	Kota Tangerang	64,97	64,52	66,08	63,39	66,02
39	BANTEN	Kota Cilegon	61,83	61,37	67,39	62,70	60,83
40	BANTEN	Kota Serang	63,79	62,16	64,80	68,44	69,10
41	BANTEN	Kota Tangerang Selatan	62,49	62,56	58,67	60,50	58,13

Data Variabel GR

No	Provinsi	Kabupaten/Kota	2020	2021	2022	2023	2024
1	DKI JAKARTA	Kep. Seribu	0,31	0,24	0,21	0,43	0,43
2	DKI JAKARTA	Kota Jakarta Selatan	0,43	0,38	0,38	0,43	0,43
3	DKI JAKARTA	Kota Jakarta Timur	0,36	0,37	0,37	0,43	0,43
4	DKI JAKARTA	Kota Jakarta Pusat	0,39	0,40	0,34	0,43	0,43
5	DKI JAKARTA	Kota Jakarta Barat	0,38	0,37	0,42	0,43	0,43
6	DKI JAKARTA	Kota Jakarta Utara	0,41	0,52	0,54	0,43	0,43
7	JAWA BARAT	Bogor	0,38	0,40	0,40	0,37	0,40
8	JAWA BARAT	Sukabumi	0,33	0,34	0,31	0,35	0,34
9	JAWA BARAT	Cianjur	0,35	0,37	0,31	0,32	0,34
10	JAWA BARAT	Bandung	0,41	0,39	0,37	0,35	0,36
11	JAWA BARAT	Garut	0,35	0,32	0,30	0,36	0,32
12	JAWA BARAT	Tasikmalaya	0,34	0,38	0,38	0,40	0,36
13	JAWA BARAT	Ciamis	0,34	0,34	0,34	0,34	0,33
14	JAWA BARAT	Kuningan	0,36	0,35	0,34	0,35	0,37
15	JAWA BARAT	Cirebon	0,34	0,34	0,35	0,36	0,37
16	JAWA BARAT	Majalengka	0,34	0,35	0,37	0,34	0,35
17	JAWA BARAT	Sumedang	0,38	0,36	0,39	0,37	0,34
18	JAWA BARAT	Indramayu	0,32	0,32	0,30	0,34	0,31
19	JAWA BARAT	Subang	0,35	0,33	0,36	0,33	0,32
20	JAWA BARAT	Purwakarta	0,41	0,39	0,37	0,37	0,40
21	JAWA BARAT	Karawang	0,33	0,37	0,35	0,39	0,38
22	JAWA BARAT	Bekasi	0,37	0,34	0,37	0,40	0,37
23	JAWA BARAT	Bandung Barat	0,38	0,36	0,37	0,37	0,40
24	JAWA BARAT	Pangandaran	0,34	0,32	0,29	0,32	0,29
25	JAWA BARAT	Kota Bogor	0,41	0,44	0,43	0,46	0,48
26	JAWA BARAT	Kota Sukabumi	0,40	0,44	0,48	0,46	0,44
27	JAWA BARAT	Kota Bandung	0,40	0,43	0,46	0,46	0,44
28	JAWA BARAT	Kota Cirebon	0,42	0,49	0,47	0,47	0,42
29	JAWA BARAT	Kota Bekasi	0,35	0,40	0,44	0,41	0,43
30	JAWA BARAT	Kota Depok	0,34	0,37	0,39	0,40	0,40
31	JAWA BARAT	Kota Cimahi	0,38	0,41	0,41	0,39	0,40
32	JAWA BARAT	Kota Tasikmalaya	0,37	0,41	0,41	0,43	0,36
33	JAWA BARAT	Kota Banjar	0,31	0,34	0,36	0,34	0,32
34	BANTEN	Pandeglang	0,30	0,27	0,30	0,24	0,24
35	BANTEN	Lebak	0,31	0,33	0,31	0,27	0,26
36	BANTEN	Tangerang	0,28	0,29	0,28	0,29	0,27
37	BANTEN	Serang	0,30	0,26	0,28	0,27	0,27
38	BANTEN	Kota Tangerang	0,34	0,34	0,38	0,38	0,33
39	BANTEN	Kota Cilegon	0,37	0,36	0,32	0,36	0,36
40	BANTEN	Kota Serang	0,35	0,34	0,35	0,31	0,32
41	BANTEN	Kota Tangerang Selatan	0,37	0,37	0,36	0,36	0,40

Lampiran 2 Kode Program Aplikasi Dashboard

```
from dash import Dash, dcc, html, Input, Output, State, dash_table,
    exceptions, ctx
import base64
import dash
import geopandas as gpd
import io
import json
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import scipy.stats as stats
import os
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
EXTERNAL_STYLESHEETS = ['https://codepen.io/chriddyp/pen/bWLwgP.css']
external_scripts = [
    "https://cdn.jsdelivr.net/npm/mathjax@3/es5/tex-mml-chtml.js"
]

app = Dash(
    __name__,
    external_stylesheets=EXTERNAL_STYLESHEETS,
    external_scripts=external_scripts,
    suppress_callback_exceptions=True
)
server = app.server

# --- GLOBAL MAP INITIALIZATION ---
try:
    print("Sedang membaca Shapefile...")
    # Load shapefile (ensure gadm41_IDN_2.shp exists in project root)
    try:
        gdf = gpd.read_file("gadm41_IDN_2.shp", engine="pyogrio")
    except Exception as geo_e:
        print(f"Gagal memuat dengan pyogrio, mencoba default: {geo_e}")
        gdf = gpd.read_file("gadm41_IDN_2.shp")

    print("Mengoptimalkan/Menyederhanakan geometri peta...")
    gdf['geometry'] = gdf['geometry'].simplify(tolerance=0.005,
       preserve_topology=True)

    # Create matching keys (Eksak sesuai GADM)
    def get_raw_name(row):
        return row.get('NAME_2') or row.get('WADMKK') or
           row.get('NAMOBJ') or row.get('Name') or ""

    gdf['id_cocok'] = gdf.apply(get_raw_name, axis=1)

    indonesia_geojson = json.loads(gdf.to_json())
    print(f"Peta berhasil dimuat! Total Wilayah: {len(gdf)}")
```

```
except Exception as e:
    print(f"Gagal memuat Shapefile: {e}")
    gdf = None
    indonesia_geojson = None

# --- AESTHETIC STYLES ---
math_style = {
    "padding": "24px",
    "backgroundColor": "#F1F5F9",
    "borderLeft": "6px solid #2563EB",
    "marginTop": "15px",
    "borderRadius": "12px",
    "overflowX": "auto",
    "fontSize": "19px",
    "boxShadow": "0 4px 6px -1px rgb(0 0 0 / 0.1)"
}
summary_style = {
    "fontWeight": "bold",
    "cursor": "pointer",
    "color": "#1E40AF",
    "fontSize": "18px",
    "padding": "12px 18px",
    "backgroundColor": "#DBEAFE",
    "borderRadius": "8px",
    "transition": "all 0.2s ease"
}

def render_step(title, content):
    return html.Div([
        html.H6(title, style={"fontWeight": "bold", "marginTop":
            "10px", "color": "#1E293B"}),
        dcc.Markdown(children=content, mathjax=True,
            id=f'math-{hash(content)}')
    ])


def render_diagnostic_formulas(n=None, df_k=None, r_sq=None,
    q_sq=None, kmo_val=None, kmo_str="", bart_chi=None, bart_p=None,
    bart_str=""):

    # --- KMO ---
    kmo_f1 = r"""
$$
KMO = \frac{\sum_{j \neq k} r_{jk}^2}{\sum_{j \neq k} r_{jk}^2 +
    \sum_{j \neq k} q_{jk}^2}
$$
"""
    kmo_f2 = fr"""
$$
KMO = \frac{{{r_sq:.4f}}}{{{r_sq:.4f} + {q_sq:.4f}}}
$$
""" if r_sq is not None else r"""
$$
KMO = \cdots
$$
"""
```

```
    kmo_f3 = fr"""
$$
KMO = {kmo_val:.4f} \quad \text{{({kmo_str})}}
$$
""" if kmo_val is not None else r"""
$$
KMO = \cdots
$$
"""

    # --- Bartlett ---
    bart_f1 = r"""
$$
\chi^2 \approx - \left( n - 1 - \frac{2p + 5}{6} \right) \ln |R|
$$
"""
    if n is not None and df_k is not None:
        bart_f2 = fr"""
$$
\chi^2 \approx - \left( {n} - 1 - \frac{{2({int(df_k)}) + 5}}{{6}}
   \right) \ln |R| = {bart_chi:.4f}
$$
""" if bart_chi is not None else r"""
$$
\cdots
$$
"""
    else:
        bart_f2 = r"""
$$
\chi^2 \approx \cdots
$$
"""

    bart_f3 = fr"""
$$
p = {bart_p} \quad \text{{({bart_str})}}
$$
""" if bart_p is not None else r"""
$$
p = \cdots
$$
"""

    return html.Div([
        html.Details([
            html.Summary(dcc.Markdown("1. Uji Kaiser-Meyer-Olkin (KMO)
               Measure of Sampling Adequacy", mathjax=True,
               style={'display':'inline-block',
               'verticalAlign':'middle'}), style=summary_style),
            html.Div([
                html.P("Tujuan: Mengukur kelayakan data untuk analisis
                   multivariat.", style={'fontStyle': 'italic'}),
                render_step("1. General Formula", kmo_f1),
                render_step("2. Substitution", kmo_f2),
                render_step("3. Final Result", kmo_f3),
```

```
            ], style=math_style)
        ], style={'marginBottom': '10px', 'marginTop': '10px'}),

        html.Details([
            html.Summary(dcc.Markdown("2. Bartlett's Test of
                Sphericity", mathjax=True,
                style={'display':'inline-block',
                'verticalAlign':'middle'}), style=summary_style),
            html.Div([
                html.P("Tujuan: Menguji apakah matriks korelasi
                    berbeda secara signifikan dari matriks
                    identitas.", style={'fontStyle': 'italic'}),
                render_step("1. General Formula", bart_f1),
                render_step("2. Substitution", bart_f2),
                render_step("3. Final Result", bart_f3),
            ], style=math_style)
        ], style={'marginBottom': '10px'})
    ])

def render_statis_formulas(alpha_vals=None, years=None,
    w_c_slice=None, inter_rv=None, t1=None, t2=None):

    # -- RV --
    rv_f1 = r"""
$$
RV(\mathbf{W}_A, \mathbf{W}_B) = \frac{tr(\mathbf{W}_A
    \mathbf{W}_B)}{\sqrt{tr(\mathbf{W}_A^2) tr(\mathbf{W}_B^2)}}
$$
"""
    t_a, t_b = (years[0], years[1]) if years and len(years) > 1 else
        ("A", "B")
    rv_val = inter_rv[0,1] if (inter_rv is not None and
        inter_rv.shape[0]>1) else 0.0

    rv_f2 = fr"""
$$
RV(\mathbf{{W}}_{{{t_a}}}, \mathbf{{W}}_{{{t_b}}}) = \cdots
$$
"""
    rv_f3 = fr"""
$$
RV = {rv_val:.4f}
$$
""" if (inter_rv is not None) else r"""
$$
RV = \cdots
$$
"""

    # -- Wc --
    wc_f1 = r"""
$$
W_c = \sum_{t=1}^{K} \alpha_t W_t
$$
"""
    if alpha_vals is not None and years is not None:
```

```
200         sub_parts = [f"({a:.3f})W_{{{years[i]}}}" for i, a in
                enumerate(alpha_vals)]
201         wc_f2 = fr"""
202 $$
203 W_c = {' + ' .join(sub_parts)}
204 $$
205 """
206     else:
207         wc_f2 = r"""
208 $$
209 W_c = \alpha_1 W_1 + \alpha_2 W_2 \cdots
210 $$
211 """
212
213     if w_c_slice is not None:
214         matrix_rows = [" & ".join([f"{float(v):.2f}" for v in r]) for
                r in w_c_slice]
215         matrix_str = ' \\\\ '.join(matrix_rows)
216         wc_f3 = fr"""
217 $$
218 W_c \approx \begin{{bmatrix}} {matrix_str} \end{{bmatrix}}
219 $$
220 """
221     else:
222         wc_f3 = r"""
223 $$
224 W_c = \cdots
225 $$
226 """
227
228     return html.Div([
229         html.Details([
230             html.Summary(dcc.Markdown("1. Peta Kemiripan Tahunan
                    (Intra-struktur $\mathbf{W}_t$)", mathjax=True,
                    style={'display':'inline-block',
                    'verticalAlign':'middle'}), style=summary_style),
231             html.Div([
232                 html.P("Mengubah data z-score mentah menjadi
                        konfigurasi jarak spasial antar wilayah.",
                        style={'fontStyle': 'italic'}),
233                 render_step("1. General Formula", r"""
234 $$
235 \mathbf{W}_t = \mathbf{Z}_t \mathbf{Z}_t^T
236 $$
237 """)
238             ], style=math_style)
239         ], style={'marginBottom': '10px', 'marginTop': '10px'}),
240
241         html.Details([
242             html.Summary(dcc.Markdown("2. Hubungan Antar-Tahun
                    (Koefisien RV & Bobot $\\alpha$)", mathjax=True,
                    style={'display':'inline-block',
                    'verticalAlign':'middle'}), style=summary_style),
243             html.Div([
244                 html.P("Mengukur tingkat kesamaan (korelasi matriks)
                        antara tiap tahun pengamatan.",
```

```
                    style={'fontStyle': 'italic'}),
                render_step("1. General Formula", rv_f1),
                render_step(f"2. Substitution (Tahun {t_a} vs {t_b})",
                    rv_f2),
                render_step("3. Final Result", rv_f3),
            ], style=math_style)
        ], style={'marginBottom': '10px'}),

        html.Details([
            html.Summary(dcc.Markdown("3. Matriks Kompromi ($W_c$)",
                mathjax=True, style={'display':'inline-block',
                'verticalAlign':'middle'}), style=summary_style),
            html.Div([
                html.P("Dihitung sebagai kombinasi linear dari semua
                    matriks W tahunan dikali bobot utamanya.",
                    style={'fontStyle': 'italic'}),
                render_step("1. General Formula", wc_f1),
                render_step("2. Substitution", wc_f2),
                render_step("3. Final Result (Slice 3x3)", wc_f3),
            ], style=math_style)
        ], style={'marginBottom': '10px'}),

        html.Details([
            html.Summary(dcc.Markdown("4. Dekomposisi Singular (SVD)
                Kompromi", mathjax=True,
                style={'display':'inline-block',
                'verticalAlign':'middle'}), style=summary_style),
            html.Div([
                html.P("Mendapatkan titik kordinat akhir untuk
                    memetakan objek di grafik Biplot.",
                    style={'fontStyle': 'italic'}),
                render_step("1. General Formula (SVD)", r"""
$$
W_c = U S V^T
$$
"""),
                render_step("2. Final Coordinates", r"""
$$
\mathbf{F}_c = U \sqrt{S}
$$
""")
            ], style=math_style)
        ], style={'marginBottom': '10px'})
    ])

def render_cluster_formulas(moran_i=None, moran_exp=None, n=None,
    sum_w=None, sum_w_dev=None, sum_dev_sq=None):

    m_f1 = r"""
$$
I = \frac{N}{\sum_i \sum_j w_{ij}} \frac{\sum_i \sum_j w_{ij} (x_i -
    \bar{x})(x_j - \bar{x})}{\sum_i (x_i - \bar{x})^2}
$$
"""

    if all(v is not None for v in [n, sum_w, sum_w_dev, sum_dev_sq]):
```

```
        m_f2 = fr"""
$$
I = \left( \frac{{{n}}}{{{sum_w:.2f}}} \right)
    \frac{{{sum_w_dev:.2f}}}{{{sum_dev_sq:.2f}}}
$$
"""
    else:
        m_f2 = r"""
$$
I = \cdots
$$
"""

    if moran_i is not None and moran_exp is not None:
        m_f3 = fr"""
$$
I_{{obs}} = {moran_i:.4f} \quad \text{{(Ekspektasi: }} {moran_exp:.4f}
    \text{{)}}
$$
"""
    else:
        m_f3 = r"""
$$
I_{{obs}} = \cdots
$$
"""

    return html.Div([
        html.Details([
            html.Summary(dcc.Markdown("1. Global Moran\'s I
                (Autokorelasi Spasial PC1)", mathjax=True,
                style={'display':'inline-block',
                'verticalAlign':'middle'}), style=summary_style),
            html.Div([
                render_step("1. General Formula", m_f1),
                render_step("2. Substitution", m_f2),
                render_step("3. Final Result", m_f3),
            ], style=math_style)
        ], style={'marginBottom': '10px', 'marginTop': '10px'}),

        html.Details([
            html.Summary(dcc.Markdown("2. Fungsi Objektif K-Means",
                mathjax=True, style={'display':'inline-block',
                'verticalAlign':'middle'}), style=summary_style),
            html.Div([
                render_step("1. General Formula", r"""
$$
J = \sum_{j=1}^{k} \sum_{i \in C_j} ||\mathbf{x}_i^{(traj)} -
    \mathbf{\mu}_j||^2
$$
""")
            ], style=math_style)
        ], style={'marginBottom': '10px'}),

        html.Details([
```

```
335             html.Summary(dcc.Markdown("3. Jarak Trajektori
                    Spasio-Temporal", mathjax=True,
                    style={'display':'inline-block',
                    'verticalAlign':'middle'}), style=summary_style),
336             html.Div([
337                 html.P("Mengukur seberapa jauh perbedaan lintas objek
                        sepanjang tahun", style={'fontStyle': 'italic'}),
338                 render_step("1. General Formula", r"""
339 $$
340 d(i,j) = \sqrt{\sum_t (x_{it}-x_{jt})^2}
341 $$
342 """)
343             ], style=math_style)
344         ], style={'marginBottom': '10px'})
345     ])
346
347 def render_permutation_formulas(p_val=None, lambda_obs=None,
        n_perm=None, interpretation=""):
348
349     f1 = r"""
350 $$
351 P(\lambda \geq \lambda_{obs}) = \frac{\#\{\lambda^* \geq
        \lambda_{obs}\} + 1}{N_{perm} + 1}
352 $$
353 """
354
355     if p_val is not None and lambda_obs is not None and n_perm is not
            None:
356         count_val = int(round(p_val * (n_perm + 1))) - 1
357         f2 = fr"""
358 $$
359 P(\lambda \geq {lambda_obs:.4f}) = \frac{{{count_val} + 1}}{{{n_perm}
        + 1}} = {p_val:.4f}
360 $$
361 """
362     else:
363         f2 = r"""
364 $$
365 P(\lambda \geq \lambda_{obs}) = \cdots
366 $$
367 """
368
369     f3 = fr"""
370 $$
371 \text{{P-value}} = {p_val:.4f}
372 $$
373 """ if p_val is not None else r"""
374 $$
375 \text{P-value} = \cdots
376 $$
377 """
378
379     decision = ""
380     if p_val is not None:
381         if p_val < 0.05:
382             decision = fr"""
```

```
$$
\text{{Keputusan: Tolak }} H_0 \text{{ (Signifikan/Stabil)}}
$$
"""
        else:
            decision = fr"""
$$
\text{{Keputusan: Gagal Tolak }} H_0 \text{{ (Tidak Signifikan/Acak)}}
$$
"""

    return html.Div([
        html.Details([
            html.Summary(dcc.Markdown("1. Formula Uji Permutasi",
                mathjax=True, style={'display':'inline-block',
                'verticalAlign':'middle'}), style=summary_style),
            html.Div([
                render_step("1. Probabilitas Empiris", f1)
            ], style=math_style)
        ], style={'marginBottom': '10px', 'marginTop': '10px'}),

        html.Details([
            html.Summary(dcc.Markdown("2. Substitusi & Perhitungan",
                mathjax=True, style={'display':'inline-block',
                'verticalAlign':'middle'}), style=summary_style),
            html.Div([
                render_step("1. Nilai Observasi & Jumlah Sampel", f2)
            ], style=math_style)
        ], style={'marginBottom': '10px'}),

        html.Details([
            html.Summary(dcc.Markdown("3. Hasil Akhir & Keputusan
                Hipotesis", mathjax=True,
                style={'display':'inline-block',
                'verticalAlign':'middle'}), style=summary_style),
            html.Div([
                render_step("1. Final P-value", f3),
                render_step("2. Kesimpulan Statistik", decision)
            ], style=math_style)
        ], style={'marginBottom': '10px'})
    ])


def render_descriptive_formulas():
    """Returns a collapsible section showing formulas for all
       descriptive statistics."""
    formulas = {
        "Rata-rata (Mean)": (
            r"""
$$
\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i
$$
""",
            "Nilai rata-rata dari seluruh observasi pada setiap
                variabel."
        ),
```

```
        "Simpangan Baku (Std. Dev)": (
            r"""
$$
s = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2}
$$
""",
            "Ukuran penyebaran data terhadap rata-ratanya."
        ),
        "Nilai Minimum & Maksimum": (
            r"""
$$
\text{Min} = \min(x_1, \ldots, x_n), \quad \text{Max} = \max(x_1,
    \ldots, x_n)
$$
""",
            "Nilai terkecil dan terbesar dalam data."
        ),
        "Median (Q2)": (
            r"""
$$
Q_2 = \begin{cases} x_{\left(\frac{n+1}{2}\right)} & n \text{ ganjil}
    \\ \frac{x_{\left(\frac{n}{2}\right)} +
    x_{\left(\frac{n}{2}+1\right)}}{2} & n \text{ genap} \end{cases}
$$
""",
            "Nilai tengah data setelah diurutkan."
        ),
        "Kuartil & IQR": (
            r"""
$$
IQR = Q_3 - Q_1
$$
""",
            "Jarak antar-kuartil  ukuran penyebaran yang robust
                terhadap pencilan."
        ),
        "Skewness (Kemiringan)": (
            r"""
$$
g_1 = \frac{\frac{1}{n}\sum_{i=1}^n (x_i - \bar{x})^3}{s^3}
$$
""",
            "$g_1 > 0$: condong kanan (right-skewed); $g_1 < 0$:
                condong kiri."
        ),
        "Kurtosis (Keruncingan)": (
            r"""
$$
g_2 = \frac{\frac{1}{n}\sum_{i=1}^n (x_i - \bar{x})^4}{s^4} - 3
$$
""",
            "$g_2 > 0$: lebih runcing dari normal (leptokurtik); $g_2
                < 0$: lebih datar (platikurtik)."
        ),
        "Koefisien Variasi (CV)": (
            r"""
```

```
$$
CV = \frac{s}{\bar{x}} \times 100\%
$$
""",
            "Ukuran variasi relatif  berguna untuk membandingkan variabelitas antar-variabel bersatuan berbeda."
        ),
    }

    items = []
    for i, (name, (formula, desc)) in enumerate(formulas.items(), 1):
        items.append(
            html.Details([
                html.Summary(
                    dcc.Markdown(f"{i}. {name}", mathjax=True,
                                 style={'display': 'inline-block',
                                        'verticalAlign': 'middle'}),
                    style=summary_style
                ),
                html.Div([
                    dcc.Markdown(desc, mathjax=True,
                                 style={'fontStyle': 'italic',
                                        'color': '#455A64',
                                        'marginBottom': '5px'}),
                    render_step("Formula", formula)
                ], style=math_style)
            ], style={'marginBottom': '8px'})
        )

    return html.Div(items, style={'marginTop': '20px'})


# ========================================

# Import Library

COLOR_ACCENT = '#1976D2'




# Helper Functions

# ----------------------------
def get_rotation_angle(df_pc, df_load, baseline_type, baseline_name, year):
    try:
        vec = np.array([0.0, 0.0])
        if baseline_type == 'skor':
            sub = df_pc[(df_pc['Kabupaten/Kota'] == baseline_name) & (df_pc['Tahun'] == year)]
            if not sub.empty: vec = sub[['PC1', 'PC2']].iloc[0].values
        elif baseline_type == 'loading':
            sub = df_load[(df_load['features'] == baseline_name) & (df_load['Tahun'] == year)]
            if not sub.empty: vec = sub[['PC1', 'PC2']].iloc[0].values
```

```
        if np.allclose(vec, 0): return 0.0
        return -np.arctan2(vec[1], vec[0])
    except: return 0.0

def rotate_dataframe(df, angle, cols=['PC1', 'PC2']):
    df_rot = df.copy()
    if len(cols) < 2: return df_rot
    c, s = np.cos(angle), np.sin(angle)
    R = np.array([[c, -s], [s, c]])
    vals = df_rot[cols].values
    df_rot[cols] = vals.dot(R.T)
    return df_rot

def _calculate_R_matrix(Ws_list, k):
    R = np.zeros((k, k))
    for i in range(k):
        for j in range(k):
            num = np.trace(Ws_list[i].T @ Ws_list[j])
            den = np.sqrt(np.trace(Ws_list[i].T @ Ws_list[i]) *
                np.trace(Ws_list[j].T @ Ws_list[j]))
            R[i, j] = 0.0 if den == 0 else num / den
    return R

def _calculate_moran_manual(gdf, value_analytics, obs_names_analytics):
    """
    Hitung Moran's I secara manual menggunakan Geopandas adjacency
        (Touches).
    Tanpa library pysal/esda agar ringan.
    """
    try:
        if gdf is None: return None, None, None, None, None, None

        # 1. Match Data dengan GeoData (Eksak)
        geom_map = {}
        for _, row in gdf.iterrows():
            name = str(row['id_cocok'])
            if name: geom_map[name] = row['geometry']

        # 2. Re-construct Series data yang urut
        y, geoms = [], []
        for i, name in enumerate(obs_names_analytics):
            if name in geom_map:
                y.append(value_analytics[i])
                geoms.append(geom_map[name])

        if len(y) < 5:
            return None, None, None, None, None, None

        y = np.array(y)
        n = len(y)

        # 3. Buat Weight Matrix (W) berdasarkan sentuhan (Queen
            Contiguity)
        W = np.zeros((n, n))
        temp_gdf = gpd.GeoDataFrame({'geometry': geoms})
        for i in range(n):
```

```
582             for j in range(n):
583                 if i != j:
584                     if
                           temp_gdf.iloc[i].geometry.touches(temp_gdf.iloc[j].geometry):
585                         W[i, j] = 1.0
586
587         # Row Standardization
588         row_sums = W.sum(axis=1)
589         with np.errstate(divide='ignore', invalid='ignore'):
590             W_std = W / row_sums[:, None]
591         W_std[np.isnan(W_std)] = 0.0
592
593         # 4. Hitung Moran's I
594         y_bar = np.mean(y)
595         y_dev = y - y_bar
596         sum_sq_dev = np.sum(y_dev**2)
597
598         sum_w = np.sum(W)
599         if sum_w == 0 or sum_sq_dev == 0: return 0.0, -1.0/(n-1), n,
               0.0, 0.0, 0.0
600
601         numerator = 0.0
602         for i in range(n):
603             for j in range(n):
604                 numerator += W_std[i, j] * y_dev[i] * y_dev[j]
605
606         moran_i = (n / sum_w) * (numerator / sum_sq_dev)
607         expected_i = -1.0 / (n - 1)
608
609         return moran_i, expected_i, n, sum_w, numerator, sum_sq_dev
610
611     except Exception as e:
612         print(f"Moran Error: {e}")
613         return None, None, None, None, None, None
614
615 def _get_first_eigenvalue(R_matrix):
616     vals = np.linalg.eigvals(R_matrix)
617     return np.sort(np.real(vals))[-1]
618
619 # --- Helper untuk Dendrogram ke Plotly ---
620 # --- Redundant function removed ---
621
622 def create_dendrogram_plot(X, labels):
623     # Hitung Linkage Matrix
624     Z = linkage(X, 'ward')
625     # Create dendrogram figure factory is heavy, manual approach for
          speed:
626     # Kita pakai Plotly Figure Factory (ff) kalau ada, tapi biar
          ringan kita pakai Scipy text logic atau simple plot.
627     # Agar simple dan cantik, kita pakai library plotly.figure_factory
          jika diinstall.
628     # TAPI KARENA SAYA TIDAK TAHU APAKAH ANDA PUNYA, SAYA PAKAI LOGIKA
          MANUAL YANG AMAN:
629
630     import plotly.figure_factory as ff
631     try:
```

```
632         fig = ff.create_dendrogram(X, labels=labels,
                orientation='left')
633         fig.update_layout(template='plotly_white', margin=dict(l=150,
                r=20, t=20, b=20), height=450)
634         return fig
635     except:
636         # Fallback jika library kurang lengkap (jarang terjadi di env
                standar)
637         fig = go.Figure()
638         fig.add_annotation(text="Mohon install 'scipy' agar dendrogram
                muncul.", showarrow=False)
639         return fig
640
641 # ------------------------------
642
643 # LATEX EXPORT HELPERS
644
645 # ------------------------------
646 def export_core_latex(res):
647     try:
648         lines = []
649         lines.append(r"% Generated by final.py")
650         lines.append(r"\newcommand{\statisDatasetName}{Data
                Statistik}")
651
652         # Alphas
653         alphas = res.get('alphas', [])
654         years = res.get('sheet_names', [])
655         labels = ['A', 'B', 'C', 'D', 'E', 'F', 'G']
656         for i, (yr, val) in enumerate(zip(years, alphas)):
657             if i < len(labels):
658                 lines.append(fr"\newcommand{{\statisAlphaTahun{labels[i]}}}{{{val:.4f}
                        % {yr}")
659
660         # Inertia (Dim 1)
661         eig_vals = np.array(res.get('eig_vals_c', [0]))
662         if len(eig_vals) > 0:
663             val = eig_vals[0]
664             pct = 100 * val / np.sum(eig_vals)
665             lines.append(fr"\newcommand{{\statisEigenA}}{{{val:.4f}}}")
666             lines.append(fr"\newcommand{{\statisInertiaA}}{{{pct:.2f}}}")
667
668         # Simpan di parent directory agar sejajar dengan file LaTeX
                utama
669         os.makedirs("../huggingface", exist_ok=True)
670         with open("../huggingface/output_core_statis.tex", "w") as f:
671             f.write("\n".join(lines))
672         print("LaTeX Core Exported to ../output_core_statis.tex!")
673     except Exception as e:
674         print(f"Failed to export LaTeX Core: {e}")
675
676 def export_tables_latex(res):
677     """
678     Exports STATIS tables to SEPARATE LaTeX files for contextual
            embedding.
679     """
```

```
680     try:
681         # Helper to write individual table file
682         def write_single_table(data_list, filename, caption, label,
                columns=None, float_fmt="{:.4f}", longtable=False):
683             if not data_list: return
684 
685             # Sanitizer for LaTeX
686             def escape_tex(val):
687                 if isinstance(val, (int, float)): return val
688                 s = str(val)
689                 chars = {'_': r'\_', '%': r'\%', '$': r'\$', '#':
                        r'\#', '&': r'\&'}
690                 for c, r in chars.items():
691                     s = s.replace(c, r)
692                 return s
693 
694             df = pd.DataFrame(data_list)
695             if columns: df = df[columns]
696 
697             lines = []
698             if longtable:
699                 # LONGTABLE logic
700                 lines.append(r"\begin{longtable}{|" +
                        "c|"*len(df.columns) + "}")
701                 lines.append(r"\caption{" + caption + "} \label{tab:"
                        + label + "} \\\\")
702                 lines.append(r"\hline")
703                 # Escape headers
704                 header = " & ".join([f"\\textbf{{{escape_tex(c)}}}"
                        for c in df.columns]) + " \\\\"
705                 lines.append(header)
706                 lines.append(r"\hline")
707                 lines.append(r"\endfirsthead")
708                 lines.append(r"\hline")
709                 lines.append(header)
710                 lines.append(r"\hline")
711                 lines.append(r"\endhead")
712                 lines.append(r"\hline")
713                 lines.append(r"\endfoot")
714 
715                 for _, row in df.iterrows():
716                     # Escape row values
717                     row_str = " & ".join([float_fmt.format(val) if
                            isinstance(val, (int, float)) else
                            escape_tex(val) for val in row])
718                     lines.append(row_str + " \\\\")
719 
720                 lines.append(r"\hline")
721                 lines.append(r"\end{longtable}")
722             else:
723                 # STANDARD TABLE logic
724                 lines.append(r"\begin{table}[H]")
725                 lines.append(r"\centering")
726                 lines.append(r"\caption{" + caption + "} \label{tab:"
                        + label + "}")
```

```
                lines.append(r"\begin{tabular}{|" +
                    "c|"*len(df.columns) + "}")
                lines.append(r"\hline")
                # Escape headers
                header = " & ".join([f"\\textbf{{{escape_tex(c)}}}"
                    for c in df.columns]) + " \\\\"
                lines.append(header)
                lines.append(r"\hline")

                for _, row in df.iterrows():
                    # Escape row values
                    row_str = " & ".join([float_fmt.format(val) if
                        isinstance(val, (int, float)) else
                        escape_tex(val) for val in row])
                    lines.append(row_str + " \\\\")

                lines.append(r"\hline")
                lines.append(r"\end{tabular}")
                lines.append(r"\end{table}")

            # Write to specific file
            os.makedirs("../huggingface", exist_ok=True)
            with open(f"../huggingface/output_table_{filename}.tex",
                "w") as f:
                f.write("\n".join(lines))
            print(f"Exported ../output_table_{filename}.tex")

        # 1. Bobot Alpha
        write_single_table(res.get('Bobot (Alpha)'), "alpha", "Nilai
            Bobot (Alpha)", "alpha")

        # 2. Nilai Eigen
        write_single_table(res.get('Nilai Eigen (Lambda)'), "eigen",
            "Dekomposisi Nilai Eigen", "eigen")

        # 3. Matriks R
        write_single_table(res.get('Matriks Inter-struktur (R)'),
            "rv", "Koefisien RV Antar-tahun", "rv")

        # 4. Loadings
        write_single_table(res.get('Loadings Kompromi (Gc)'),
            "loadings", "Loadings Kompromi", "loadings")

        # 5. Skor (Longtable)
        write_single_table(res.get('Skor Kompromi (Fc)'), "scores",
            "Koordinat Skor Kompromi", "scores", longtable=True)

    except Exception as e:
        print(f"Failed to export Split Tables: {e}")

def export_perm_latex(p_val, desc):
    try:
        lines = []
        lines.append(r"% Generated by final.py (Permutation)")
        lines.append(fr"\newcommand{{\statisPValue}}{{{p_val:.4f}}}")
        lines.append(fr"\newcommand{{\statisKet}}{{{desc}}}")
```

```
773
774         os.makedirs("../huggingface", exist_ok=True)
775         with open("../huggingface/output_perm_statis.tex", "w") as f:
776             f.write("\n".join(lines))
777         print("LaTeX Perm Exported to ../output_perm_statis.tex!")
778     except Exception as e:
779         print(f"Failed to export LaTeX Perm: {e}")
780
781 # ----------------------------
782
783 # PLOT HELPERS (With Visual Scaling)
784
785 # ----------------------------
786 def apply_visual_adjustments(scores, loadings, norm_scores,
    scale_loadings):
787     """
788     Melakukan scaling HANYA untuk plotting, tidak mengubah data tabel.
789     """
790     plot_scores = scores.copy()
791     plot_loadings = loadings.copy()
792
793     # 1. Normalize Scores (Optional)
794     if norm_scores:
795         max_val = np.max(np.abs(plot_scores))
796         if max_val > 0: plot_scores = plot_scores / max_val
797
798     # 2. Scale Loadings (Visual scaling to match score magnitude)
799     if scale_loadings:
800         max_score_mag = np.max(np.abs(plot_scores))
801         max_load_mag = np.max(np.abs(plot_loadings))
802         if max_load_mag > 0:
803             scale_factor = (max_score_mag / max_load_mag) * 0.8
804             plot_loadings = plot_loadings * scale_factor
805
806     return plot_scores, plot_loadings
807
808 def create_biplot(scores, loadings, labels, vars, title,
    color=COLOR_ACCENT, show_loadings=True, show_points=True,
    show_arrows=True):
809     fig = go.Figure()
810
811     # Points
812     if show_points:
813         fig.add_trace(go.Scatter(x=scores[:,0], y=scores[:,1],
                mode='markers+text',
814                                  text=labels, textposition='top
                                     center',
815                                  marker=dict(size=8, color=color,
                                     opacity=0.8), name='Objek'))
816
817     # Vectors
818     if show_loadings and len(vars) > 0:
819         for i, var in enumerate(vars):
820             marker_opts = dict(size=6, symbol='arrow-bar-up') if
                show_arrows else dict(size=0)
```

```
821             fig.add_trace(go.Scatter(x=[0, loadings[i,0]], y=[0,
                    loadings[i,1]],
822                                      mode='lines+markers+text',
                                             marker=marker_opts,
823                                      line=dict(color='#D81B60',
                                             width=2), text=[None, var],
824                                      textposition='top center',
                                             name=f'{var}',
825                                      hoverinfo='text',
                                             hovertext=f"<b>{var}</b><br>Loading:
                                             ({loadings[i, 0]:.4f},
                                             {loadings[i, 1]:.4f})"))
826
827     fig.update_layout(title=title, xaxis=dict(constrain='domain'),
            yaxis=dict(scaleanchor='x', scaleratio=1),
828                        width=600, height=600, autosize=False,
                               margin=dict(l=40, r=40, t=60, b=40),
                               showlegend=False,
829                        template="plotly_white")
830     return fig
831
832 # ----------------------------
833 # DATA DIAGNOSTICS HELPERS
834 # ----------------------------
835
836 def calc_kmo_bartlett(df_dict, variables):
837     try:
838         kmos, p_vals = [], []
839         n, p, r_sq, q_sq, chi2_stat = 0, 0, 0, 0, 0
840         for sheet, df in df_dict.items():
841             if len(variables) < 2: return 0, 1.0, "Need >1 var", "Need
                    >1 var"
842             data = df[variables].values
843             R = np.corrcoef(data.T)
844
845             # KMO
846             try:
847                 inv_R = np.linalg.inv(R)
848                 D = np.diag(1.0 / np.sqrt(np.diag(inv_R)))
849                 Q = D @ inv_R @ D
850                 r_sq = np.sum(R**2) - np.sum(np.diag(R**2))
851                 q_sq = np.sum(Q**2) - np.sum(np.diag(Q**2))
852                 kmos.append(r_sq / (r_sq + q_sq) if (r_sq + q_sq) > 0
                        else 0)
853             except: kmos.append(0)
854
855             # Bartlett
856             n, p = data.shape
857             det_R = np.linalg.det(R)
858             if det_R > 0:
859                 chi2_stat = - (n - 1 - (2*p + 5)/6) * np.log(det_R)
860                 df_chi = p * (p - 1) / 2
861                 p_val = stats.chi2.sf(chi2_stat, df_chi)
862                 p_vals.append(p_val)
863             else: p_vals.append(1.0)
864
```

```
865         mean_kmo = np.mean(kmos)
866         mean_p = np.mean(p_vals)
867         kmo_interp = "Excellent" if mean_kmo >= 0.9 else "Great" if
                mean_kmo >= 0.8 else "Good" if mean_kmo >= 0.7 else
                "Acceptable" if mean_kmo >= 0.5 else "Poor"
868         bart_interp = "Significant (Suitable)" if mean_p < 0.05 else
                "Not Significant (Orthogonal)"
869         return mean_kmo, mean_p, kmo_interp, bart_interp, n, p, r_sq,
                q_sq, chi2_stat
870     except: return 0.0, 1.0, "Error", "Error", 0, 0, 0, 0, 0
871
872 # ----------------------------
873
874
875 # ========================================
876
877 # Import Library
878
879
880
881 # STATIS Core Logic
882 # DATA DIAGNOSTICS HELPERS
883
884 # ----------------------------
885
886 # ----------------------------
887
888 # ----------------------------
889
890
891 # ----------------------------
892 def run_statis_core(uploaded_content_string, selected_vars,
        sheet_names):
893     """
894     Menjalankan perhitungan STATIS Lengkap (Matriks, Skor, Loading).
895     Mengembalikan data siap plot dan siap tabel.
896     """
897     if uploaded_content_string is None: raise Exception("File belum
            di-upload.")
898
899     # 1. Load Data
900     try:
901         _, content_string = uploaded_content_string.split(',')
902         decoded = base64.b64decode(content_string)
903         excel_file_stream = io.BytesIO(decoded)
904     except Exception as e: raise Exception(f"Error decoding file: {e}")
905
906     tables = []
907     for s in sheet_names:
908         excel_file_stream.seek(0)
909         df_s = pd.read_excel(excel_file_stream,
                sheet_name=s).set_index("Kabupaten/Kota")
910         vars_in_sheet = [v for v in selected_vars if v in df_s.columns]
911         if vars_in_sheet: tables.append(df_s[vars_in_sheet].copy())
912
```

```
    if not tables: raise Exception("Data tidak valid atau variabel tidak ditemukan.")

    # 2. Align Indices
    if not all(tables[0].index.equals(t.index) for t in tables):
        common = tables[0].index
        for t in tables: common = common.intersection(t.index)
        tables = [t.loc[common].copy() for t in tables]

    obs_labels = tables[0].index.tolist()
    selected_vars_final = tables[0].columns.tolist()
    k = len(tables)

    # 3. Standardisasi & Intra-struktur (W)
    scaler_statis = StandardScaler()
    tables_std, X_matrices, Ws = [], [], []

    for t in tables:
        arr = t.values.astype(float)
        arr_scaled = scaler_statis.fit_transform(arr) # Z-score
        tables_std.append(pd.DataFrame(arr_scaled, index=t.index, columns=t.columns))
        X_matrices.append(arr_scaled)
        Ws.append(arr_scaled @ arr_scaled.T) # W = Z * Z'

    # 4. Inter-struktur (R)
    R = _calculate_R_matrix(Ws, k)
    vals_R, vecs_R = np.linalg.eig(R)
    vals_R, vecs_R = np.real(vals_R), np.real(vecs_R)

    # Sort Eigen
    idx = np.argsort(vals_R)[::-1]
    vals_R_sorted, vecs_R_sorted = vals_R[idx], vecs_R[:, idx]

    # Bobot (Alpha)
    p1 = vecs_R_sorted[:,0]
    if p1.sum() < 0: p1 = -p1
    alphas = p1 / p1.sum()

    # 5. Kompromi (Wc)
    Wc = np.zeros_like(Ws[0])
    for a, W in zip(alphas, Ws): Wc += a * W

    # 6. Dekomposisi Wc (Mendapatkan Skor Fc) menggunakan SVD
    # Memperbaiki error kompleks di server linux (Hugging Face)
    U, S, Vt = np.linalg.svd(Wc)
    eig_vals_c = S
    eig_vecs_c = U
    idxc = np.argsort(eig_vals_c)[::-1]
    eig_vals_c, eig_vecs_c = eig_vals_c[idxc], eig_vecs_c[:, idxc]

    # Skor Kompromi (Fc) - Raw (Sesuai Teori)
    # Ambil 2 komponen pertama untuk plot
    sq_2 = np.sqrt(np.clip(eig_vals_c[:2], a_min=0, a_max=None))
    scores_raw = eig_vecs_c[:, :2] * sq_2

```

```
    # Ambil full components untuk tabel
    sq_full = np.sqrt(np.clip(eig_vals_c, a_min=0, a_max=None))

    # 7. Loadings (Gc) - Korelasi Var vs Skor
    loadings = []
    for i, var in enumerate(selected_vars_final):
        arrs = [t[var].values for t in tables_std]
        # Rata-rata variabel antar tahun (untuk korelasi dengan
            Kompromi)
        x = np.mean(np.stack(arrs, axis=1), axis=1)
        if np.allclose(x, 0): cx, cy = 0.0, 0.0
        else:
            cx = np.corrcoef(x, scores_raw[:,0])[0,1] if np.std(x)!=0
                and np.std(scores_raw[:,0])!=0 else 0
            cy = np.corrcoef(x, scores_raw[:,1])[0,1] if np.std(x)!=0
                and np.std(scores_raw[:,1])!=0 else 0
            cx = 0.0 if np.isnan(cx) else cx
            cy = 0.0 if np.isnan(cy) else cy
        loadings.append([cx, cy])
    loadings = np.array(loadings)

    # 8. Trajectories Coordinates (Projected)
    per_year_coords = {}
    for year, dfstd in zip(sheet_names, tables_std):
        # Proyeksi Data Tahun t ke Loadings Kompromi
        per_year_coords[year] = dfstd.values @ loadings

    # --- PERSIAPAN DATAFRAME UNTUK TABEL DETAIL ---

    # A. Bobot
    df_alphas = pd.DataFrame({'Tahun': sheet_names, 'Bobot (Alpha)':
        [round(a, 4) for a in alphas]})

    # B. Eigenvalues
    var_expl = eig_vals_c / eig_vals_c.sum() * 100
    df_eigen = pd.DataFrame({
        'Dim': [f'Dim {i+1}' for i in range(len(eig_vals_c))],
        'Eigen': [round(v, 4) for v in eig_vals_c],
        'Akar Ciri': [round(v, 4) for v in sq_full],
        'Var (%)': [round(v, 2) for v in var_expl],
        'Kum (%)': [round(v, 2) for v in np.cumsum(var_expl)]
    })

    # C. Vektor Eigen (Vc)
    df_Vc = pd.DataFrame(eig_vecs_c, index=obs_labels, columns=[f"Vec
        {i+1}" for i in range(len(obs_labels))])
    df_Vc.index.name = 'Kab/Kota'; df_Vc = df_Vc.reset_index()

    # D. Skor (Fc)
    df_Fc = pd.DataFrame(scores_raw, index=obs_labels, columns=['PC1',
        'PC2'])
    df_Fc.index.name = 'Kab/Kota'; df_Fc = df_Fc.reset_index()

    # E. Loadings (Gc)
    df_Gc = pd.DataFrame(loadings, index=selected_vars_final,
        columns=['PC1', 'PC2'])
```

```
    df_Gc.index.name = 'Variabel'; df_Gc = df_Gc.reset_index()

    # F. Matriks R
    df_R = pd.DataFrame(R, index=sheet_names, columns=sheet_names)
    df_R.index.name = 'Tahun'; df_R = df_R.reset_index()

    # G. Matriks Wc
    df_Wc = pd.DataFrame(Wc, index=obs_labels, columns=obs_labels)
    df_Wc.index.name = 'Kab/Kota'; df_Wc = df_Wc.reset_index()

    # H. Dataset Gabungan
    df_dataset = pd.concat([t.add_suffix(f" ({n})") for t, n in
        zip(tables, sheet_names)], axis=1)
    df_dataset.index.name = 'Kab/Kota'; df_dataset =
        df_dataset.reset_index()

    # I. Intra-Matrices Dictionary
    intra_dicts = {}
    for i, yr in enumerate(sheet_names):
        df_wt = pd.DataFrame(Ws[i], index=obs_labels,
            columns=obs_labels)
        df_wt.index.name = 'Kab/Kota'; df_wt = df_wt.reset_index()
        intra_dicts[f"Matriks Intra ({yr})"] = df_wt.to_dict('records')

    # Return Result Dictionary
    res = {
        # Core Data
        'R': R, 'alphas': alphas, 'eig_vals_c': eig_vals_c, 'scores':
            scores_raw,
        'obs_labels': obs_labels, 'selected_vars':
            selected_vars_final, 'loadings': loadings,
        'per_year_coords': per_year_coords, 'sheet_names': sheet_names,
        'X_matrices': X_matrices, 'vals_R_sorted': vals_R_sorted, 'k':
            k, # For Permutation

        # Table Data Records
        'Dataset Asli (X)': df_dataset.to_dict('records'),
        'Bobot (Alpha)': df_alphas.to_dict('records'),
        'Nilai Eigen (Lambda)': df_eigen.to_dict('records'),
        'Vektor Eigen (Vc)': df_Vc.to_dict('records'),
        'Skor Kompromi (Fc)': df_Fc.to_dict('records'),
        'Loadings Kompromi (Gc)': df_Gc.to_dict('records'),
        'Matriks Inter-struktur (R)': df_R.to_dict('records'),
        'Matriks Kompromi (Wc)': df_Wc.to_dict('records')
    }
    res.update(intra_dicts)
    return res

def run_permutation_test(X_matrices, k, lambda_obs, n_permutations):
    dist_lambdas = []
    for _ in range(n_permutations):
        Ws_perm = []
        for X in X_matrices:
            X_perm = np.random.permutation(X) # Shuffle baris
            Ws_perm.append(X_perm @ X_perm.T)
        R_perm = _calculate_R_matrix(Ws_perm, k)
```

```
        dist_lambdas.append(_get_first_eigenvalue(R_perm))

    dist_lambdas = np.array(dist_lambdas)
    # P-value (with +1 correction)
    p_value = (np.sum(dist_lambdas >= lambda_obs) + 1) /
        (n_permutations + 1)
    return dist_lambdas, p_value

# ----------------------------


# ========================================

# Import Library



pd.set_option('display.max_columns', None)
scaler = StandardScaler()
pca = PCA(n_components=2)


COLOR_BG_MAIN = '#F0F4F8'
COLOR_CARD_BG = '#FFFFFF'
COLOR_HEADER_TEXT = '#1565C0'
COLOR_ACCENT = '#1976D2'
COLOR_BORDER = '#BBDEFB'
CARD_STYLE = {'boxShadow': '0 2px 5px 0 rgba(0,0,0,0.05)',
    'borderRadius': '8px', 'backgroundColor': COLOR_CARD_BG,
    'padding': '25px', 'marginBottom': '25px', 'border': f'1px solid
    {COLOR_BORDER}'}
HEADER_STYLE = {'textAlign': 'center', 'color': COLOR_HEADER_TEXT,
    'fontWeight': '700', 'marginBottom': '15px', 'letterSpacing':
    '0.5px'}
LABEL_STYLE = {'fontWeight': '600', 'color': '#37474F', 'fontSize':
    '1.4rem', 'marginBottom': '8px'}
BASE_URL =
    "https://huggingface.co/spaces/ammarhnfi/skripsi/resolve/main"
EXAMPLE_DATASETS = [{"name": "Data Gabungan (FINAL DATA.xlsx)", "url":
    f"{BASE_URL}/FINAL DATA.xlsx"}]
layout = html.Div(style={'backgroundColor': COLOR_BG_MAIN, 'padding':
    '30px', 'fontFamily': 'Segoe UI, Roboto, Helvetica, Arial,
    sans-serif'}, children=[
    dcc.Store(id='global-data-store'),
    dcc.Store(id='pca-results-store'),
    dcc.Store(id='statis-core-store'), # Menyimpan hasil STATIS lengkap
    dcc.Store(id='descriptive-store'), # Menyimpan hasil statistika
        deskriptif

    # --- HEADER ---
    html.Div(style=CARD_STYLE, children=[
        html.H1('Dashboard Analisis Spasio-Temporal Sosio-Ekonomi',
            style=HEADER_STYLE),
        html.H2('(Metode STATIS & Dynamic Biplot)',
            style={'textAlign': 'center', 'color': '#546E7A',
            'marginTop': '-10px'}),
```

```
        html.H3("Studi Kasus: Provinsi Banten, DKI Jakarta, dan Jawa Barat (2020-2024)", style={'textAlign': 'center', 'color': '#78909C', 'fontStyle': 'italic'})
    ]),

    # --- 1. DATA INPUT ---
    html.Div(style=CARD_STYLE, children=[
        html.H5("1. Input Data", style=LABEL_STYLE),

        # --- [TAMBAHAN] INFO FORMAT EXCEL ---
        html.Div(style={'backgroundColor': '#E3F2FD', 'borderLeft': '5px solid #2196F3', 'padding': '15px', 'marginBottom': '20px', 'borderRadius': '4px'}, children=[
            html.H6(" Ketentuan Format File Excel:", style={'fontWeight': 'bold', 'marginTop': '0', 'color': '#0D47A1'}),
            html.Ul(style={'marginBottom': '0', 'fontSize': '14px', 'color': '#1565C0'}, children=[
                html.Li("File harus memiliki kolom wajib: 'Kabupaten/Kota' dan 'Provinsi'."),
                html.Li("Setiap Sheet merepresentasikan Satu Tahun dan harus dinamai dengan angka tahun (contoh: 2020, 2021)."),
                html.Li("Pastikan tidak ada sel yang di-merge pada header.")
            ])
        ]),
        # ------------------------------------

        html.Div([
            html.Span("Download Contoh Dataset: ", style={'marginRight':'10px', 'color': '#455A64'}),
            *[html.A(d['name'], href=d['url'], target="_blank", download=d['name'], style={'marginRight':'15px', 'fontWeight':'bold', 'textDecoration':'none', 'color': COLOR_ACCENT}) for d in EXAMPLE_DATASETS]
        ], style={'marginBottom':'15px', 'textAlign':'center', 'fontSize':'14px'}),

        dcc.Upload(
            id='upload-data',
            children=html.Div([' Drag & Drop atau ', html.A('Pilih File Excel', style={'fontWeight':'bold', 'color': COLOR_ACCENT, 'textDecoration': 'underline'})]),
            style={'width': '100%', 'height': '80px', 'lineHeight': '80px', 'borderWidth': '2px', 'borderStyle': 'dashed', 'borderRadius': '12px', 'textAlign': 'center', 'borderColor': COLOR_ACCENT, 'backgroundColor': '#FAFAFA', 'cursor': 'pointer', 'transition': '0.3s'},
            multiple=False
        ),

        # Tempat Alert Muncul
        html.Div(id='output-data-upload', style={'marginTop': '20px'})
    ]),

```

```
    # --- 2. STATISTIKA DESKRIPTIF ---
    html.Div(style=CARD_STYLE, children=[
        html.H5("Statistika Deskriptif", style={'textAlign': 'center',
            'fontWeight': 'bold', 'color': COLOR_ACCENT}),
        html.P("Eksplor distribusi dan karakteristik tiap variabel
            sebelum analisis STATIS.",
               style={'textAlign': 'center', 'color': '#546E7A',
                  'marginBottom': '15px'}),
        html.Div(style={'display': 'flex', 'alignItems': 'center',
            'justifyContent': 'center',
                        'width': '100%', 'gap': '15px',
                           'marginBottom': '10px'}, children=[
            html.Label('Pilih Tahun:', style={'fontWeight': 'bold',
                'margin': '0', 'whiteSpace': 'nowrap'}),
            dcc.Dropdown(
                id='desc-year-dropdown',
                placeholder='Pilih tahun ...',
                style={'width': '180px', 'minWidth': '180px'}
            ),
            html.Button('RUN DESKRIPTIF', id='btn-run-desc',
                n_clicks=0,
                       style={'backgroundColor': '#00897B', 'color':
                          'white', 'fontWeight': 'bold',
                              'borderRadius': '5px', 'padding': '8px
                                 8px', 'border': 'none',
                              'cursor': 'pointer', 'whiteSpace':
                                 'nowrap'})
        ]),
        dcc.Loading(children=[
            html.Div(id='desc-table-output', style={'marginTop':
                '15px'}),
            html.Div(id='desc-boxplot-output', style={'marginTop':
                '10px'}),
            html.Div(id='desc-formula-output', style={'marginTop':
                '20px'})
        ]),
        # --- Multi-Year View ---
        html.Hr(style={'borderColor': '#BBDEFB', 'marginTop': '25px'}),
        html.Div(style={'display': 'flex', 'alignItems': 'center',
            'justifyContent': 'center',
                        'width': '100%', 'gap': '12px',
                           'marginBottom': '6px'}, children=[
            html.Span("Perbandingan Antar Tahun", style={'fontWeight':
                'bold', 'color': '#37474F', 'fontSize': '15px'}),
            dcc.Dropdown(
                id='desc-multi-stat',
                options=[
                    {'label': 'Mean (Rata-rata)', 'value': 'mean'},
                    {'label': 'Std Dev', 'value': 'std'},
                    {'label': 'Median', 'value': 'median'},
                    {'label': 'Q1 (Kuartil 1)', 'value': 'q1'},
                    {'label': 'Q3 (Kuartil 3)', 'value': 'q3'},
                    {'label': 'Min', 'value': 'min'},
                    {'label': 'Max', 'value': 'max'},
                    {'label': 'Skewness', 'value': 'skewness'},
                    {'label': 'Kurtosis', 'value': 'kurtosis'},
```

```
                    {'label': 'CV (%)', 'value': 'cv'},
                ],
                value='mean',
                clearable=False,
                style={'width': '200px', 'minWidth': '200px'}
            ),
            html.Button('TAMPILKAN SEMUA TAHUN',
                id='btn-run-desc-multi', n_clicks=0,
                        style={'backgroundColor': '#5C6BC0', 'color':
                            'white', 'fontWeight': 'bold',
                                'borderRadius': '5px', 'padding': '8px
                                    12px', 'border': 'none',
                                'cursor': 'pointer', 'whiteSpace':
                                    'nowrap'})
        ]),
        html.Div(style={'display': 'flex', 'justifyContent': 'center',
            'alignItems': 'center',
                        'gap': '8px', 'marginBottom': '10px'},
                            children=[
            html.Span("Skala Y:", style={'fontSize': '13px', 'color':
                '#546E7A', 'fontWeight': 'bold'}),
            dcc.RadioItems(
                id='desc-multi-scale',
                options=[
                    {'label': 'Auto (sesuai data)', 'value': 'auto'},
                    {'label': 'Mulai dari 0', 'value': 'zero'},
                ],
                value='auto',
                inline=True,
                labelStyle={'marginRight': '15px', 'fontSize': '13px'},
                inputStyle={'marginRight': '4px'}
            )
        ]),
        dcc.Loading(children=[
            html.Div(id='desc-multi-table-output', style={'marginTop':
                '10px'}),
            html.Div(id='desc-multi-trend-output', style={'marginTop':
                '10px'})
        ])
    ]),

    # --- 3. DATA DIAGNOSTICS ---
    html.Div(style=CARD_STYLE, children=[
        html.H5("Diagnostic Tests (KMO & Bartlett)",
            style={'textAlign': 'center', 'fontWeight': 'bold',
            'color': COLOR_ACCENT}),
        html.Div(style={'textAlign': 'center', 'marginBottom':
            '20px'}, children=[
            html.Button('RUN DIAGNOSTIK', id='btn-run-diag',
                n_clicks=0, style={'backgroundColor': '#FFB300',
                'color': 'white', 'fontWeight': 'bold',
                'borderRadius': '5px', 'width': '200px'})
        ]),
        dcc.Loading(children=[
            html.Div(className='row', id='diag-results-container',
                style={'display': 'none'}, children=[
```

```
                html.Div(className='six columns', children=[
                    html.H6("Kaiser-Meyer-Olkin (KMO) Test",
                        style={'fontWeight': 'bold'}),
                    html.Div(id='kmo-result', style={'fontSize':
                        '16px', 'padding': '10px', 'backgroundColor':
                        '#E8F5E9', 'borderRadius': '5px'})
                ]),
                html.Div(className='six columns', children=[
                    html.H6("Bartlett's Test of Sphericity",
                        style={'fontWeight': 'bold'}),
                    html.Div(id='bartlett-result', style={'fontSize':
                        '16px', 'padding': '10px', 'backgroundColor':
                        '#E8F5E9', 'borderRadius': '5px'})
                ])
            ])
        ]),
        html.Div(id='formula-diag-output', style={'marginTop': '20px'})
    ]),

    # --- 4. CONTROLS (STATIS Panel) ---
    html.Div(style=CARD_STYLE, children=[
        html.H4("Panel Kontrol Analisis STATIS", style={'textAlign':
            'center', 'fontWeight': 'bold', 'color': COLOR_ACCENT}),
        html.Div(className='row', children=[
            html.Div(className='three columns', children=[
                html.Label('Pilih Variabel Analisis:',
                    style={'fontWeight':'bold'}),
                dcc.Dropdown(id='variable-dropdown-final', multi=True,
                    placeholder="Pilih variabel...",
                    style={'fontSize': '14px'})
            ]),
            html.Div(className='three columns', children=[
                html.Label('Rotasi Baseline (Semua Plot):',
                    style={'fontWeight':'bold'}),
                dcc.RadioItems(id='baseline-type-final',
                    options=[{'label':'Variabel','value':'loading'},{'label':'Kab/Kota
                    value='loading',
                    labelStyle={'display':'inline-block','marginRight':'10px'}),
                dcc.Dropdown(id='baseline-name-final',
                    placeholder="Pilih referensi...",
                    style={'fontSize': '14px'})
            ]),
            html.Div(className='three columns', children=[
                html.Label('Filter Tahun:',
                    style={'fontWeight':'bold'}),
                dcc.Checklist(id='year-checklist-final', inline=True,
                    style={'marginBottom':'5px'}),
                html.Label('Mirroring (Flip PC2):',
                    style={'fontSize':'12px', 'color':'#666'}),
                dcc.Checklist(id='mirror-year-checklist-final',
                    inline=True)
            ]),
            html.Div(className='three columns', children=[
                html.Label('Highlight Trajektori:',
                    style={'fontWeight':'bold'}),
```

```
                dcc.Dropdown(id='trajectory-dropdown-final',
                    multi=True, placeholder="Pilih objek...",
                    style={'fontSize': '14px'})
            ])
        ]),
        html.Hr(style={'borderColor': '#ECEFF1'}),
        html.Div(className='row', children=[
            html.Div(className='six columns', children=[
                html.Label('Opsi Tampilan Vektor:',
                    style={'fontWeight':'bold'}),
                dcc.RadioItems(
                    id='loading-vector-scope-final',
                    options=[{'label':'Hidden','value':'none'},
                        {'label':'2D Only','value':'2d'}, {'label':'3D
                        Only','value':'3d'}, {'label':'All
                        Plots','value':'both'}],
                    value='both',
                    labelStyle={'display':'inline-block',
                        'marginRight':'15px'}
                ),
            ]),
            html.Div(className='six columns', children=[
                html.Label('Pengaturan Skala Visual:',
                    style={'fontWeight':'bold'}),
                dcc.Checklist(
                    id='visual-scaling-opts',
                    options=[
                        {'label': 'Normalisasi Skor (-1 s.d 1)',
                            'value': 'norm_score'},
                        {'label': 'Skala Visual Loadings (Agar
                            sebanding)', 'value': 'scale_load'}
                    ],
                    value=['scale_load'],
                    inline=True,
                    inputStyle={"marginRight": "5px"}
                )
            ])
        ]),
        html.Div(className='row', style={'marginTop': '15px'},
            children=[
             html.Div(className='twelve columns', children=[
                html.Label('Filter Vektor Spesifik (Visualisasi):',
                    style={'fontWeight':'bold'}),
                dcc.Dropdown(id='vector-selector-final', multi=True,
                    placeholder="Pilih vektor spesifik untuk
                    ditampilkan (Kosong = Semua)...")
            ])
        ]),
        html.Button('RUN ANALISIS STATIS & BIPLOT',
            id='btn-run-statis', n_clicks=0, style={'width': '100%',
            'marginTop': '20px', 'backgroundColor': COLOR_ACCENT,
            'color': 'white', 'fontWeight': 'bold', 'borderRadius':
            '5px'})
    ]),
```

```
    # --- 3. DYNAMIC BIPLOT ---
    html.Div(id='biplot-section-container', style={'display': 'none'},
        children=[
        html.H5("Visualisasi Dynamic Biplot (Spasio-Temporal)",
            style={'textAlign': 'center', 'fontWeight': 'bold',
            'color': COLOR_ACCENT}),
        dcc.Loading(type='cube', color=COLOR_ACCENT, children=[
            dcc.Graph(id='graph-3d-main', style={'height': '700px'})
        ]),
        html.Hr(),
        html.Div(className='row', children=[
            html.Div(className='six columns', children=[
                html.H5("Snapshot 2D per Tahun", style={'textAlign':
                    'center', 'color': '#455A64'}),
                dcc.Dropdown(id='year-dropdown-2d-single',
                    style={'width':'60%', 'margin':'0 auto',
                    'marginBottom':'10px'}),
                dcc.Loading(children=[dcc.Graph(id='graph-2d-year',
                    style={'height': '600px'})])
            ]),
            html.Div(className='six columns', children=[
                html.H5("Proyeksi Trajektori 2D", style={'textAlign':
                    'center', 'color': '#455A64'}),
                html.P("Lintasan pergerakan objek dari tahun ke
                    tahun", style={'textAlign':'center',
                    'color':'#90A4AE', 'fontSize':'12px'}),
                dcc.Loading(children=[dcc.Graph(id='graph-2d-traj',
                    style={'height': '600px'})])
            ])
        ]),
        html.Hr(),
        html.H6("Data Trajektori Terpilih (Tabel)",
            style={'textAlign':'center', 'fontWeight':'bold', 'color':
            COLOR_ACCENT}),
        dcc.Loading(children=[
            dash_table.DataTable(
                id='trajectory-table',
                export_format='xlsx',
                style_table={'overflowX': 'auto'},
                style_header={'backgroundColor': '#37474F', 'color':
                    'white', 'fontWeight': 'bold'},
                style_cell={'textAlign': 'center', 'padding': '8px',
                    'fontFamily': 'sans-serif'},
                page_size=10
            )
        ])
    ]),

    # --- 4. STATIS ANALYSIS ---
    html.Div(id='statis-section-container', style={'display': 'none'},
        children=[
        html.H5("Analisis STATIS (Struktur Kompromi)",
            style={'textAlign': 'center', 'fontWeight': 'bold',
            'color': COLOR_ACCENT}),
        dcc.Loading(children=[html.Div(id='formula-statis-output',
            style={'marginBottom': '20px'})]),
```

```
        # 4 Plot Grid -> Expanded to 8 Plot Grid
        html.Div(className='row', style={'marginBottom':'30px'},
            children=[
            html.Div(className='six columns', children=[
                dcc.Loading(children=[dcc.Graph(id='statis-scree',
                    style={'height': '400px'})])
            ]),
            html.Div(className='six columns', children=[
                dcc.Loading(children=[dcc.Graph(id='statis-rv-heatmap',
                    style={'height': '400px'})])
            ])
        ]),
        html.Div(className='row', style={'marginBottom':'30px'},
            children=[
            html.Div(className='six columns', children=[
                dcc.Loading(children=[dcc.Graph(id='statis-compromise',
                    style={'height': '500px'})])
            ]),
            html.Div(className='six columns', children=[
                dcc.Loading(children=[dcc.Graph(id='statis-biplot',
                    style={'height': '500px'})])
            ])
        ]),
        html.Div(className='row', style={'marginBottom':'30px'},
            children=[
            html.Div(className='six columns', children=[
                dcc.Loading(children=[dcc.Graph(id='statis-contribution',
                    style={'height': '400px'})])
            ]),
            html.Div(className='six columns', children=[
                dcc.Loading(children=[dcc.Graph(id='statis-cos2',
                    style={'height': '400px'})])
            ])
        ]),
        html.Div(className='row', style={'marginTop':'30px'},
            children=[
            html.Div(className='six columns', children=[
                dcc.Loading(children=[dcc.Graph(id='statis-interstructure',
                    style={'height': '500px'})])
            ]),
            html.Div(className='six columns', children=[
                dcc.Loading(children=[dcc.Graph(id='statis-trajectory',
                    style={'height': '500px'})])
            ])
        ]),

        html.Hr(style={'borderColor': '#B0BEC5', 'marginTop': '30px'}),

        # Permutation Test
        html.Div(style={'padding': '20px', 'borderRadius': '10px',
            'border': '0px solid #FFF59D'}, children=[
            html.H5("Uji Signifikansi Stabilitas (Permutasi)",
                style={'textAlign': 'center', 'fontWeight': 'bold',
                'color': COLOR_ACCENT}),
            html.Div(className='row', style={'display': 'flex',
                'justifyContent': 'center', 'alignItems': 'center',
```

```
                'marginBottom': '10px'}, children=[
1370            html.Label("Jumlah Permutasi (N): ",
                    style={'marginRight': '10px', 'fontWeight':
                    'bold'}),
1371            dcc.Input(id='perm-n-input', type='number',
                    value=1000, min=100, step=100, style={'width':
                    '100px', 'marginRight': '20px'}),
1372            html.Button("RUN PERMUTATION TEST", id='btn-run-perm',
                    n_clicks=0, style={'backgroundColor':
                    COLOR_ACCENT, 'color': 'white', 'fontWeight':
                    'bold', 'borderRadius': '5px'})
1373        ]),
1374        dcc.Loading(children=[
1375            dcc.Graph(id='perm-graph', style={'height': '350px'}),
1376            html.Div(id='perm-result-text', style={'textAlign':
                    'center', 'fontSize': '16px', 'marginTop': '10px',
                    'fontWeight': 'bold', 'color': '#333'}),
1377            dcc.Loading(children=[html.Div(id='formula-perm-output',
                    style={'marginTop': '20px'})])
1378        ])
1379    ]),
1380
1381    html.Hr(style={'borderColor': '#B0BEC5', 'marginTop': '40px'}),
1382
1383    # Detail Tables
1384    html.H5("Detail Hasil Analisis (Tabel & Matriks)",
            style={'textAlign': 'center', 'fontWeight':'bold',
            'color': COLOR_ACCENT}),
1385    html.Div(className='row', style={'display': 'flex',
            'alignItems': 'center', 'justifyContent': 'center',
            'marginBottom': '20px'}, children=[
1386        html.Label("Pilih Tabel:", style={'marginRight': '10px',
                'fontWeight': 'bold'}),
1387        dcc.Dropdown(id='statis-table-selector',
                placeholder="Pilih Tabel...", clearable=False,
                style={'width': '300px', 'marginRight': '20px'}),
1388        html.Label("Baris per Halaman:", style={'marginRight':
                '10px', 'fontWeight': 'bold'}),
1389        dcc.Input(id='table-page-size', type='number', value=10,
                min=5, max=100, step=5, style={'width': '80px'})
1390    ]),
1391
1392    html.Div(id='table-info-display', style={'textAlign':
            'center', 'marginBottom': '10px', 'fontStyle': 'italic',
            'color': '#607D8B'}),
1393
1394    dcc.Loading(children=[
1395        dash_table.DataTable(
1396            id='statis-detail-table',
1397            export_format='xlsx',
1398            style_table={'overflowX': 'auto'},
1399            style_header={'backgroundColor': '#455A64', 'color':
                    'white', 'fontWeight': 'bold'},
1400            style_cell={'textAlign': 'center', 'padding': '8px',
                    'minWidth': '100px'},
1401            style_data_conditional=[
```

```
1402                    {'if': {'row_index': 'odd'}, 'backgroundColor':
                            '#F5F5F5'}
1403                ],
1404                page_action='native', # Enable pagination
1405                page_current=0,
1406                page_size=10
1407            )
1408        ]),
1409        html.Hr(style={'borderColor': '#B0BEC5', 'marginTop': '40px'}),
1410        # --- [TAMBAHAN BARU] SECTION 5: EVALUASI JUMLAH CLUSTER
                (ELBOW & DENDROGRAM) ---
1411        html.Div(style=CARD_STYLE, children=[
1412            html.H5("Tahap Pembantu: Evaluasi Jumlah Cluster (Visual
                    Aids)", style={'textAlign': 'center', 'fontWeight':
                    'bold', 'color': COLOR_ACCENT}),
1413            html.P("Gunakan kurva Elbow dan Dendrogram di bawah ini
                    untuk menentukan jumlah cluster (k) yang optimal pada
                    suatu tahun.", style={'textAlign': 'center', 'color':
                    '#546E7A'}),
1414            html.Div(className='row', style={'display': 'flex',
                    'alignItems': 'center', 'justifyContent': 'center',
                    'marginBottom': '20px'}, children=[
1415                html.Label("Evaluasi Tahun:", style={'marginRight':
                        '10px', 'fontWeight': 'bold'}),
1416                dcc.Dropdown(id='eval-year-dropdown', clearable=False,
                        style={'width': '150px'})
1417            ]),
1418            html.Div(className='row', children=[
1419                html.Div(className='six columns', children=[
1420                    html.H6("Metode Elbow (Inersia K-Means)",
                            style={'textAlign': 'center', 'fontWeight':
                            'bold'}),
1421                    dcc.Loading(children=[dcc.Graph(id='elbow-graph',
                            style={'height': '400px'})])
1422                ]),
1423                html.Div(className='six columns', children=[
1424                    html.H6("Dendrogram (Agglomerative Hierarchical)",
                            style={'textAlign': 'center', 'fontWeight':
                            'bold'}),
1425                    dcc.Loading(children=[dcc.Graph(id='dendrogram-graph',
                            style={'height': '400px'})])
1426                ])
1427            ])
1428        ]),
1429
1430        # --- [TAMBAHAN BARU] SECTION 6: CLUSTERING TAHUNAN & MATRIKS
                TRANSISI ---
1431        html.Div(style=CARD_STYLE, children=[
1432            html.H5("Analisis Klaster Spasio-Temporal (Tahunan &
                    Migrasi)", style={'textAlign': 'center', 'fontWeight':
                    'bold', 'color': COLOR_ACCENT}),
1433            html.Div(className='row', style={'display': 'flex',
                    'alignItems': 'center', 'justifyContent': 'center',
                    'marginBottom': '20px'}, children=[
1434                html.Label("Pasangan Tahun Transisi:",
                        style={'marginRight': '10px', 'fontWeight':
```

```
                'bold'}),
            dcc.Dropdown(id='trans-year-dropdown',
                clearable=False, style={'width': '200px',
                'marginRight': '20px'}),
            html.Label("Jumlah Cluster (k):",
                style={'marginRight': '10px', 'fontWeight':
                'bold'}),
            dcc.Input(id='trans-k-input', type='number', value=3,
                min=2, max=10, step=1, style={'width': '60px',
                'marginRight': '20px'}),
            html.Button("RUN CLUSTERING TAHUNAN",
                id='btn-run-trans', n_clicks=0,
                        style={'backgroundColor': COLOR_ACCENT,
                            'color': 'white', 'fontWeight':
                            'bold', 'border': 'none',
                            'borderRadius': '5px'})
        ]),
        html.Div(className='row', children=[
            html.Div(className='six columns', children=[
                html.H6(id='title-trans-map-t1',
                    style={'textAlign': 'center', 'fontWeight':
                    'bold'}),
                dcc.Loading(children=[dcc.Graph(id='trans-map-t1',
                    style={'height': '500px'})])
            ]),
            html.Div(className='six columns', children=[
                html.H6(id='title-trans-map-t2',
                    style={'textAlign': 'center', 'fontWeight':
                    'bold'}),
                dcc.Loading(children=[dcc.Graph(id='trans-map-t2',
                    style={'height': '500px'})])
            ])
        ]),
        html.Div(className='row', children=[
            html.Div(className='six columns', children=[
                html.H6(id='title-trans-scatter-t1',
                    style={'textAlign': 'center', 'fontWeight':
                    'bold'}),
                dcc.Loading(children=[dcc.Graph(id='trans-scatter-t1',
                    style={'height': '500px'})])
            ]),
            html.Div(className='six columns', children=[
                html.H6(id='title-trans-scatter-t2',
                    style={'textAlign': 'center', 'fontWeight':
                    'bold'}),
                dcc.Loading(children=[dcc.Graph(id='trans-scatter-t2',
                    style={'height': '500px'})])
            ])
        ]),
        html.Hr(),
        html.H6("Matriks Transisi (Perpindahan Kabupaten/Kota)",
            style={'textAlign': 'center', 'fontWeight': 'bold',
            'color': COLOR_ACCENT}),
        html.Div(style={'display': 'flex', 'justifyContent':
            'center', 'marginBottom': '10px'}, children=[
```

```
                dcc.RadioItems(id='trans-view-mode',
                    options=[{'label': ' Tampilkan Jumlah', 'value':
                    'count'}, {'label': ' Tampilkan Nama Daerah',
                    'value': 'names'}], value='count', inline=True,
                    inputStyle={"marginRight": "5px", "marginLeft":
                    "15px"})
            ]),
            dcc.Loading(children=[
                dash_table.DataTable(id='trans-matrix-table',
                    style_table={'overflowX': 'auto'},
                    style_header={'backgroundColor': '#37474F',
                    'color': 'white', 'fontWeight': 'bold'},
                    style_cell={'textAlign': 'center', 'padding':
                    '8px', 'fontFamily': 'sans-serif'})
            ])
        ]),

        # --- SECTION 7: CLUSTERING TRAJEKTORI KESELURUHAN ---
        html.Div(style=CARD_STYLE, children=[
            html.H5("Analisis Lanjutan: Clustering Trajektori
                Keseluruhan (STATIS)", style={'textAlign': 'center',
                'fontWeight': 'bold', 'color': COLOR_ACCENT}),
            html.P("Mengelompokkan Kabupaten/Kota berdasarkan
                kemiripan pola pergerakan (nasib) selama periode
                penuh.",
                    style={'textAlign': 'center', 'color': '#546E7A',
                        'marginBottom': '20px'}),

            html.Div(className='row', style={'display': 'flex',
                'alignItems': 'center', 'justifyContent': 'center',
                'marginBottom': '20px'}, children=[
                html.Label("Jumlah Cluster (k):",
                    style={'marginRight': '10px', 'fontWeight':
                    'bold'}),
                dcc.Input(id='cluster-k-input', type='number',
                    value=3, min=2, max=10, step=1, style={'width':
                    '60px', 'marginRight': '20px'}),
                html.Button("RUN CLUSTERING TRAJEKTORI",
                    id='btn-run-cluster', n_clicks=0,
                            style={'backgroundColor': COLOR_ACCENT,
                                'color': 'white', 'fontWeight':
                                'bold', 'border': 'none',
                                'borderRadius': '5px'})
            ]),

            html.Div(className='row', children=[
                # Kiri: Scatter Plot
                html.Div(className='six columns', children=[
                    html.H5("Peta Sebaran (Scatter Plot)",
                        style={'textAlign': 'center', 'fontSize':
                        '14px', 'fontWeight': 'bold'}),
                    dcc.Loading(children=[dcc.Graph(id='cluster-scatter',
                        style={'height': '800px'})])
                ]),
                # Kanan: Dendrogram
                html.Div(className='six columns', children=[
```

```
                    html.H5("Dendrogram Kemiripan",
                        style={'textAlign': 'center', 'fontSize':
                        '14px', 'fontWeight': 'bold'}),
                    dcc.Loading(children=[dcc.Graph(id='cluster-dendrogram',
                        style={'height': '800px'})])
                ])
            ]),

            # --- [TAMBAHAN BARU: PETA SPASIAL] ---
            html.Hr(style={'borderColor': '#E0E0E0', 'marginTop':
                '20px', 'marginBottom': '20px'}),
            html.H5("Distribusi Geografis Cluster (Peta Wilayah)",
                style={'textAlign': 'center', 'fontWeight': 'bold',
                'color': COLOR_ACCENT}),
            dcc.Loading(type='cube', color=COLOR_ACCENT, children=[
                dcc.Graph(id='cluster-map', style={'height': '500px'})
            ]),
            # ------------------------------------

            html.Div(id='cluster-interpretation', style={'marginTop':
                '15px', 'padding': '15px', 'backgroundColor':
                '#E3F2FD', 'borderRadius': '5px', 'borderLeft': '5px
                solid #1565C0', 'fontStyle': 'italic', 'color':
                '#0D47A1'}),
            dcc.Loading(children=[html.Div(id='formula-cluster-output',
                style={'marginTop': '20px'})])
        ]),

        # --- [TAMBAHAN BARU] SECTION 8: BASELINE MATRIKS TRANSISI
            (IPM/UNDP) ---
        html.Div(style=CARD_STYLE, children=[
            html.H5("Disclaimer & Baseline: Matriks Transisi
                Berdasarkan Kategori IPM (UNDP)", style={'textAlign':
                'center', 'fontWeight': 'bold', 'color': '#E64A19'}),
            html.P("Sebelum menentukan jumlah klaster k-means, Anda
                dapat melihat kondisi riil perpindahan wilayah
                berdasarkan 4 kategori Indeks Pembangunan Manusia
                (UNDP): Sangat Tinggi (80), Tinggi (70-79.99), Sedang
                (55-69.99), dan Rendah (<55).",
                    style={'textAlign': 'center', 'color': '#546E7A',
                        'marginBottom': '20px'}),

            html.Div(className='row', style={'display': 'flex',
                'alignItems': 'center', 'justifyContent': 'center',
                'marginBottom': '20px'}, children=[
                html.Label("Pasangan Tahun Transisi IPM:",
                    style={'marginRight': '10px', 'fontWeight':
                    'bold'}),
                dcc.Dropdown(id='ipm-trans-year-dropdown',
                    clearable=False, style={'width': '200px',
                    'marginRight': '20px'}),
                html.Button("TAMPILKAN BASELINE IPM",
                    id='btn-run-ipm-trans', n_clicks=0,
                            style={'backgroundColor': '#E64A19',
                                'color': 'white', 'fontWeight':
                                'bold', 'border': 'none',
```

```
                                'borderRadius': '5px'})
1520            ]),
1521
1522            html.Div(id='ipm-disclaimer-text', style={'textAlign':
                    'center', 'marginBottom': '20px', 'fontStyle':
                    'italic', 'fontWeight': 'bold'}),
1523
1524            html.Div(style={'display': 'flex', 'justifyContent':
                    'center', 'marginBottom': '10px'}, children=[
1525                dcc.RadioItems(id='ipm-trans-view-mode', options=[
1526                    {'label': ' Tampilkan Jumlah', 'value': 'count'},
1527                    {'label': ' Tampilkan Nama Daerah', 'value':
                            'names'}
1528                ], value='count', inline=True,
                        inputStyle={"marginRight": "5px", "marginLeft":
                        "15px"})
1529            ]),
1530
1531            dcc.Loading(children=[
1532                dash_table.DataTable(id='ipm-trans-matrix-table',
                        style_table={'overflowX': 'auto'},
1533                                     style_header={'backgroundColor':
                                            '#37474F', 'color': 'white',
                                            'fontWeight': 'bold'},
1534                                     style_cell={'textAlign':
                                            'center', 'padding': '8px',
                                            'fontFamily': 'sans-serif'})
1535            ])
1536        ]),
1537
1538        # --- [TAMBAHAN BARU] SECTION 9: PERBANDINGAN CLUSTER VS IPM
                (CROSS-TABULATION) ---
1539        html.Div(style=CARD_STYLE, children=[
1540            html.H5("Perbandingan Anggota Cluster K-Means vs Kategori
                    IPM (UNDP)", style={'textAlign': 'center',
                    'fontWeight': 'bold', 'color': '#00695C'}),
1541            html.P("Bagian ini menampilkan tabel tabulasi silang
                    (cross-tabulation) antara cluster yang dibentuk oleh
                    algoritma K-Means dengan pengelompokan baku dari UNDP
                    berdasarkan skor IPM. Ini berguna untuk mengevaluasi
                    apakah cluster data-driven sejalan dengan standar
                    institusi global.",
1542                   style={'textAlign': 'center', 'color': '#546E7A',
                           'marginBottom': '20px'}),
1543
1544            html.Div(className='row', style={'display': 'flex',
                    'alignItems': 'center', 'justifyContent': 'center',
                    'marginBottom': '20px'}, children=[
1545                html.Label("Tahun:", style={'marginRight': '10px',
                        'fontWeight': 'bold'}),
1546                dcc.Dropdown(id='cross-year-dropdown',
                        clearable=False, style={'width': '120px',
                        'marginRight': '20px'}),
1547                html.Label("Jumlah Cluster K-Means (k):",
                        style={'marginRight': '10px', 'fontWeight':
                        'bold'}),
```

```
                dcc.Input(id='cross-k-input', type='number', value=3,
                    min=2, max=10, step=1, style={'width': '60px',
                    'marginRight': '20px'}),
                html.Button("TAMPILKAN PERBANDINGAN",
                    id='btn-run-cross', n_clicks=0,
                            style={'backgroundColor': '#00897B',
                                'color': 'white', 'fontWeight':
                                'bold', 'border': 'none',
                                'borderRadius': '5px'})
            ]),

            html.Div(style={'display': 'flex', 'justifyContent':
                'center', 'marginBottom': '10px'}, children=[
                dcc.RadioItems(id='cross-view-mode', options=[
                    {'label': ' Tampilkan Jumlah', 'value': 'count'},
                    {'label': ' Tampilkan Nama Daerah', 'value':
                        'names'}
                ], value='count', inline=True,
                    inputStyle={"marginRight": "5px", "marginLeft":
                    "15px"})
            ]),

            dcc.Loading(children=[
                dash_table.DataTable(id='cross-table',
                    style_table={'overflowX': 'auto'},
                                style_header={'backgroundColor':
                                    '#37474F', 'color': 'white',
                                    'fontWeight': 'bold'},
                                style_cell={'textAlign':
                                    'center', 'padding': '8px',
                                    'fontFamily': 'sans-serif'})
            ])
        ])
    ])
])

# ------------------------------


# ========================================

# Import Library



COLOR_ACCENT = '#1976D2'


# ------------------------------

# 1. Upload & Validasi
@app.callback(
    Output('global-data-store', 'data'),
    Output('output-data-upload', 'children'),
    Input('upload-data', 'contents'),
    State('upload-data', 'filename')
```

```
)
def cb_upload(contents, filename):
    if not contents:
        raise exceptions.PreventUpdate

    try:
        _, content_string = contents.split(',')
        decoded = base64.b64decode(content_string)
        # Baca semua sheet
        all_sheets = pd.read_excel(io.BytesIO(decoded),
            sheet_name=None)

        all_data, sheet_names = [], []
        for name, df in all_sheets.items():
            # Validasi Nama Sheet (Harus Integer Tahun)
            try:
                y = int(name)
                df['Tahun'] = y
                all_data.append(df)
                sheet_names.append(name)
            except:
                pass # Skip sheet yang bukan tahun

        if not all_data:
            return None, html.Div([
                html.Strong("Gagal!"), " Tidak ditemukan sheet dengan
                    nama Tahun (angka) yang valid."
            ], style={'backgroundColor': '#FFEBEE', 'color':
                '#C62828', 'padding': '15px', 'borderRadius': '5px',
                'border': '1px solid #EF9A9A', 'textAlign': 'center'})

        df_concat = pd.concat(all_data, ignore_index=True)

        # Validasi Kolom Wajib
        required_cols = ['Kabupaten/Kota', 'Provinsi']
        missing = [c for c in required_cols if c not in
            df_concat.columns]

        if missing:
            return None, html.Div([
                html.Strong("Format Salah!"), f" Kolom berikut tidak
                    ditemukan: {', '.join(missing)}"
            ], style={'backgroundColor': '#FFEBEE', 'color':
                '#C62828', 'padding': '15px', 'borderRadius': '5px',
                'border': '1px solid #EF9A9A', 'textAlign': 'center'})

        # Cleaning
        cols = df_concat.columns.drop(['Provinsi', 'Kabupaten/Kota',
            'Tahun'])
        for c in cols:
            if df_concat[c].dtype == 'object':
                df_concat[c] =
                    pd.to_numeric(df_concat[c].astype(str).str.replace(',',
                    '.').str.replace(' ', ''), errors='coerce')
        df_concat.dropna(inplace=True)

```

```
        # Store
        base_features = sorted(list(set(cols)))
        years = sorted(list(df_concat['Tahun'].unique()))
        kab_kota = df_concat['Kabupaten/Kota'].unique().tolist()

        # --- [NEW] VALIDASI NAMA EKSAK DENGAN SHAPEFILE ---
        import difflib
        valid_peta = gdf['id_cocok'].unique().tolist() if gdf is not
            None else []
        matches = [k for k in kab_kota if k in valid_peta]
        mismatches = [k for k in kab_kota if k not in valid_peta]

        report_items = []

        # Bagian 1: Ringkasan & Sukses
        report_items.append(html.Div([
            html.Strong(f" Berhasil Memuat: {filename}"),
            html.P(f"Total: {len(kab_kota)} Wilayah di Excel |
                {len(matches)} Cocok Eksak dengan Peta."),
        ], style={'marginBottom': '15px'}))

        if matches:
            report_items.append(html.Div([
                html.Span("Daftar Wilayah Sesuai (Eksak): ",
                    style={'fontWeight': 'bold', 'color': '#2E7D32'}),
                html.Span(", ".join(matches), style={'fontSize':
                    '13px'})
            ], style={'backgroundColor': '#E8F5E9', 'padding': '10px',
                'borderRadius': '5px', 'marginBottom': '10px'}))

        # Bagian 2: Mismatch & Saran
        if mismatches:
            mismatch_ui = []
            for m in mismatches:
                suggestions = difflib.get_close_matches(m, valid_peta,
                    n=1, cutoff=0.6)
                saran_txt = f" (Saran: '{suggestions[0]}')" if
                    suggestions else " (Tidak ditemukan saran)"
                mismatch_ui.append(html.Li([
                    html.Span(f"'{m}'", style={'fontWeight': 'bold',
                        'color': '#C62828'}),
                    html.Span(saran_txt, style={'fontStyle': 'italic',
                        'color': '#546E7A'})
                ]))

            report_items.append(html.Div([
                html.Strong(" Wilayah Belum Sesuai (Penting untuk
                    Moran & Map):", style={'color': '#D84315'}),
                html.P("Pastikan nama di Excel persis seperti saran di
                    bawah agar dapat dipetakan:", style={'fontSize':
                    '13px', 'marginTop': '5px'}),
                html.Ul(mismatch_ui, style={'fontSize': '13px',
                    'columnCount': '2' if len(mismatches) > 10 else
                    '1'})
            ], style={'backgroundColor': '#FFF3E0', 'padding': '15px',
                'borderRadius': '5px', 'border': '1px solid #FFE0B2',
```

```
                'marginTop': '10px'}))

        store = {
            'df_json': df_concat.to_json(orient='split'),
            'base_features': base_features,
            'kab_kota': kab_kota,
            'years': years,
            'sheet_names': sheet_names
        }

        return store, html.Div(report_items, style={'padding': '15px',
            'borderRadius': '8px', 'border': '1px solid #CFD8DC',
            'backgroundColor': 'white'})

    except Exception as e:
        error_msg = html.Div([
            html.Strong("Error!"), f" Terjadi kesalahan saat memproses
                file: {str(e)}"
        ], style={'backgroundColor': '#FFEBEE', 'color': '#C62828',
            'padding': '15px', 'borderRadius': '5px', 'border': '1px
            solid #EF9A9A', 'textAlign': 'center'})
        return None, error_msg

# 2. Controls & Dropdowns Sync
@app.callback(
    Output('variable-dropdown-final', 'options'),
        Output('variable-dropdown-final', 'value'),
    Output('trajectory-dropdown-final', 'options'),
        Output('year-checklist-final', 'options'),
        Output('year-checklist-final', 'value'),
    Output('mirror-year-checklist-final', 'options'),
        Output('year-dropdown-2d-single', 'options'),
        Output('year-dropdown-2d-single', 'value'),
    Output('baseline-name-final', 'options'),
    Output('eval-year-dropdown', 'options'),
        Output('eval-year-dropdown', 'value'),
    Output('trans-year-dropdown', 'options'),
        Output('trans-year-dropdown', 'value'),
    Output('ipm-trans-year-dropdown', 'options'),
        Output('ipm-trans-year-dropdown', 'value'),
    Output('cross-year-dropdown', 'options'),
        Output('cross-year-dropdown', 'value'),
    Input('global-data-store', 'data'), Input('baseline-type-final',
        'value')
)
def cb_controls(data, base_type):
    if not data: raise exceptions.PreventUpdate
    feats, kabs, years = data['base_features'], data['kab_kota'],
        data['years']
    traj_opts = ([{'label': f'{k} (Kab/Kota)', 'value': f'skor_{k}'}
        for k in kabs] +
                 [{'label': f'{f} (Variabel)', 'value':
                    f'loading_{f}'} for f in feats])
    y_opts = [{'label': str(y), 'value': y} for y in years]
    base_opts = [{'label': f, 'value': f} for f in (feats if base_type
        == 'loading' else kabs)]
```

```
1711
1712        trans_y_opts = []
1713        for i in range(1, len(years)):
1714            trans_y_opts.append({'label': f"{years[i-1]} / {years[i]}",
                'value': years[i]})
1715
1716        return [{'label': f, 'value': f} for f in feats], feats,
               traj_opts, y_opts, years, y_opts, y_opts, years[0], base_opts,
               y_opts, years[-1], trans_y_opts, (trans_y_opts[-1]['value'] if
               trans_y_opts else None), trans_y_opts,
               (trans_y_opts[-1]['value'] if trans_y_opts else None), y_opts,
               years[-1]
1717
1718    # 2b. Sync Vector Selector with Selected Vars
1719    @app.callback(
1720        Output('vector-selector-final', 'options'),
1721        Input('variable-dropdown-final', 'value')
1722    )
1723    def cb_sync_vector_selector(selected_vars):
1724        if not selected_vars: return []
1725        return [{'label': v, 'value': v} for v in selected_vars]
1726
1727    # 3. Dynamic PCA Calculation
1728    @app.callback(
1729        Output('pca-results-store', 'data'),
1730        Input('btn-run-statis', 'n_clicks'),
1731        State('variable-dropdown-final', 'value'),
1732        State('global-data-store', 'data')
1733    )
1734    def cb_pca_calc(n_clicks, vars, data):
1735        if not n_clicks or not vars or not data: raise
               exceptions.PreventUpdate
1736        df = pd.read_json(io.StringIO(data['df_json']), orient='split')
1737        pca = PCA(n_components=2)
1738        scaler = StandardScaler()
1739        pc_list, load_list = [], []
1740        for y in data['years']:
1741            df_y = df[df['Tahun'] == y]
1742            if df_y.empty: continue
1743            X = df_y[vars]
1744            coords = pca.fit_transform(scaler.fit_transform(X))
1745            d_sc = df_y[['Kabupaten/Kota', 'Provinsi', 'Tahun']].copy()
1746            d_sc['PC1'], d_sc['PC2'] = coords[:,0], coords[:,1]
1747            pc_list.append(d_sc)
1748            d_load = pd.DataFrame(pca.components_.T, columns=['PC1',
                   'PC2'])
1749            d_load['features'], d_load['Tahun'] = vars, y
1750            load_list.append(d_load)
1751        return {'pc_scores': pd.concat(pc_list).to_json(orient='split'),
               'loadings': pd.concat(load_list).to_json(orient='split')}
1752
1753    # 4. STATIS Core Calculation
1754    @app.callback(
1755        Output('statis-core-store', 'data'),
1756        Input('btn-run-statis', 'n_clicks'),
1757        State('variable-dropdown-final', 'value'),
```

```
    State('upload-data', 'contents'), State('global-data-store',
        'data')
)
def cb_statis_calc(n_clicks, vars, contents, global_data):
    if not n_clicks or not vars or not contents: raise
        exceptions.PreventUpdate
    try: return run_statis_core(contents, vars,
        global_data['sheet_names'])
    except: return None

# 4c. SHOW/HIDE SECTIONS CALLBACK
@app.callback(
    Output('biplot-section-container', 'style'),
    Output('statis-section-container', 'style'),
    Input('btn-run-statis', 'n_clicks'),
    prevent_initial_call=True
)
def cb_show_sections(n_clicks):
    if n_clicks and n_clicks > 0:
        return CARD_STYLE, CARD_STYLE
    return {'display': 'none'}, {'display': 'none'}

# 4b. DIAGNOSTICS & MATH FORMULAS CALLBACK
@app.callback(
    Output('kmo-result', 'children'),
    Output('kmo-result', 'style'),
    Output('bartlett-result', 'children'),
    Output('bartlett-result', 'style'),
    Output('diag-results-container', 'style'),
    Output('formula-diag-output', 'children'),
    Input('btn-run-diag', 'n_clicks'),
    State('global-data-store', 'data'),
    State('variable-dropdown-final', 'value')
)
def cb_diag_math(n_clicks, data_store, vars):
    if not n_clicks or not data_store or not vars:
        return "Wait...", {}, "Wait...", {}, {'display': 'none'},
            render_diagnostic_formulas()

    # 1. Diagnostics
    df = pd.read_json(io.StringIO(data_store['df_json']),
        orient='split')
    df_dict = {y: df[df['Tahun'] == y] for y in data_store['years']}
    kmo_val, p_val, kmo_int, bart_int, n, df_k, r_sq, q_sq, chi2_stat
        = calc_kmo_bartlett(df_dict, vars)

    kmo_txt = f"KMO = {kmo_val:.3f} ({kmo_int})"
    bart_txt = f"P-value = {p_val} ({bart_int})" # No Rounding!

    kmo_style = {'fontSize': '16px', 'padding': '10px',
        'borderRadius': '5px', 'backgroundColor': '#E8F5E9' if kmo_val
        >= 0.5 else '#FFEBEE'}
    bart_style = {'fontSize': '16px', 'padding': '10px',
        'borderRadius': '5px', 'backgroundColor': '#E8F5E9' if p_val <
        0.05 else '#FFEBEE'}

```

```
1804     formula_ui = render_diagnostic_formulas(n, df_k, r_sq, q_sq,
             kmo_val, kmo_int, chi2_stat, p_val, bart_int)
1805
1806     return kmo_txt, kmo_style, bart_txt, bart_style, {'display':
             'flex'}, formula_ui
1807
1808 # 5. DYNAMIC BIPLOT PLOTS
1809 @app.callback(
1810     Output('graph-3d-main', 'figure'), Output('graph-2d-year',
             'figure'), Output('graph-2d-traj', 'figure'),
1811     Output('trajectory-table', 'data'), Output('trajectory-table',
             'columns'),
1812     Input('pca-results-store', 'data'), Input('year-checklist-final',
             'value'), Input('mirror-year-checklist-final', 'value'),
1813     Input('baseline-type-final', 'value'),
             Input('baseline-name-final', 'value'),
             Input('trajectory-dropdown-final', 'value'),
1814     Input('year-dropdown-2d-single', 'value'),
             Input('loading-vector-scope-final', 'value'),
1815     Input('visual-scaling-opts', 'value'),
             Input('vector-selector-final', 'value')
1816 )
1817 def cb_update_plots(pca_data, years_filter, mirror_years, base_type,
         base_name, traj_sel, year_2d, load_scope, vis_opts,
         selected_vectors_to_show):
1818     if not pca_data or not years_filter: return {}, {}, {}, [], []
1819
1820     norm_score = 'norm_score' in vis_opts if vis_opts else False
1821     scale_load = 'scale_load' in vis_opts if vis_opts else False
1822
1823     df_pc = pd.read_json(io.StringIO(pca_data['pc_scores']),
             orient='split')
1824     df_load = pd.read_json(io.StringIO(pca_data['loadings']),
             orient='split')
1825     df_pc = df_pc[df_pc['Tahun'].isin(years_filter)]
1826     df_load = df_load[df_load['Tahun'].isin(years_filter)]
1827
1828     if selected_vectors_to_show and len(selected_vectors_to_show) > 0:
1829          df_load =
                 df_load[df_load['features'].isin(selected_vectors_to_show)]
1830
1831     df_pc_fin, df_load_fin = df_pc.copy(), df_load.copy()
1832     for y in sorted(df_pc['Tahun'].unique()):
1833         angle = get_rotation_angle(df_pc, df_load, base_type,
                 base_name, y) if base_name else 0
1834         mask_p, mask_l = df_pc_fin['Tahun'] == y, df_load_fin['Tahun']
                 == y
1835         if angle != 0:
1836             df_pc_fin.loc[mask_p, ['PC1', 'PC2']] =
                     rotate_dataframe(df_pc.loc[mask_p], angle)
1837             df_load_fin.loc[mask_l, ['PC1', 'PC2']] =
                     rotate_dataframe(df_load.loc[mask_l], angle)
1838         if mirror_years and y in mirror_years:
1839             df_pc_fin.loc[mask_p, 'PC2'] *= -1
1840             df_load_fin.loc[mask_l, 'PC2'] *= -1
1841
```

```
    # Visual Scaling
    pc_plot, load_plot =
        apply_visual_adjustments(df_pc_fin[['PC1','PC2']].values,
        df_load_fin[['PC1','PC2']].values, norm_score, scale_load)
    df_pc_plt = df_pc_fin.copy(); df_pc_plt[['PC1','PC2']] = pc_plot
    df_load_plt = df_load_fin.copy(); df_load_plt[['PC1','PC2']] =
        load_plot

    # 3D
    fig_3d = px.scatter_3d(df_pc_plt, x='Tahun', y='PC1', z='PC2',
        color='Provinsi', hover_name='Kabupaten/Kota')
    fig_3d.update_traces(marker=dict(size=4))
    if load_scope in ['3d', 'both']:
        for y in sorted(df_pc['Tahun'].unique()):
            d_l = df_load_plt[df_load_plt['Tahun'] == y]
            d_l_raw = df_load_fin[df_load_fin['Tahun'] == y]
            for idx, row in d_l.iterrows():
                raw = d_l_raw.loc[idx]
                fig_3d.add_trace(go.Scatter3d(
                    x=[y, y], y=[0, row['PC1']], z=[0, row['PC2']],
                    mode='lines+text', line=dict(color='red',
                        width=1), opacity=0.5,
                    text=[None, str(row['features'])],
                        textposition='top center', showlegend=False,
                    hoverinfo='text', hovertext=f"Loading:
                        {row['features']}<br>Raw: ({raw['PC1']:.2f},
                        {raw['PC2']:.2f})"
                ))
    if traj_sel:
        for sel in traj_sel:
            t_type, t_name = sel.split('_', 1)
            if t_type == 'skor':
                d_t = df_pc_plt[df_pc_plt['Kabupaten/Kota'] ==
                    t_name].sort_values('Tahun')
                fig_3d.add_trace(go.Scatter3d(x=d_t['Tahun'],
                    y=d_t['PC1'], z=d_t['PC2'], mode='lines+markers',
                    name=t_name))
            elif t_type == 'loading':
                d_t = df_load_plt[df_load_plt['features'] ==
                    t_name].sort_values('Tahun')
                if not d_t.empty:
                    fig_3d.add_trace(go.Scatter3d(x=d_t['Tahun'],
                    y=d_t['PC1'], z=d_t['PC2'], mode='lines+markers',
                    name=t_name))
    fig_3d.update_layout(title='Visualisasi 3D (Rotated)',
        margin=dict(l=0, r=0, b=0, t=30),
        scene=dict(aspectmode='manual', aspectratio=dict(x=2, y=1,
        z=1)))

    # 2D
    fig_2d_y = go.Figure()
    if year_2d:
        d_p, d_l = df_pc_plt[df_pc_plt['Tahun']==year_2d],
            df_load_plt[df_load_plt['Tahun']==year_2d]
        d_l_raw = df_load_fin[df_load_fin['Tahun']==year_2d]
```

```
1878            fig_2d_y.add_trace(go.Scatter(x=d_p['PC1'], y=d_p['PC2'],
                    mode='markers+text', text=d_p['Kabupaten/Kota'],
                    textposition='top center', name='Skor'))
1879            if load_scope in ['2d', 'both']:
1880                for idx, r in d_l.iterrows():
1881                    raw = d_l_raw.loc[idx]
1882                    fig_2d_y.add_trace(go.Scatter(x=[0, r['PC1']], y=[0,
                        r['PC2']], mode='lines+text', text=[None,
                        r['features']], textposition='top center',
                        line=dict(color='red'), showlegend=False,
                        hoverinfo='text', hovertext=f"Var:
                        {r['features']}<br>Raw: ({r['PC1']:.2f},
                        {r['PC2']:.2f})"))
1883        fig_2d_y.update_layout(title=f"Snapshot {year_2d}",
                xaxis=dict(scaleanchor='y', scaleratio=1),
                template='plotly_white')
1884
1885        # Traj 2D
1886        fig_2d_t = go.Figure()
1887        table_rows = []
1888        if traj_sel:
1889            for sel in traj_sel:
1890                t_type, t_name = sel.split('_', 1)
1891                if t_type == 'skor':
1892                    d_t = df_pc_plt[df_pc_plt['Kabupaten/Kota'] ==
                            t_name].sort_values('Tahun')
1893                    d_t_raw = df_pc_fin[df_pc_fin['Kabupaten/Kota'] ==
                            t_name].sort_values('Tahun')
1894                    fig_2d_t.add_trace(go.Scatter(x=d_t['PC1'],
                            y=d_t['PC2'], mode='lines+markers+text',
                            text=d_t['Tahun'], name=t_name))
1895                    for _, r in d_t.iterrows():
                            table_rows.append({'Nama':t_name,
                            'Tahun':r['Tahun'], 'PC1':round(r['PC1'],4),
                            'PC2':round(r['PC2'],4)})
1896                elif t_type == 'loading':
1897                    d_t = df_load_plt[df_load_plt['features'] ==
                            t_name].sort_values('Tahun')
1898                    d_t_raw = df_load_fin[df_load_fin['features'] ==
                            t_name].sort_values('Tahun')
1899                    fig_2d_t.add_trace(go.Scatter(x=d_t['PC1'],
                            y=d_t['PC2'], mode='lines+markers+text',
                            text=d_t['Tahun'], name=t_name))
1900                    for _, r in d_t.iterrows():
                            table_rows.append({'Nama':t_name,
                            'Tahun':r['Tahun'], 'PC1':round(r['PC1'],4),
                            'PC2':round(r['PC2'],4)})
1901        fig_2d_t.update_layout(title="Trajektori 2D",
                xaxis=dict(scaleanchor='y', scaleratio=1),
                template='plotly_white')
1902
1903        return fig_3d, fig_2d_y, fig_2d_t, table_rows, [{"name": i, "id":
                i} for i in pd.DataFrame(table_rows).columns] if table_rows
                else []
1904
1905    # 6. STATIS PLOTS & TABLES
```

```
@app.callback(
    Output('statis-scree', 'figure'), Output('statis-rv-heatmap', 'figure'),
    Output('statis-contribution', 'figure'), Output('statis-cos2', 'figure'),
    Output('statis-compromise', 'figure'), Output('statis-biplot', 'figure'),
    Output('statis-interstructure', 'figure'), Output('statis-trajectory', 'figure'),
    Output('statis-table-selector', 'options'), Output('statis-detail-table', 'data'), Output('statis-detail-table', 'columns'),
    Output('table-info-display', 'children'), Output('formula-statis-output', 'children'),
    Input('statis-core-store', 'data'), Input('statis-table-selector', 'value'), Input('visual-scaling-opts', 'value'),
    Input('table-page-size', 'value')
)
def cb_statis_viz(res, table_key, vis_opts, page_size):
    if not res: return {}, {}, {}, {}, {}, {}, {}, {}, [], [], [], "", render_statis_formulas()

    # Cek Opsi Visualisasi
    norm_score = 'norm_score' in vis_opts if vis_opts else False
    scale_load = 'scale_load' in vis_opts if vis_opts else False

    # Ambil Data Dasar
    scores = np.array(res['scores'])
    loadings = np.array(res['loadings'])

    # --- 1. Scaling untuk Compromise & Biplot ---
    plot_scores, plot_loadings = apply_visual_adjustments(scores, loadings, norm_score, scale_load)

    # Label Sumbu (Persentase Variansi)
    eig_vals = np.array(res['eig_vals_c'])
    var_pct = eig_vals / eig_vals.sum() * 100
    ax_x = f"Compromise 1 ({var_pct[0]:.2f}%)"
    ax_y = f"Compromise 2 ({var_pct[1]:.2f}%)"

    # Hitung Faktor Normalisasi Global (Penting untuk Trajektori)
    # Agar Trajektori punya skala yang sama dengan Peta Kompromi
    global_max_val = 1.0
    if norm_score:
        global_max_val = np.max(np.abs(scores))
        if global_max_val == 0: global_max_val = 1.0

    # PLOT 1: Compromise Loading (Ganti RV Heatmap di baris atas)
    fig_load = create_biplot(np.zeros((len(res['selected_vars']), 2)), plot_loadings, [], res['selected_vars'],
                             "Compromise Loading", show_points=False, show_arrows=False)
    fig_load.update_layout(xaxis_title=ax_x, yaxis_title=ax_y)

    # PLOT 1b: Interstructure Heatmap (Masih dikirim ke placeholder interstructure)
```

```
    fig_inter = px.imshow(res['R'], text_auto=True,
        x=res['sheet_names'], y=res['sheet_names'],
                          color_continuous_scale='Blues', title="RV
                              Coefficient Matrix (Heatmap)")

    # Placeholder fig_rv (Sesuai output callback lama)
    fig_rv = fig_load

    # NEW PLOTS: Scree Plot, Contribution, Cos2
    fig_scree = go.Figure()
    fig_scree.add_trace(go.Scatter(x=list(range(1, len(eig_vals)+1)),
        y=eig_vals, mode="lines+markers", marker=dict(color='#E91E63',
        size=10)))
    fig_scree.update_layout(title="Scree Plot (Eigenvalue Compromise
        Matrix)", xaxis_title="Component", yaxis_title="Eigenvalue",
        template="plotly_white")

    contrib = (loadings**2) / np.sum(loadings**2, axis=0) #
        Normalization
    fig_contrib = px.bar(x=res['selected_vars'], y=contrib[:, 0],
        labels={'x': 'Variable', 'y': 'Contribution to PC1'},
        title="Variable Contribution (PC1)")
    fig_contrib.update_layout(template="plotly_white")

    cos2 = loadings**2
    fig_cos2 = px.bar(x=res['selected_vars'], y=cos2[:, 0],
        labels={'x': 'Variable', 'y': 'Cos (PC1)'}, title="Cos
        Variable Representation (PC1)")
    fig_cos2.update_layout(template="plotly_white")
    # PLOT 2: Compromise Map
    fig_comp = create_biplot(plot_scores, np.zeros_like(loadings),
        res['obs_labels'], [],
                             "Compromise Map", color='#1E88E5')
    fig_comp.update_layout(xaxis_title=ax_x, yaxis_title=ax_y)

    # PLOT 3: Biplot (Hapus Tanda Panah)
    fig_bi = create_biplot(plot_scores, plot_loadings,
        res['obs_labels'], res['selected_vars'],
                           "Compromise Biplot", show_loadings=True,
                               show_arrows=False)
    fig_bi.update_layout(xaxis_title=ax_x, yaxis_title=ax_y)

    # PLOT 4: Trajectory (Perbaikan Logika)
    coords_dict = res['per_year_coords']
    df_traj_list = []

    for yr, coords in coords_dict.items():
        c = np.array(coords)

        # Terapkan Normalisasi jika User Meminta
        # Kita bagi dengan Max Value dari GLOBAL SCORES agar
            proporsional
        if norm_score:
            c = c / global_max_val

        # Build DF untuk Plotly Express
```

```
            df_t = pd.DataFrame(c[:, :2], columns=['PC1', 'PC2'])
            df_t['Kab/Kota'] = res['obs_labels']
            df_t['Tahun'] = yr
            df_traj_list.append(df_t)

        df_traj = pd.concat(df_traj_list)

        # Gunakan Line Plot (Garis terhubung per Kota)
        fig_traj = px.line(df_traj, x='PC1', y='PC2', color='Kab/Kota',
            markers=True,
                           title='STATIS Trajectories (Projected)')

        # Pastikan layout konsisten
        fig_traj.update_layout(template='plotly_white',
            xaxis=dict(scaleanchor='y', scaleratio=1),
                               xaxis_title=ax_x, yaxis_title=ax_y)

        # 5. Tables Logic
        # Filter key internal agar tidak muncul di dropdown
        exclude_keys = ['R', 'alphas', 'eig_vals_c', 'scores', 'loadings',
            'per_year_coords',
                        'sheet_names', 'X_matrices', 'vals_R_sorted', 'k',
                            'obs_labels',
                        'selected_vars', 'perm_lambda_obs',
                            'perm_distribution', 'perm_p_value']

        table_opts = [{'label': k.replace('table_', '').replace('_', '
            ').title(), 'value': k}
                      for k in res.keys() if k not in exclude_keys]

        tbl_data, tbl_cols, info_text = [], [], ""
        if table_key:
            tbl_data = res.get(table_key, [])
            if tbl_data:
                tbl_cols = [{"name": i, "id": i} for i in
                    tbl_data[0].keys()]
                info_text = f"Menampilkan Tabel: {table_key}"

        # --- AUTO EXPORT LATEX ---
        export_core_latex(res)
        export_tables_latex(res)


        # Render Statis Formulas
        wc = np.array(res['Matriks Kompromi (Wc)'])
        w_slice = None
        if wc.shape[0] > 0:
            # ambil top 3x3 numeric part of it
            w_slice = [[float(v) for v in list(r.values())[1:4]] for r in
                wc[:3]]

        r_matrix = np.array(res['R'])
        form_statis_ui = render_statis_formulas(res['alphas'],
            res['sheet_names'], w_slice, r_matrix)

```

```
2036        return fig_scree, fig_rv, fig_contrib, fig_cos2, fig_comp, fig_bi,
                fig_inter, fig_traj, table_opts, tbl_data, tbl_cols,
                info_text, form_statis_ui
2037
2038
2039    # 7. Permutation Test
2040    @app.callback(
2041        Output('perm-graph', 'figure'), Output('perm-result-text',
                'children'),
2042        Output('formula-perm-output', 'children'),
2043        Input('btn-run-perm', 'n_clicks'), State('perm-n-input', 'value'),
                State('statis-core-store', 'data')
2044    )
2045    def cb_run_perm(n_clicks, n_perm, res):
2046        if not n_clicks or not res: return {}, ""
2047        X_matrices = [np.array(m) for m in res['X_matrices']]
2048        lambda_obs = res['vals_R_sorted'][0]
2049        dist, p_val = run_permutation_test(X_matrices, res['k'],
                lambda_obs, n_perm)
2050
2051        fig = px.histogram(x=dist, title=f"Distribusi Null (N={n_perm})",
                color_discrete_sequence=['#90A4AE'])
2052        fig.add_vline(x=lambda_obs, line_color="#D32F2F",
                line_dash="dash", annotation_text=f"Observed:
                {lambda_obs:.3f}")
2053        fig.update_layout(template='plotly_white')
2054
2055        interp = "Signifikan (Struktur Stabil)" if p_val < 0.05 else
                "Tidak Signifikan (Acak)"
2056
2057        # --- AUTO EXPORT LATEX ---
2058        export_perm_latex(p_val, interp)
2059
2060        # Render Result Formulas
2061        form_ui = render_permutation_formulas(p_val, lambda_obs, n_perm,
                interp)
2062
2063        return fig, f"P-value = {p_val}. Interpretasi: {interp}", form_ui
2064
2065
2066    # 7b. Populate desc-year-dropdown from global-data-store
2067    @app.callback(
2068        Output('desc-year-dropdown', 'options'),
2069        Output('desc-year-dropdown', 'value'),
2070        Input('global-data-store', 'data')
2071    )
2072    def cb_update_desc_years(data):
2073        if not data: return [], None
2074        years = data.get('sheet_names', [])
2075        opts = [{'label': str(y), 'value': y} for y in years]
2076        return opts, (years[0] if years else None)
2077
2078
2079    # 7c. Run Descriptive Statistics
2080    @app.callback(
2081        Output('desc-table-output', 'children'),
```

```
    Output('desc-boxplot-output', 'children'),
    Output('desc-formula-output', 'children'),
    Input('btn-run-desc', 'n_clicks'),
    State('desc-year-dropdown', 'value'),
    State('global-data-store', 'data')
)
def cb_run_desc(n_clicks, selected_year, data):
    if not n_clicks or not data or not selected_year:
        return "", "", ""

    try:
        df_all = pd.read_json(io.StringIO(data['df_json']),
            orient='split')
        # Fix: match year dtype  Tahun may be int or str depending on
            JSON
        tahun_col = df_all['Tahun']
        try:
            mask = tahun_col == int(selected_year)
        except (ValueError, TypeError):
            mask = tahun_col.astype(str) == str(selected_year)
        df_year = df_all[mask].copy()

        # Drop metadata columns
        drop_cols = ['Kabupaten/Kota', 'Provinsi', 'Tahun']
        df_clean = df_year.drop(columns=[c for c in drop_cols if c in
            df_year.columns], errors='ignore')

        # Detect numeric vs non-numeric among remaining columns
        all_cols = df_clean.columns.tolist()
        non_numeric =
            df_clean.select_dtypes(exclude='number').columns.tolist()
        df = df_clean.select_dtypes(include='number').dropna(axis=1,
            how='all')
    except Exception as e:
        return html.P(f"Error membaca data: {e}", style={'color':
            'red'}), "", ""

    # --- Diagnostic info bar ---
    if df_year.empty:
        diag = html.Div(
            f" Tidak ada data untuk tahun {selected_year}. Pastikan
                kolom 'Tahun' di data cocok dengan pilihan.",
            style={'backgroundColor': '#FFF3CD', 'color': '#856404',
                'padding': '10px',
                    'borderRadius': '6px', 'marginBottom': '10px',
                        'fontWeight': 'bold'}
        )
        return diag, "", ""

    info_parts = [f" {len(df.columns)} kolom numerik terdeteksi"]
    if non_numeric:
        info_parts.append(f" {len(non_numeric)} kolom non-numerik
            diabaikan: {', '.join(non_numeric)}")
    diag_color = '#E8F5E9' if not non_numeric else '#FFF8E1'
    diag_bar = html.Div(info_parts[0] + ("  |  " + info_parts[1] if
        len(info_parts) > 1 else ""),
```

```
2127        style={'backgroundColor': diag_color, 'padding': '8px 14px',
                'borderRadius': '6px',
2128                'marginBottom': '12px', 'fontSize': '13px', 'color':
                    '#333'})
2129
2130    if df.empty:
2131        return html.Div([
2132            diag_bar,
2133            html.P("Tidak ada kolom numerik yang dapat dianalisis.",
                    style={'color': 'red'})
2134        ]), "", ""
2135
2136    from scipy.stats import skew, kurtosis as kurt
2137    # Build summary stats
2138    rows = []
2139    for col in df.columns:
2140        vals = df[col].dropna().values
2141        n = len(vals)
2142        mean_v = np.mean(vals)
2143        std_v = np.std(vals, ddof=1)
2144        rows.append({
2145            'Variabel': col,
2146            'N': n,
2147            'Mean': round(mean_v, 4),
2148            'Std Dev': round(std_v, 4),
2149            'Min': round(np.min(vals), 4),
2150            'Q1': round(np.percentile(vals, 25), 4),
2151            'Median': round(np.median(vals), 4),
2152            'Q3': round(np.percentile(vals, 75), 4),
2153            'Max': round(np.max(vals), 4),
2154            'Skewness': round(skew(vals), 4),
2155            'Kurtosis': round(kurt(vals, fisher=True), 4),
2156            'CV (%)': round(std_v / mean_v * 100, 2) if mean_v != 0
                    else None
2157        })
2158
2159    df_stats = pd.DataFrame(rows)
2160
2161    # --- TABLE ---
2162    tbl = dash_table.DataTable(
2163        columns=[{"name": c, "id": c} for c in df_stats.columns],
2164        data=df_stats.to_dict('records'),
2165        style_table={'overflowX': 'auto'},
2166        style_cell={'textAlign': 'center', 'fontSize': '13px',
                'padding': '6px'},
2167        style_header={'backgroundColor': '#DBEAFE', 'fontWeight':
                'bold', 'color': '#1E3A5F', 'textAlign': 'center'},
2168        style_data_conditional=[{'if': {'row_index': 'odd'},
                'backgroundColor': '#F8FAFC'}],
2169        page_size=20
2170    )
2171
2172    tbl_section = html.Div([
2173        diag_bar,
2174        html.H6(f"Tabel Statistika Deskriptif  Tahun {selected_year}",
```

```
                style={'textAlign': 'center', 'fontWeight': 'bold',
                   'color': '#1565C0', 'marginBottom': '10px'}),
        tbl
    ])

    # --- BOXPLOTS (one per column) ---
    figs = []
    for col in df.columns:
        fig = px.box(df, y=col, points='all', title=col,
                     color_discrete_sequence=[COLOR_ACCENT])
        fig.update_layout(
            template='plotly_white', height=300,
            title_x=0.5, margin=dict(l=20, r=20, t=40, b=20),
            showlegend=False
        )
        figs.append(html.Div(dcc.Graph(figure=fig), className='three
           columns'))

    boxplot_section = html.Div([
        html.H6("Distribusi Variabel (Boxplot)", style={'textAlign':
           'center', 'fontWeight': 'bold',
                                                        'color':
                                                           '#1565C0',
                                                           'marginBottom':
                                                           '10px'}),
        html.Div(className='row', style={'display': 'flex',
           'flexWrap': 'wrap'}, children=figs)
    ])

    # --- FORMULAS ---
    formula_section = html.Div([
        html.Hr(style={'borderColor': '#BBDEFB'}),
        html.H6("Rumus Statistika Deskriptif", style={'textAlign':
           'center', 'fontWeight': 'bold',
                                                      'color':
                                                         '#1565C0',
                                                         'marginBottom':
                                                         '5px'}),
        render_descriptive_formulas()
    ])

    return tbl_section, boxplot_section, formula_section


# 8. Update Table Page Size
@app.callback(
    Output('statis-detail-table', 'page_size'),
    Input('table-page-size', 'value')
)
def update_page_size(size):
    return size if size else 10


# 7d. Multi-Year Descriptive Statistics
@app.callback(
    Output('desc-multi-table-output', 'children'),
```

```
    Output('desc-multi-trend-output', 'children'),
    Input('btn-run-desc-multi', 'n_clicks'),
    State('desc-multi-stat', 'value'),
    State('desc-multi-scale', 'value'),
    State('global-data-store', 'data')
)
def cb_run_desc_multi(n_clicks, stat_key, scale_mode, data):
    if not n_clicks or not data:
        return "", ""

    from scipy.stats import skew, kurtosis as kurt

    try:
        df_all = pd.read_json(io.StringIO(data['df_json']),
            orient='split')
        drop_cols = ['Kabupaten/Kota', 'Provinsi']
        years = sorted(df_all['Tahun'].unique())
    except Exception as e:
        return html.P(f"Error: {e}", style={'color': 'red'}), ""

    STAT_LABELS = {
        'mean': 'Mean (Rata-rata)', 'std': 'Std Dev', 'median':
            'Median',
        'q1': 'Q1 (Kuartil 1)', 'q3': 'Q3 (Kuartil 3)',
        'min': 'Min', 'max': 'Max',
        'skewness': 'Skewness', 'kurtosis': 'Kurtosis', 'cv': 'CV (%)'
    }

    # Build per-year stats
    year_dfs = {}
    for yr in years:
        try:
            mask = df_all['Tahun'] == int(yr)
        except Exception:
            mask = df_all['Tahun'].astype(str) == str(yr)
        df_yr = df_all[mask].drop(columns=[c for c in drop_cols +
            ['Tahun'] if c in df_all.columns], errors='ignore')
        df_yr = df_yr.select_dtypes(include='number')
        if df_yr.empty:
            continue

        if stat_key == 'mean':
            year_dfs[yr] = df_yr.mean()
        elif stat_key == 'std':
            year_dfs[yr] = df_yr.std()
        elif stat_key == 'median':
            year_dfs[yr] = df_yr.median()
        elif stat_key == 'q1':
            year_dfs[yr] = df_yr.quantile(0.25)
        elif stat_key == 'q3':
            year_dfs[yr] = df_yr.quantile(0.75)
        elif stat_key == 'min':
            year_dfs[yr] = df_yr.min()
        elif stat_key == 'max':
            year_dfs[yr] = df_yr.max()
        elif stat_key == 'skewness':
```

```
            year_dfs[yr] = df_yr.apply(lambda col: skew(col.dropna()))
        elif stat_key == 'kurtosis':
            year_dfs[yr] = df_yr.apply(lambda col: kurt(col.dropna(),
                fisher=True))
        elif stat_key == 'cv':
            m = df_yr.mean()
            s = df_yr.std()
            year_dfs[yr] = (s / m * 100).where(m != 0)

    if not year_dfs:
        return html.P("Tidak ada data yang dapat ditampilkan.",
            style={'color': 'red'}), ""

    df_pivot = pd.DataFrame(year_dfs).T
    df_pivot.index.name = 'Tahun'
    df_pivot = df_pivot.reset_index().round(4)

    # --- Pivot TABLE ---
    tbl = dash_table.DataTable(
        columns=[{"name": str(c), "id": str(c)} for c in
            df_pivot.columns],
        data=df_pivot.to_dict('records'),
        style_table={'overflowX': 'auto'},
        style_cell={'textAlign': 'center', 'fontSize': '12px',
            'padding': '5px'},
        style_header={'backgroundColor': '#EDE7F6', 'fontWeight':
            'bold', 'color': '#311B92', 'textAlign': 'center'},
        style_data_conditional=[{'if': {'row_index': 'odd'},
            'backgroundColor': '#F3E5F5'}],
        export_format='xlsx'
    )

    tbl_section = html.Div([
        html.H6(f"{STAT_LABELS.get(stat_key, stat_key)} per Variabel
            Semua Tahun",
                style={'textAlign': 'center', 'fontWeight': 'bold',
                    'color': '#4527A0', 'marginBottom': '8px'}),
        tbl
    ])

    # --- TREND LINE CHARTS ---
    vars_to_plot = [c for c in df_pivot.columns if c != 'Tahun']
    trend_figs = []
    for var in vars_to_plot:
        fig = px.line(df_pivot, x='Tahun', y=var, markers=True,
            title=var,
                      color_discrete_sequence=['#5C6BC0'])
        fig.update_traces(line=dict(width=2.5), marker=dict(size=8))
        yaxis_opts = dict(tickmode='auto')
        if scale_mode == 'zero':
            yaxis_opts['rangemode'] = 'tozero'
        fig.update_layout(
            template='plotly_white', height=260, title_x=0.5,
            margin=dict(l=20, r=20, t=40, b=20),
            xaxis=dict(tickmode='array',
                tickvals=list(df_pivot['Tahun'])),
```

```
            yaxis=yaxis_opts,
            showlegend=False
        )
        trend_figs.append(
            html.Div(dcc.Graph(figure=fig, config={'displayModeBar': False}),
                     style={'width': '25%', 'display': 'inline-block', 'padding': '4px'})
        )

    trend_section = html.Div([
        html.H6(f"Tren {STAT_LABELS.get(stat_key, stat_key)} per Variabel (20202024)",
                style={'textAlign': 'center', 'fontWeight': 'bold', 'color': '#4527A0',
                       'marginTop': '20px', 'marginBottom': '8px'}),
        html.Div(trend_figs, style={'display': 'flex', 'flexWrap': 'wrap'})
    ])

    return tbl_section, trend_section


# 8. Update Table Page Size
@app.callback(
    Output('elbow-graph', 'figure'),
    Output('dendrogram-graph', 'figure'),
    Input('eval-year-dropdown', 'value'),
    State('statis-core-store', 'data')
)
def cb_eval_cluster(eval_year, res):
    if not eval_year or not res: return {}, {}
    coords = np.array(res['per_year_coords'][str(eval_year)])
    obs_labels = res['obs_labels']

    # 1. ELBOW
    sse = []
    k_range = range(1, min(11, len(coords)))
    for k in k_range:
        km = KMeans(n_clusters=k, random_state=42, n_init=10)
        km.fit(coords)
        sse.append(km.inertia_)

    fig_elbow = go.Figure()
    fig_elbow.add_trace(go.Scatter(x=list(k_range), y=sse,
        mode='lines+markers', line=dict(color=COLOR_ACCENT, width=3),
        marker=dict(size=8)))
    fig_elbow.update_layout(title=f"Metode Elbow (Tahun {eval_year})",
        xaxis_title="Jumlah Cluster (k)", yaxis_title="Sum of Squared Errors (SSE)", template='plotly_white')
    import os
    os.makedirs("../huggingface", exist_ok=True)
    try:
        fig_elbow.write_image(f"../huggingface/elbow_plot_{eval_year}.png")
    except: pass

```

```
    # 2. DENDROGRAM
    try:
        import plotly.figure_factory as ff
        from scipy.cluster.hierarchy import linkage
        Z = linkage(coords, method='ward')
        fig_dendro = ff.create_dendrogram(coords, labels=obs_labels,
            orientation='left', linkagefun=lambda x: Z,
            colorscale=px.colors.qualitative.Bold)
        fig_dendro.update_layout(title=f"Dendrogram Hierarchical
            (Tahun {eval_year})", template='plotly_white',
            margin=dict(l=150))
        try:
            fig_dendro.write_image(f"../huggingface/dendrogram_{eval_year}.png")
        except: pass
    except Exception as e:
        fig_dendro = go.Figure().add_annotation(text=f"Error
            Dendrogram: {e}", showarrow=False)

    return fig_elbow, fig_dendro


# 10. CLUSTERING TAHUNAN & MATRIKS TRANSISI
@app.callback(
    Output('title-trans-map-t1', 'children'),
    Output('trans-map-t1', 'figure'),
    Output('title-trans-map-t2', 'children'),
    Output('trans-map-t2', 'figure'),
    Output('title-trans-scatter-t1', 'children'),
    Output('trans-scatter-t1', 'figure'),
    Output('title-trans-scatter-t2', 'children'),
    Output('trans-scatter-t2', 'figure'),
    Output('trans-matrix-table', 'data'),
    Output('trans-matrix-table', 'columns'),
    Input('btn-run-trans', 'n_clicks'),
    Input('trans-view-mode', 'value'),
    State('trans-year-dropdown', 'value'),
    State('trans-k-input', 'value'),
    State('statis-core-store', 'data')
)
def cb_run_trans(n_clicks, view_mode, t2_year, k, res):
    if not n_clicks or not t2_year or not res: return "", {}, "", {},
        "", {}, "", {}, [], []

    years = sorted(list(res['per_year_coords'].keys()))
    t2_idx = years.index(str(t2_year))
    if t2_idx == 0: return "Pilih tahun selain tahun pertama.", {},
        "", {}, "", {}, "", {}, [], []
    t1_year = years[t2_idx - 1]

    coords_t1 = np.array(res['per_year_coords'][t1_year])
    coords_t2 = np.array(res['per_year_coords'][str(t2_year)])
    obs_labels = res['obs_labels']

    km_t1 = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels_t1 = km_t1.fit_predict(coords_t1)
    km_t2 = KMeans(n_clusters=k, random_state=42, n_init=10)
```

```
    labels_t2 = km_t2.fit_predict(coords_t2)
    str_labels_t1 = [f"Cluster {l+1}" for l in labels_t1]
    str_labels_t2 = [f"Cluster {l+1}" for l in labels_t2]

    # SCATTER T1
    fig_scatter_t1 = px.scatter(x=coords_t1[:,0], y=coords_t1[:,1],
        color=str_labels_t1, text=obs_labels, title="",
        color_discrete_sequence=px.colors.qualitative.Bold)
    fig_scatter_t1.update_traces(marker=dict(size=9),
        textposition='top center')
    fig_scatter_t1.update_layout(template='plotly_white',
        xaxis_title="PC1", yaxis_title="PC2")

    # SCATTER T2
    fig_scatter_t2 = px.scatter(x=coords_t2[:,0], y=coords_t2[:,1],
        color=str_labels_t2, text=obs_labels, title="",
        color_discrete_sequence=px.colors.qualitative.Bold)
    fig_scatter_t2.update_traces(marker=dict(size=9),
        textposition='top center')
    fig_scatter_t2.update_layout(template='plotly_white',
        xaxis_title="PC1", yaxis_title="PC2")

    import os
    os.makedirs("../huggingface", exist_ok=True)
    try:
        fig_scatter_t1.write_image(f"../huggingface/cluster_biplot_{t1_year}.png")
    except: pass
    try:
        fig_scatter_t2.write_image(f"../huggingface/cluster_biplot_{t2_year}.png")
    except: pass

    # MAP T1 & T2
    fig_map_t1 = go.Figure()
    fig_map_t2 = go.Figure()
    if indonesia_geojson:
        # Menyesuaikan Center & Zoom sesuai request user
        # mapbox_center = {"lat": -6.9, "lon": 108.5} # Geser ke kiri
            (bujur lebih tinggi maka map ketarik ke kiri relatif
            terhadap frame)
        # zoom sekitar 6.2 dari asalnya 7
        mapbox_center = {"lat": -6.9, "lon": 107}
        mapbox_zoom = 6.5

        df_map_t1 = pd.DataFrame({'Kab/Kota': obs_labels, 'Cluster':
            str_labels_t1, 'match_key': obs_labels})
        df_map_t2 = pd.DataFrame({'Kab/Kota': obs_labels, 'Cluster':
            str_labels_t2, 'match_key': obs_labels})

        for feature in indonesia_geojson['features']:
            props = feature['properties']
            raw_name = props.get('NAME_2') or props.get('kab_kota') or
                props.get('NAMOBJ') or props.get('Name') or ""
            feature['properties']['id_cocok'] = str(raw_name)

        fig_map_t1 = px.choropleth_mapbox(df_map_t1,
            geojson=indonesia_geojson, locations='match_key',
```

```
            featureidkey='properties.id_cocok', color='Cluster',
            color_discrete_sequence=px.colors.qualitative.Bold,
            hover_name='Kab/Kota', mapbox_style="carto-positron",
            center=mapbox_center, zoom=mapbox_zoom, opacity=0.7)
        fig_map_t1.update_layout(title="",
            margin={"r":0,"t":0,"l":0,"b":0})
        # Add Text Labels
        lon_list, lat_list, text_list = [], [], []
        for _, row in df_map_t1.iterrows():
            area = row['Kab/Kota']
            geom = gdf[gdf['id_cocok'] == area]
            if not geom.empty:
                lon_list.append(geom.geometry.centroid.x.values[0])
                lat_list.append(geom.geometry.centroid.y.values[0])
                text_list.append(area)
        if text_list:
            fig_map_t1.add_trace(go.Scattermapbox(lon=lon_list,
                lat=lat_list, mode='text', text=text_list,
                textfont=dict(size=9, color='black'),
                showlegend=False))

        fig_map_t2 = px.choropleth_mapbox(df_map_t2,
            geojson=indonesia_geojson, locations='match_key',
            featureidkey='properties.id_cocok', color='Cluster',
            color_discrete_sequence=px.colors.qualitative.Bold,
            hover_name='Kab/Kota', mapbox_style="carto-positron",
            center=mapbox_center, zoom=mapbox_zoom, opacity=0.7)
        fig_map_t2.update_layout(title="",
            margin={"r":0,"t":0,"l":0,"b":0})
        # Add Text Labels
        lon_list2, lat_list2, text_list2 = [], [], []
        for _, row in df_map_t2.iterrows():
            area = row['Kab/Kota']
            geom = gdf[gdf['id_cocok'] == area]
            if not geom.empty:
                lon_list2.append(geom.geometry.centroid.x.values[0])
                lat_list2.append(geom.geometry.centroid.y.values[0])
                text_list2.append(area)
        if text_list2:
            fig_map_t2.add_trace(go.Scattermapbox(lon=lon_list2,
                lat=lat_list2, mode='text', text=text_list2,
                textfont=dict(size=9, color='black'),
                showlegend=False))

        try:
            fig_map_t1.write_image(f"../huggingface/cluster_map_{t1_year}.png")
        except: pass
        try:
            fig_map_t2.write_image(f"../huggingface/cluster_map_{t2_year}.png")
        except: pass

    # MATRIKS TRANSISI
    def build_matrix(mode):
        matrix_data, matrix_tex = [], []
        matrix_tex.append(r"% !TeX root = ../induk.tex")
        matrix_tex.append(r"\begin{table}[H]")
```

```
2490            matrix_tex.append(r"\centering")
2491            matrix_tex.append(r"\caption{Matriks Transisi Klaster Tahun "
                    + str(t1_year) + r" ke " + str(t2_year) + r"}
                    \label{tab:transisi_" + str(t1_year) + r"_" + str(t2_year)
                    + r"}")
2492            matrix_tex.append(r"\begin{tabular}{|c|" + "p{2cm}|"*k + "}")
2493            matrix_tex.append(r"\hline")
2494
2495            header_row = [f"\\makecell{{\\textbf{{Tahun {t1_year}}} \\\\
                    \\textbf{{{t2_year}}}}}"] + [f"\\textbf{{Cluster {j+1}
                    ({t2_year})}}" for j in range(k)]
2496            matrix_tex.append(" & ".join(header_row) + r" \\ \hline")
2497
2498            for i in range(k):
2499                row_dict = {f"Tahun {t1_year} \\ {t2_year}": f"Cluster
                        {i+1} ({t1_year})"}
2500                tex_row = [f"\\textbf{{Cluster {i+1} ({t1_year})}}"]
2501                for j in range(k):
2502                    intersect_indices = [idx for idx, (l1, l2) in
                            enumerate(zip(labels_t1, labels_t2)) if l1 == i
                            and l2 == j]
2503                    if mode == 'count': val = str(len(intersect_indices))
2504                    else: val = ", ".join([obs_labels[idx] for idx in
                            intersect_indices]) if intersect_indices else "-"
2505                    row_dict[f"Cluster {j+1} ({t2_year})"] = val
2506                    tex_row.append(val.replace('&', r'\\&').replace('%',
                            r'\\%'))
2507                matrix_data.append(row_dict)
2508                matrix_tex.append(" & ".join(tex_row) + r" \\ \hline")
2509
2510            matrix_tex.append(r"\end{tabular}")
2511            matrix_tex.append(r"\end{table}")
2512            return matrix_data, matrix_tex
2513
2514        data_count, tex_count = build_matrix('count')
2515        data_names, tex_names = build_matrix('names')
2516
2517        try:
2518            with
                    open(f"../huggingface/output_table_transition_matrix_sum_{t1_year}x{t2_yea
                    "w") as f: f.write("\n".join(tex_count))
2519            with
                    open(f"../huggingface/output_table_transition_matrix_name_{t1_year}x{t2_ye
                    "w") as f: f.write("\n".join(tex_names))
2520        except: pass
2521
2522        req_data = data_count if view_mode == 'count' else data_names
2523        cols = [{"name": c, "id": c} for c in req_data[0].keys()] if
                req_data else []
2524
2525        return (
2526            f"Peta Sebaran Cluster (Tahun {t1_year})", fig_map_t1,
2527            f"Peta Sebaran Cluster (Tahun {t2_year})", fig_map_t2,
2528            f"Biplot Cluster (Tahun {t1_year})", fig_scatter_t1,
2529            f"Biplot Cluster (Tahun {t2_year})", fig_scatter_t2,
2530            req_data, cols
```

```
    )


# 11. CLUSTERING TRAJEKTORI CALLBACK (STATIS FULL)
@app.callback(
    Output('cluster-scatter', 'figure'),
    Output('cluster-dendrogram', 'figure'),
    Output('cluster-map', 'figure'),
    Output('cluster-interpretation', 'children'),
    Output('formula-cluster-output', 'children'),
    Input('btn-run-cluster', 'n_clicks'),
    State('cluster-k-input', 'value'),
    State('statis-core-store', 'data')
)
def cb_run_clustering_trajectory(n_clicks, k_clusters, res):
    if not n_clicks or not res:
        return {}, {}, {}, "Silakan klik tombol 'RUN CLUSTERING'.",
           render_cluster_formulas()

    obs_labels = res['obs_labels']
    coords_dict = res['per_year_coords']
    years = sorted(list(coords_dict.keys()))

    # --- 1. Clustering Logic ---
    features = []
    for i in range(len(obs_labels)):
        city_feats = []
        for yr in years:
            coords = np.array(coords_dict[str(yr)])
            city_feats.extend(coords[i, :])
        features.append(city_feats)

    X_traj = np.array(features)
    kmeans = KMeans(n_clusters=k_clusters, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_traj)
    str_labels = [f"Cluster {l+1}" for l in labels]

    # --- 2. Scatter Plot ---
    scores_comp = np.array(res['scores'])
    fig_scatter = px.scatter(
        x=scores_comp[:, 0], y=scores_comp[:, 1], color=str_labels,
        text=obs_labels, title="Cluster pada Ruang Kompromi",
        color_discrete_sequence=px.colors.qualitative.Bold
    )
    fig_scatter.update_traces(marker=dict(size=8), textposition='top
       center')
    fig_scatter.update_layout(template='plotly_white', width=800,
       height=800)
    import os
    os.makedirs("../huggingface", exist_ok=True)
    try:
       fig_scatter.write_image("../huggingface/cluster_compromise_biplot.png")
    except: pass

    # --- 3. Dendrogram (Dengan Pewarnaan Sesuai K) ---
    try:
```

```
2583            import plotly.figure_factory as ff
2584            from scipy.cluster.hierarchy import linkage
2585
2586            # 1. Hitung Linkage Matrix manual untuk mencari threshold
2587            Z = linkage(X_traj, method='ward')
2588
2589            # 2. Hitung Threshold Jarak agar terbentuk persis K cluster
2590            # Logika: Kita cari jarak merge pada langkah ke-(k-1) dari
                    atas.
2591            # Kita set threshold sedikit di bawah jarak tersebut.
2592            if k_clusters > 1 and k_clusters < len(X_traj):
2593                # Ambil rata-rata jarak antara merge k dan k-1 agar aman
2594                dist_k = Z[-k_clusters, 2]      # Jarak saat masih k+1
                        cluster
2595                dist_k_minus_1 = Z[-(k_clusters-1), 2] # Jarak saat jadi k
                        cluster
2596                color_thresh = 0.5 * (dist_k + dist_k_minus_1)
2597            else:
2598                color_thresh = 0 # Default jika k=1 atau error
2599
2600            # 3. Buat Dendrogram dengan threshold tersebut
2601            fig_dendro = ff.create_dendrogram(
2602                X_traj,
2603                labels=obs_labels,
2604                orientation='left',
2605                linkagefun=lambda x: Z, # Gunakan Z yang sama agar
                        konsisten
2606                color_threshold=color_thresh, # <--- INI KUNCINYA
2607                colorscale=px.colors.qualitative.Bold
2608            )
2609
2610            fig_dendro.update_layout(title="Dendrogram Kemiripan
                    Trajektori", template='plotly_white', margin=dict(l=150))
2611            import os
2612            os.makedirs("../huggingface", exist_ok=True)
2613            try:
                    fig_dendro.write_image("../huggingface/dendrogram_compromise.png")
2614            except: pass
2615        except Exception as e:
2616            fig_dendro = go.Figure().add_annotation(text=f"Error
                    Dendrogram: {e}", showarrow=False)
2617
2618        # --- 3B. Hitung Moran's I untuk PC1 Kompromi (Sumbu Ekonomi
                Utama) ---
2619        moran_text = ""
2620        formulas_m_i = None
2621        formulas_m_exp = None
2622        if res and indonesia_geojson and gdf is not None:
2623            try:
2624                # Ambil skor PC1
2625                scores = np.array(res['scores'])
2626                pc1_scores = scores[:, 0]
2627                obs_labels_mc = res['obs_labels']
2628
2629                # Hitung
```

```
            m_i, m_exp, n, sum_w, sum_w_dev, sum_dev_sq =
                _calculate_moran_manual(gdf, pc1_scores, obs_labels_mc)

            if m_i is not None:
                status = "Berkelompok (Clustered)" if m_i > m_exp else
                    "Menyebar (Dispersed/Random)"
                moran_text = f" | Global Moran's I (PC1): {m_i:.4f}
                    (Exp: {m_exp:.4f}) -> Pola: {status}"
                formulas_m_i, formulas_m_exp, formulas_n,
                    formulas_sum_w, formulas_sum_w_dev,
                    formulas_sum_dev_sq = m_i, m_exp, n, sum_w,
                    sum_w_dev, sum_dev_sq
        except Exception as e:
            print(f"Gagal hitung Moran di callback: {e}")

    # --- 4. PETA CHOROPLETH (UPDATED MATCHING LOGIC) ---
    fig_map = go.Figure()

    if indonesia_geojson:
        # A. Siapkan DataFrame
        df_map = pd.DataFrame({'Kab/Kota': obs_labels, 'Cluster':
            str_labels})
        df_map['match_key'] = df_map['Kab/Kota']

        # B. Inject Key ke GeoJSON (In-Memory)
        found_matches = 0

        for feature in indonesia_geojson['features']:
            props = feature['properties']

            # Cari nama di GeoJSON (Eksak)
            clean_geo_name = props.get('id_cocok') or ""

            # Cek apakah cocok dengan data Excel
            if clean_geo_name in df_map['match_key'].values:
                found_matches += 1

        # C. Plotting
        title_text = f"Peta Sebaran Cluster (Cocok: {found_matches}
            Wilayah)"
        if found_matches == 0:
            title_text += " - PERINGATAN: Nama wilayah tidak ada yang
                cocok!"

        fig_map = px.choropleth_mapbox(
            df_map,
            geojson=indonesia_geojson,
            locations='match_key',           # Eksak
            featureidkey='properties.id_cocok',
            color='Cluster',
            color_discrete_sequence=px.colors.qualitative.Bold,
            hover_name='Kab/Kota',
            mapbox_style="carto-positron",
            center={"lat": -6.9, "lon": 107}, # Geser ke kiri (zoom
                out 2x)
            zoom=6.5, opacity=0.7
```

```
        )
        fig_map.update_layout(title=title_text,
            margin={"r":0,"t":40,"l":0,"b":0})
        # Add Text Labels
        lon_list_c, lat_list_c, text_list_c = [], [], []
        for _, row in df_map.iterrows():
            area = row['Kab/Kota']
            geom = gdf[gdf['id_cocok'] == area]
            if not geom.empty:
                lon_list_c.append(geom.geometry.centroid.x.values[0])
                lat_list_c.append(geom.geometry.centroid.y.values[0])
                text_list_c.append(area)
        if text_list_c:
            fig_map.add_trace(go.Scattermapbox(lon=lon_list_c,
                lat=lat_list_c, mode='text', text=text_list_c,
                textfont=dict(size=9, color='black'),
                showlegend=False))
        import os
        os.makedirs("../huggingface", exist_ok=True)
        try:
            fig_map.write_image("../huggingface/cluster_compromise_map.png")
        except: pass
    else:
        fig_map.add_annotation(text="Gagal memuat data GeoJSON.",
            showarrow=False)

    # 5. Interpretasi
    counts = np.bincount(labels)
    interp_text = [html.Strong("Interpretasi Cluster: "), html.Br()]
    if moran_text:
        interp_text.append(html.Span(moran_text, style={'color':
            '#D81B60', 'fontWeight': 'bold'}))
        interp_text.append(html.Br())

    for i, count in enumerate(counts):
        interp_text.append(f"Cluster {i+1}: {count} Wilayah.")
        if i < len(counts)-1: interp_text.append(" | ")

    # --- 6. Formulasi ---
    if formulas_m_i is not None:
        math_ui = render_cluster_formulas(formulas_m_i,
            formulas_m_exp, formulas_n, formulas_sum_w,
            formulas_sum_w_dev, formulas_sum_dev_sq)
    else:
        math_ui = render_cluster_formulas()

    return fig_scatter, fig_dendro, fig_map, interp_text, math_ui

# 12. IPM MATRIX CALLBACK (UNDP CATEGORIES)
@app.callback(
    Output('ipm-disclaimer-text', 'children'),
    Output('ipm-trans-matrix-table', 'data'),
    Output('ipm-trans-matrix-table', 'columns'),
    Input('btn-run-ipm-trans', 'n_clicks'),
    Input('ipm-trans-view-mode', 'value'),
    State('ipm-trans-year-dropdown', 'value'),
```

```
    State('global-data-store', 'data')
)
def cb_run_ipm_trans(n_clicks, view_mode, t2_year, data):
    if not n_clicks or not t2_year or not data: return "", [], []

    df = pd.read_json(io.StringIO(data['df_json']), orient='split')
    years = data['years']

    if 'IPM' not in df.columns:
        return "Kolom IPM tidak ditemukan pada data yang diunggah.", [], []

    t2_idx = years.index(t2_year)
    if t2_idx == 0: return "Pilih tahun selain tahun pertama.", [], []
    t1_year = years[t2_idx - 1]

    # Filter Data untuk T1 dan T2
    df_t1 = df[df['Tahun'] == t1_year].set_index('Kabupaten/Kota')
    df_t2 = df[df['Tahun'] == t2_year].set_index('Kabupaten/Kota')

    common_kabs = df_t1.index.intersection(df_t2.index)
    if len(common_kabs) == 0: return "Tidak ada data Kabupaten/Kota yang cocok antara dua tahun.", [], []

    df_t1 = df_t1.loc[common_kabs]
    df_t2 = df_t2.loc[common_kabs]

    # Kategori UNDP
    def categorize_ipm(val):
        if val >= 80: return "Sangat Tinggi"
        elif val >= 70: return "Tinggi"
        elif val >= 55: return "Sedang"
        else: return "Rendah"

    df_t1['IPM_Cat'] = df_t1['IPM'].apply(categorize_ipm)
    df_t2['IPM_Cat'] = df_t2['IPM'].apply(categorize_ipm)

    categories = ["Sangat Tinggi", "Tinggi", "Sedang", "Rendah"]

    # Hitung Disclaimer / Summary
    counts_t1 = df_t1['IPM_Cat'].value_counts()
    counts_t2 = df_t2['IPM_Cat'].value_counts()

    disclaimer_parts = []
    disclaimer_parts.append(html.Span(f"Tahun {t1_year}: ", style={'color': '#1565C0'}))
    for cat in categories:
        if cat in counts_t1: disclaimer_parts.append(f"{cat} ({counts_t1[cat]}), ")
    disclaimer_parts.append(html.Br())
    disclaimer_parts.append(html.Span(f"Tahun {t2_year}: ", style={'color': '#D81B60'}))
    for cat in categories:
        if cat in counts_t2: disclaimer_parts.append(f"{cat} ({counts_t2[cat]}), ")

```

```
    # Bangun Matriks
    def build_ipm_matrix(mode):
        matrix_data, matrix_tex = [], []
        matrix_tex.append(r"% !TeX root = ../induk.tex")
        matrix_tex.append(r"\begin{table}[H]")
        matrix_tex.append(r"\centering")
        matrix_tex.append(r"\caption{Matriks Transisi IPM Tahun " +
            str(t1_year) + r" ke " + str(t2_year) + r"}
            \label{tab:ipm_transisi_" + str(t1_year) + r"_" +
            str(t2_year) + r"}")
        matrix_tex.append(r"\begin{tabular}{|c|" +
            "p{2cm}|"*len(categories) + "}")
        matrix_tex.append(r"\hline")

        header_row = [f"\\makecell{{\\textbf{{Tahun {t1_year}}} \\\\
            \\textbf{{{t2_year}}}}}"] + [f"\\textbf{{{cat}
            ({t2_year})}}" for cat in categories]
        matrix_tex.append(" & ".join(header_row) + r" \\ \hline")

        for cat_t1 in categories:
            row_dict = {f"Tahun {t1_year} \\ {t2_year}": f"{cat_t1}
                ({t1_year})"}
            tex_row = [f"\\textbf{{{cat_t1} ({t1_year})}}"]
            for cat_t2 in categories:
                # Ambil daerah yang pindah dari cat_t1 ke cat_t2
                mask = (df_t1['IPM_Cat'] == cat_t1) &
                    (df_t2['IPM_Cat'] == cat_t2)
                kabs = df_t1[mask].index.tolist()

                if mode == 'count': val = str(len(kabs))
                else: val = ", ".join(kabs) if kabs else "-"
                row_dict[f"{cat_t2} ({t2_year})"] = val
                tex_row.append(val.replace('&', r'\\&').replace('%',
                    r'\\%'))

            matrix_data.append(row_dict)
            matrix_tex.append(" & ".join(tex_row) + r" \\ \hline")

        matrix_tex.append(r"\end{tabular}")
        matrix_tex.append(r"\end{table}")
        return matrix_data, matrix_tex

    data_count, tex_count = build_ipm_matrix('count')
    data_names, tex_names = build_ipm_matrix('names')

    import os
    os.makedirs("../huggingface", exist_ok=True)
    try:
        with
            open(f"../huggingface/output_table_ipm_matrix_sum_{t1_year}x{t2_year}.tex"
            "w") as f: f.write("\n".join(tex_count))
        with
            open(f"../huggingface/output_table_ipm_matrix_name_{t1_year}x{t2_year}.tex
            "w") as f: f.write("\n".join(tex_names))
    except: pass
```

```
    req_data = data_count if view_mode == 'count' else data_names
    cols = [{"name": c, "id": c} for c in req_data[0].keys()] if
        req_data else []

    return disclaimer_parts, req_data, cols


# 13. CROSS TABULATION CALLBACK (K-MEANS VS IPM)
@app.callback(
    Output('cross-table', 'data'),
    Output('cross-table', 'columns'),
    Input('btn-run-cross', 'n_clicks'),
    Input('cross-view-mode', 'value'),
    State('cross-year-dropdown', 'value'),
    State('cross-k-input', 'value'),
    State('global-data-store', 'data'),
    State('statis-core-store', 'data')
)
def cb_run_cross_tab(n_clicks, view_mode, sel_year, k, data_store,
    statis_store):
    if not n_clicks or not sel_year or not data_store or not
        statis_store: return [], []

    # 1. Ambil IPM kategories dari data asli
    df = pd.read_json(io.StringIO(data_store['df_json']),
        orient='split')
    if 'IPM' not in df.columns: return [], []

    df_yr = df[df['Tahun'] == sel_year].set_index('Kabupaten/Kota')
    def categorize_ipm(val):
        if val >= 80: return "Sangat Tinggi"
        elif val >= 70: return "Tinggi"
        elif val >= 55: return "Sedang"
        else: return "Rendah"
    df_yr['IPM_Cat'] = df_yr['IPM'].apply(categorize_ipm)

    # 2. Ambil label K-Means dari statis store
    obs_labels = statis_store['obs_labels']
    str_year = str(sel_year)
    if str_year not in statis_store['per_year_coords']: return [], []

    coords = np.array(statis_store['per_year_coords'][str_year])
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(coords)

    # Gabung
    df_clusters = pd.DataFrame({'Cluster_KMeans': [f"Cluster {l+1}"
        for l in labels]}, index=obs_labels)
    df_merged =
        df_yr.join(df_clusters).dropna(subset=['Cluster_KMeans',
        'IPM_Cat'])

    # Tabulasi Silang
    categories = ["Sangat Tinggi", "Tinggi", "Sedang", "Rendah"]

    def build_cross_matrix(mode):
```

```
2865            matrix_data, matrix_tex = [], []
2866            matrix_tex.append(r"% !TeX root = ../induk.tex")
2867            matrix_tex.append(r"\begin{table}[H]")
2868            matrix_tex.append(r"\centering")
2869            matrix_tex.append(r"\caption{Perbandingan Anggota Cluster
                    dengan Kategori IPM (" + str(sel_year) + r")}
                    \label{tab:ipm_cross_" + str(sel_year) + r"}")
2870            matrix_tex.append(r"\begin{tabular}{|c|" +
                    "p{2cm}|"*len(categories) + "}")
2871            matrix_tex.append(r"\hline")
2872
2873            header_row = ["Cluster K-Means"] + categories
2874            matrix_tex.append(" & ".join([f"\\textbf{{{x}}}" for x in
                    header_row]) + r" \\ \hline")
2875
2876            for c_idx in range(k):
2877                c_name = f"Cluster {c_idx+1}"
2878                row_dict = {"Cluster K-Means": c_name}
2879                tex_row = [f"\\textbf{{{c_name}}}"]
2880                for cat in categories:
2881                    kabs = df_merged[(df_merged['Cluster_KMeans'] ==
                            c_name) & (df_merged['IPM_Cat'] ==
                            cat)].index.tolist()
2882                    if mode == 'count': val = str(len(kabs))
2883                    else: val = ", ".join(kabs) if kabs else "-"
2884                    row_dict[cat] = val
2885                    tex_row.append(val.replace('&', r'\\&').replace('%',
                            r'\\%'))
2886                matrix_data.append(row_dict)
2887                matrix_tex.append(" & ".join(tex_row) + r" \\ \hline")
2888
2889            matrix_tex.append(r"\end{tabular}")
2890            matrix_tex.append(r"\end{table}")
2891            return matrix_data, matrix_tex
2892
2893        data_count, tex_count = build_cross_matrix('count')
2894        data_names, tex_names = build_cross_matrix('names')
2895
2896        import os
2897        os.makedirs("../huggingface", exist_ok=True)
2898        try:
2899            with
                    open(f"../huggingface/output_table_cross_matrix_sum_{sel_year}.tex",
                    "w") as f: f.write("\n".join(tex_count))
2900            with
                    open(f"../huggingface/output_table_cross_matrix_name_{sel_year}.tex",
                    "w") as f: f.write("\n".join(tex_names))
2901        except: pass
2902
2903        req_data = data_count if view_mode == 'count' else data_names
2904        cols = [{"name": c, "id": c} for c in req_data[0].keys()] if
                req_data else []
2905        return req_data, cols
2906
2907
2908    app.layout = layout
```

```
2909
2910 if __name__ == '__main__':
2911     # Run on port 7860 for Hugging Face Spaces compatibility
2912     app.run_server(host='0.0.0.0', port=7860, debug=False)
```